

**UNIVERSITY OF TRANSPORT HO CHI MINH
CITY
INSTITUTE OF INFORMATION TECHNOLOGY**

images/logo UTH.jpg

FINAL TERM REPORT

Software Technology Learning Course

PROJECT TOPIC

**PERSONALIZED CAREER
ORIENTATION
& LEARNING ROADMAP PLATFORM
FOR SOFTWARE ENGINEERING
STUDENTS**

Group 2

Ho Chi Minh City, July 2026

Instructor Nguyen Van Chien

Students Nguyen Van Bao

Phan Thi Duy Chung

Vo Thi Kiem Dung

Tran Thi Bich Diem

Vo Trong Nguyen

Vu Le Hoang Nhat

Vo Bao Phuc

Contents

I	Project Introduction	1
1	Overview	1
1.1	Project Information	1
1.2	Project Team	2
2	Product Background	2
3	Existing Systems	3
4	Business Opportunity	3
5	Software Product Vision	4
6	Project Scope & Limitations	5
6.1	Major Features	5
6.1.1	Major Features for Administrators	5
6.1.2	Major Features for Student	5
6.1.3	Major Features for Academic Counselor	6
6.1.4	Major Features for Industry Mentor	6
6.2	Limitations & Exclusions	6
II	Project Management Plan	8
1	Overview	8
1.1	Project Scope and Estimation	8
1.2	Testing Objectives	9
1.3	Project Risks	9
2	Management Approach	10
2.1	Development Methodology	10
2.2	Quality Management	10
2.3	Training Plan	10
3	Project Deliverables	11
4	Responsibility Assignments	11

5	Project Communications	12
6	Configuration Management	12
6.1	Document Management	12
6.2	Source Code Management	12
6.3	Version Control Policy	12
III Software Requirement Specification		14
1	Product Overview	14
2	User Requirements	15
2.1	Actors	15
2.2	Use Cases	17
2.2.1	Diagram(s)	17
2.2.2	Descriptions	17
3	Functional Requirements	21
3.1	System Functional Requirements	21
3.1.1	Screens Flow	21
3.1.2	Screen Descriptions	25
3.1.3	Screen Authorization	30
3.1.4	Non-Screen Functions	32
3.1.5	Entity Relationship Diagram	34
3.2	Authentication	34
3.2.1	Login	34
3.3	User Management	35
3.3.1	Manage Users	35
3.3.2	Import Students	36
3.3.3	Import Mentors	36
3.4	Career Assessment Management	37
3.4.1	Student Take Career Assessment	37
3.4.2	View Assessment Result	38
3.4.3	Mentor Review Assessment	38
3.4.4	Manage Assessment Questions	39
3.5	Mentorship Management	40
3.5.1	Student Request Mentor	40
3.5.2	Mentor Review Request	41
3.5.3	Mentor Respond Request	41

3.5.4	Schedule Mentoring Session	42
3.5.5	View Mentoring History	42
3.6	Learning Roadmap Management	43
3.6.1	Generate Learning Roadmap	43
3.6.2	Update Learning Progress	44
3.6.3	Export Learning Roadmap	45
3.7	Learning Resource Management	45
3.7.1	Generate Learning Resource Recommendations	46
3.7.2	Manage Learning Resources	46
3.8	Timeline Management	47
3.8.1	Manage Career Development Timeline	47
3.9	Blog Management	49
3.9.1	Create Blog Post	50
3.9.2	Interact with Blog Post	52
3.9.3	Apply for Internship or Team Recruitment	53
3.10	Semester Management	55
3.10.1	Manage Semester	55
3.11	Criteria Form Management	57
3.11.1	Manage Criteria Form	57
IV Software Design Description		60
1	System Design	60
1.1	System Architecture	60
1.1.1	Frontend	61
1.1.2	Backend	62
1.1.3	AI and Third-party Services	64
1.2	Package Diagram	64
1.2.1	Frontend Package Structure	64
1.2.2	Backend Package Structure	65
2	Database Design	66
2.1	Database Tables Overview	67
2.2	Users Table	67
2.3	CareerPaths Table	67
2.4	SkillNodes Table	67
2.5	LearningResources Table	67

2.6	StudentProfiles Table	68
2.7	StaffProfiles Table	68
2.8	StudentCourses Table	68
2.9	CareerPathSkills Table	69
2.10	Roadmaps Table	69
2.11	RoadmapProgress Table	69
2.12	Portfolios Table	69
2.13	GitHubRepositories Table	70
2.14	GitHubRepoLanguages Table	70
2.15	ConsultationSessions Table	70
2.16	ConsultationFeedbacks Table	71
2.17	JobTrends Table	71
2.18	AIConfigurations Table	71
2.19	AIAnalysisLogs Table	71
2.20	Notifications Table	72
3	Detailed Design	72
3.1	Use Case to Design Coverage Summary	72
3.2	Authentication and Account Access	75
3.2.1	UC01 Register	75
3.2.2	UC02 Login	77
3.2.3	UC03 Login with Google	80
3.2.4	UC04 Forgot Password	82
3.2.5	UC05 Logout	85
3.3	Student Profile and GitHub Integration	87
3.3.1	UC06 Manage Profile	88
3.3.2	UC07 Upload Avatar	90
3.3.3	UC08 Update Contact Information	92
3.3.4	UC09 Change Password	95
3.3.5	UC10 Upload Transcript	97
3.3.6	UC11 Upload CV	100
3.3.7	UC12 Link GitHub	102
3.3.8	UC13 Sync GitHub Repository	105
3.3.9	UC14 View GitHub Analysis	107
3.3.10	UC15 View Skill Gap Report	110

3.4	Career Path and Skill Management	112
3.4.1	UC16 Select Career Role	113
3.4.2	UC17 Update Career Goal	115
3.4.3	UC18 View Recommended Roles	117
3.4.4	UC19 Compare Career Paths	120
3.4.5	UC20 View Required Skills	122
3.5	Learning Roadmap and Course Management	125
3.5.1	UC21 Generate Learning Roadmap	125
3.5.2	UC22 View Learning Roadmap	127
3.5.3	UC23 Update Roadmap Progress	130
3.5.4	UC24 View Learning Progress	132
3.5.5	UC25 Receive Learning Reminder	135
3.5.6	UC26 Search Learning Resources	137
3.5.7	UC27 View Recommended Courses	140
3.5.8	UC28 Save Favorite Courses	142
3.5.9	UC29 Register External Course	145
3.5.10	UC30 Track Course Completion	147
3.6	Market Trend Management	150
3.6.1	UC31 View Market Trends	150
3.6.2	UC32 Search Job Opportunities	152
3.6.3	UC33 View Salary Information	155
3.6.4	UC34 View Technology Trends	157
3.6.5	UC35 Receive Job Recommendation	160
3.7	AI Mentor and Analysis Logging	162
3.7.1	UC36 Chat with AI Mentor	163
3.7.2	UC37 Ask Career Questions	165
3.7.3	UC38 Receive AI Feedback	167
3.8	E-Portfolio and Mentor Review	170
3.8.1	UC39 Generate E-Portfolio	170
3.8.2	UC40 Share Portfolio	172
3.9	Authentication and Account Access	175
3.9.1	UC41 Login	175
3.10	Consultation and Feedback Management	177
3.10.1	UC42 View Student Profile	178

3.10.2	UC43 Review Skill Gap	180
3.10.3	UC44 Recommend Career Path	182
3.10.4	UC45 Recommend Courses	185
3.10.5	UC46 Track Student Progress	187
3.10.6	UC47 Comment Learning Plan	190
3.10.7	UC48 Schedule Consultation	192
3.10.8	UC49 View Consultation History	195
3.11	Authentication and Account Access	197
3.11.1	UC50 Logout	198
3.11.2	UC51 Login	200
3.12	E-Portfolio and Mentor Review	202
3.12.1	UC52 View Student Portfolio	203
3.12.2	UC53 Review Projects	205
3.12.3	UC54 Recommend Technologies	207
3.12.4	UC55 Give Technical Advice	210
3.12.5	UC56 Evaluate Progress	212
3.12.6	UC57 Comment Projects	215
3.12.7	UC58 Approve Portfolio	217
3.12.8	UC59 Suggest Improvements	220
3.13	Authentication and Account Access	222
3.13.1	UC60 Logout	223
3.13.2	UC61 Login	225
3.14	Administration and Resource Management	227
3.14.1	UC62 Manage Users	228
3.15	Career Path and Skill Management	230
3.15.1	UC63 Manage Career Roles	230
3.15.2	UC64 Manage Skill Nodes	232
3.16	Administration and Resource Management	235
3.16.1	UC65 Manage Courses	235
3.16.2	UC66 Manage Learning Resources	237
3.17	Market Trend Management	240
3.17.1	UC67 Manage Job Trend Data	240
3.18	AI and Market Configuration	242
3.18.1	UC68 Manage AI Configuration	243

3.19	Administration and Resource Management	245
3.19.1	UC69 Manage Reports	245
3.20	Authentication and Account Access	247
3.20.1	UC70 Logout	248
3.21	AI Mentor and Analysis Logging	250
3.21.1	UC71 Analyze Transcript	250
3.21.2	UC72 Analyze GitHub Repository	252
3.22	Learning Roadmap and Course Management	255
3.22.1	UC73 Generate Personalized Roadmap	255
3.22.2	UC74 Analyze Skill Gap	257
3.22.3	UC75 Recommend Courses	260
3.23	AI Mentor and Analysis Logging	262
3.23.1	UC76 Generate Portfolio Summary	263
3.24	Authentication and Account Access	265
3.24.1	UC77 Authenticate User	265
3.25	Student Profile and GitHub Integration	267
3.25.1	UC78 Retrieve Repository	268
3.25.2	UC79 Retrieve README	270
3.25.3	UC80 Retrieve Tech Stack	272
3.26	Market Trend Management	275
3.26.1	UC81 Retrieve Job Data	275
3.26.2	UC82 Analyze Market Trends	277
V	Software Testing Documentation	281
VI	Requirement Appendix	282
1	Business Rules	282
2	Assumptions and Dependencies	284
2.1	Assumptions	284
2.2	Dependencies	284
3	Data Dictionary	285
3.1	Student Profile	285
3.2	Tech Path	285
3.3	Skill Node	285
3.4	Course Repository	286

3.5	Job Trend	286
4	Glossary	286
5	Requirement Traceability Matrix	287
VI Release Package & User Guides		288
1	Deliverable Package	288
2	Installation Guides	290
2.1	System Requirements	290
2.2	Installation Instructions	290
2.2.1	Option 1 — Access the Online Demo	290
2.2.2	Option 2 — Run the Project Locally	290
2.2.3	Deployment	291
3	User Manual	291
3.1	Overview	291
3.2	Guideline	292
3.2.1	Login	292
3.2.2	Chat with the AI Virtual Mentor (FR1)	294
3.2.3	View and Update the Dynamic Learning Roadmap (FR2)	295
3.2.4	Run a Skill Gap Analysis (FR3)	295
3.2.5	Check Market Pulse (FR4)	296
3.2.6	Build and Share an E-Portfolio (FR5)	296
3.2.7	Academic Counselor — Review a Student	296
3.2.8	Industry Mentor — Evaluate a Portfolio	296

List of Figures

III.1 Context Diagram	15
III.2 Use Case Diagram of the Personalized Career Orientation and Learning Roadmap Platform	17
III.3 Administrator Screen Flow	22
III.4 Student Screen Flow	23
III.5 Academic Counselor Screen Flow	24
III.6 Industry Mentor Screen Flow	25
IV.1 System Architecture Diagram	61
IV.2 Frontend Architecture Diagram	62
IV.3 Backend Architecture Diagram	63
IV.4 Entity Relationship Diagram	66
IV.5 Class diagram for UC01 Register	77
IV.6 Sequence diagram for UC01 Register	77
IV.7 Class diagram for UC02 Login	79
IV.8 Sequence diagram for UC02 Login	80
IV.9 Class diagram for UC03 Login with Google	82
IV.10Sequence diagram for UC03 Login with Google	82
IV.11Class diagram for UC04 Forgot Password	84
IV.12Sequence diagram for UC04 Forgot Password	85
IV.13Class diagram for UC05 Logout	87
IV.14Sequence diagram for UC05 Logout	87
IV.15Class diagram for UC06 Manage Profile	89
IV.16Sequence diagram for UC06 Manage Profile	90
IV.17Class diagram for UC07 Upload Avatar	92
IV.18Sequence diagram for UC07 Upload Avatar	92
IV.19Class diagram for UC08 Update Contact Information	94

IV.20Sequence diagram for UC08 Update Contact Information	95
IV.21Class diagram for UC09 Change Password	97
IV.22Sequence diagram for UC09 Change Password	97
IV.23Class diagram for UC10 Upload Transcript	99
IV.24Sequence diagram for UC10 Upload Transcript	100
IV.25Class diagram for UC11 Upload CV	102
IV.26Sequence diagram for UC11 Upload CV	102
IV.27Class diagram for UC12 Link GitHub	104
IV.28Sequence diagram for UC12 Link GitHub	105
IV.29Class diagram for UC13 Sync GitHub Repository	107
IV.30Sequence diagram for UC13 Sync GitHub Repository	107
IV.31Class diagram for UC14 View GitHub Analysis	109
IV.32Sequence diagram for UC14 View GitHub Analysis	110
IV.33Class diagram for UC15 View Skill Gap Report	112
IV.34Sequence diagram for UC15 View Skill Gap Report	112
IV.35Class diagram for UC16 Select Career Role	114
IV.36Sequence diagram for UC16 Select Career Role	115
IV.37Class diagram for UC17 Update Career Goal	117
IV.38Sequence diagram for UC17 Update Career Goal	117
IV.39Class diagram for UC18 View Recommended Roles	119
IV.40Sequence diagram for UC18 View Recommended Roles	120
IV.41Class diagram for UC19 Compare Career Paths	122
IV.42Sequence diagram for UC19 Compare Career Paths	122
IV.43Class diagram for UC20 View Required Skills	124
IV.44Sequence diagram for UC20 View Required Skills	125
IV.45Class diagram for UC21 Generate Learning Roadmap	127
IV.46Sequence diagram for UC21 Generate Learning Roadmap	127
IV.47Class diagram for UC22 View Learning Roadmap	129
IV.48Sequence diagram for UC22 View Learning Roadmap	130
IV.49Class diagram for UC23 Update Roadmap Progress	132
IV.50Sequence diagram for UC23 Update Roadmap Progress	132
IV.51Class diagram for UC24 View Learning Progress	134
IV.52Sequence diagram for UC24 View Learning Progress	135
IV.53Class diagram for UC25 Receive Learning Reminder	137

IV.54	Sequence diagram for UC25 Receive Learning Reminder	137
IV.55	Class diagram for UC26 Search Learning Resources	139
IV.56	Sequence diagram for UC26 Search Learning Resources	140
IV.57	Class diagram for UC27 View Recommended Courses	142
IV.58	Sequence diagram for UC27 View Recommended Courses	142
IV.59	Class diagram for UC28 Save Favorite Courses	144
IV.60	Sequence diagram for UC28 Save Favorite Courses	145
IV.61	Class diagram for UC29 Register External Course	147
IV.62	Sequence diagram for UC29 Register External Course	147
IV.63	Class diagram for UC30 Track Course Completion	149
IV.64	Sequence diagram for UC30 Track Course Completion	150
IV.65	Class diagram for UC31 View Market Trends	152
IV.66	Sequence diagram for UC31 View Market Trends	152
IV.67	Class diagram for UC32 Search Job Opportunities	154
IV.68	Sequence diagram for UC32 Search Job Opportunities	155
IV.69	Class diagram for UC33 View Salary Information	157
IV.70	Sequence diagram for UC33 View Salary Information	157
IV.71	Class diagram for UC34 View Technology Trends	159
IV.72	Sequence diagram for UC34 View Technology Trends	160
IV.73	Class diagram for UC35 Receive Job Recommendation	162
IV.74	Sequence diagram for UC35 Receive Job Recommendation	162
IV.75	Class diagram for UC36 Chat with AI Mentor	164
IV.76	Sequence diagram for UC36 Chat with AI Mentor	165
IV.77	Class diagram for UC37 Ask Career Questions	167
IV.78	Sequence diagram for UC37 Ask Career Questions	167
IV.79	Class diagram for UC38 Receive AI Feedback	169
IV.80	Sequence diagram for UC38 Receive AI Feedback	170
IV.81	Class diagram for UC39 Generate E-Portfolio	172
IV.82	Sequence diagram for UC39 Generate E-Portfolio	172
IV.83	Class diagram for UC40 Share Portfolio	174
IV.84	Sequence diagram for UC40 Share Portfolio	175
IV.85	Class diagram for UC41 Login	177
IV.86	Sequence diagram for UC41 Login	177
IV.87	Class diagram for UC42 View Student Profile	179

IV.88	Sequence diagram for UC42 View Student Profile	180
IV.89	Class diagram for UC43 Review Skill Gap	182
IV.90	Sequence diagram for UC43 Review Skill Gap	182
IV.91	Class diagram for UC44 Recommend Career Path	184
IV.92	Sequence diagram for UC44 Recommend Career Path	185
IV.93	Class diagram for UC45 Recommend Courses	187
IV.94	Sequence diagram for UC45 Recommend Courses	187
IV.95	Class diagram for UC46 Track Student Progress	189
IV.96	Sequence diagram for UC46 Track Student Progress	190
IV.97	Class diagram for UC47 Comment Learning Plan	192
IV.98	Sequence diagram for UC47 Comment Learning Plan	192
IV.99	Class diagram for UC48 Schedule Consultation	194
IV.100	Sequence diagram for UC48 Schedule Consultation	195
IV.100	Class diagram for UC49 View Consultation History	197
IV.100	Sequence diagram for UC49 View Consultation History	197
IV.100	Class diagram for UC50 Logout	199
IV.100	Sequence diagram for UC50 Logout	200
IV.100	Class diagram for UC51 Login	202
IV.100	Sequence diagram for UC51 Login	202
IV.100	Class diagram for UC52 View Student Portfolio	204
IV.100	Sequence diagram for UC52 View Student Portfolio	205
IV.100	Class diagram for UC53 Review Projects	207
IV.110	Sequence diagram for UC53 Review Projects	207
IV.110	Class diagram for UC54 Recommend Technologies	209
IV.110	Sequence diagram for UC54 Recommend Technologies	210
IV.110	Class diagram for UC55 Give Technical Advice	212
IV.110	Sequence diagram for UC55 Give Technical Advice	212
IV.110	Class diagram for UC56 Evaluate Progress	214
IV.110	Sequence diagram for UC56 Evaluate Progress	215
IV.110	Class diagram for UC57 Comment Projects	217
IV.110	Sequence diagram for UC57 Comment Projects	217
IV.110	Class diagram for UC58 Approve Portfolio	219
IV.120	Sequence diagram for UC58 Approve Portfolio	220
IV.120	Class diagram for UC59 Suggest Improvements	222

IV.12	Sequence diagram for UC59 Suggest Improvements	222
IV.12	Class diagram for UC60 Logout	224
IV.12	Sequence diagram for UC60 Logout	225
IV.12	Class diagram for UC61 Login	227
IV.12	Sequence diagram for UC61 Login	227
IV.12	Class diagram for UC62 Manage Users	229
IV.12	Sequence diagram for UC62 Manage Users	230
IV.12	Class diagram for UC63 Manage Career Roles	232
IV.13	Sequence diagram for UC63 Manage Career Roles	232
IV.13	Class diagram for UC64 Manage Skill Nodes	234
IV.13	Sequence diagram for UC64 Manage Skill Nodes	235
IV.13	Class diagram for UC65 Manage Courses	237
IV.13	Sequence diagram for UC65 Manage Courses	237
IV.13	Class diagram for UC66 Manage Learning Resources	239
IV.13	Sequence diagram for UC66 Manage Learning Resources	240
IV.13	Class diagram for UC67 Manage Job Trend Data	242
IV.13	Sequence diagram for UC67 Manage Job Trend Data	242
IV.13	Class diagram for UC68 Manage AI Configuration	244
IV.14	Sequence diagram for UC68 Manage AI Configuration	245
IV.14	Class diagram for UC69 Manage Reports	247
IV.14	Sequence diagram for UC69 Manage Reports	247
IV.14	Class diagram for UC70 Logout	249
IV.14	Sequence diagram for UC70 Logout	250
IV.14	Class diagram for UC71 Analyze Transcript	252
IV.14	Sequence diagram for UC71 Analyze Transcript	252
IV.14	Class diagram for UC72 Analyze GitHub Repository	254
IV.14	Sequence diagram for UC72 Analyze GitHub Repository	255
IV.14	Class diagram for UC73 Generate Personalized Roadmap	257
IV.15	Sequence diagram for UC73 Generate Personalized Roadmap	257
IV.15	Class diagram for UC74 Analyze Skill Gap	259
IV.15	Sequence diagram for UC74 Analyze Skill Gap	260
IV.15	Class diagram for UC75 Recommend Courses	262
IV.15	Sequence diagram for UC75 Recommend Courses	262
IV.15	Class diagram for UC76 Generate Portfolio Summary	264

IV.156	Sequence diagram for UC76 Generate Portfolio Summary	265
IV.157	Class diagram for UC77 Authenticate User	267
IV.158	Sequence diagram for UC77 Authenticate User	267
IV.159	Class diagram for UC78 Retrieve Repository	269
IV.160	Sequence diagram for UC78 Retrieve Repository	270
IV.161	Class diagram for UC79 Retrieve README	272
IV.162	Sequence diagram for UC79 Retrieve README	272
IV.163	Class diagram for UC80 Retrieve Tech Stack	274
IV.164	Sequence diagram for UC80 Retrieve Tech Stack	275
IV.165	Class diagram for UC81 Retrieve Job Data	277
IV.166	Sequence diagram for UC81 Retrieve Job Data	277
IV.167	Class diagram for UC82 Analyze Market Trends	279
IV.168	Sequence diagram for UC82 Analyze Market Trends	280
VII.1	Login Screen	292
VII.2	Continue with Google Authentication	293
VII.3	Student Dashboard	294
VII.4	AI Mentor Interface	295

List of Tables

I.1	Project Team	2
II.1	Work Breakdown Structure (WBS) and Project Effort Estimation . . .	8
II.2	Testing Objectives	9
II.3	Project Risks	9
II.4	Training Plan	10
II.5	Project Deliverables	11
II.6	Responsibility Assignment Matrix	11
II.7	Project Communications	12
IV.1	Frontend Package Description	64
IV.2	Backend Package Description	65
IV.3	Users Table Description	67
IV.4	CareerPaths Table Description	67
IV.5	SkillNodes Table Description	67
IV.6	LearningResources Table Description	68
IV.7	StudentProfiles Table Description	68
IV.8	StaffProfiles Table Description	68
IV.9	StudentCourses Table Description	69
IV.10	CareerPathSkills Table Description	69
IV.11	Roadmaps Table Description	69
IV.12	RoadmapProgress Table Description	69
IV.13	Portfolios Table Description	70
IV.14	GitHubRepositories Table Description	70
IV.15	GitHubRepoLanguages Table Description	70
IV.16	ConsultationSessions Table Description	70
IV.17	ConsultationFeedbacks Table Description	71
IV.18	JobTrends Table Description	71

IV.19AIConfigurations Table Description	71
IV.20AIAnalysisLogs Table Description	72
IV.21Notifications Table Description	72
IV.22Use Case to Detailed Design Coverage	72
VI.1 Business Rules	283
VI.2 External Dependencies	284
VI.3 Student Profile Entity	285
VI.4 Tech Path Entity	285
VI.5 Skill Node Entity	285
VI.6 Course Repository Entity	286
VI.7 Job Trend Entity	286
VI.8 Glossary	286
VI.9 Requirement Traceability Matrix	287
VII.1Deliverable Package	289
VII.2System Requirements	290

I. Project Introduction

1. Overview

1.1 Project Information

- **Project name:** Personalized Career Orientation & Learning Roadmap Platform for Software Engineering Students
- **Project code:**
- **Group name:** Group 2
- **Software Type:** Web application

The rapid development of Information Technology has significantly transformed the way students learn, develop professional skills, and prepare for future careers. In particular, Software Engineering students are required to continuously update their technical knowledge, practical experience, and industry trends to remain competitive in the job market.

However, many students face difficulties in determining a suitable career path, identifying the required technical competencies, evaluating their current skill levels, and planning an effective learning roadmap. They often rely on scattered learning resources, inconsistent career advice, or personal experiences without systematic guidance. Consequently, students may spend considerable time learning technologies that are not aligned with their career goals or current industry demands.

The **Personalized Career Orientation & Learning Roadmap Platform for Software Engineering Students** is proposed to address these challenges. The platform provides intelligent career guidance by analyzing students' academic background, technical skills, career interests, and learning progress. Based on this information, the system generates personalized learning roadmaps, recommends appropriate learning resources, identifies missing competencies through skill gap analysis, and helps students build professional portfolios using their GitHub projects.

By integrating Artificial Intelligence, career planning, learning management, portfolio generation, and job market analysis into a unified platform, the proposed system aims to improve students' learning efficiency and enhance their career readiness.

1.2 Project Team

Table I.1: Project Team

Full Name	Role	Email	Mobile
Nguyen Van Bao	Project Manager		
Phan Thi Duy Chung	Business Analyst		
Vo Thi Kiem Dung	System Analyst		
Tran Thi Bich Diem	UI/UX Designer		
Vo Trong Nguyen	Backend Developer		
Vu Le Hoang Nhat	Frontend Developer		
Vo Bao Phuc	Tester		

2. Product Background

Software Engineering education focuses not only on theoretical knowledge but also on practical skills, project experience, and continuous learning. Modern software industries require graduates to possess diverse competencies, including programming, database management, software architecture, cloud computing, DevOps, artificial intelligence, and communication skills.

Despite the availability of numerous online learning platforms, students often encounter several challenges:

- Lack of personalized career guidance.
- Difficulty selecting an appropriate Software Engineering specialization.
- Limited understanding of industry-required technical skills.
- Inability to evaluate their own competencies objectively.
- Difficulty organizing learning plans according to career objectives.
- Lack of professional portfolios for internship and job applications.

Currently, students usually collect learning materials from various websites such as Microsoft Learn, Coursera, Udemy, roadmap.sh, GitHub, YouTube, and Stack Overflow. Since these resources are independent and not integrated, students must manually determine what to study, when to study, and how to evaluate their progress.

Therefore, a comprehensive platform that combines career orientation, personalized learning roadmap generation, AI-assisted mentoring, skill assessment, portfolio management, and labor market analysis can significantly improve students' learning outcomes and employment opportunities.

3. Existing Systems

Several existing platforms provide partial support for career planning or online learning; however, none of them fully integrate all required functionalities for Software Engineering students.

Roadmap.sh

Roadmap.sh provides structured technology roadmaps for different software engineering career paths, including Backend Development, Frontend Development, DevOps, AI Engineering, and Mobile Development. Although it offers detailed technology trees, it does not personalize learning paths based on students' individual competencies or academic performance.

Coursera

Coursera offers thousands of online courses from universities and technology companies. Students can obtain certificates after completing courses. However, course recommendations are primarily based on browsing history rather than comprehensive career analysis.

LinkedIn Learning

LinkedIn Learning provides professional courses and integrates with LinkedIn profiles. Nevertheless, it lacks personalized skill gap analysis and automatic roadmap generation tailored to Software Engineering students.

GitHub

GitHub serves as the largest source code hosting platform, allowing students to manage software projects and demonstrate programming experience. However, GitHub itself does not evaluate students' technical competencies or generate learning recommendations.

ChatGPT and Gemini

Artificial Intelligence chatbots such as ChatGPT and Gemini can answer programming and career-related questions. Nevertheless, they do not maintain long-term learning progress or automatically generate structured career roadmaps integrated with students' academic profiles.

Therefore, there is still a need for an integrated platform specifically designed for Software Engineering students.

4. Business Opportunity

The software industry continues to experience rapid growth worldwide. Companies increasingly demand graduates with practical technical skills rather than solely academic qualifications.

Universities are also shifting toward outcome-based education, emphasizing competency assessment, project-based learning, and career preparation. Consequently, educational institutions require systems capable of supporting students throughout their

learning journeys.

The proposed platform provides several business opportunities:

- Supporting universities in career counseling.
- Assisting students in planning personalized learning paths.
- Helping employers evaluate candidates through dynamic portfolios.
- Assisting academic advisors in monitoring students' learning progress.
- Providing technology trend analysis based on labor market demands.
- Integrating Artificial Intelligence into educational support systems.

Furthermore, the platform can be expanded in the future to support multiple universities, online certification providers, internship management systems, and enterprise recruitment platforms.

5. Software Product Vision

The vision of the project is to become an intelligent career development platform that accompanies Software Engineering students throughout their academic journey and professional development.

The system aims to:

- Provide personalized career recommendations.
- Generate intelligent learning roadmaps.
- Continuously analyze students' skill gaps.
- Recommend high-quality learning resources.
- Build professional online portfolios automatically.
- Analyze current technology trends and employment demands.
- Improve students' employability and career readiness through data-driven recommendations.

By integrating Artificial Intelligence, GitHub repositories, academic information, and industry trends, the platform creates a comprehensive ecosystem for career development.

6. Project Scope & Limitations

6.1 Major Features

6.1.1 Major Features for Administrators

FE-01: Login/Logout.

FE-02: Manage user accounts (Students, Academic Counselors, Industry Mentors).

FE-03: Manage career paths (Backend Developer, Frontend Developer, Full Stack Developer, DevOps Engineer, AI Engineer, Data Engineer, Mobile Developer, QA Engineer, etc.).

FE-04: Manage skill categories and technical skills.

FE-05: Manage learning resources (courses, books, documentation, videos, practice websites).

FE-06: Manage AI prompt templates and system configurations.

FE-07: Manage job market data and technology trend information.

FE-08: Manage notifications and announcements.

FE-09: View system reports and analytics.

FE-10: Monitor user activities and system logs.

FE-11: Manage role-based access permissions.

6.1.2 Major Features for Student

FE-01: Register, login, logout, and authenticate using Google OAuth.

FE-02: Manage personal profile, including academic information, technical skills, career interests, and learning goals.

FE-03: Complete skill assessment to evaluate current competencies.

FE-04: Receive AI-powered career consultation through the AI Virtual Mentor.

FE-05: Select a target software engineering career path.

FE-06: Generate a personalized learning roadmap based on selected career goals.

FE-07: View and update learning progress for each roadmap milestone.

FE-08: Perform skill gap analysis to identify missing competencies.

FE-09: Receive recommended learning resources based on skill gaps.

FE-10: Connect and synchronize GitHub repositories.

FE-11: Automatically generate and maintain an online professional portfolio.

FE-12: View job market trends, technology demand, and career insights.

FE-13: Receive notifications about roadmap updates, learning recommendations, and career opportunities.

6.1.3 Major Features for Academic Counselor

FE-01: Login/Logout.

FE-02: View student profiles and academic information.

FE-03: Monitor students' learning roadmap progress.

FE-04: Review students' skill gap analysis reports.

FE-05: Provide academic advice and learning recommendations.

FE-06: Track students' learning performance and completion status.

FE-07: Send notifications or reminders to students.

FE-08: View reports summarizing students' learning progress.

6.1.4 Major Features for Industry Mentor

FE-01: Login/Logout.

FE-02: Review students' career roadmaps.

FE-03: Evaluate students' professional portfolios.

FE-04: Recommend industry-required technical skills.

FE-05: Provide career advice and industry feedback.

FE-06: Suggest learning priorities according to current market demand.

FE-07: Review students' project experiences and GitHub repositories.

FE-08: View students' competency development reports.

6.2 Limitations & Exclusions

LI-01: The first version of the system is designed exclusively for Software Engineering students and does not currently support students from other academic disciplines.

LI-02: AI career recommendations depend on external AI services (such as OpenAI or Gemini). Recommendation quality may vary depending on the AI model and available information.

LI-03: GitHub integration supports only repositories that are publicly accessible or explicitly authorized by users.

LI-04: Job market analysis depends on external recruitment platforms and publicly available datasets. Real-time accuracy cannot be fully guaranteed.

LI-05: The system currently supports predefined Software Engineering career paths only. Users cannot create custom career paths.

LI-06: The platform is implemented as a web application. Native Android and iOS applications are not included in this version.

LI-07: The platform requires a stable Internet connection because several core services depend on external APIs (AI services, Google Authentication, GitHub API, and job

market data).

LI-08: The current version does not provide online coding practice, automatic code assessment, or integrated programming environments.

II. Project Management Plan

1. Overview

1.1 Project Scope and Estimation

The **Personalized Career Orientation & Learning Roadmap Platform for Software Engineering Students** is a web-based system designed to assist Software Engineering students in identifying suitable career paths and generating personalized learning roadmaps based on their academic performance, technical skills, interests, and career goals.

The project consists of the following major work packages.

Table II.1: Work Breakdown Structure (WBS) and Project Effort Estimation

WBS	Task	Complexity	Estimated Effort (Man-days)
1	Project Initiation	Medium	10
1.1	Requirement Gathering	Medium	5
1.2	Feasibility Analysis	Medium	5
2	Planning	Medium	8
2.1	Project Planning	Medium	4
2.2	Risk Planning	Medium	4
3	Analysis and Design	Complex	22
3.1	Requirement Analysis	Medium	8
3.2	Database Design	Medium	6
3.3	System Architecture	Complex	8
4	Implementation	Complex	80
4.1	Authentication Module	Simple	6
4.2	Student Profile Module	Medium	8
4.3	Career Assessment Module	Complex	12
4.4	Career Recommendation Engine	Complex	16
4.5	Learning Roadmap Generator	Complex	16
4.6	Learning Resource Management	Medium	10
4.7	Dashboard and Progress Tracking	Medium	12
5	Testing	Complex	24
5.1	Unit Testing	Medium	8
5.2	Integration Testing	Complex	8
5.3	System Testing	Complex	8
6	Deployment	Medium	8
6.1	Deployment	Medium	4
6.2	User Acceptance Testing	Medium	4

Total Estimated Effort: 152 Man-days

The objectives of the project are as follows.

- Develop a web-based personalized career orientation platform.
- Recommend suitable Software Engineering career paths.
- Generate personalized learning roadmaps.
- Track students' learning progress.
- Recommend appropriate learning resources.
- Improve students' career planning capability.
- Complete the project within one academic semester.

1.2 Testing Objectives

Table II.2: Testing Objectives

Testing Stage	Coverage	Target Defects
Code Review	Coding Standard	<10
Unit Testing	Business Logic	<8
Integration Testing	Module Communication	<5
System Testing	Complete Workflow	<5
Acceptance Testing	User Requirements	<3

1.3 Project Risks

Table II.3: Project Risks

Risk	Impact	Probability	Mitigation
Changing Requirements	High	Medium	Weekly meetings with supervisor
Implementation Delay	High	Medium	Sprint planning and monitoring
Incorrect Recommendation Results	Medium	Medium	Validation and testing
Database Performance Issues	Medium	Low	Query optimization
Communication Problems	Medium	Medium	Weekly meetings and GitHub tracking
Member Absence	High	Low	Knowledge sharing and backup assignments

2. Management Approach

2.1 Development Methodology

The project adopts the Agile Scrum methodology.

The reasons include:

- Continuous customer feedback
- Flexible requirement changes
- Incremental software delivery
- Continuous testing
- Better project monitoring

Each sprint lasts two weeks.

2.2 Quality Management

Quality assurance activities include:

- Requirement Review
- Code Review
- Unit Testing
- Integration Testing
- System Testing
- User Acceptance Testing

2.3 Training Plan

Table II.4: Training Plan

Technology	Participants	Duration
ASP.NET Core	All Members	1 Week
ReactJS	Front-end Developers	1 Week
SQL Server	All Members	3 Days
Git and GitHub	All Members	2 Days
Figma	UI Designer	2 Days

3. Project Deliverables

Table II.5: Project Deliverables

Sprint	Deliverables
1	Requirement Analysis
2	Database Design and UI Prototype
3	Authentication Module
4	Student Profile Module
5	Career Assessment Module
6	Career Recommendation Module
7	Learning Roadmap Module
8	Learning Resource Module
9	Dashboard and Progress Tracking
10	Integration Testing
11	System Testing
12	Deployment and Final Report

4. Responsibility Assignments

Table II.6: Responsibility Assignment Matrix

Task	Member 1	Member 2	Member 3	Member 4
Requirement Analysis	D	R	S	S
Project Planning	D	S	R	S
Database Design	R	D	S	S
Backend Development	S	D	R	S
Frontend Development	S	R	D	S
Testing	R	S	D	S
Documentation	S	R	S	D

Where

- D = Do
- R = Review
- S = Support

5. Project Communications

Table II.7: Project Communications

Communication	Participants	Frequency	Tool
Daily Meeting	Development Team	Daily	Google Meet
Sprint Planning	All Members	Every 2 Weeks	Google Meet
Sprint Review	Supervisor	Every Sprint	Offline/Meet
Issue Discussion	Related Members	As Needed	Discord
Document Sharing	All Members	Continuous	Google Drive
Source Code Review	Developers	Continuous	GitHub

6. Configuration Management

6.1 Document Management

The project documentation is maintained using:

- Microsoft Word
- Microsoft Excel
- Google Drive

6.2 Source Code Management

The source code is managed using GitHub and Git.

The following branching strategy is adopted:

- main
- develop
- feature/*
- release/*
- hotfix/*

6.3 Version Control Policy

- Every feature is developed in an independent branch.
- Pull Requests are mandatory before merging.

- Code review is required before approval.
- Semantic Versioning is applied.

Example versions:

- Version 1.0.0
- Version 1.1.0
- Version 2.0.0

III. Software Requirement Specification

1. Product Overview

The **Personalized Career Orientation & Learning Roadmap Platform for Software Engineering Students** is an intelligent web-based software platform designed to support Software Engineering students in planning their academic development and preparing for future careers. The system replaces the traditional approach of manually searching for career information, collecting learning resources from multiple websites, and creating self-designed learning plans without structured guidance. Instead, it provides a centralized platform that integrates AI-powered career consultation, personalized learning roadmap generation, skill gap analysis, portfolio management, and technology trend analysis into a single environment. The platform enables students to identify suitable career paths, monitor learning progress, build professional portfolios, and receive personalized recommendations based on their academic background, technical skills, and career aspirations.

The context diagram below illustrates the external entities and system interfaces for **Release 1.0** of the proposed platform. In this release, the system interacts with students, academic counselors, industry mentors, administrators, Artificial Intelligence services, GitHub, Google Authentication, and external job market data providers to deliver personalized career orientation and learning support. Future releases are expected to incorporate additional features such as internship recommendation, online certification integration, collaboration among students, mobile applications, and deeper integration with university information systems to provide a more comprehensive learning and career development ecosystem.

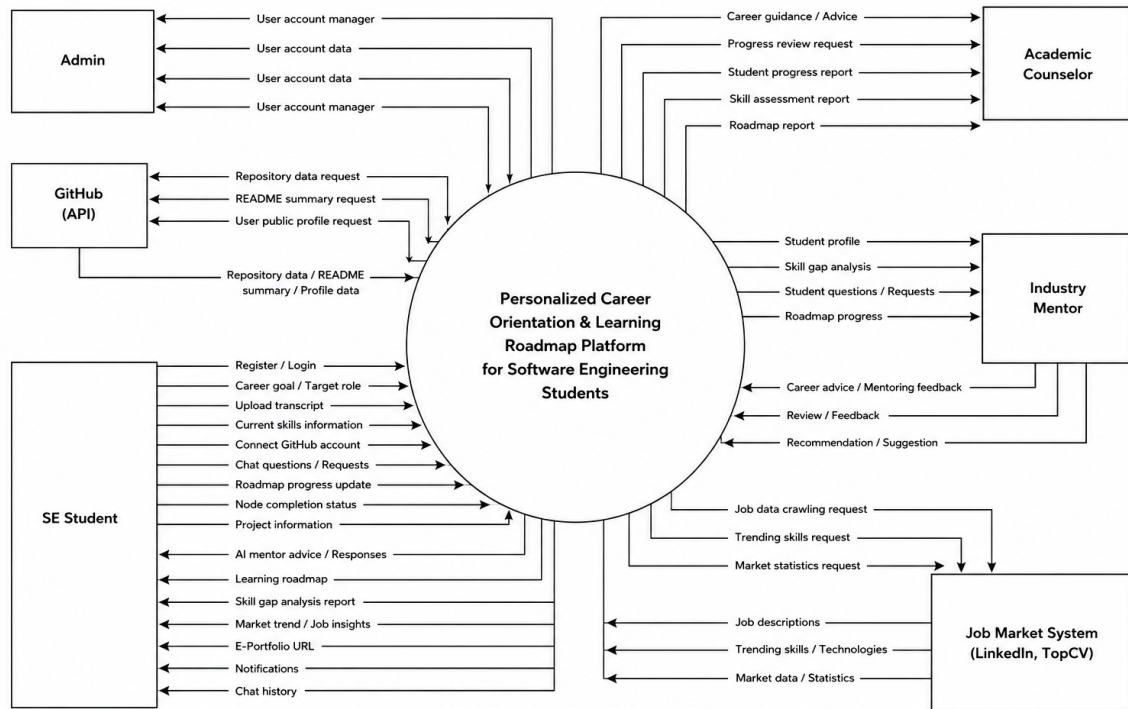


Figure III.1: Context Diagram

2. User Requirements

This section describes the users who interact with the system and the major use cases supported by the Personalized Career Orientation and Learning Roadmap Platform. These requirements define what each actor expects from the system and serve as the foundation for the functional requirements presented in the next section.

2.1 Actors

[HTML]F4D7AD7A #	Actor	Description
1	Administrator	Responsible for managing the entire platform, including user account management, role assignment, authentication settings, AI service configuration, learning resource management, and monitoring system performance. The Administrator ensures platform security, maintains data integrity, and guarantees reliable system operation.

2	SE Student	The primary user of the platform. Students create and manage personal profiles, define target career roles, upload academic transcripts, connect GitHub accounts, perform skill assessments, interact with the AI Virtual Mentor, follow personalized learning roadmaps, monitor skill gap analysis, explore job market trends, and manage AI-generated e-portfolios.
3	Academic Counselor	Supports students throughout their academic and career development. Academic Counselors review students' learning progress, evaluate roadmap completion, analyze skill development, recommend suitable learning resources, provide career guidance, and assist students in selecting appropriate Software Engineering career paths.
4	Industry Mentor	Represents experienced professionals from the software industry who provide practical career guidance and technical mentoring. Industry Mentors review student profiles, evaluate skill gap reports, answer career-related questions, recommend industry best practices, suggest learning priorities, and provide feedback based on current recruitment demands and real-world software development experience.

2.2 Use Cases

2.2.1 Diagram(s)

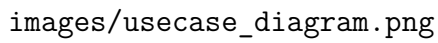
A large rectangular box with a thin black border. Inside the box, the text "images/usecase_diagram.png" is written in a monospaced font, positioned in the lower-left area of the box.

Figure III.2: Use Case Diagram of the Personalized Career Orientation and Learning Roadmap Platform

2.2.2 Descriptions

The following use cases describe the authentication and account management functions that are shared by all users of the platform. These functions ensure secure access, account protection, and identity verification before users interact with the Personalized Career Orientation and Learning Roadmap Platform.

[HTML] UC ID	Use Case	Actors	Use Case Description
UC01	Register	SE Student	Create a new account.
UC02	Login	All authenticated users	Login to the platform.
UC03	Login with Google	SE Student	Authenticate using Google OAuth.
UC04	Logout	All authenticated users	Logout from the system.
UC05	Reset Password	All users	Recover forgotten password.
UC06	Change Password	Authenticated users	Change current password.
UC07	View Profile	Student	View personal profile.
UC08	Update Profile	Student	Edit personal information.
UC09	Upload Avatar	Student	Upload profile picture.
UC10	Upload Transcript	Student	Upload academic transcript.
UC11	View Transcript	Student	View uploaded transcript.
UC12	Update Transcript	Student	Replace transcript.
UC13	Delete Transcript	Student	Remove transcript.
UC14	Select Career Goal	Student	Select desired career role.
UC15	View Career Roles	Student	Browse available career paths.
UC16	Connect GitHub	Student	Link GitHub account.
UC17	Disconnect GitHub	Student	Remove GitHub account.
UC18	View GitHub Projects	Student	Display repositories.
UC19	Chat with AI Mentor	Student	Ask AI questions.
UC20	Receive AI Advice	Student	Receive AI responses.
UC21	View Personalized Roadmap	Student	Display roadmap.
UC22	Update Roadmap Progress	Student	Update completed skills.
UC23	View Skill Gap Report	Student	Review missing skills.
UC24	View Learning Progress	Student	Monitor completion percentage.
UC25	Search Learning Resources	Student	Search learning materials.
UC26	View Recommended Courses	Student	Display suggested courses.

UC27	View Market Trends	Student	View technology trends.
UC28	View Job Insights	Student	Browse recruitment demand.
UC29	Generate Portfolio	Student	Generate e-portfolio.
UC30	View Portfolio	Student	Display portfolio.
UC31	Share Portfolio	Student	Share public portfolio.
UC32	Download Portfolio	Student	Export PDF portfolio.
UC33	View Notifications	Student	Receive notifications.
UC34	View Student Profile	Counselor	View student information.
UC35	Review Transcript	Counselor	Review transcript.
UC36	Review Skill Gap	Counselor	Analyze skill gaps.
UC37	Review Learning Progress	Counselor	Monitor progress.
UC38	Recommend Career Path	Counselor	Suggest career path.
UC39	Recommend Learning Plan	Counselor	Recommend roadmap improvements.
UC40	Comment Learning Plan	Counselor	Leave comments.
UC41	Send Feedback	Counselor	Send feedback.
UC42	View Reports	Counselor	View progress reports.
UC43	Manage Consultation	Counselor	Schedule consultation.
UC44	View Student Portfolio	Mentor	View portfolio.
UC45	Review GitHub Repository	Mentor	Review repositories.
UC46	Review Projects	Mentor	Evaluate projects.
UC47	Recommend Technologies	Mentor	Suggest technologies.
UC48	Provide Technical Advice	Mentor	Give technical guidance.
UC49	Evaluate Progress	Mentor	Evaluate learning progress.
UC50	Comment Portfolio	Mentor	Comment portfolio.
UC51	View Skill Gap Report	Mentor	Review skill analysis.
UC52	Recommend Industry Skills	Mentor	Recommend market skills.

UC53	Send Mentoring Feedback	Mentor	Submit mentoring feedback.
UC54	Manage Users	Administrator	Manage user accounts.
UC55	Add User	Administrator	Create user.
UC56	Update User	Administrator	Update user.
UC57	Delete User	Administrator	Delete user.
UC58	View Users	Administrator	View users.
UC59	Manage Career Roles	Administrator	CRUD career roles.
UC60	Manage Skill Nodes	Administrator	CRUD skill tree.
UC61	Manage Skill Categories	Administrator	Manage skill groups.
UC62	Manage Learning Resources	Administrator	CRUD resources.
UC63	Manage Courses	Administrator	CRUD courses.
UC64	Manage Job Trend Data	Administrator	Update market data.
UC65	Manage AI Models	Administrator	Configure AI.
UC66	Configure Recommendation Engine	Administrator	Tune recommendation engine.
UC67	Manage Notifications	Administrator	Send notifications.
UC68	View Analytics	Administrator	View dashboards.
UC69	Generate Reports	Administrator	Export reports.
UC70	Backup Database	Administrator	Backup system.
UC71	Restore Database	Administrator	Restore data.
UC72	Analyze Transcript	AI Engine	Analyze transcript.
UC73	Analyze GitHub Repository	AI Engine	Analyze GitHub.
UC74	Analyze Skill Gap	AI Engine	Detect missing skills.
UC75	Generate Personalized Roadmap	AI Engine	Generate roadmap.
UC76	Recommend Courses	AI Engine	Recommend learning resources.
UC77	Generate Portfolio Summary	AI Engine	Summarize portfolio.
UC78	Authenticate User	Google OAuth	Verify identity.
UC79	Retrieve Repository	GitHub API	Retrieve repositories.

UC80	Retrieve README	GitHub API	Retrieve README.
UC81	Retrieve Tech Stack	GitHub API	Retrieve technologies.
UC82	Retrieve Job Data	Job Portal API	Retrieve recruitment data.
UC83	Analyze Market Trends	Job Portal API	Analyze job market.
UC84	Retrieve Salary Statistics	Job Portal API	Retrieve salary data.

3. Functional Requirements

3.1 System Functional Requirements

3.1.1 Screens Flow

Administrator Screen Flow

images/Administrator_ScreenFlow.png

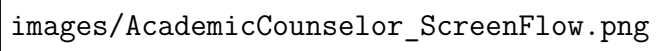
Figure III.3: Administrator Screen Flow

Student Screen Flow

images/Student_ScreenFlow.png

Figure III.4: Student Screen Flow

Academic Counselor Screen Flow

The image is a placeholder for a screen flow diagram for an Academic Counselor. It contains the text "images/AcademicCounselor_ScreenFlow.png" which likely refers to the source of the diagram.

images/AcademicCounselor_ScreenFlow.png

Figure III.5: Academic Counselor Screen Flow

Industry Mentor Screen Flow

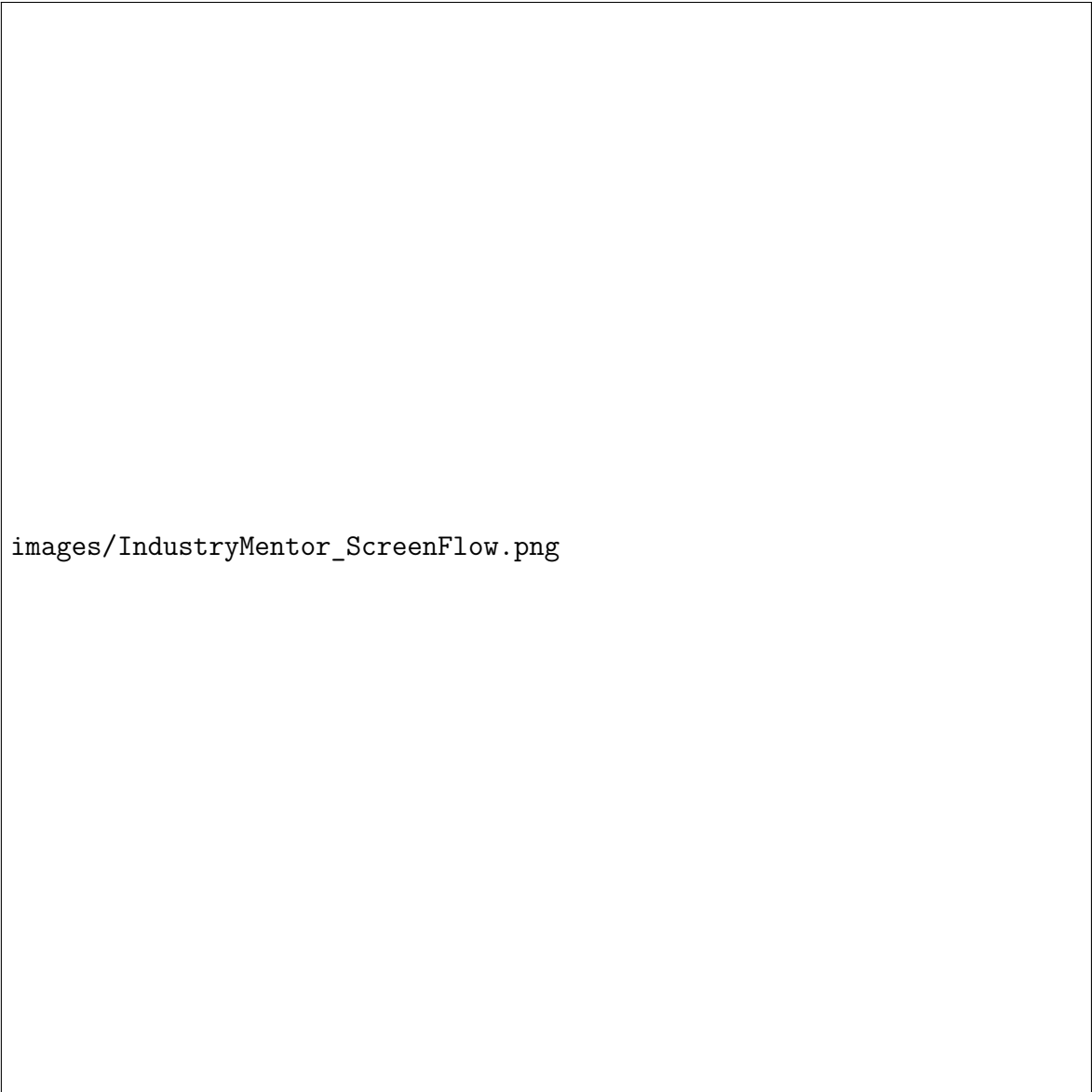


Figure III.6: Industry Mentor Screen Flow

3.1.2 Screen Descriptions

[HTML Page #]	Feature	Screen	Description
1	Authentication	Login Page	Allows Administrator, Academic Counselor, Industry Mentor and SE Student to authenticate using email/password or Google OAuth before accessing the system.

2	Dashboard	Administrator Dashboard	Displays system statistics, user activities, AI recommendation status, learning resources and administrative shortcuts.
3	User Management	Manage Users	Allows administrators to create, search, update, delete and manage user accounts together with role assignment.
4	Create User	Create User	Allows administrators to register new student, counselor or mentor accounts.
5	View Users	View Users	Displays all registered users with filtering and searching functions.
6	Update User	Update User	Allows administrators to modify user profile information and account status.
7	Delete User	Delete User	Allows administrators to remove inactive or unnecessary user accounts.
8	Career Management	Manage Career Roles	Allows administrators to create, update and maintain Software Engineering career roles.
9	Skill Management	Manage Skill Nodes	Allows administrators to organize the hierarchical Software Engineering skill tree.
10	Course Management	Manage Courses	Allows administrators to manage recommended courses and learning resources.
11	Learning Resources	Manage Learning Resources	Allows administrators to add, update and remove books, MOOCs, tutorials and documentation.
12	Market Data	Manage Job Trend Data	Allows administrators to synchronize and maintain recruitment market information collected from Job Portal APIs.
13	AI Configuration	Configure AI Engine	Allows administrators to configure AI recommendation parameters and external AI services.
14	Analytics	View Analytics	Displays statistical reports, recommendation accuracy and platform usage.
15	Reports	Generate Reports	Allows administrators to export system reports into PDF or Excel.
16	Notifications	Send Notifications	Allows administrators to broadcast announcements and notifications to users.
17	Database Backup	Backup Database	Creates backup copies of the system database for disaster recovery.
18	Database Restore	Restore Database	Restores system data from previously created backups.

19	Logout	Logout	Ends the administrator session securely.
20	Profile Management	Student Dashboard	Displays the student's personalized dashboard including learning progress, career roadmap, AI recommendations, GitHub integration status and recent notifications.
21	Profile	View Profile	Allows students to view their personal information, academic information and career preferences.
22	Profile	Update Profile	Allows students to update personal information such as fullname, phone number, biography and career interests.
23	Avatar Management	Upload Avatar	Allows students to upload or replace their profile picture.
24	Career Planning	Select Career Goal	Allows students to choose their desired Software Engineering career path for AI recommendation.
25	Career Planning	View Career Roles	Displays available Software Engineering career paths together with required skills and descriptions.
26	GitHub Integration	Connect GitHub	Allows students to connect their GitHub account for automatic repository analysis.
27	GitHub Integration	View GitHub Projects	Displays public repositories, programming languages and project information retrieved from GitHub.
28	GitHub Integration	Disconnect GitHub	Allows students to remove GitHub account integration from the platform.
29	AI Virtual Mentor	Chat with AI	Allows students to ask questions related to programming, learning plans and career orientation.
30	AI Recommendation	Receive AI Advice	Displays personalized recommendations generated by the AI Recommendation Engine.
31	Learning Roadmap	View Roadmap	Displays the personalized learning roadmap generated based on career goals and current skills.
32	Learning Roadmap	Update Progress	Allows students to mark completed skills, courses and certifications.
33	Skill Analysis	Skill Gap Report	Displays missing skills required for the selected Software Engineering career path.

34	Learning Resources	Learning Resources	Displays recommended books, online courses, documentation and tutorials suggested by the AI engine.
35	Portfolio	Generate Portfolio	Automatically generates an AI-powered e-Portfolio based on academic records, GitHub projects and completed skills.
36	Portfolio	View Portfolio	Allows students to preview, download and share their professional e-Portfolio.
37	Job Market Analysis	Market Trends	Displays current Software Engineering technology trends and hiring demands collected from Job Portal APIs.
38	Job Market Analysis	Job Insights	Displays recruitment statistics, required technologies, salary trends and industry demand.
39	Notifications	Notifications	Displays system notifications, AI reminders, counselor feedback and learning updates.
40	Authentication	Logout	Allows students to securely logout from the platform.
41	Dashboard	Academic Counselor Dashboard	Displays an overview of assigned students, consultation requests, learning progress, skill gap summaries and recent academic activities.
42	Student Management	View Student Profile	Allows Academic Counselors to view students' personal profiles, academic transcripts, GitHub information and learning roadmaps.
43	Transcript Review	Review Transcript	Allows Academic Counselors to review uploaded academic transcripts and evaluate students' academic performance.
44	Skill Assessment	Review Skill Gap	Displays AI-generated skill gap analysis and identifies missing competencies required for the selected career path.
45	Career Guidance	Recommend Career Path	Allows Academic Counselors to recommend appropriate Software Engineering career paths based on students' abilities, interests and academic performance.
46	Learning Recommendation	Recommend Learning Plan	Allows Academic Counselors to suggest personalized learning roadmaps, recommended courses and certification plans.

47	Progress Monitoring	Review Learning Progress	Displays students' learning progress, completed skills, completed courses and roadmap completion status.
48	Learning Feedback	Comment Learning Plan	Allows Academic Counselors to provide comments, suggestions and improvement recommendations for students' learning plans.
49	Consultation	Manage Consultation	Allows Academic Counselors to schedule career consultation sessions, manage appointments and communicate with students.
50	Feedback	Send Feedback	Allows Academic Counselors to send academic evaluations, career advice and consultation feedback directly to students.
51	Reports	View Reports	Displays learning reports, roadmap completion statistics and student performance summaries.
52	Authentication	Logout	Allows Academic Counselors to securely logout from the platform after completing advising activities.
53	Dashboard	Industry Mentor Dashboard	Displays an overview of assigned students, mentoring requests, GitHub project summaries, portfolio evaluations and mentoring activities.
54	Student Portfolio	View Student Portfolio	Allows Industry Mentors to view students' e-Portfolios, technical skills, completed projects and academic achievements.
55	Repository Review	Review GitHub Repository	Allows Industry Mentors to review students' GitHub repositories, programming languages, project quality and contribution history.
56	Project Evaluation	Review Projects	Allows Industry Mentors to evaluate students' software projects based on implementation quality, coding practices and project completeness.
57	Technology Recommendation	Recommend Technologies	Allows Industry Mentors to recommend modern programming languages, frameworks, development tools and industry best practices.
58	Technical Mentoring	Provide Technical Advice	Allows Industry Mentors to provide technical guidance, answer programming questions and recommend software engineering solutions.

59	Learning Evaluation	Evaluate Progress	Allows Industry Mentors to monitor students' technical growth, completed roadmap milestones and overall learning progress.
60	Portfolio Feedback	Comment Portfolio	Allows Industry Mentors to leave comments and suggestions for improving students' e-Portfolios and software projects.
61	Skill Assessment	View Skill Gap Report	Displays AI-generated skill gap analysis to help mentors identify missing technical competencies.
62	Industry Recommendation	Recommend Industry Skills	Allows Industry Mentors to recommend practical industry skills, emerging technologies and certifications required by employers.
63	Mentoring Feedback	Send Mentoring Feedback	Allows Industry Mentors to submit mentoring reports, technical evaluations and personalized improvement recommendations.
64	Authentication	Logout	Allows Industry Mentors to securely logout from the platform after completing mentoring activities.

3.1.3 Screen Authorization

The following table describes the authorization of each screen for every user role in the system.

[HTML Screen #]	Screen	Admin	Student	Counselor	Mentor
1	Login	✓	✓	✓	✓
2	Logout	✓	✓	✓	✓
3	Administrator Dashboard	✓			
4	Manage Users	✓			
5	Create User	✓			
6	View Users	✓			
7	Update User	✓			
8	Delete User	✓			
9	Manage Career Roles	✓			
10	Manage Skill Nodes	✓			
11	Manage Courses	✓			

12	Manage Learning Resources	✓			
13	Manage Job Trend Data	✓			
14	Configure AI Engine	✓			
15	View Analytics	✓			
16	Generate Reports	✓			
17	Backup Database	✓			
18	Restore Database	✓			
19	Student Dashboard		✓		
20	View Profile		✓		
21	Update Profile		✓		
22	Upload Avatar		✓		
23	Select Career Goal		✓		
24	View Career Roles		✓		
25	Connect GitHub		✓		
26	View GitHub Projects		✓		
27	Disconnect GitHub		✓		
28	Chat with AI		✓		
29	Receive AI Advice		✓		
30	View Roadmap		✓		
31	Update Progress		✓		
32	Skill Gap Report		✓		
33	Learning Resources		✓		
34	Generate Portfolio		✓		
35	View Portfolio		✓		
36	Market Trends		✓		
37	Job Insights		✓		
38	Notifications		✓		
39	Academic Counselor Dashboard			✓	
40	View Student Profile			✓	✓
41	Review Transcript			✓	
42	Review Skill Gap			✓	✓

43	Recommend Career Path			✓	
44	Recommend Learning Plan			✓	
45	Review Learning Progress			✓	✓
46	Comment Learning Plan			✓	
47	Manage Consultation			✓	
48	Send Feedback			✓	
49	View Reports			✓	
50	Industry Mentor Dashboard				✓
51	View Student Portfolio				✓
52	Review GitHub Repository				✓
53	Review Projects				✓
54	Recommend Technologies				✓
55	Provide Technical Advice				✓
56	Evaluate Progress				✓
57	Comment Portfolio				✓
58	Recommend Industry Skills				✓
59	Send Mentoring Feedback				✓

3.1.4 Non-Screen Functions

Some important functions of the system are executed automatically in the background without direct user interaction. These functions support AI recommendation, external system integration, data synchronization and system maintenance. The following table summarizes all non-screen functions implemented by the platform.

[HTML] #	Feature	Module	Description
1	Analyze Academic Transcript	AI Engine	Automatically analyzes uploaded academic transcripts to identify completed subjects, academic performance and knowledge areas.
2	Analyze GitHub Repository	GitHub Analyzer	Retrieves repository information, programming languages, commits and project activities from GitHub.
3	Detect Skill Gap	AI Recommendation Engine	Compares the student's current competencies with the selected career path to identify missing technical skills.
4	Generate Personalized Roadmap	AI Recommendation Engine	Automatically generates an individualized learning roadmap according to career goals, skill gap analysis and market demand.
5	Recommend Learning Resources	AI Recommendation Engine	Selects suitable courses, books, tutorials and certifications based on students' learning progress.
6	Generate Portfolio Summary	Portfolio Generator	Automatically summarizes academic achievements, projects and technical skills into a professional e-Portfolio.
7	Google Authentication	Google OAuth Service	Authenticates users securely using Google OAuth without storing user passwords.
8	Retrieve GitHub Repository	GitHub API	Retrieves public repositories, programming languages and repository statistics.
9	Retrieve README Files	GitHub API	Downloads README files for project analysis and portfolio generation.
10	Detect Technology Stack	GitHub API	Identifies programming languages, frameworks and technologies used in repositories.
11	Retrieve Job Market Data	Job Portal API	Collects recruitment information from external job portals such as LinkedIn and TopCV.
12	Analyze Market Trends	Job Market Analyzer	Analyzes recruitment demand, trending technologies and required skills.
13	Retrieve Salary Statistics	Job Portal API	Collects salary ranges and employment statistics for different Software Engineering positions.
14	Send Notifications	Notification Service	Automatically sends notifications related to learning progress, roadmap updates, counselor feedback and mentoring activities.

15	Email Notification	Mail Service	Sends verification emails, password recovery emails and important announcements.
16	Daily Data Synchronization	Scheduler	Synchronizes GitHub repositories, learning resources and job market information periodically.
17	Backup Database	Backup Service	Creates scheduled backups of the system database for disaster recovery.
18	Restore Database	Recovery Service	Restores the system database from backup files when necessary.
19	Generate System Reports	Reporting Service	Automatically generates statistical reports related to users, learning progress, AI recommendations and platform usage.
20	System Logging	Logging Service	Records user activities, API requests, AI execution logs and system errors for monitoring and auditing purposes.

3.1.5 Entity Relationship Diagram

3.2 Authentication

Authentication is responsible for verifying user identities before allowing access to the Personalized Career Orientation & Learning Roadmap Platform. This module provides secure login, user registration, password recovery, and logout functions. Only authenticated users can access system resources according to their assigned roles.

3.2.1 Login

Role:

Administrator, Student, Industry Mentor, Academic Counselor .

Function Trigger:

The user enters a registered email address and password, then clicks the **Login** button.

Function Description:

- **Purpose:**

Authenticate users before granting access to the system.

- **Data Processing:**

The system validates the entered credentials, verifies the account status, authenticates the user, determines the user role, and redirects the user to the corresponding dashboard.

Function Details:

- If the login credentials are correct, the system redirects the user to the corresponding dashboard.
- If the email address or password is incorrect, the system displays an error message.
- If the account has not been verified, the system requests email verification before login.
- If the account is temporarily locked because of multiple unsuccessful login attempts, the system informs the user accordingly.

3.3 User Management

User Management is responsible for managing user accounts and personal information within the Personalized Career Orientation & Learning Roadmap Platform. This module enables administrators to maintain user accounts, while users can update their own profile information. The system also supports searching, viewing, editing, and deleting user accounts according to role-based access permissions.

3.3.1 Manage Users

Role:

Administrator.

Function Trigger:

The administrator logs into the system and navigates to the **User Management** page.

Function Description:

- **Purpose:**
Manage all user accounts in the system.
- **Data Processing:**
Create, edit, delete, search, and update user account information.

Function Details:

- The administrator can create new user accounts.
- The administrator can update user information.
- The administrator can activate or deactivate user accounts.
- Search users by Student ID, Full Name, or Email Address.
- Assign roles such as Student, Mentor, Career Counselor, or Administrator.

3.3.2 Import Students

Role:

Administrator.

Function Trigger:

The administrator logs into the system, opens the **User Management** module, and selects the **Import Students** function.

Function Description:

- **Purpose:**
Import student accounts into the system.
- **Data Processing:**
Read student information from an Excel file and automatically create user accounts.

Function Details:

- Import student information from an Excel file.
- Validate duplicated Student IDs and email addresses.
- Automatically assign the Student role.
- Display the import result after processing.

3.3.3 Import Mentors

Role:

Administrator.

Function Trigger:

The administrator logs into the system, opens the **User Management** module, and selects the **Import Mentors** function.

Function Description:

- **Purpose:**
Import mentor accounts into the system.
- **Data Processing:**
Read mentor information from an Excel file and create mentor accounts automatically.

Function Details:

- Import mentor information from an Excel file.
- Validate duplicated email addresses.
- Automatically assign the Mentor role.
- Display the import result after processing.

3.4 Career Assessment Management

Career Assessment Management allows students to evaluate their technical skills, interests, and career orientation through assessment questionnaires. The assessment results are analyzed by the system and used as the basis for generating personalized career recommendations and learning roadmaps.

3.4.1 Student Take Career Assessment

Function Trigger:

Student logs in → navigates to "Career Assessment" → clicks "Start Assessment".

Function Description:**Actors:**

Student

Purpose:

Allow students to evaluate their technical skills, interests, and career orientation.

Interface:

An assessment page containing multiple-choice questions related to programming, software engineering knowledge, career interests, and technical skills.

Data Processing:

The student's answers are saved in the database. The system calculates the assessment score and generates an assessment result for future AI recommendations.

Function Details:**Validation:**

- All questions must be answered.

Business Rules:

- Students can take the assessment once per evaluation period.

- Assessment results are stored for future comparison.

Normal Case:

Student answers all questions → clicks "**Submit**" → system calculates the score and saves the result.

Abnormal Case:

If required questions are unanswered, the system requests the student to complete all questions before submission.

3.4.2 View Assessment Result

Function Trigger:

Student completes the assessment → selects "**View Result**".

Function Description:

Actors:

Student

Purpose:

Allow students to review their assessment results.

Interface:

A result page displaying assessment score, strengths, weaknesses, and suggested career paths.

Data Processing:

Retrieve assessment results from the database and display them graphically.

Function Details:

Business Rules:

- Students can only view their own assessment results.

Normal Case:

Student selects "**View Result**" → system displays assessment report.

Abnormal Case:

If no assessment exists, the system requests the student to complete an assessment first.

3.4.3 Mentor Review Assessment

Function Trigger:

Mentor logs in → navigates to "**Student Assessments**" → selects a student.

Function Description:

Actors:

Mentor

Purpose:

Review students' assessment results and provide career guidance.

Interface:

A dashboard displaying assessment reports, skill analysis, and feedback form.

Data Processing:

Store mentor comments and recommendations in the database.

Function Details:

Business Rules:

- Mentors can only review students assigned to them.

Normal Case:

Mentor reviews assessment → enters comments → clicks "**Save Feedback**".

Abnormal Case:

If the selected student has not completed an assessment, the system displays a notification.

3.4.4 Manage Assessment Questions

Function Trigger:

Administrator logs in → navigates to "**Assessment Management**".

Function Description:

Actors:

Administrator

Purpose:

Manage assessment questions used by the career assessment module.

Interface:

A management page allowing administrators to create, edit, delete, and organize assessment questions.

Data Processing:

Save updated assessment questions to the database.

Function Details:

Business Rules:

- Only administrators can modify assessment questions.

Normal Case:

Administrator creates or updates questions → system saves the changes.

Abnormal Case:

If required fields are missing, the system displays validation messages.

3.5 Mentorship Management

Mentorship Management allows students to connect with mentors for career guidance and learning support. Students can request mentorship, while mentors can review, accept, or decline requests. The system also enables both parties to manage mentorship relationships throughout the learning process.

3.5.1 Student Request Mentor

Function Trigger:

Student logs in → navigates to **"Find Mentors"** → selects a mentor → clicks **"Request Mentorship"**.

Function Description:

Actors:

Student

Purpose:

Allow students to request mentorship from a mentor.

Interface:

A mentor profile page displaying mentor information, expertise, availability, and a "Request Mentorship" button.

Data Processing:

The system creates a mentorship request with the status **Pending** and notifies the selected mentor.

Function Details:

Validation:

All required fields must be completed.

Business Rules:

A student can only have one active mentor at a time.

A mentorship request cannot be duplicated.

Normal Case:

Student submits a mentorship request → system stores the request and displays a success message.

Abnormal Case:

Duplicate request or mentor unavailable → display an error message.

3.5.2 Mentor Review Request**Function Trigger:**

Mentor logs in → opens "Mentorship Requests".

Function Description:**Actors:**

Mentor

Purpose:

Allow mentors to review mentorship requests.

Interface:

A list of pending mentorship requests with student information and request details.

Data Processing:

The system retrieves pending requests and displays them for review.

Function Details:

Mentors can review student profiles before making a decision.

3.5.3 Mentor Respond Request**Function Trigger:**

Mentor selects a mentorship request → clicks "Accept" or "Reject".

Function Description:**Actors:**

Mentor

Purpose:

Approve or reject a student's mentorship request.

Interface:

A mentorship request detail page with Accept and Reject buttons.

Data Processing:

Update the request status and notify the student.

Function Details:**Business Rules:**

A mentor cannot exceed the maximum number of active mentees.

Normal Case:

Mentor accepts the request → mentorship relationship is created.

Abnormal Case:

Maximum mentee limit reached → display an error message.

3.5.4 Schedule Mentoring Session

Function Trigger:

Student or Mentor logs in → selects "Schedule Session".

Function Description:**Actors:**

Student, Mentor

Purpose:

Schedule mentoring sessions between students and mentors.

Interface:

A scheduling calendar with available dates and meeting information.

Data Processing:

Save meeting information and notify both participants.

Function Details:

Meeting schedules are synchronized for both users.

3.5.5 View Mentoring History

Function Trigger:

Student or Mentor logs in → opens "Mentoring History".

Function Description:

Actors:

Student, Mentor

Purpose:

Display previous mentoring sessions and recommendations.

Interface:

A table showing meeting history, mentor feedback, and follow-up recommendations.

Data Processing:

Retrieve mentoring records from the database.

Function Details:

Users can search and filter mentoring history by date.

3.6 Learning Roadmap Management

Learning Roadmap Management enables students to generate, view, and update personalized learning roadmaps based on their career goals, current skills, and AI recommendations. The module helps students monitor their learning progress and achieve the competencies required for their desired career paths.

3.6.1 Generate Learning Roadmap

Function Trigger:

Student logs in → navigates to "**Learning Roadmap**" → clicks "**Generate Roadmap**".

Function Description:**Actors:**

Student

Purpose:

Generate a personalized learning roadmap based on the student's assessment results and career objectives.

Interface:

A page allowing students to select a career path, preferred learning duration, and learning pace before generating the roadmap.

Data Processing:

The system analyzes assessment results and AI recommendations to generate a customized learning roadmap, which is then stored in the database.

Function Details:

Validation:

The student must complete the career assessment before generating a roadmap.

Business Rules:

Each student can maintain only one active learning roadmap for each selected career path.

Normal Case:

Student selects career preferences → clicks "**Generate Roadmap**" → the system creates and displays the personalized roadmap.

Abnormal Case:

If the student has not completed the assessment, the system requests completion before generating the roadmap.

3.6.2 Update Learning Progress

Function Trigger:

Student logs in → opens the learning roadmap → marks a learning activity as completed.

Function Description:**Actors:**

Student

Purpose:

Allow students to update their learning progress.

Interface:

A roadmap page displaying learning milestones with progress indicators and completion checkboxes.

Data Processing:

The system updates the student's learning progress and recalculates the completion percentage.

Function Details:**Validation:**

Only authenticated students can update their own learning progress.

Business Rules:

Completed learning activities cannot be modified unless reset by the student.

Normal Case:

Student marks a task as completed → system updates the progress successfully.

Abnormal Case:

If the selected task does not exist, the system displays an error message.

3.6.3 Export Learning Roadmap

Function Trigger:

Student logs in → opens the learning roadmap → clicks "**Export**".

Function Description:**Actors:**

Student

Purpose:

Allow students to export their personalized learning roadmap for future reference.

Interface:

An export dialog allowing students to select PDF or Excel format.

Data Processing:

The system generates the selected document format and downloads it to the user's device.

Function Details:**Validation:**

The student must have an existing learning roadmap before exporting.

Business Rules:

Only the owner of the roadmap can export it.

Normal Case:

Student selects the export format → clicks "**Download**" → system generates and downloads the file.

Abnormal Case:

If no learning roadmap exists, the system displays an appropriate notification.

3.7 Learning Resource Management

Learning Resource Management enables students to discover, access, and manage learning resources recommended by the system. These resources include online courses, books, tutorials, technical documentation, and certification programs that support students in achieving their career goals.

3.7.1 Generate Learning Resource Recommendations

Function Trigger:

Student logs in → navigates to "Learning Resources" → selects a career path or learning roadmap.

Function Description:**Actors:**

Student

Purpose:

Generate personalized learning resource recommendations based on the student's career objectives, assessment results, and learning roadmap.

Interface:

A recommendation page displaying filters for career path, technology, difficulty level, and learning platform.

Data Processing:

The system analyzes the student's profile and retrieves suitable learning resources from the database and external learning platforms.

Function Details:**Validation:**

The student must complete a career assessment before receiving personalized recommendations.

Business Rules:

Only active and verified learning resources are recommended.

The recommendation list is updated whenever the student's learning roadmap changes.

Normal Case:

Student selects a career path → system generates recommended learning resources → recommendation list is displayed successfully.

Abnormal Case:

If no suitable learning resources are found, the system displays an appropriate notification.

3.7.2 Manage Learning Resources

Function Trigger:

Administrator logs in → navigates to "Learning Resource Management".

Function Description:

Actors:

Administrator

Purpose:

Manage learning resources available on the platform.

Interface:

A management page allowing administrators to add, edit, delete, search, and categorize learning resources.

Data Processing:

The system validates the entered information and updates the learning resource database.

Function Details:**Validation:**

All required fields must be completed.

Resource links must be valid URLs.

Business Rules:

Only administrators can modify learning resources.

Duplicate learning resources are not allowed.

Normal Case:

Administrator enters resource information → clicks "**Save**" → the system stores the resource successfully.

Abnormal Case:

If required information is missing or the resource already exists, the system displays an error message.

3.8 Timeline Management

The Timeline Management module allows administrators to define and manage important milestones throughout students' career development process. It enables administrators to configure schedules for career assessments, AI recommendation periods, mentoring sessions, learning roadmap updates, portfolio reviews, and career consultation activities. Proper timeline management ensures that all users receive notifications and complete required activities according to the academic calendar.

3.8.1 Manage Career Development Timeline

Function Trigger:

Administrator logs in → navigates to "Timeline Management" → selects an academic semester or training period.

Function Description:**Actors:**

Administrator

Purpose:

Allow administrators to create, update, and manage important milestones for career orientation, AI assessments, mentoring activities, and learning roadmap execution.

Interface:

The interface contains:

- Academic Semester
- Career Assessment Period
- AI Recommendation Schedule
- Learning Roadmap Update Period
- Mentor Consultation Schedule
- Portfolio Submission Deadline
- Career Fair Schedule
- Save button
- Cancel button

Data Processing:

The system retrieves the existing academic timeline from the database. After validation, updated schedules are stored and notifications are automatically sent to affected users.

Function Details:**Validation:**

- All required timeline information must be completed.
- Dates must follow chronological order.
- Start dates must be earlier than end dates.
- Activities cannot overlap if conflicts exist.

Business Rules:

- Each semester contains only one official career development timeline.

- Career assessment must be completed before AI recommendation generation.
- Learning roadmap generation is available only after assessment completion.
- Mentor consultation sessions must occur after learning roadmap generation.
- Portfolio evaluation takes place near the end of the semester.
- All timeline updates are recorded in the audit log.

Normal Case:

1. Administrator opens Timeline Management.
2. Selects an academic semester.
3. Updates important schedules.
4. Clicks **Save**.
5. The system validates all dates.
6. Timeline information is saved successfully.
7. Notifications are sent to related users.
8. The system displays:
“Timeline updated successfully.”

Abnormal Case:

- Invalid date format.
“Please enter a valid date.”
- End date is earlier than start date.
“End date must be after start date.”
- Timeline conflicts with existing activities.
“Timeline conflict detected.”
- Database error.
“Unable to update the timeline. Please try again later.”

3.9 Blog Management

The Blog Management module allows students, mentors, and career counselors to share knowledge, learning experiences, career guidance, technology trends, internship opportunities, and other software engineering related content. Users can create blog posts, interact with published articles, and participate in knowledge sharing within the platform community.

3.9.1 Create Blog Post

Function Trigger:

Student or Mentor logs in → navigates to “Blog” → clicks the **Create New Post** button.

Function Description:**Actors:**

Student, Mentor

Purpose:

Allow authorized users to publish articles related to career orientation, learning experiences, technical knowledge, internship opportunities, software engineering technologies, and academic activities.

Interface:

The blog creation page contains:

- Blog Title
- Blog Category
- Rich Text Editor
- Upload Cover Image
- Upload Attachment (Optional)
- Publish Button
- Save Draft Button

Data Processing:

The system validates all submitted information before saving the blog post into the database. If the post is published successfully, it is displayed on the Blog page and notifications may be sent to followers or subscribed users.

Function Details:**Validation:**

- Blog title is required.
- Blog content is required.
- Uploaded images must be JPG, JPEG or PNG format.
- Maximum image size is 5 MB.

Business Rules:

- Only authenticated students and mentors can create blog posts.
- Blog posts may be reviewed by administrators if reported.
- Draft posts are only visible to their authors.
- Published posts become immediately visible to all users.

Normal Case:

1. User opens the Blog module.
2. Clicks **Create New Post**.
3. Enters the title and content.
4. Uploads an optional cover image.
5. Clicks **Publish**.
6. The system validates all information.
7. The blog post is stored successfully.
8. The post appears on the Blog page.

Abnormal Case:

- Missing title.
“Please enter the blog title.”
- Missing content.
“Please enter blog content.”
- Unsupported image format.
“Only JPG, JPEG or PNG images are accepted.”
- Uploaded image exceeds the maximum size.
“Maximum image size is 5 MB.”
- System error.
“Unable to publish the blog post. Please try again later.”

3.9.2 Interact with Blog Post

Function Trigger:

User logs in → navigates to the “Blog” page → selects a blog post → clicks the **Like** button or submits a comment.

Function Description:**Actors:**

Student, Mentor, Career Counselor

Purpose:

Allow users to interact with blog posts by expressing appreciation through likes and participating in discussions by posting comments.

Interface:

The blog detail page contains:

- Blog content
- Like button
- Total number of likes
- Comment section
- Comment text box
- Submit Comment button

Data Processing:

The system records user interactions in the database. Likes are counted immediately, while comments are stored and displayed below the corresponding blog post. Notifications may be sent to the blog author when new interactions occur.

Function Details:**Validation:**

- Comments cannot be empty.
- The selected blog post must exist.
- Only authenticated users can interact with blog posts.

Business Rules:

- Each user can like a blog post only once.

- Clicking the Like button again removes the previous like.
- Comments containing inappropriate language may be automatically filtered.
- Deleted blog posts cannot receive new likes or comments.

Normal Case:

1. User opens a blog post.
2. Clicks the Like button or enters a comment.
3. Clicks Submit Comment.
4. The system validates the submitted information.
5. The interaction is saved successfully.
6. The updated number of likes or comments is displayed immediately.

Abnormal Case:

- Empty comment.
“Please enter your comment.”
- The selected blog post no longer exists.
“This blog post is no longer available.”
- Network or database error.
“Unable to process your request. Please try again later.”

3.9.3 Apply for Internship or Team Recruitment

Function Trigger:

Student logs in → navigates to the “Blog” page → opens an internship or team recruitment post → clicks the **Apply** button.

Function Description:

Actors:

Student

Purpose:

Allow students to apply for internship opportunities, project teams, research groups, or mentoring programs by submitting their CV and optional cover letter.

Interface:

The application page contains:

- Recruitment Information
- Upload CV
- Cover Letter
- Apply Button
- Cancel Button

Data Processing:

The system validates the uploaded CV, stores the application in the database, links the application to the corresponding recruitment post, and notifies the recruiter or mentor.

Function Details:**Validation:**

- CV must be uploaded.
- Accepted file formats are PDF and DOCX.
- Maximum file size is 5 MB.
- Cover Letter is optional but cannot exceed 1000 characters.

Business Rules:

- A student may submit only one application for each recruitment post.
- Applications can be updated only before the recruitment deadline.
- Students must have a completed personal profile before submitting an application.
- Recruiters receive notifications immediately after a successful application.

Normal Case:

1. Student opens a recruitment blog post.
2. Uploads a valid CV.
3. Enters an optional cover letter.
4. Clicks the Apply button.
5. The system validates the submitted information.
6. The application is stored successfully.
7. A confirmation message is displayed.

8. The recruiter receives a notification.

Abnormal Case:

- No CV is uploaded.
“Please upload your CV.”
- Unsupported file format.
“Only PDF and DOCX files are accepted.”
- Uploaded file exceeds the maximum size.
“The maximum allowed file size is 5 MB.”
- The student has already applied.
“You have already applied for this opportunity.”
- System error.
“Unable to submit your application. Please try again later.”

3.10 Semester Management

The Semester Management module allows administrators or managers to create, configure, and maintain academic semesters used throughout the system. Each semester contains important information such as the semester period, review schedules, defense schedules, and other academic milestones. Proper semester management ensures that all project activities are organized according to the official academic timeline.

3.10.1 Manage Semester

Role:

Administrator, Manager.

Function Trigger:

The Manager or Administrator logs into the system, navigates to the **Semester Management** module, and selects the **Manage Semester** function.

Function Description:

Purpose:

Allow authorized users to create, edit, update, and manage semester information, including review schedules, defense schedules, and important academic milestones.

Interface:

The system displays a semester management page containing:

- Semester list.
- Add Semester button.
- Edit Semester button.
- Delete Semester button.
- Semester information form including:
 - Semester Name
 - Start Date
 - End Date
 - Review Schedule
 - Defense Schedule
 - Additional Notes

Data Processing:

The system validates all entered information before saving it into the Semester database. After successful validation, the system updates the semester information and synchronizes related review and defense schedules.

Function Details:

Validation:

- All required fields must be completed.
- Semester dates must follow the correct date format.
- Review and defense schedules must fall within the semester duration.
- Semester periods must not overlap with existing semesters.

Business Rules:

- Only Managers and Administrators are allowed to manage semesters.
- Every semester must contain exactly one academic timeline.
- Review sessions must be scheduled before defense sessions.
- All schedule changes are recorded in the system log.

Normal Case:

1. The Manager opens the Semester Management page.

2. The Manager enters or edits semester information.
3. The Manager clicks the **Save** button.
4. The system validates all information.
5. The semester information is stored successfully.
6. The system displays the message:
"Semester information has been updated successfully."

Abnormal Case:

- Missing required information.
- Invalid date format.
- Semester dates overlap with an existing semester.
- Review or defense schedules are outside the semester period.
- Unexpected system error during data saving.

3.11 Criteria Form Management

The Criteria Form Management module enables managers to create, modify, and maintain evaluation forms and assessment criteria used throughout the project lifecycle. These forms are applied during review and defense sessions to ensure that all evaluations follow standardized criteria and grading rules.

3.11.1 Manage Criteria Form

Role:

Manager.

Function Trigger:

The Manager logs into the system, navigates to the **Form Management** module, and selects the **Add/Edit Form, Criteria** function.

Function Description:

Purpose:

Allow managers to create, update, and manage evaluation forms and assessment criteria used for reviews and defense sessions in each semester.

Interface:

The system displays a management page containing:

- List of existing evaluation forms.
- Add Form button.
- Edit Form button.
- Delete Form button.
- Form information including:
 - Form Title
 - Applicable Semester
 - Evaluation Criteria
 - Question
 - Score Type
 - Weight
 - Description

Data Processing:

The system validates all entered information before storing it in the Form and Criteria database tables. Each form is associated with a semester and can later be reused in future semesters if required.

Function Details:

Validation:

- All mandatory fields must be completed.
- The form title must be unique within the same semester.
- Each criterion must contain a valid question and score type.
- Mandatory criteria cannot be left empty.
- The applicable semester must exist in the system.

Business Rules:

- Only Managers are allowed to create, edit, or delete evaluation forms.
- Evaluation forms may be reused in multiple semesters.
- Mandatory criteria must always be included in every evaluation form.
- Lecturers are authorized to evaluate students using the assigned forms.
- Council members are authorized to evaluate lecturers during defense sessions.

Normal Case:

1. The Manager opens the Form Management page.
2. The Manager creates or edits an evaluation form.
3. The Manager enters all required criteria.
4. The Manager clicks the **Save** button.
5. The system validates all information.
6. The evaluation form is stored successfully.
7. The system displays the message:
"Evaluation form has been saved successfully."

Abnormal Case:

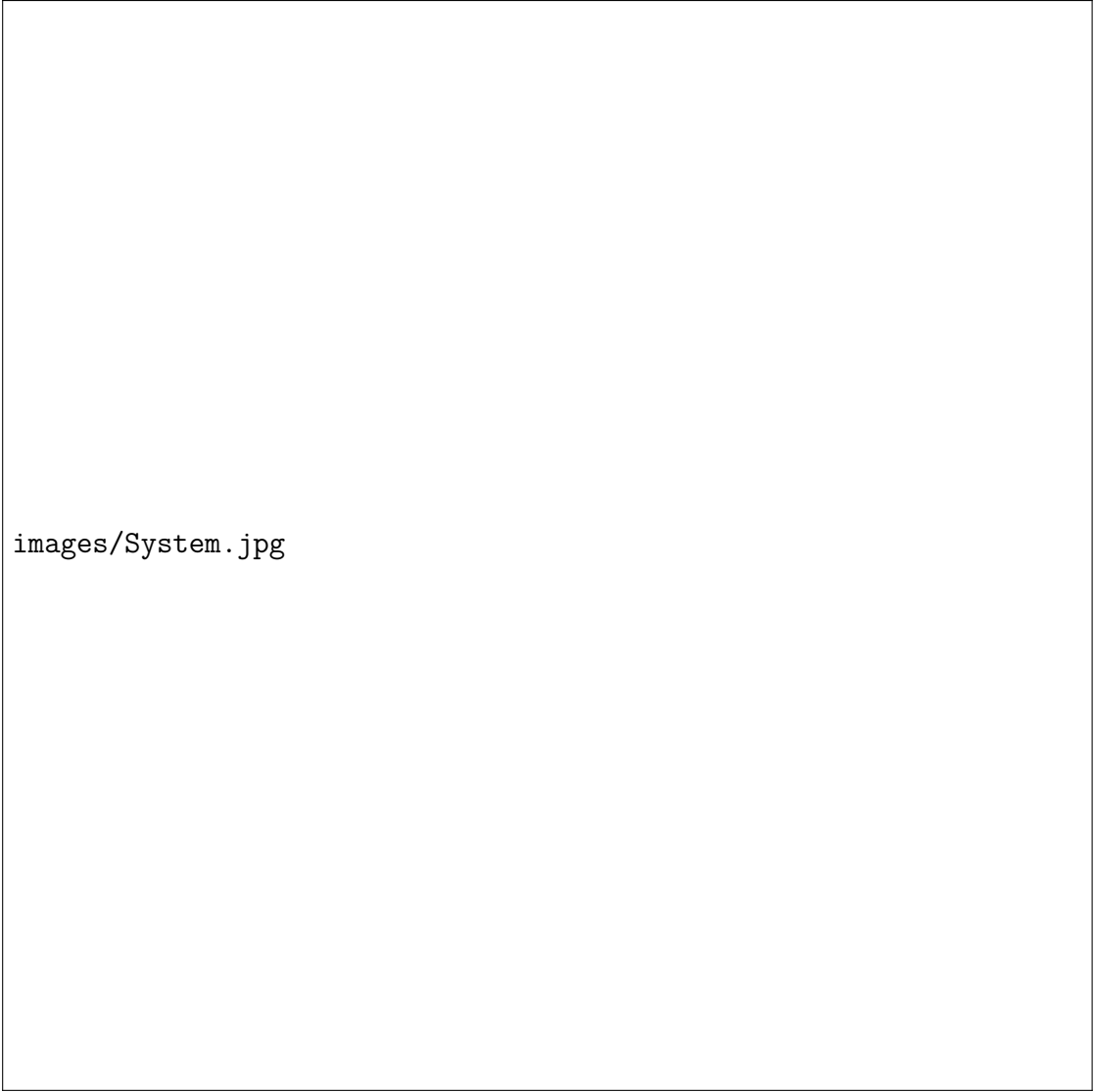
- Required information is missing.
- Duplicate form title within the same semester.
- Invalid criterion configuration.
- Invalid semester selection.
- Unexpected system error during data saving.

IV. Software Design Description

1. System Design

1.1 System Architecture

The system is designed as a web-based platform based on a layered client–server architecture. The frontend provides interactive interfaces for students, counselors, mentors, and administrators, while the backend exposes RESTful APIs, implements business logic, integrates external services, and manages persistent data. LLM APIs, the GitHub API, Google OAuth, and job portal crawlers are integrated to personalize recommendations and enrich career information.



images/System.jpg

Figure IV.1: System Architecture Diagram

1.1.1 Frontend

The frontend layer presents information, collects user input, displays roadmap progress, and visualizes skill gap and market trend reports.

- **ReactJS:** Builds a component-based single-page application with reusable user interface modules.
- **TypeScript:** Provides static typing to reduce runtime errors and improve maintainability in complex user interfaces.
- **TailwindCSS:** Supports responsive and consistent styling for dashboards, roadmap trees, cards, forms, and charts.
- **Chart Library:** Visualizes skill gap charts, learning progress statistics, and technology trend graphs.

- **Axios or Fetch API:** Communicates with backend REST APIs for authentication, profile management, roadmap data, AI chat messages, and portfolio information.

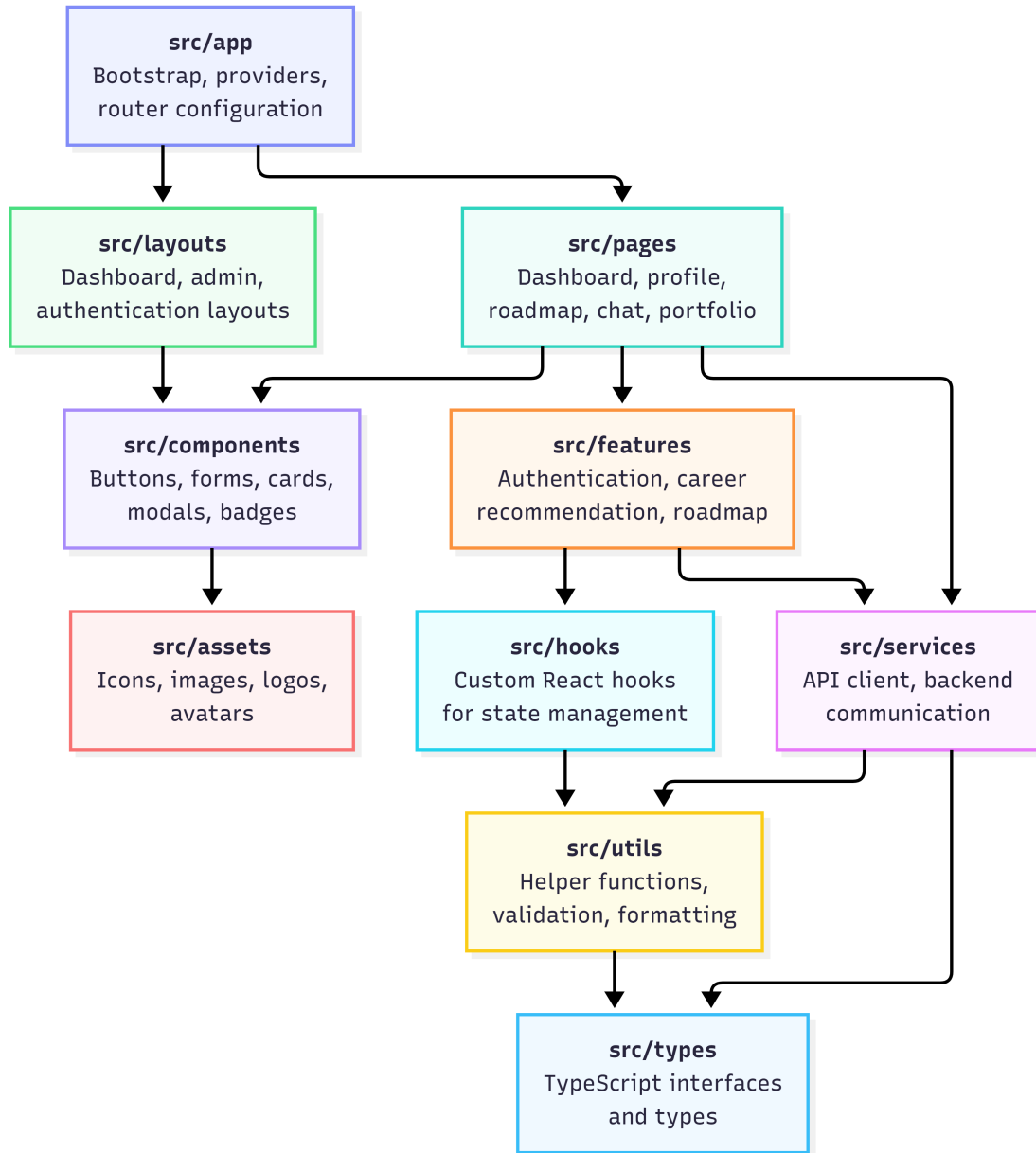


Figure IV.2: Frontend Architecture Diagram

1.1.2 Backend

The backend layer processes the core business logic and coordinates data flow among the frontend, database, AI services, GitHub, and job market data sources.

- **Spring Boot:** Serves as the main backend framework for RESTful APIs, service classes, dependency injection, validation, and security configuration.

- **Spring Security and JWT:** Protects API endpoints, manages login sessions, and supports role-based access control for students, counselors, mentors, and administrators.
- **MySQL:** Stores accounts, student profiles, skill nodes, roadmaps, progress records, mentor sessions, job trends, and portfolio records.
- **JPA/Hibernate:** Maps Java domain entities to database tables and simplifies CRUD operations.
- **Scheduler Service:** Executes daily jobs for collecting labor market data and updating technology trend statistics.
- **File Processing Service:** Handles transcript and CV uploads, extracts important information, and stores file metadata securely.

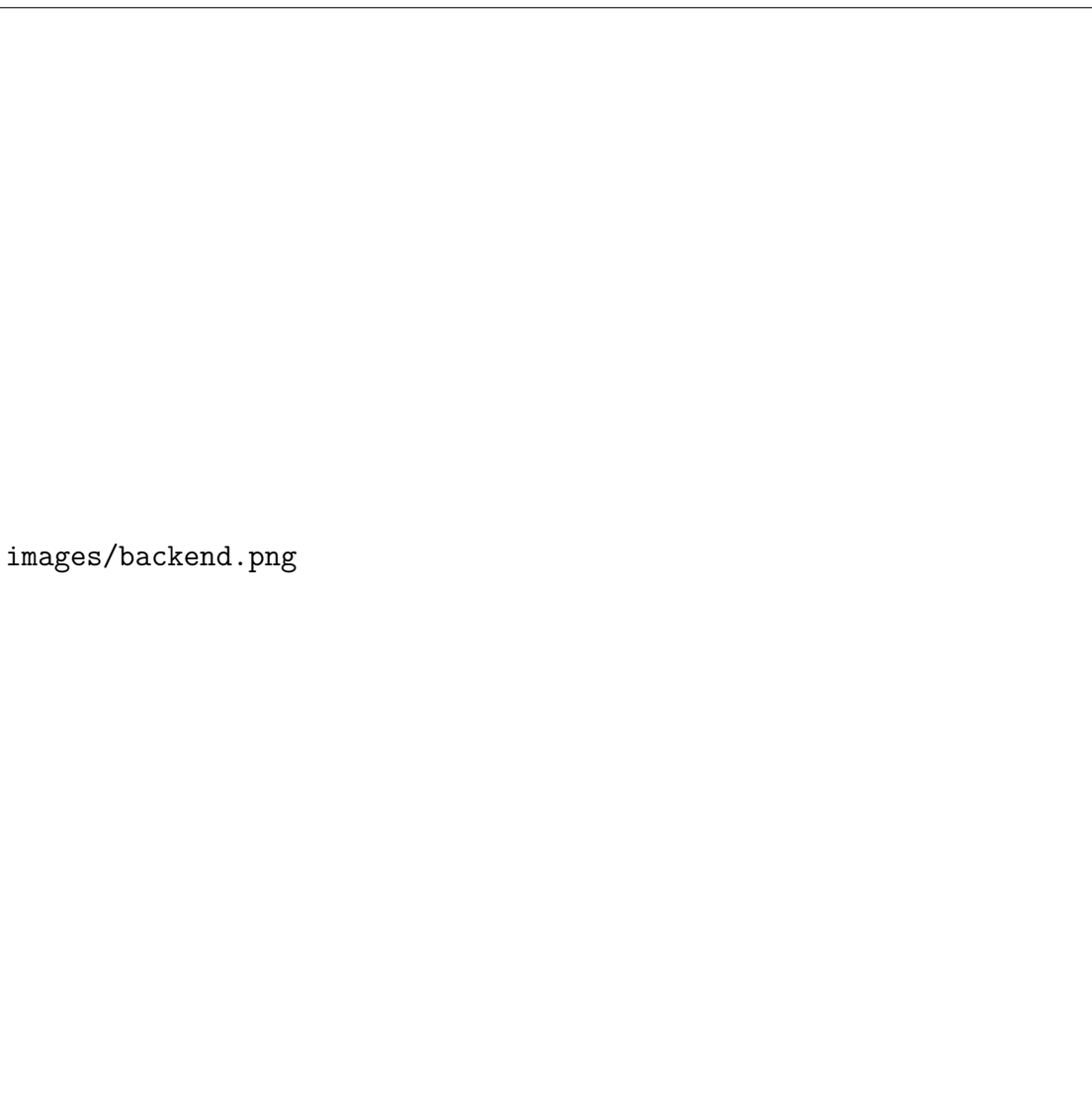


Figure IV.3: Backend Architecture Diagram

1.1.3 AI and Third-party Services

External services are integrated to support personalization, authentication, portfolio analysis, and market data collection.

- **LLM API:** Processes natural language questions and generates AI mentor responses, career advice, skill explanations, and portfolio summaries.
- **Google OAuth 2.0:** Allows users to authenticate with Google accounts in addition to email and password login.
- **GitHub API:** Retrieves public repositories, README files, programming languages, and contribution data for portfolio generation and skill analysis.
- **Job Portal Crawler/API:** Collects job descriptions from IT recruitment platforms to identify trending technologies and required competencies.
- **Cloud Deployment Platform:** Hosts frontend, backend, database, and scheduled services to ensure accessibility and scalability.

1.2 Package Diagram

This section describes the logical package organization of the frontend and backend applications. The structure separates responsibilities, improves maintainability, and supports future extension.

1.2.1 Frontend Package Structure

Table IV.1: Frontend Package Description

No.	Package	Description
1	<code>src/app</code>	Contains application bootstrap files, global providers, router configuration, and application-level initialization.
2	<code>src/pages</code>	Contains main screens such as dashboard, profile, roadmap, AI mentor chat, market pulse, portfolio, and administration pages.
3	<code>src/components</code>	Contains reusable UI components such as buttons, forms, cards, modals, skill badges, roadmap nodes, and progress indicators.
4	<code>src/features</code>	Groups feature-specific modules including authentication, career recommendation, roadmap management, skill analysis, market trends, and e-portfolio.
5	<code>src/services</code>	Provides API client functions for communicating with backend endpoints and third-party integration callbacks.

No.	Package	Description
6	src/hooks	Stores custom React hooks for authentication state, form handling, roadmap progress, chat sessions, and chart data loading.
7	src/layouts	Defines shared layouts such as student dashboard layout, administrator layout, authentication layout, and public portfolio layout.
8	src/utils	Contains helper functions for date formatting, validation, skill score calculation, file size checking, and URL generation.
9	src/assets	Stores static assets such as icons, images, logos, illustrations, and default avatar files.
10	src/types	Defines TypeScript interfaces for users, roadmaps, skill nodes, job trends, chat messages, repositories, and portfolio items.

1.2.2 Backend Package Structure

Table IV.2: Backend Package Description

No.	Package	Description
1	src/config	Contains application configuration, environment settings, CORS configuration, scheduler configuration, and external API settings.
2	src/controllers	Defines REST controllers that receive HTTP requests and return responses for each functional module.
3	src/security	Handles JWT generation, token validation, password encryption, authentication filters, and role-based authorization.
4	src/services	Contains business logic for profile management, AI mentoring, roadmap generation, skill gap analysis, market pulse, and portfolio generation.
5	src/repositories	Provides data access interfaces for querying and updating database tables through JPA/Hibernate.
6	src/entities	Defines domain entities such as User, StudentProfile, TechPath, SkillNode, Roadmap, JobTrend, MentorSession, and Portfolio.
7	src/dto	Contains request and response objects used to transfer data between frontend and backend without exposing internal entities.
8	src/mappers	Converts entities to DTOs and DTOs to entities to keep the controller, service, and persistence layers independent.
9	src/integrations	Implements clients for LLM API, GitHub API, Google OAuth, and job portal data collection.
10	src/exceptions	Manages custom exceptions and standardized error responses for validation, authentication, integration, and business rule errors.

No.	Package	Description
11	<code>src/utils</code>	Stores helper classes for file processing, text cleaning, keyword extraction, pagination, and common validation logic.
12	<code>src/schedulers</code>	Contains scheduled jobs for daily job market crawling, trend analysis updates, and reminder notification generation.

2. Database Design

The database is designed as a relational schema that maintains consistency among user accounts, student and staff profiles, career paths, skill nodes, learning resources, roadmaps, GitHub repositories, consultation sessions, AI analysis records, notifications, job trend data, and e-portfolio records. The main entities are shown in Figure IV.4.

images/DB Design.jpg

Figure IV.4: Entity Relationship Diagram

2.1 Database Tables Overview

The database contains 19 main tables. The tables cover user authentication, student and staff profiles, career paths, skill requirements, learning resources, roadmaps, portfolios, GitHub repository analysis, consultation sessions, AI configuration and analysis logs, notifications, and job market trends.

2.2 Users Table

Table IV.3: Users Table Description

Field	Type	Description
UserID	INT, PK, IDENTITY	Unique identifier of the user account.
Email	NVARCHAR(150), UNIQUE, NOT NULL	Email address used for login.
PasswordHash	NVARCHAR(255), NULL	Password hash; null when OAuth is used.
Role	NVARCHAR(50), NOT NULL	Role of the user in the platform.
CreatedAt	DATETIME	Account creation time.
IsActive	BIT	Account active status.

2.3 CareerPaths Table

Table IV.4: CareerPaths Table Description

Field	Type	Description
CareerPathID	INT, PK, IDENTITY	Unique identifier of the career path.
Title	NVARCHAR(100), NOT NULL	Career path title.
Description	NVARCHAR(MAX), NULL	Detailed career path description.
IsActive	BIT	Indicates whether the career path is active.

2.4 SkillNodes Table

Table IV.5: SkillNodes Table Description

Field	Type	Description
SkillNodeID	INT, PK, IDENTITY	Unique identifier of the skill node.
SkillName	NVARCHAR(100), NOT NULL	Name of the skill.
Category	NVARCHAR(100), NULL	Skill category.
Description	NVARCHAR(MAX), NULL	Detailed skill description.

2.5 LearningResources Table

Table IV.6: LearningResources Table Description

Field	Type	Description
ResourceID	INT, PK, IDENTITY	Unique identifier of the learning resource.
SkillNodeID	INT, FK, NOT NULL	Reference to the related skill node.
Title	NVARCHAR(255), NOT NULL	Learning resource title.
ResourceType	NVARCHAR(50), NOT NULL	Type such as Course, Book, Documentation, or Video.
Provider	NVARCHAR(100), NULL	Resource provider.
Url	NVARCHAR(500), NOT NULL	External resource URL.
Description	NVARCHAR(MAX), NULL	Additional resource description.

2.6 StudentProfiles Table

Table IV.7: StudentProfiles Table Description

Field	Type	Description
StudentID	INT, PK, FK	One-to-one reference to Users (UserID).
FullName	NVARCHAR(100), NOT NULL	Full name of the student.
AvatarUrl	NVARCHAR(255), NULL	Student avatar image URL.
PhoneNumber	VARCHAR(20), NULL	Student phone number.
Address	NVARCHAR(255), NULL	Student address.
AcademicBackground	NVARCHAR(MAX), NULL	Academic background information.
TranscriptUrl	NVARCHAR(255), NULL	Uploaded transcript file path.
CvUrl	NVARCHAR(255), NULL	Uploaded CV file path.
GitHubUsername	NVARCHAR(100), NULL	Linked GitHub username.
GitHubLinkedAt	DATETIME, NULL	Time when GitHub was linked.

2.7 StaffProfiles Table

Table IV.8: StaffProfiles Table Description

Field	Type	Description
StaffID	INT, PK, FK	One-to-one reference to Users (UserID).
FullName	NVARCHAR(100), NOT NULL	Full name of the staff member.
Specialization	NVARCHAR(255), NULL	Professional specialization.
Organization	NVARCHAR(255), NULL	Organization name.
Bio	NVARCHAR(MAX), NULL	Staff biography or introduction.

2.8 StudentCourses Table

Table IV.9: StudentCourses Table Description

Field	Type	Description
StudentID	INT, PK, FK	Reference to the student profile.
ResourceID	INT, PK, FK	Reference to the learning resource.
IsFavorite	BIT	Favorite resource flag.
Status	NVARCHAR(50)	Course status such as Enrolled or Completed.
RegisteredAt	DATETIME	Registration time.
LastAccessedAt	DATETIME, NULL	Last time the student accessed the resource.

2.9 CareerPathSkills Table

Table IV.10: CareerPathSkills Table Description

Field	Type	Description
CareerPathID	INT, PK, FK	Reference to CareerPaths(CareerPathID) .
SkillNodeID	INT, PK, FK	Reference to SkillNodes(SkillNodeID) .
RequiredLevel	NVARCHAR(50)	Required level such as Basic, Intermediate, or Advanced.

2.10 Roadmaps Table

Table IV.11: Roadmaps Table Description

Field	Type	Description
RoadmapID	INT, PK, IDENTITY	Unique identifier of the roadmap.
StudentID	INT, FK, NOT NULL	Reference to the student profile.
CareerPathID	INT, FK, NOT NULL	Reference to the selected career path.
TargetGoal	NVARCHAR(MAX), NULL	Personal target goal.
GeneratedAt	DATETIME	Roadmap generation time.
Status	NVARCHAR(50)	Roadmap status.

2.11 RoadmapProgress Table

Table IV.12: RoadmapProgress Table Description

Field	Type	Description
ProgressID	INT, PK, IDENTITY	Unique identifier of the progress record.
RoadmapID	INT, FK, NOT NULL	Reference to the roadmap.
SkillNodeID	INT, FK, NOT NULL	Reference to the skill node.
Status	NVARCHAR(50)	Learning status.
LastUpdated	DATETIME	Last progress update time.

2.12 Portfolios Table

Table IV.13: Portfolios Table Description

Field	Type	Description
PortfolioID	INT, PK, IDENTITY	Unique identifier of the portfolio.
StudentID	INT, UNIQUE, FK, NOT NULL	Reference to the portfolio owner.
Summary	NVARCHAR(MAX), NULL	AI-generated professional summary.
ShareToken	VARCHAR(100), UNIQUE, NULL	Token for public portfolio sharing.
IsApproved	BIT	Mentor approval status.
CreatedAt	DATETIME	Portfolio creation time.

2.13 GitHubRepositories Table

Table IV.14: GitHubRepositories Table Description

Field	Type	Description
RepoID	INT, PK, IDENTITY	Unique identifier of the repository record.
StudentID	INT, FK, NOT NULL	Reference to the student profile.
RepoName	NVARCHAR(150), NOT NULL	GitHub repository name.
HtmlUrl	NVARCHAR(255), NULL	Repository web URL.
Description	NVARCHAR(MAX), NULL	Repository description.
PrimaryLanguage	NVARCHAR(50), NULL	Main programming language.
ReadmeContent	NVARCHAR(MAX), NULL	README content used for portfolio analysis.
SyncedAt	DATETIME	Last synchronization time.

2.14 GitHubRepoLanguages Table

Table IV.15: GitHubRepoLanguages Table Description

Field	Type	Description
RepoID	INT, PK, FK	Reference to GitHubRepositories(RepoID) .
LanguageName	NVARCHAR(50), PK	Programming language name.
BytesWritten	INT, NOT NULL	Number of bytes written in the language.

2.15 ConsultationSessions Table

Table IV.16: ConsultationSessions Table Description

Field	Type	Description
SessionID	INT, PK, IDENTITY	Unique identifier of the session.
StudentID	INT, FK, NOT NULL	Reference to the student profile.
CounselorOrMentorID	INT, FK, NOT NULL	Reference to StaffProfiles(StaffID) .
ScheduledAt	DATETIME, NOT NULL	Scheduled consultation time.
Status	NVARCHAR(50)	Session status such as Pending, Completed, or Cancelled.
MeetingLink	NVARCHAR(255), NULL	Online meeting link.

Field	Type	Description
NotesOrComments	NVARCHAR(MAX), NULL	Counselor or mentor notes and comments.

2.16 ConsultationFeedbacks Table

Table IV.17: ConsultationFeedbacks Table Description

Field	Type	Description
FeedbackID	INT, PK, IDENTITY	Unique identifier of the feedback record.
SessionID	INT, UNIQUE, FK, NOT NULL	Reference to the consultation session.
MentorFeedback	NVARCHAR(MAX), NULL	Feedback submitted by the mentor or counselor.
StudentRating	INT, NULL	Rating submitted by the student.
SubmittedAt	DATETIME	Feedback submission time.

2.17 JobTrends Table

Table IV.18: JobTrends Table Description

Field	Type	Description
TrendID	INT, PK, IDENTITY	Unique identifier of the job trend record.
CareerPathID	INT, FK, NOT NULL	Reference to the career path.
AverageSalary	DECIMAL(18,2), NULL	Average salary information.
DemandLevel	NVARCHAR(50), NULL	Demand level.
TopRequired Technologies	NVARCHAR(MAX), NULL	Most required technologies from job market data.
LastUpdated	DATETIME	Last trend update time.

2.18 AIConfigurations Table

Table IV.19: AIConfigurations Table Description

Field	Type	Description
ConfigID	INT, PK, IDENTITY	Unique identifier of the AI configuration.
EngineKey	NVARCHAR(100), UNIQUE, NOT NULL	Key used to identify the AI engine or feature.
ModelVersion	NVARCHAR(50)	Model version used by the AI feature.
PromptTemplate	NVARCHAR(MAX), NOT NULL	Prompt template for AI requests.
Temperature	FLOAT	AI generation temperature.
LastModified	DATETIME	Last configuration update time.

2.19 AIAnalysisLogs Table

Table IV.20: AIAAnalysisLogs Table Description

Field	Type	Description
LogID	INT, PK, IDENTITY	Unique identifier of the AI analysis log.
StudentID	INT, FK, NOT NULL	Reference to the student profile.
AnalysisType	NVARCHAR(100), NOT NULL	Type of AI analysis.
InputSnapshot	NVARCHAR(MAX), NULL	Input data snapshot sent to AI.
OutputResult	NVARCHAR(MAX), NOT NULL	AI output result.
TokensUsed	INT, NULL	Number of tokens used.
ExecutedAt	DATETIME	Execution time of the AI analysis.

2.20 Notifications Table

Table IV.21: Notifications Table Description

Field	Type	Description
NotificationID	INT, PK, IDENTITY	Unique identifier of the notification.
UserID	INT, FK, NOT NULL	Reference to the receiver user.
Title	NVARCHAR(200), NOT NULL	Notification title.
Message	NVARCHAR(MAX), NOT NULL	Notification message content.
IsRead	BIT	Read status of the notification.
CreatedAt	DATETIME	Notification creation time.

3. Detailed Design

This section provides detailed design coverage for all 82 use cases described in Chapter III. Each use case is traced to the related actor, database tables, controller, service, entity, repository, processing flow, exception handling, class diagram, and sequence diagram. This ensures that the detailed design is consistent with the system architecture, package design, database design, and class design presented earlier in Chapter IV.

3.1 Use Case to Design Coverage Summary

Table IV.22: Use Case to Detailed Design Coverage

UC	Use Case	Actor	Design Group
UC01	Register	Student	Authentication and Account Access
UC02	Login	Student	Authentication and Account Access
UC03	Login with Google	Student	Authentication and Account Access
UC04	Forgot Password	Student	Authentication and Account Access
UC05	Logout	Student	Authentication and Account Access
UC06	Manage Profile	Student	Student Profile and GitHub Integration
UC07	Upload Avatar	Student	Student Profile and GitHub Integration

UC	Use Case	Actor	Design Group
UC08	Update Contact Information	Student	Student Profile and GitHub Integration
UC09	Change Password	Student	Student Profile and GitHub Integration
UC10	Upload Transcript	Student	Student Profile and GitHub Integration
UC11	Upload CV	Student	Student Profile and GitHub Integration
UC12	Link GitHub	Student	Student Profile and GitHub Integration
UC13	Sync GitHub Repository	Student	Student Profile and GitHub Integration
UC14	View GitHub Analysis	Student	Student Profile and GitHub Integration
UC15	View Skill Gap Report	Student	Student Profile and GitHub Integration
UC16	Select Career Role	Student	Career Path and Skill Management
UC17	Update Career Goal	Student	Career Path and Skill Management
UC18	View Recommended Roles	Student	Career Path and Skill Management
UC19	Compare Career Paths	Student	Career Path and Skill Management
UC20	View Required Skills	Student	Career Path and Skill Management
UC21	Generate Learning Roadmap	Student	Learning Roadmap and Course Management
UC22	View Learning Roadmap	Student	Learning Roadmap and Course Management
UC23	Update Roadmap Progress	Student	Learning Roadmap and Course Management
UC24	View Learning Progress	Student	Learning Roadmap and Course Management
UC25	Receive Learning Reminder	Student	Learning Roadmap and Course Management
UC26	Search Learning Resources	Student	Learning Roadmap and Course Management
UC27	View Recommended Courses	Student	Learning Roadmap and Course Management
UC28	Save Favorite Courses	Student	Learning Roadmap and Course Management
UC29	Register External Course	Student	Learning Roadmap and Course Management
UC30	Track Course Completion	Student	Learning Roadmap and Course Management
UC31	View Market Trends	Student	Market Trend Management
UC32	Search Job Opportunities	Student	Market Trend Management
UC33	View Salary Information	Student	Market Trend Management
UC34	View Technology Trends	Student	Market Trend Management
UC35	Receive Job Recommendation	Student	Market Trend Management
UC36	Chat with AI Mentor	Student	AI Mentor and Analysis Logging
UC37	Ask Career Questions	Student	AI Mentor and Analysis Logging
UC38	Receive AI Feedback	Student	AI Mentor and Analysis Logging
UC39	Generate E-Portfolio	Student	E-Portfolio and Mentor Review
UC40	Share Portfolio	Student	E-Portfolio and Mentor Review
UC41	Login	Academic Counselor	Authentication and Account Access
UC42	View Student Profile	Academic Counselor	Consultation and Feedback Management

UC	Use Case	Actor	Design Group
UC43	Review Skill Gap	Academic Counselor	Consultation and Feedback Management
UC44	Recommend Career Path	Academic Counselor	Consultation and Feedback Management
UC45	Recommend Courses	Academic Counselor	Consultation and Feedback Management
UC46	Track Student Progress	Academic Counselor	Consultation and Feedback Management
UC47	Comment Learning Plan	Academic Counselor	Consultation and Feedback Management
UC48	Schedule Consultation	Academic Counselor	Consultation and Feedback Management
UC49	View Consultation History	Academic Counselor	Consultation and Feedback Management
UC50	Logout	Academic Counselor	Authentication and Account Access
UC51	Login	Industry Mentor	Authentication and Account Access
UC52	View Student Portfolio	Industry Mentor	E-Portfolio and Mentor Review
UC53	Review Projects	Industry Mentor	E-Portfolio and Mentor Review
UC54	Recommend Technologies	Industry Mentor	E-Portfolio and Mentor Review
UC55	Give Technical Advice	Industry Mentor	E-Portfolio and Mentor Review
UC56	Evaluate Progress	Industry Mentor	E-Portfolio and Mentor Review
UC57	Comment Projects	Industry Mentor	E-Portfolio and Mentor Review
UC58	Approve Portfolio	Industry Mentor	E-Portfolio and Mentor Review
UC59	Suggest Improvements	Industry Mentor	E-Portfolio and Mentor Review
UC60	Logout	Industry Mentor	Authentication and Account Access
UC61	Login	Administrator	Authentication and Account Access
UC62	Manage Users	Administrator	Administration and Resource Management
UC63	Manage Career Roles	Administrator	Career Path and Skill Management
UC64	Manage Skill Nodes	Administrator	Career Path and Skill Management
UC65	Manage Courses	Administrator	Administration and Resource Management
UC66	Manage Learning Resources	Administrator	Administration and Resource Management
UC67	Manage Job Trend Data	Administrator	Market Trend Management
UC68	Manage AI Configuration	Administrator	AI and Market Configuration
UC69	Manage Reports	Administrator	Administration and Resource Management
UC70	Logout	Administrator	Authentication and Account Access
UC71	Analyze Transcript	AI Recommendation Engine	AI Mentor and Analysis Logging
UC72	Analyze GitHub Repository	AI Recommendation Engine	AI Mentor and Analysis Logging
UC73	Generate Personalized Roadmap	AI Recommendation Engine	Learning Roadmap and Course Management
UC74	Analyze Skill Gap	AI Recommendation Engine	Learning Roadmap and Course Management
UC75	Recommend Courses	AI Recommendation Engine	Learning Roadmap and Course Management

UC	Use Case	Actor	Design Group
UC76	Generate Portfolio Summary	AI Recommendation Engine	AI Mentor and Analysis Logging
UC77	Authenticate User	Google OAuth	Authentication and Account Access
UC78	Retrieve Repository	GitHub API	Student Profile and GitHub Integration
UC79	Retrieve README	GitHub API	Student Profile and GitHub Integration
UC80	Retrieve Tech Stack	GitHub API	Student Profile and GitHub Integration
UC81	Retrieve Job Data	Job Portal API	Market Trend Management
UC82	Analyze Market Trends	Job Portal API	Market Trend Management

3.2 Authentication and Account Access

This group contains use cases related to authentication and account access. The design uses the layered architecture introduced in the system design: React boundary pages, Spring Boot controllers, services, repositories, database entities, and external gateways when required.

3.2.1 UC01 Register

3.2.1.1 Requirement Traceability

- **Use case ID:** UC01.
- **Use case name:** Register.
- **Actor:** Student.
- **Requirement description:** Create a new student account.
- **Related table(s):** Users.

3.2.1.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.2.1.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.2.1.4 Main Processing Flow

1. The actor opens **RegisterPage** and performs the action: create a new student account..
2. The frontend validates required fields, prepares the request DTO, and sends it to **AuthController**.
3. **AuthController** checks authentication, authorization, request format, and path parameters.
4. **AuthService** loads related data through **UserRepository** and applies business rules for Register.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in **Users** and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.2.1.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.2.1.6 Participating Components

- **Boundary/UI – RegisterPage:** collects input and displays the final result for Register.
- **Controller – AuthController:** receives REST requests and validates request DTOs.
- **Service – AuthService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – User:** represents the main persisted domain object.
- **Repository – UserRepository:** provides data access operations for the related table(s).

3.2.1.7 Class Diagram

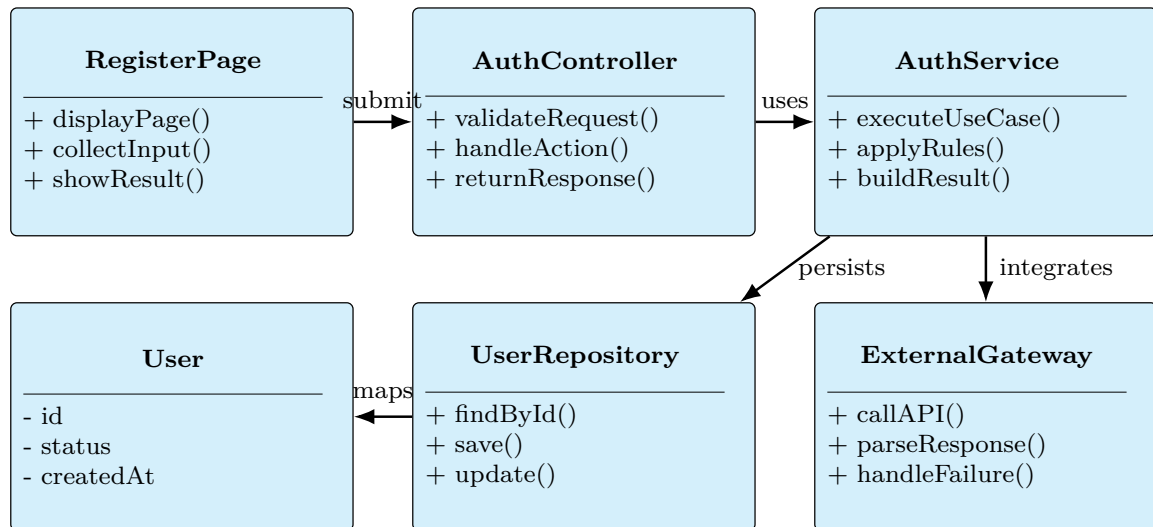


Figure IV.5: Class diagram for UC01 Register

3.2.1.8 Sequence Diagram

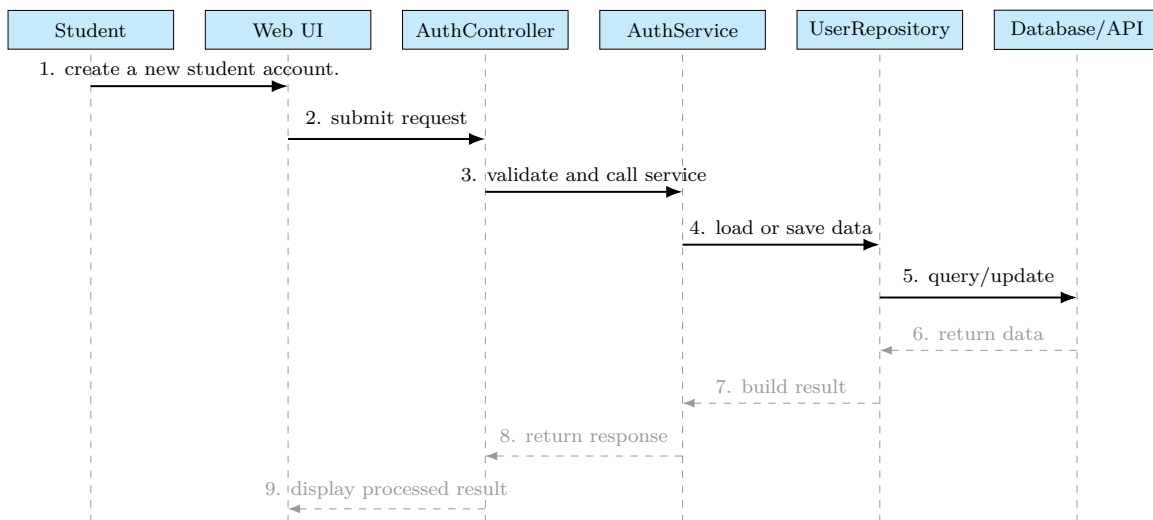


Figure IV.6: Sequence diagram for UC01 Register

3.2.2 UC02 Login

3.2.2.1 Requirement Traceability

- **Use case ID:** UC02.
- **Use case name:** Login.
- **Actor:** Student.
- **Requirement description:** Login using email and password.
- **Related table(s):** Users.

3.2.2.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.2.2.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.2.2.4 Main Processing Flow

1. The actor opens `LoginPage` and performs the action: login using email and password..
2. The frontend validates required fields, prepares the request DTO, and sends it to `AuthController`.
3. `AuthController` checks authentication, authorization, request format, and path parameters.
4. `AuthService` loads related data through `UserRepository` and applies business rules for Login.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in `Users` and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.2.2.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.2.2.6 Participating Components

- **Boundary/UI – LoginPage:** collects input and displays the final result for Login.
- **Controller – AuthController:** receives REST requests and validates request DTOs.
- **Service – AuthService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – User:** represents the main persisted domain object.
- **Repository – UserRepository:** provides data access operations for the related table(s).

3.2.2.7 Class Diagram

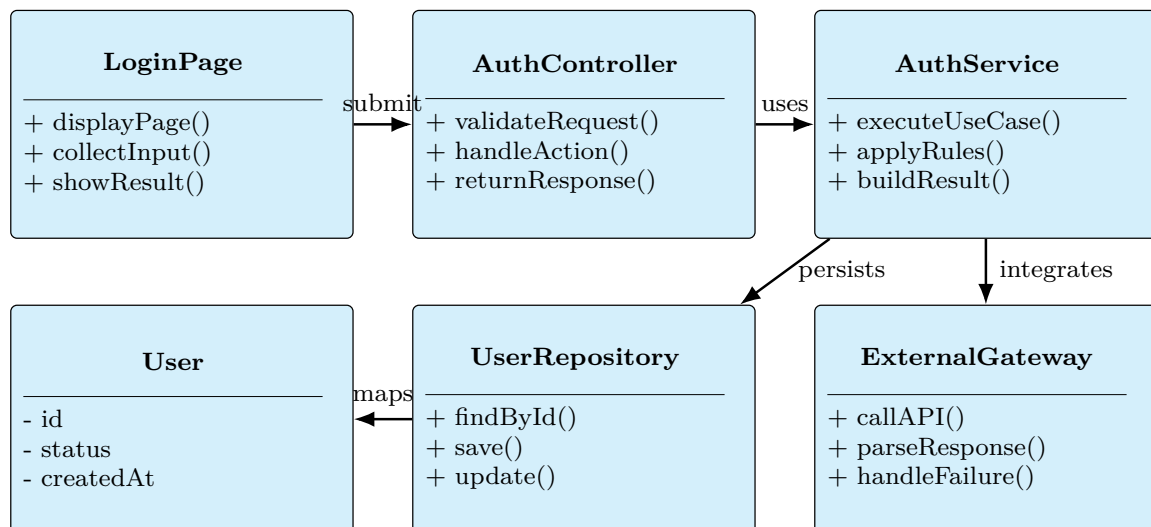


Figure IV.7: Class diagram for UC02 Login

3.2.2.8 Sequence Diagram

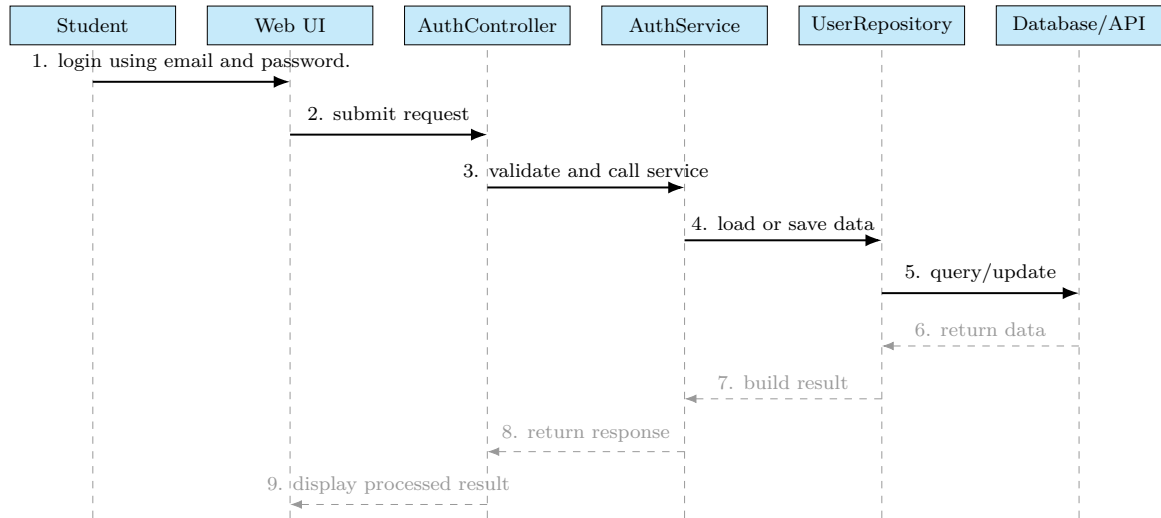


Figure IV.8: Sequence diagram for UC02 Login

3.2.3 UC03 Login with Google

3.2.3.1 Requirement Traceability

- **Use case ID:** UC03.
- **Use case name:** Login with Google.
- **Actor:** Student.
- **Requirement description:** Authenticate via Google OAuth.
- **Related table(s):** Users.

3.2.3.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.2.3.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.2.3.4 Main Processing Flow

1. The actor opens `LoginwithGooglePage` and performs the action: authenticate via Google OAuth..

2. The frontend validates required fields, prepares the request DTO, and sends it to **AuthController**.
3. **AuthController** checks authentication, authorization, request format, and path parameters.
4. **AuthService** loads related data through **UserRepository** and applies business rules for Login with Google.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in **Users** and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.2.3.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.2.3.6 Participating Components

- **Boundary/UI – LoginwithGooglePage:** collects input and displays the final result for Login with Google.
- **Controller – AuthController:** receives REST requests and validates request DTOs.
- **Service – AuthService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – User:** represents the main persisted domain object.
- **Repository – UserRepository:** provides data access operations for the related table(s).

3.2.3.7 Class Diagram

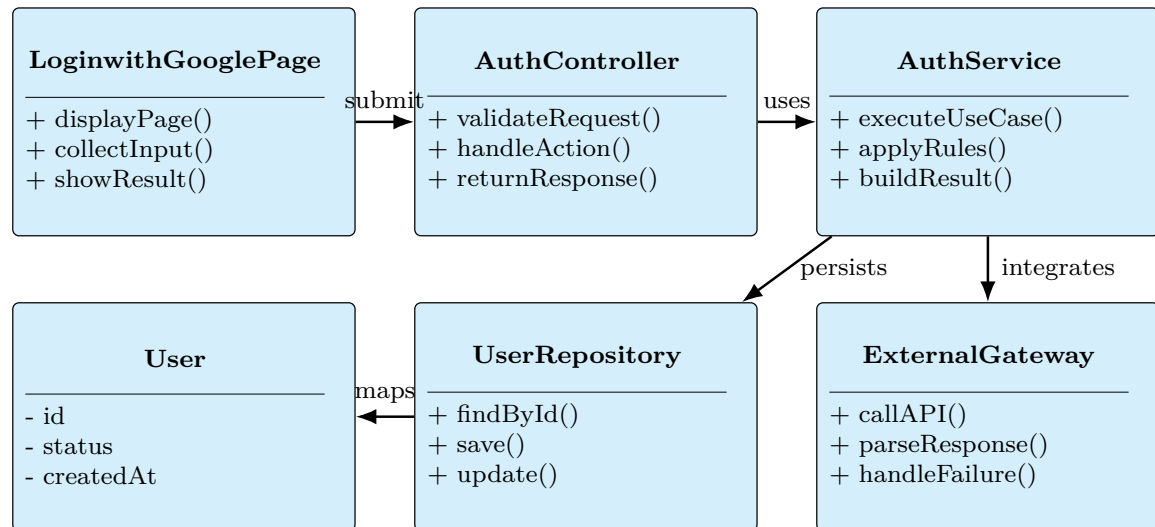


Figure IV.9: Class diagram for UC03 Login with Google

3.2.3.8 Sequence Diagram

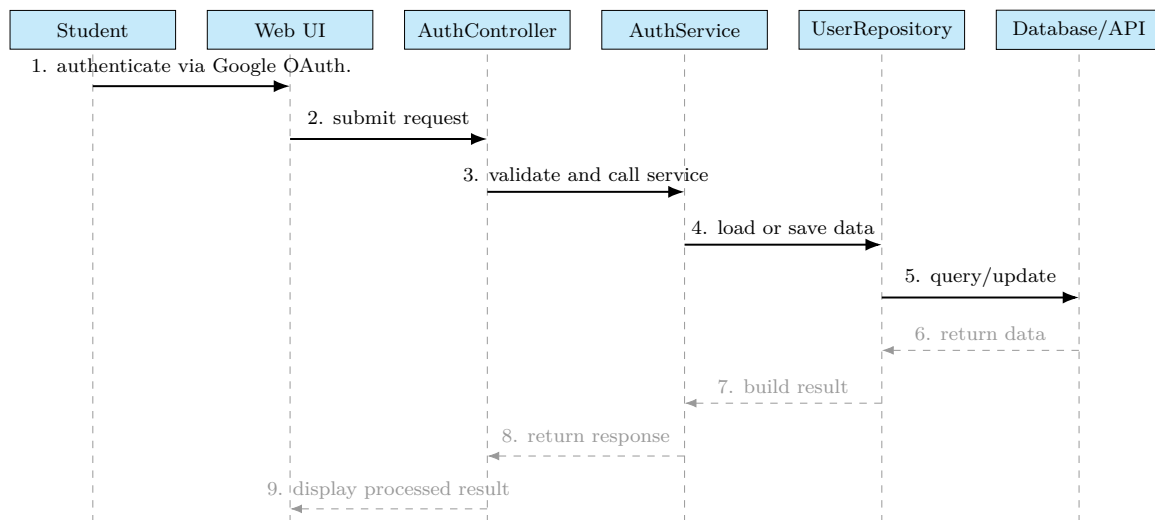


Figure IV.10: Sequence diagram for UC03 Login with Google

3.2.4 UC04 Forgot Password

3.2.4.1 Requirement Traceability

- **Use case ID:** UC04.
- **Use case name:** Forgot Password.
- **Actor:** Student.
- **Requirement description:** Recover forgotten password.
- **Related table(s):** Users.

3.2.4.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.2.4.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.2.4.4 Main Processing Flow

1. The actor opens `ForgotPasswordPage` and performs the action: recover forgotten password..
2. The frontend validates required fields, prepares the request DTO, and sends it to `AuthController`.
3. `AuthController` checks authentication, authorization, request format, and path parameters.
4. `AuthService` loads related data through `UserRepository` and applies business rules for Forgot Password.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in `Users` and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.2.4.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.2.4.6 Participating Components

- **Boundary/UI – ForgotPasswordPage:** collects input and displays the final result for Forgot Password.
- **Controller – AuthController:** receives REST requests and validates request DTOs.
- **Service – AuthService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – User:** represents the main persisted domain object.
- **Repository – UserRepository:** provides data access operations for the related table(s).

3.2.4.7 Class Diagram

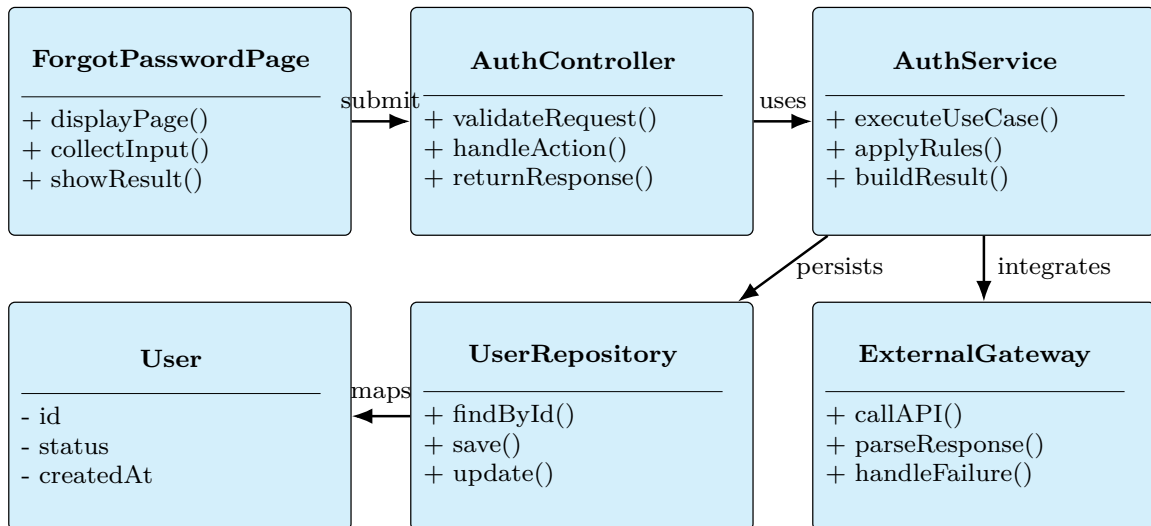


Figure IV.11: Class diagram for UC04 Forgot Password

3.2.4.8 Sequence Diagram

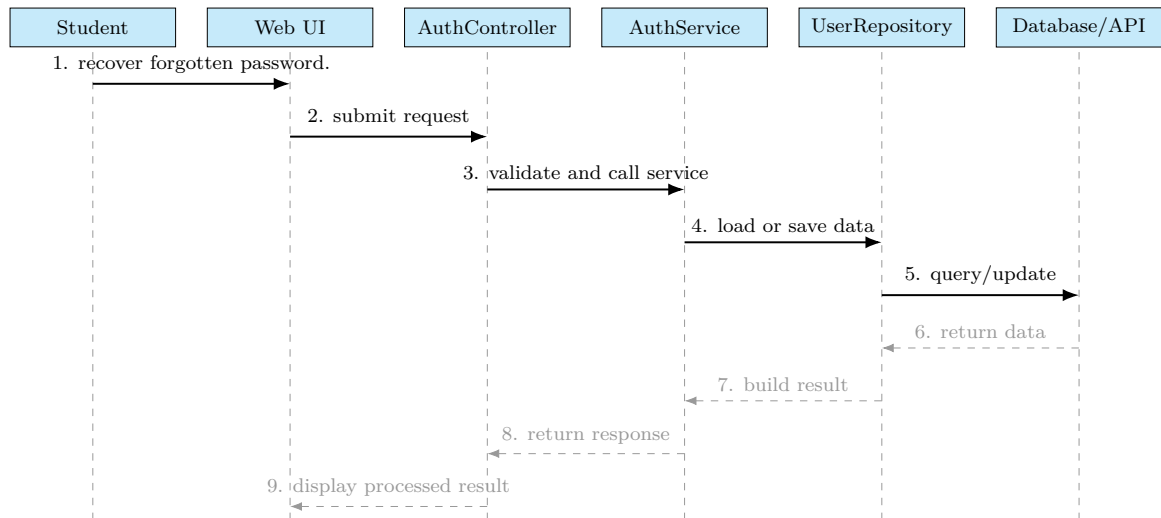


Figure IV.12: Sequence diagram for UC04 Forgot Password

3.2.5 UC05 Logout

3.2.5.1 Requirement Traceability

- **Use case ID:** UC05.
- **Use case name:** Logout.
- **Actor:** Student.
- **Requirement description:** Logout from the system.
- **Related table(s):** Users.

3.2.5.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.2.5.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.2.5.4 Main Processing Flow

1. The actor opens `LogoutPage` and performs the action: logout from the system..

2. The frontend validates required fields, prepares the request DTO, and sends it to **AuthController**.
3. **AuthController** checks authentication, authorization, request format, and path parameters.
4. **AuthService** loads related data through **UserRepository** and applies business rules for Logout.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in **Users** and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.2.5.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.2.5.6 Participating Components

- **Boundary/UI – LogoutPage:** collects input and displays the final result for Logout.
- **Controller – AuthController:** receives REST requests and validates request DTOs.
- **Service – AuthService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – User:** represents the main persisted domain object.
- **Repository – UserRepository:** provides data access operations for the related table(s).

3.2.5.7 Class Diagram

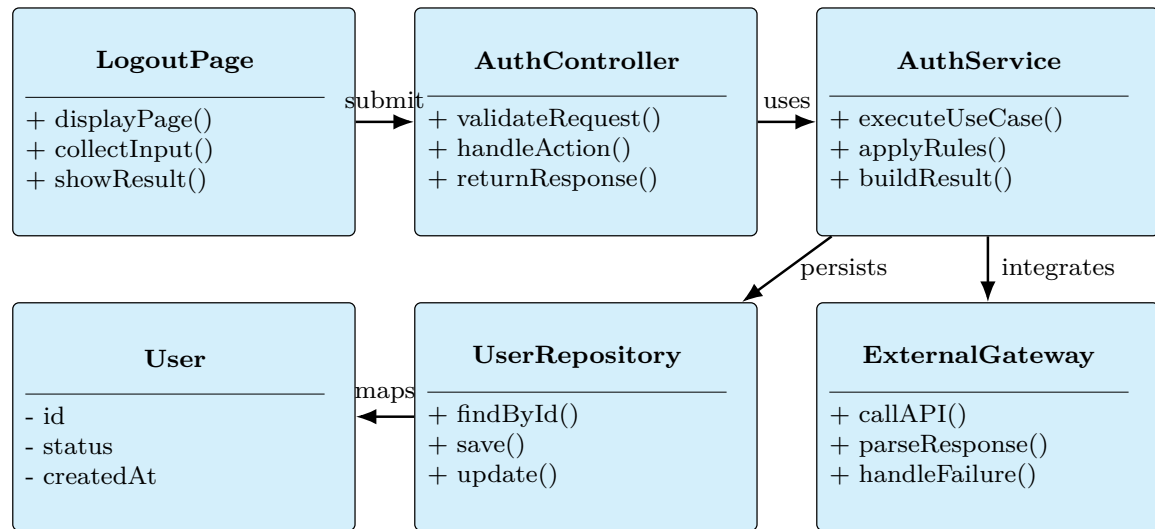


Figure IV.13: Class diagram for UC05 Logout

3.2.5.8 Sequence Diagram

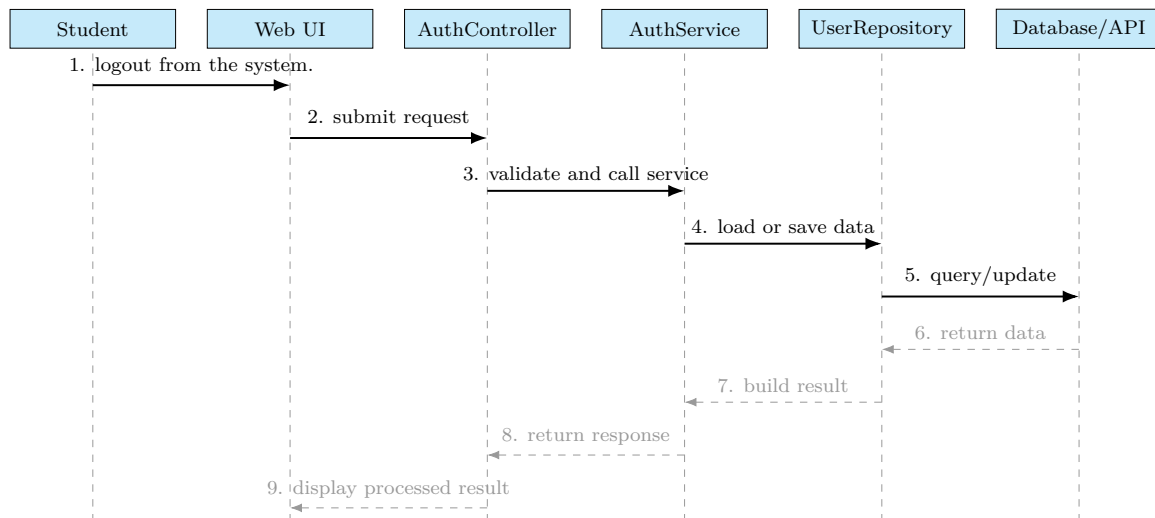


Figure IV.14: Sequence diagram for UC05 Logout

3.3 Student Profile and GitHub Integration

This group contains use cases related to student profile and github integration. The design uses the layered architecture introduced in the system design: React boundary pages, Spring Boot controllers, services, repositories, database entities, and external gateways when required.

3.3.1 UC06 Manage Profile

3.3.1.1 Requirement Traceability

- **Use case ID:** UC06.
- **Use case name:** Manage Profile.
- **Actor:** Student.
- **Requirement description:** Update personal information.
- **Related table(s):** StudentProfiles.

3.3.1.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in **Users.Role** must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.3.1.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.3.1.4 Main Processing Flow

1. The actor opens **ManageProfilePage** and performs the action: update personal information..
2. The frontend validates required fields, prepares the request DTO, and sends it to **ProfileController**.
3. **ProfileController** checks authentication, authorization, request format, and path parameters.
4. **ProfileService** loads related data through **StudentProfileRepo** and applies business rules for Manage Profile.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in **StudentProfiles** and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.3.1.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.3.1.6 Participating Components

- **Boundary/UI – ManageProfilePage:** collects input and displays the final result for Manage Profile.
- **Controller – ProfileController:** receives REST requests and validates request DTOs.
- **Service – ProfileService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – StudentProfile:** represents the main persisted domain object.
- **Repository – StudentProfileRepo:** provides data access operations for the related table(s).

3.3.1.7 Class Diagram

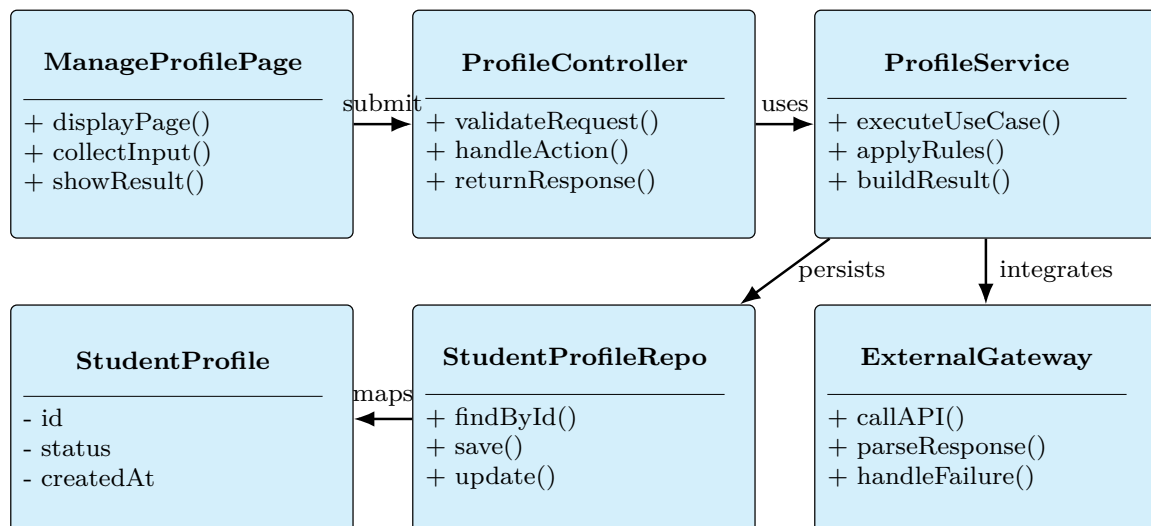


Figure IV.15: Class diagram for UC06 Manage Profile

3.3.1.8 Sequence Diagram

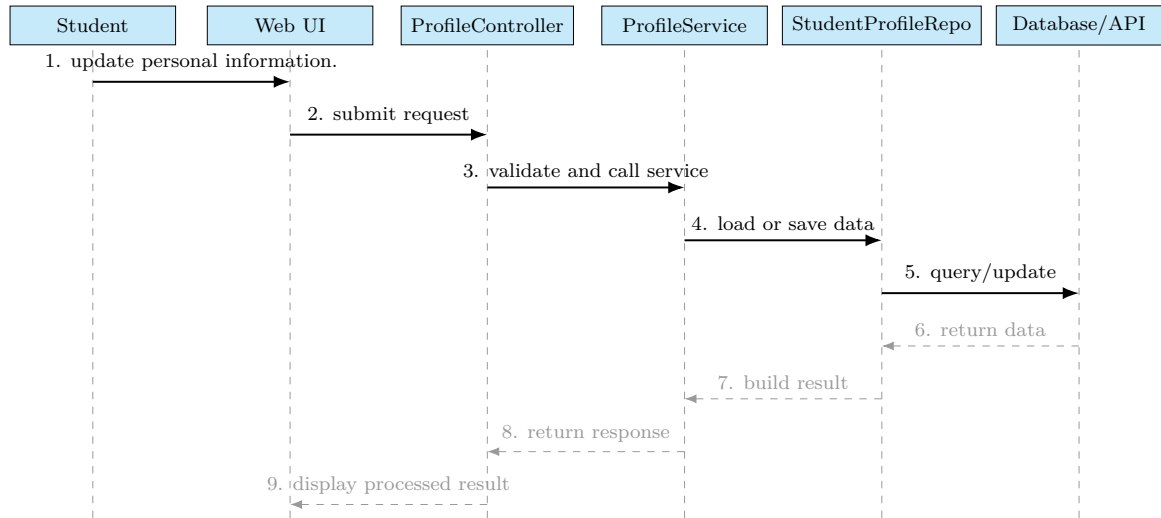


Figure IV.16: Sequence diagram for UC06 Manage Profile

3.3.2 UC07 Upload Avatar

3.3.2.1 Requirement Traceability

- **Use case ID:** UC07.
- **Use case name:** Upload Avatar.
- **Actor:** Student.
- **Requirement description:** Upload profile image.
- **Related table(s):** StudentProfiles.

3.3.2.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.3.2.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.3.2.4 Main Processing Flow

1. The actor opens `UploadAvatarPage` and performs the action: upload profile image..

2. The frontend validates required fields, prepares the request DTO, and sends it to **ProfileController**.
3. **ProfileController** checks authentication, authorization, request format, and path parameters.
4. **ProfileService** loads related data through **StudentProfileRepo** and applies business rules for Upload Avatar.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in **StudentProfiles** and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.3.2.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.3.2.6 Participating Components

- **Boundary/UI – UploadAvatarPage:** collects input and displays the final result for Upload Avatar.
- **Controller – ProfileController:** receives REST requests and validates request DTOs.
- **Service – ProfileService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – StudentProfile:** represents the main persisted domain object.
- **Repository – StudentProfileRepo:** provides data access operations for the related table(s).

3.3.2.7 Class Diagram

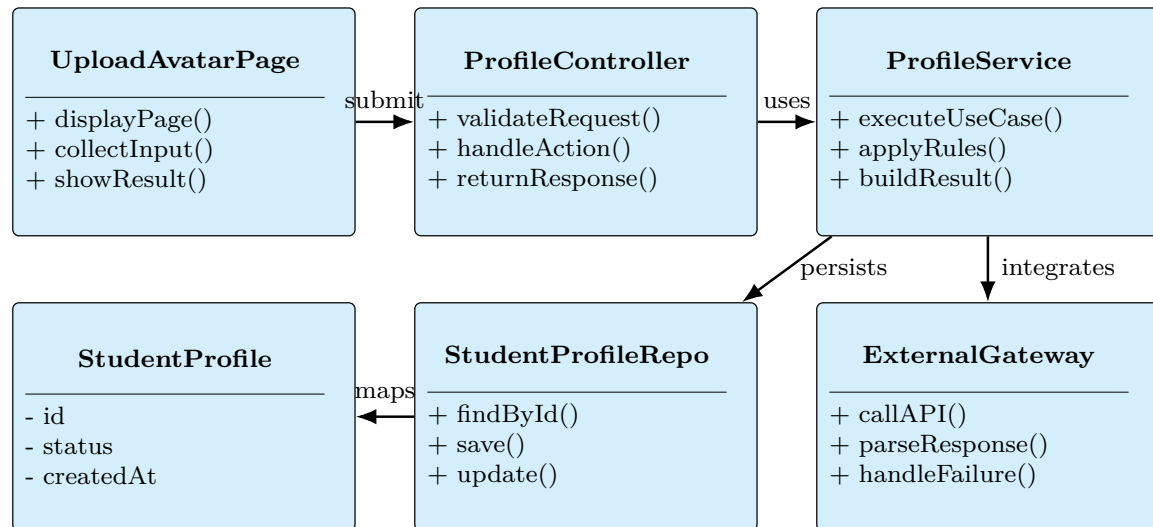


Figure IV.17: Class diagram for UC07 Upload Avatar

3.3.2.8 Sequence Diagram

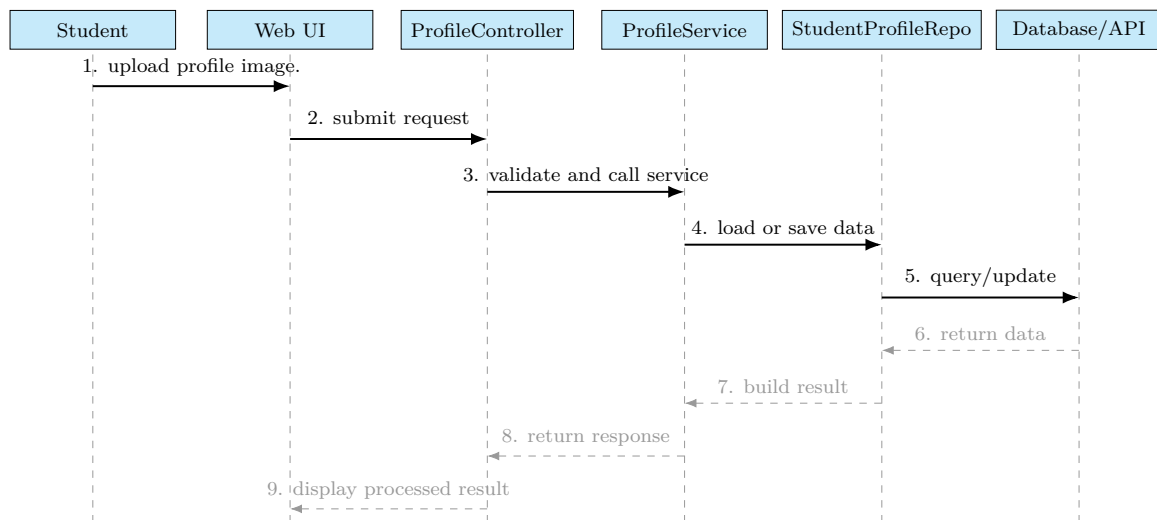


Figure IV.18: Sequence diagram for UC07 Upload Avatar

3.3.3 UC08 Update Contact Information

3.3.3.1 Requirement Traceability

- **Use case ID:** UC08.
- **Use case name:** Update Contact Information.
- **Actor:** Student.
- **Requirement description:** Edit phone number and address.
- **Related table(s):** StudentProfiles.

3.3.3.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.3.3.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.3.3.4 Main Processing Flow

1. The actor opens `UpdateContactInforPage` and performs the action: edit phone number and address..
2. The frontend validates required fields, prepares the request DTO, and sends it to `ProfileController`.
3. `ProfileController` checks authentication, authorization, request format, and path parameters.
4. `ProfileService` loads related data through `StudentProfileRepo` and applies business rules for Update Contact Information.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in `StudentProfiles` and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.3.3.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.3.3.6 Participating Components

- **Boundary/UI – UpdateContactInforPage:** collects input and displays the final result for Update Contact Information.
- **Controller – ProfileController:** receives REST requests and validates request DTOs.
- **Service – ProfileService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – StudentProfile:** represents the main persisted domain object.
- **Repository – StudentProfileRepo:** provides data access operations for the related table(s).

3.3.3.7 Class Diagram

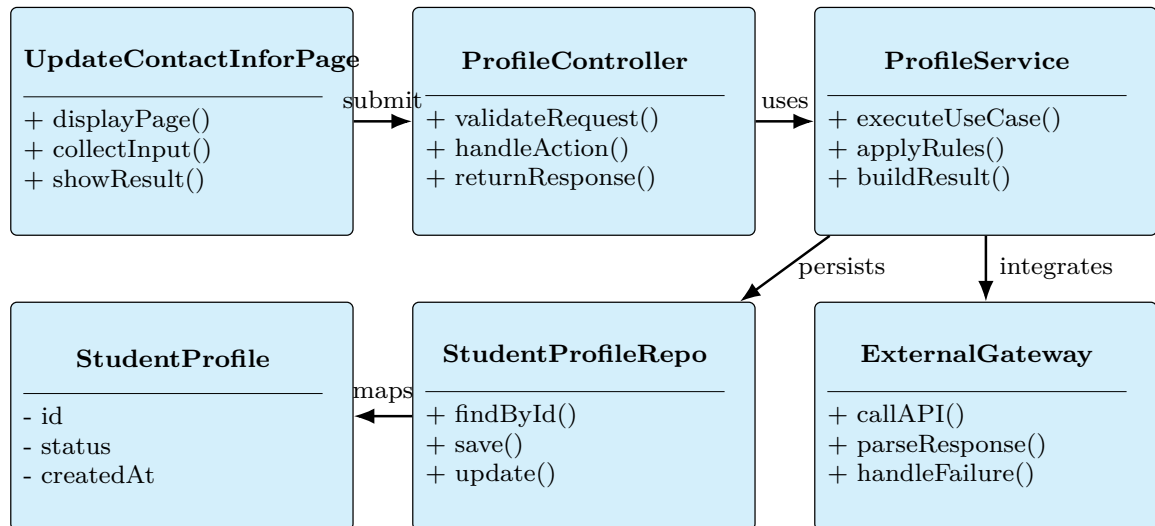


Figure IV.19: Class diagram for UC08 Update Contact Information

3.3.3.8 Sequence Diagram

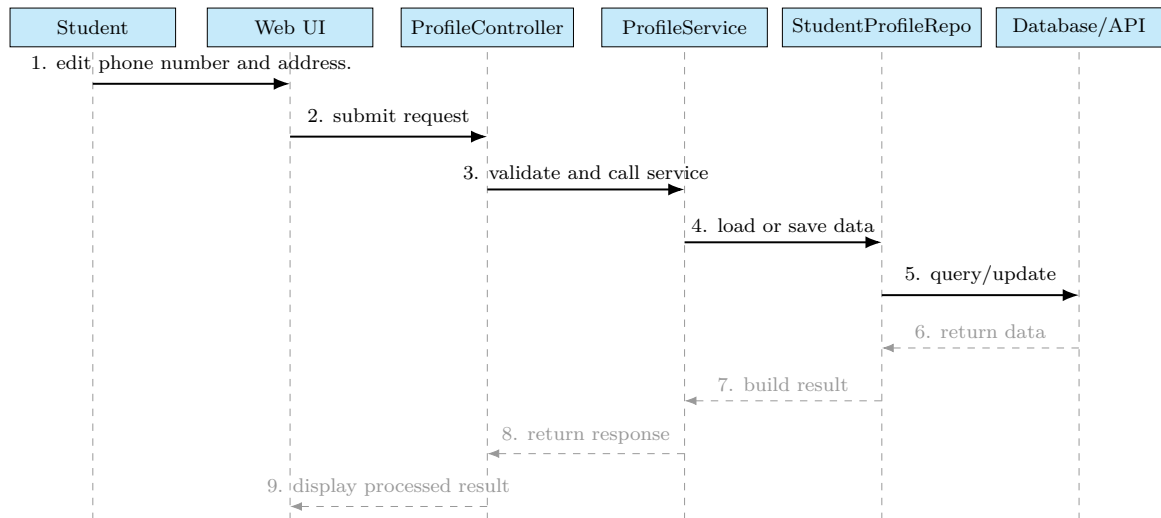


Figure IV.20: Sequence diagram for UC08 Update Contact Information

3.3.4 UC09 Change Password

3.3.4.1 Requirement Traceability

- **Use case ID:** UC09.
- **Use case name:** Change Password.
- **Actor:** Student.
- **Requirement description:** Update account password.
- **Related table(s):** StudentProfiles.

3.3.4.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.3.4.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.3.4.4 Main Processing Flow

1. The actor opens `ChangePasswordPage` and performs the action: update account password..

2. The frontend validates required fields, prepares the request DTO, and sends it to **ProfileController**.
3. **ProfileController** checks authentication, authorization, request format, and path parameters.
4. **ProfileService** loads related data through **StudentProfileRepo** and applies business rules for Change Password.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in **StudentProfiles** and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.3.4.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.3.4.6 Participating Components

- **Boundary/UI – ChangePasswordPage:** collects input and displays the final result for Change Password.
- **Controller – ProfileController:** receives REST requests and validates request DTOs.
- **Service – ProfileService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – StudentProfile:** represents the main persisted domain object.
- **Repository – StudentProfileRepo:** provides data access operations for the related table(s).

3.3.4.7 Class Diagram

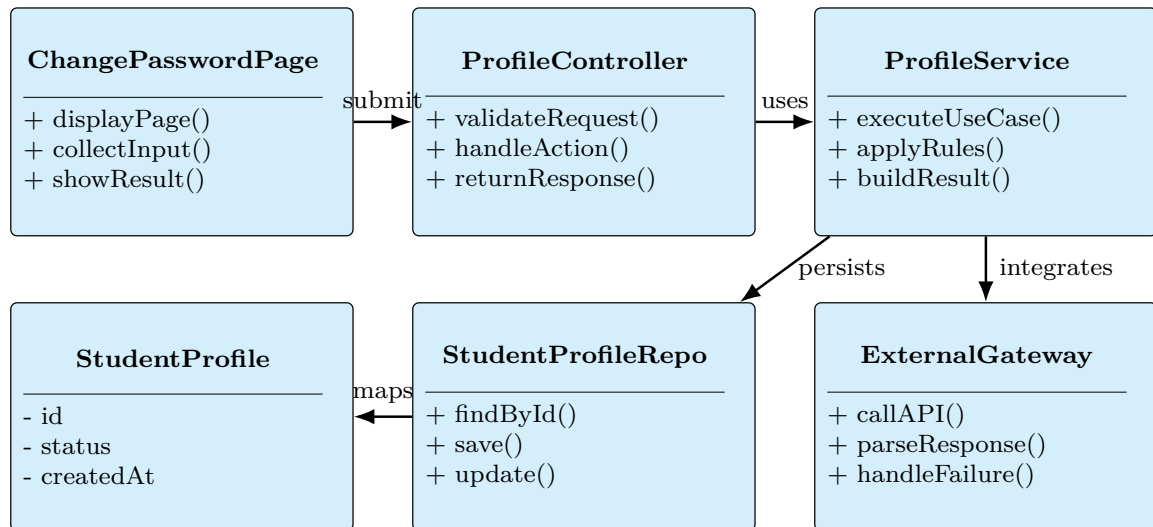


Figure IV.21: Class diagram for UC09 Change Password

3.3.4.8 Sequence Diagram

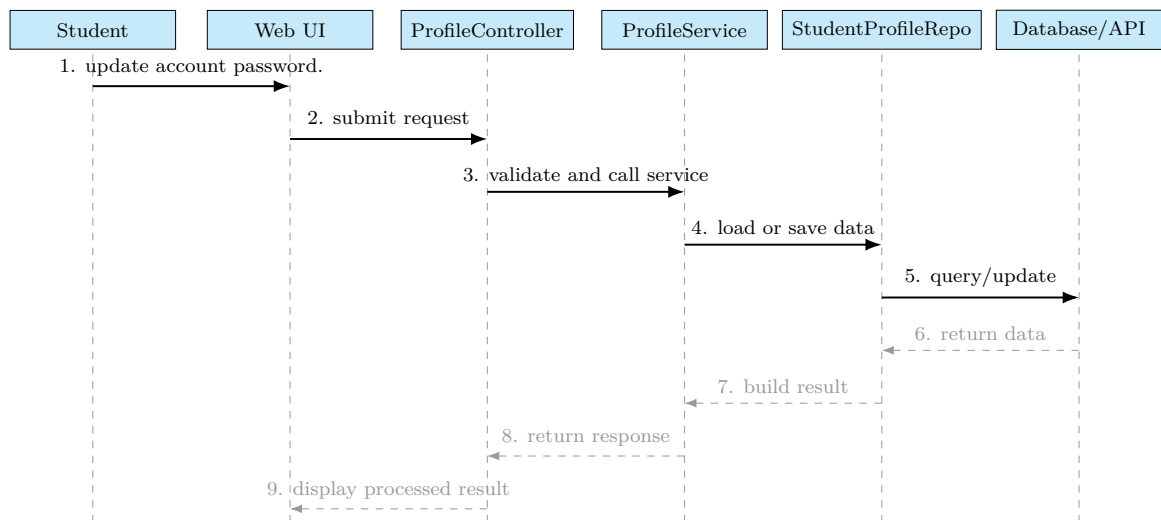


Figure IV.22: Sequence diagram for UC09 Change Password

3.3.5 UC10 Upload Transcript

3.3.5.1 Requirement Traceability

- **Use case ID:** UC10.
- **Use case name:** Upload Transcript.
- **Actor:** Student.
- **Requirement description:** Upload academic transcript.
- **Related table(s):** StudentProfiles.

3.3.5.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.3.5.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.3.5.4 Main Processing Flow

1. The actor opens `UploadTranscriptPage` and performs the action: upload academic transcript..
2. The frontend validates required fields, prepares the request DTO, and sends it to `ProfileController`.
3. `ProfileController` checks authentication, authorization, request format, and path parameters.
4. `ProfileService` loads related data through `StudentProfileRepo` and applies business rules for Upload Transcript.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in `StudentProfiles` and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.3.5.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.3.5.6 Participating Components

- **Boundary/UI – UploadTranscriptPage:** collects input and displays the final result for Upload Transcript.
- **Controller – ProfileController:** receives REST requests and validates request DTOs.
- **Service – ProfileService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – StudentProfile:** represents the main persisted domain object.
- **Repository – StudentProfileRepo:** provides data access operations for the related table(s).

3.3.5.7 Class Diagram

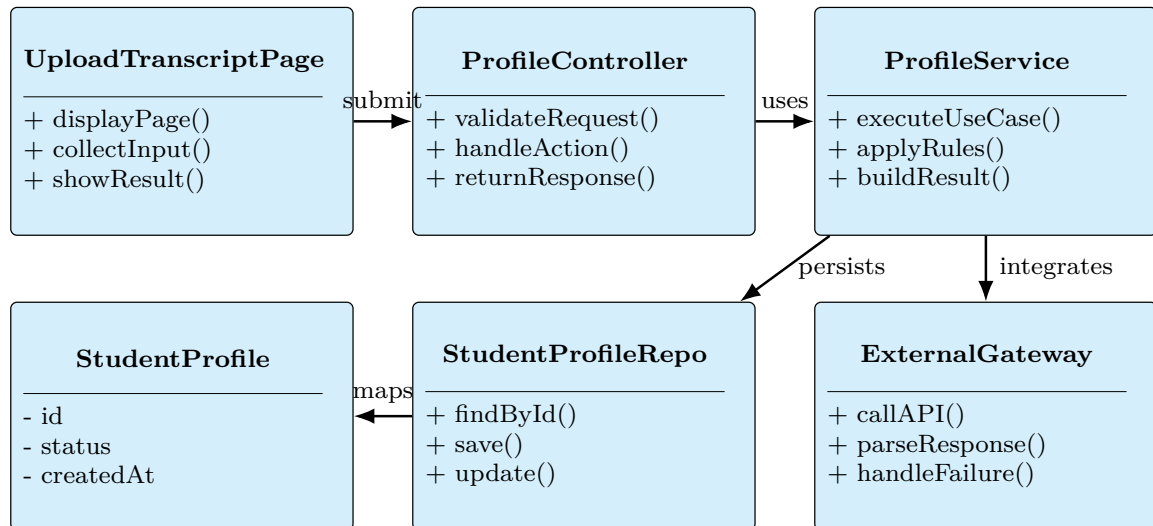


Figure IV.23: Class diagram for UC10 Upload Transcript

3.3.5.8 Sequence Diagram

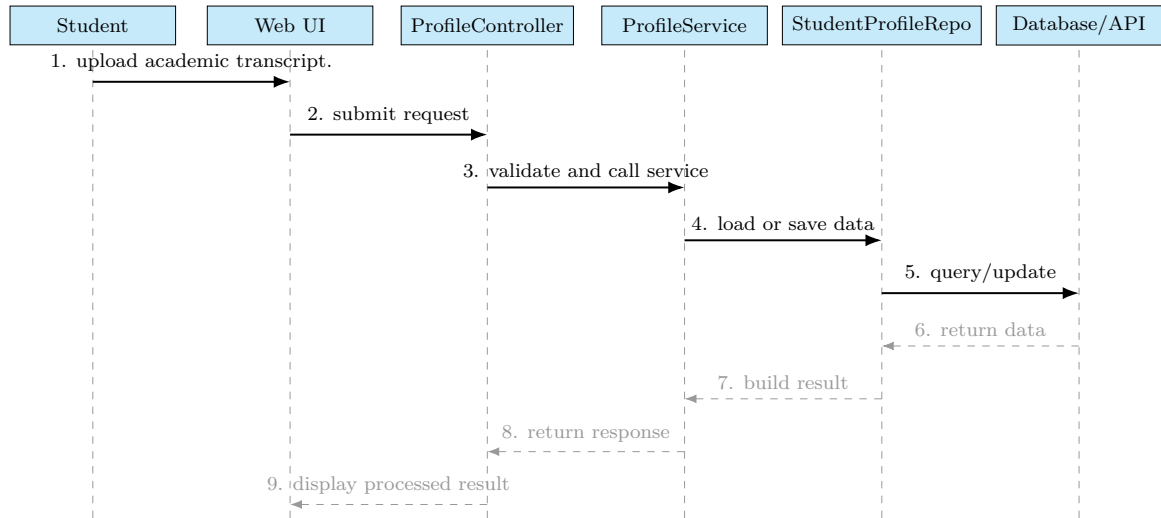


Figure IV.24: Sequence diagram for UC10 Upload Transcript

3.3.6 UC11 Upload CV

3.3.6.1 Requirement Traceability

- **Use case ID:** UC11.
- **Use case name:** Upload CV.
- **Actor:** Student.
- **Requirement description:** Upload curriculum vitae.
- **Related table(s):** StudentProfiles.

3.3.6.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.3.6.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.3.6.4 Main Processing Flow

1. The actor opens `UploadCVPPage` and performs the action: upload curriculum vitae..

2. The frontend validates required fields, prepares the request DTO, and sends it to **ProfileController**.
3. **ProfileController** checks authentication, authorization, request format, and path parameters.
4. **ProfileService** loads related data through **StudentProfileRepo** and applies business rules for Upload CV.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in **StudentProfiles** and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.3.6.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.3.6.6 Participating Components

- **Boundary/UI – UploadCVPage:** collects input and displays the final result for Upload CV.
- **Controller – ProfileController:** receives REST requests and validates request DTOs.
- **Service – ProfileService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – StudentProfile:** represents the main persisted domain object.
- **Repository – StudentProfileRepo:** provides data access operations for the related table(s).

3.3.6.7 Class Diagram

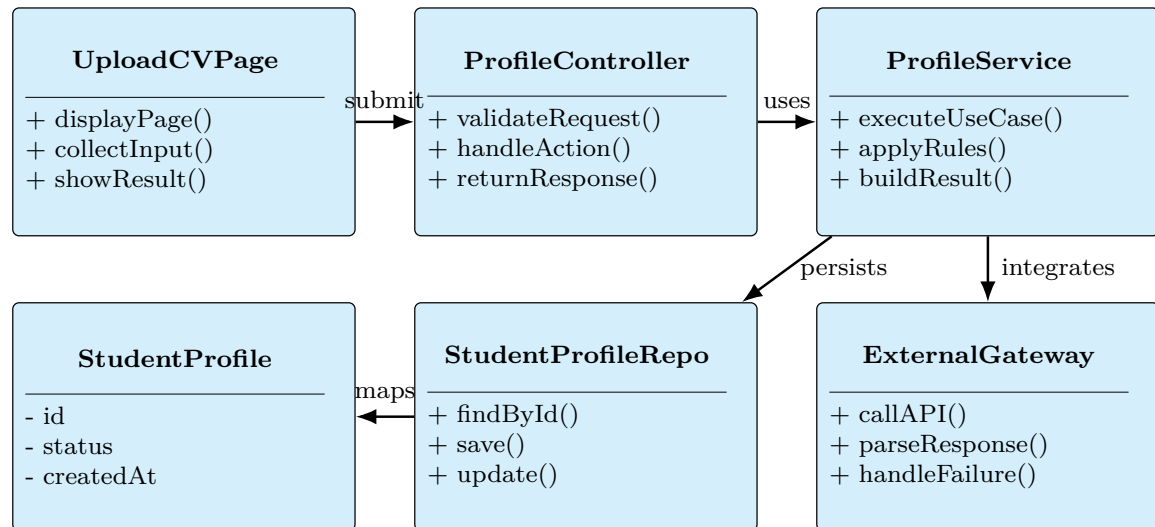


Figure IV.25: Class diagram for UC11 Upload CV

3.3.6.8 Sequence Diagram

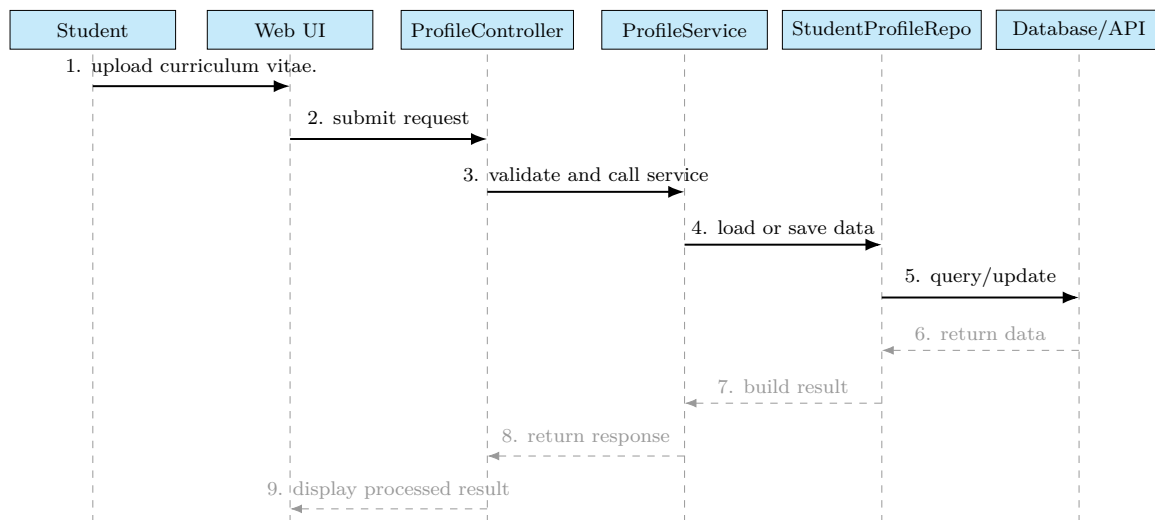


Figure IV.26: Sequence diagram for UC11 Upload CV

3.3.7 UC12 Link GitHub

3.3.7.1 Requirement Traceability

- **Use case ID:** UC12.
- **Use case name:** Link GitHub.
- **Actor:** Student.
- **Requirement description:** Connect GitHub account.
- **Related table(s):** StudentProfiles, GitHubRepositories, GitHubRepoLanguages.

3.3.7.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.3.7.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.3.7.4 Main Processing Flow

1. The actor opens `LinkGitHubPage` and performs the action: connect GitHub account..
2. The frontend validates required fields, prepares the request DTO, and sends it to `GitHubController`.
3. `GitHubController` checks authentication, authorization, request format, and path parameters.
4. `GitHubSyncService` loads related data through `GitHubRepositoryRepo` and applies business rules for Link GitHub.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in `StudentProfiles`, `GitHubRepositories`, `GitHubRepoLanguages` and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.3.7.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.3.7.6 Participating Components

- **Boundary/UI – LinkGitHubPage:** collects input and displays the final result for Link GitHub.
- **Controller – GitHubController:** receives REST requests and validates request DTOs.
- **Service – GitHubSyncService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – GitHubRepository:** represents the main persisted domain object.
- **Repository – GitHubRepositoryRepo:** provides data access operations for the related table(s).

3.3.7.7 Class Diagram

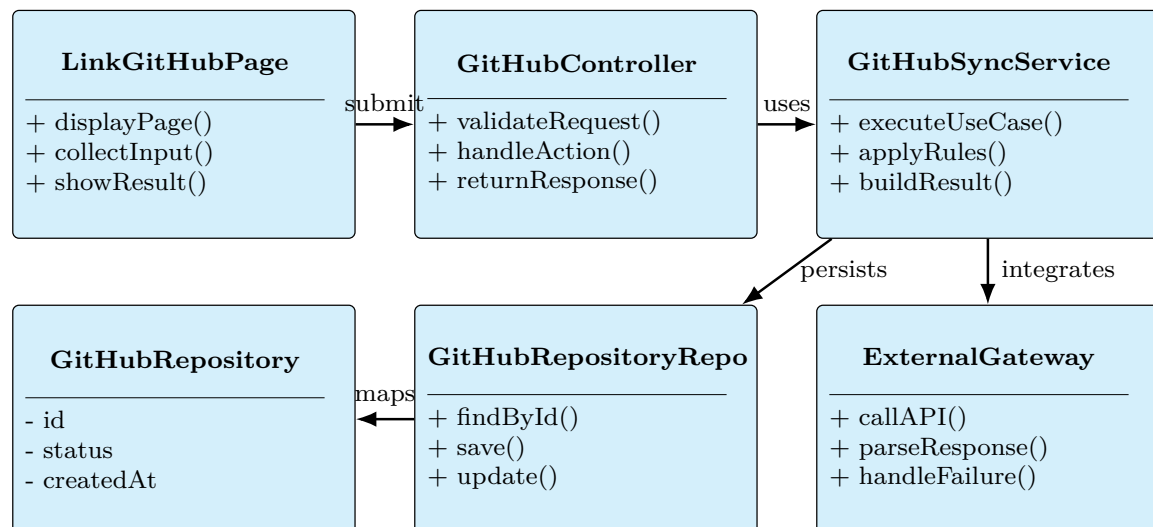


Figure IV.27: Class diagram for UC12 Link GitHub

3.3.7.8 Sequence Diagram

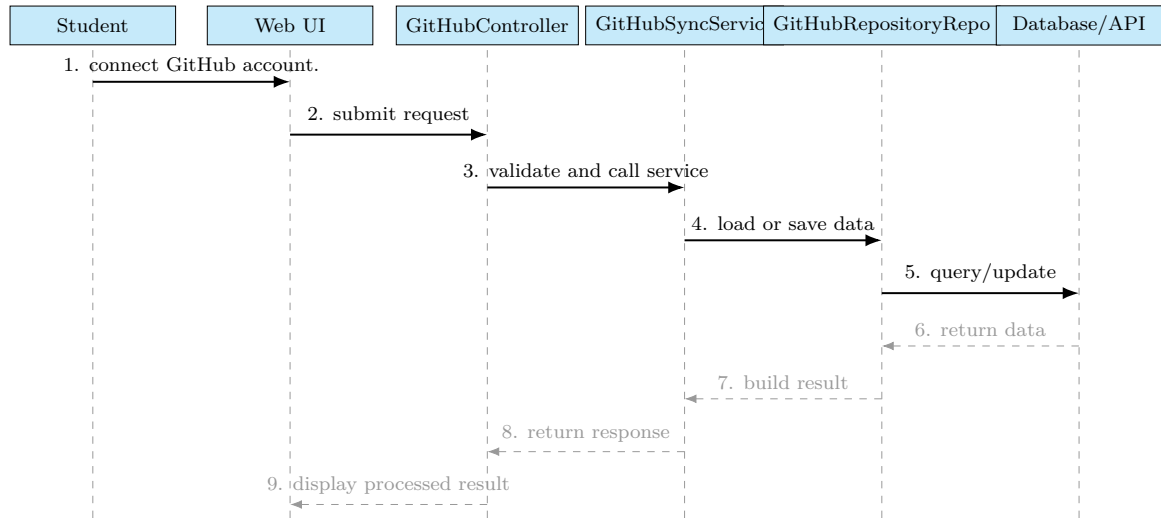


Figure IV.28: Sequence diagram for UC12 Link GitHub

3.3.8 UC13 Sync GitHub Repository

3.3.8.1 Requirement Traceability

- **Use case ID:** UC13.
- **Use case name:** Sync GitHub Repository.
- **Actor:** Student.
- **Requirement description:** Synchronize repository information.
- **Related table(s):** StudentProfiles, GitHubRepositories, GitHubRepoLanguages.

3.3.8.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.3.8.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.3.8.4 Main Processing Flow

1. The actor opens `SyncGitHubRepositoryPage` and performs the action: synchronize repository information..

2. The frontend validates required fields, prepares the request DTO, and sends it to `GitHubController`.
3. `GitHubController` checks authentication, authorization, request format, and path parameters.
4. `GitHubSyncService` loads related data through `GitHubRepositoryRepo` and applies business rules for Sync GitHub Repository.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in `StudentProfiles`, `GitHubRepositories`, `GitHubRepoLanguages` and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.3.8.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.3.8.6 Participating Components

- **Boundary/UI – `SyncGitHubRepositoryPage`:** collects input and displays the final result for Sync GitHub Repository.
- **Controller – `GitHubController`:** receives REST requests and validates request DTOs.
- **Service – `GitHubSyncService`:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – `GitHubRepository`:** represents the main persisted domain object.
- **Repository – `GitHubRepositoryRepo`:** provides data access operations for the related table(s).

3.3.8.7 Class Diagram

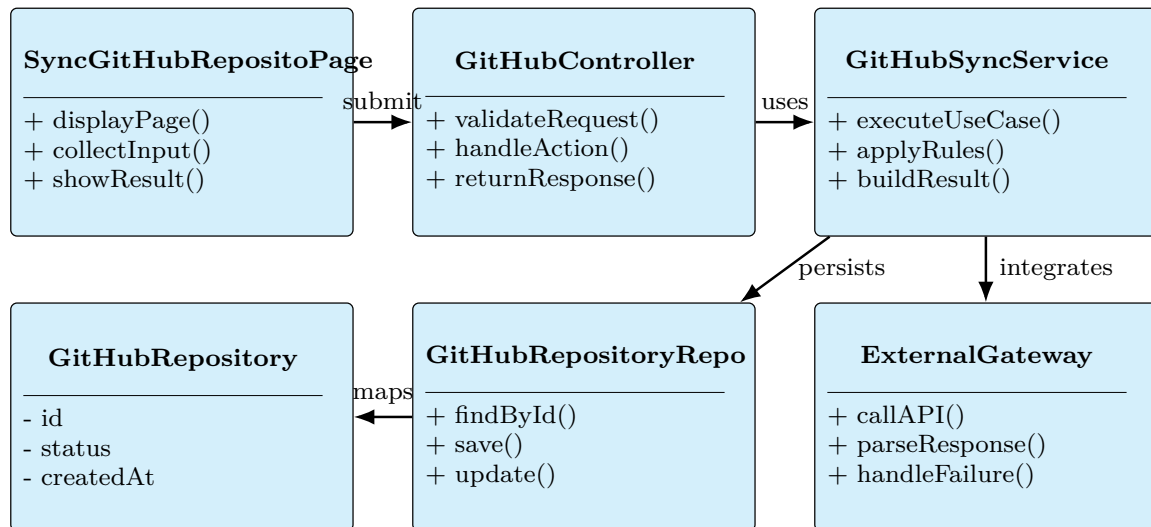


Figure IV.29: Class diagram for UC13 Sync GitHub Repository

3.3.8.8 Sequence Diagram

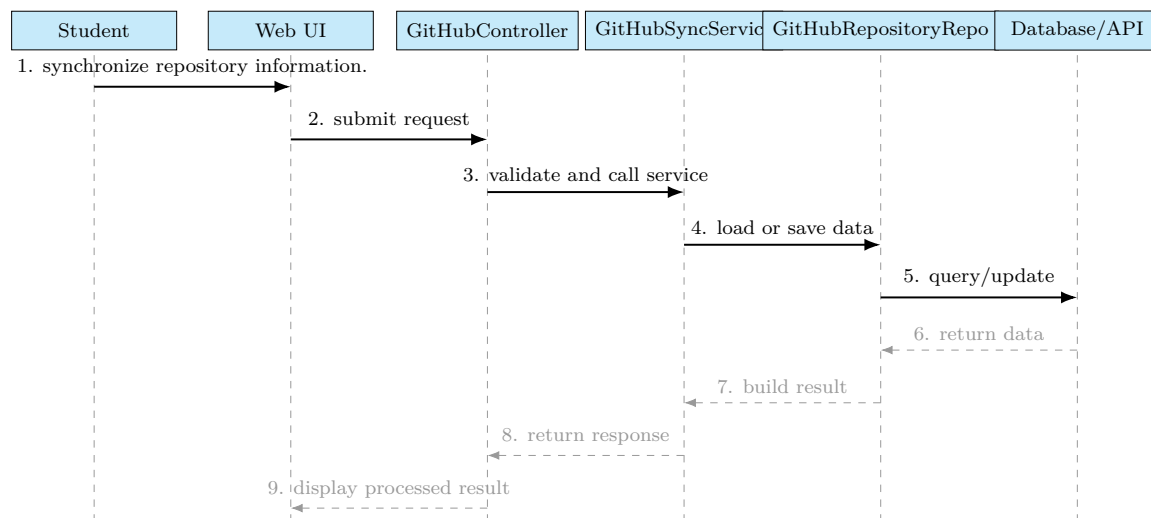


Figure IV.30: Sequence diagram for UC13 Sync GitHub Repository

3.3.9 UC14 View GitHub Analysis

3.3.9.1 Requirement Traceability

- **Use case ID:** UC14.
- **Use case name:** View GitHub Analysis.
- **Actor:** Student.
- **Requirement description:** View programming skill analysis.
- **Related table(s):** StudentProfiles, GitHubRepositories, GitHubRepoLanguages.

3.3.9.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.3.9.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.3.9.4 Main Processing Flow

1. The actor opens `ViewGitHubAnalysisPage` and performs the action: view programming skill analysis..
2. The frontend validates required fields, prepares the request DTO, and sends it to `GitHubController`.
3. `GitHubController` checks authentication, authorization, request format, and path parameters.
4. `GitHubSyncService` loads related data through `GitHubRepositoryRepo` and applies business rules for View GitHub Analysis.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in `StudentProfiles`, `GitHubRepositories`, `GitHubRepoLanguages` and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.3.9.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.3.9.6 Participating Components

- **Boundary/UI – ViewGitHubAnalysisPage:** collects input and displays the final result for View GitHub Analysis.
- **Controller – GitHubController:** receives REST requests and validates request DTOs.
- **Service – GitHubSyncService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – GitHubRepository:** represents the main persisted domain object.
- **Repository – GitHubRepositoryRepo:** provides data access operations for the related table(s).

3.3.9.7 Class Diagram

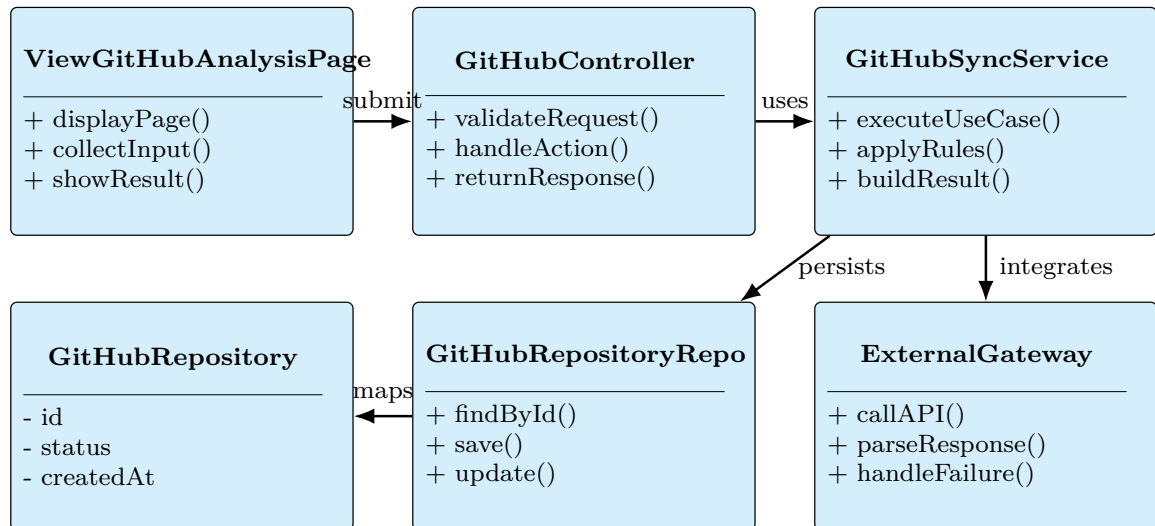


Figure IV.31: Class diagram for UC14 View GitHub Analysis

3.3.9.8 Sequence Diagram

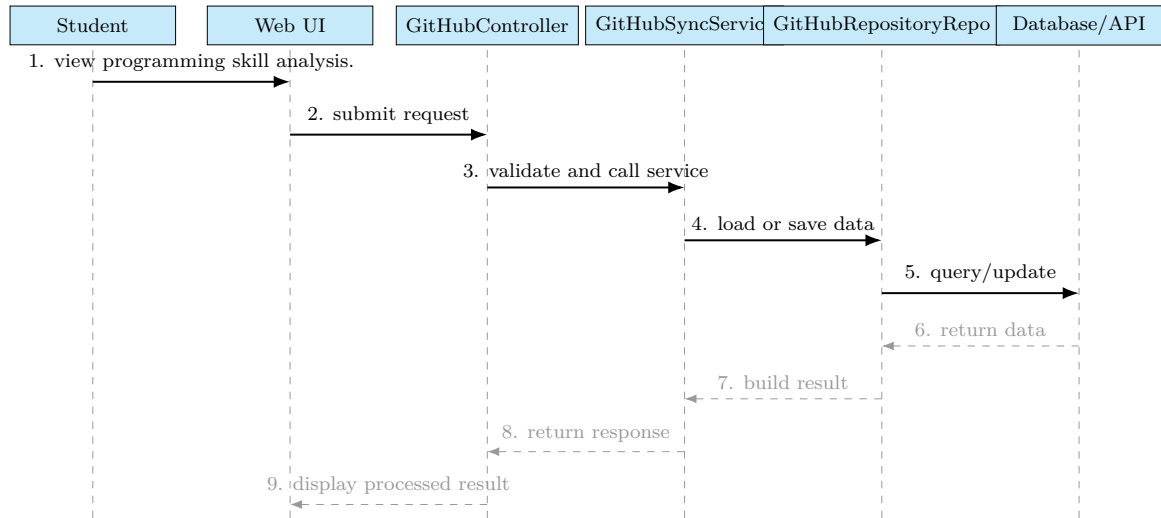


Figure IV.32: Sequence diagram for UC14 View GitHub Analysis

3.3.10 UC15 View Skill Gap Report

3.3.10.1 Requirement Traceability

- **Use case ID:** UC15.
- **Use case name:** View Skill Gap Report.
- **Actor:** Student.
- **Requirement description:** View missing skills analysis.
- **Related table(s):** StudentProfiles.

3.3.10.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.3.10.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.3.10.4 Main Processing Flow

1. The actor opens `ViewSkillGapReportPage` and performs the action: view missing skills analysis..

2. The frontend validates required fields, prepares the request DTO, and sends it to **ProfileController**.
3. **ProfileController** checks authentication, authorization, request format, and path parameters.
4. **ProfileService** loads related data through **StudentProfileRepo** and applies business rules for View Skill Gap Report.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in **StudentProfiles** and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.3.10.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.3.10.6 Participating Components

- **Boundary/UI – ViewSkillGapReportPage:** collects input and displays the final result for View Skill Gap Report.
- **Controller – ProfileController:** receives REST requests and validates request DTOs.
- **Service – ProfileService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – StudentProfile:** represents the main persisted domain object.
- **Repository – StudentProfileRepo:** provides data access operations for the related table(s).

3.3.10.7 Class Diagram

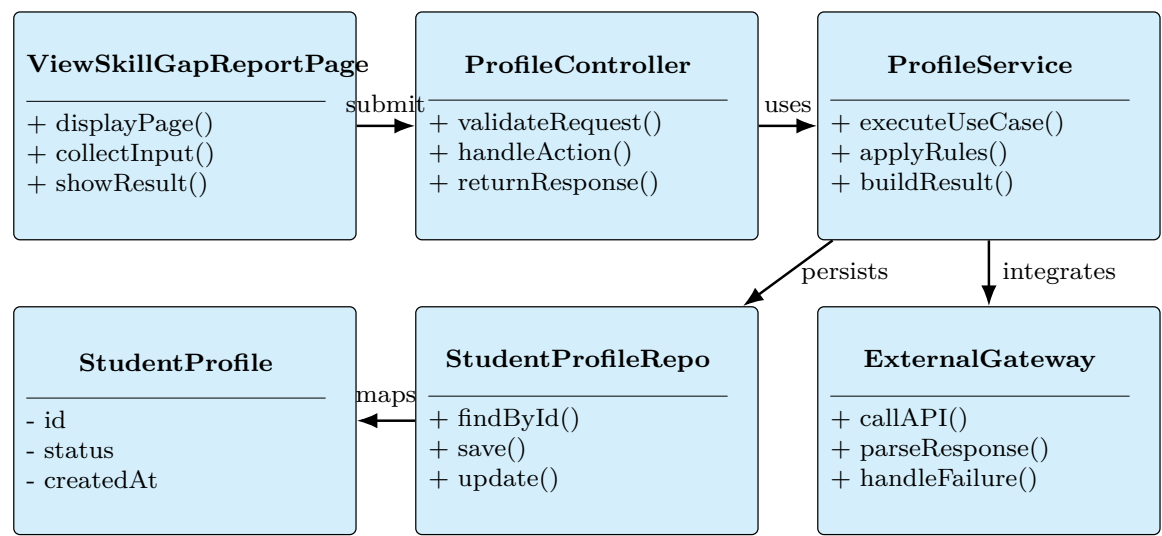


Figure IV.33: Class diagram for UC15 View Skill Gap Report

3.3.10.8 Sequence Diagram

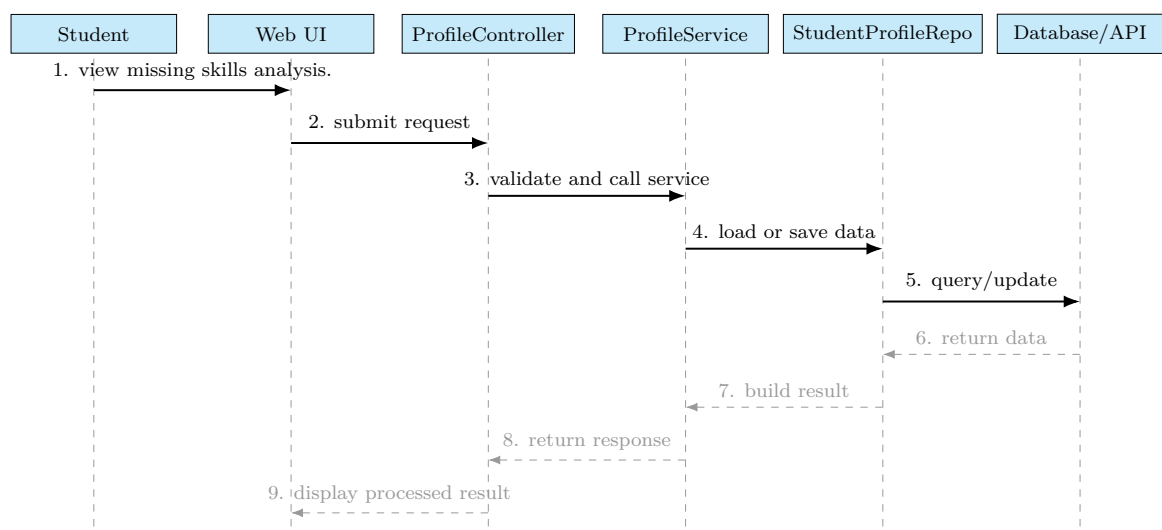


Figure IV.34: Sequence diagram for UC15 View Skill Gap Report

3.4 Career Path and Skill Management

This group contains use cases related to career path and skill management. The design uses the layered architecture introduced in the system design: React boundary pages, Spring Boot controllers, services, repositories, database entities, and external gateways when required.

3.4.1 UC16 Select Career Role

3.4.1.1 Requirement Traceability

- **Use case ID:** UC16.
- **Use case name:** Select Career Role.
- **Actor:** Student.
- **Requirement description:** Select desired career path.
- **Related table(s):** CareerPaths, SkillNodes, CareerPathSkills.

3.4.1.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.4.1.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.4.1.4 Main Processing Flow

1. The actor opens `SelectCareerRolePage` and performs the action: select desired career path..
2. The frontend validates required fields, prepares the request DTO, and sends it to `CareerPathController`.
3. `CareerPathController` checks authentication, authorization, request format, and path parameters.
4. `CareerPathService` loads related data through `CareerPathRepository` and applies business rules for Select Career Role.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in `CareerPaths`, `SkillNodes`, `CareerPathSkills` and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.4.1.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.4.1.6 Participating Components

- **Boundary/UI – SelectCareerRolePage:** collects input and displays the final result for Select Career Role.
- **Controller – CareerPathController:** receives REST requests and validates request DTOs.
- **Service – CareerPathService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – CareerPath:** represents the main persisted domain object.
- **Repository – CareerPathRepository:** provides data access operations for the related table(s).

3.4.1.7 Class Diagram

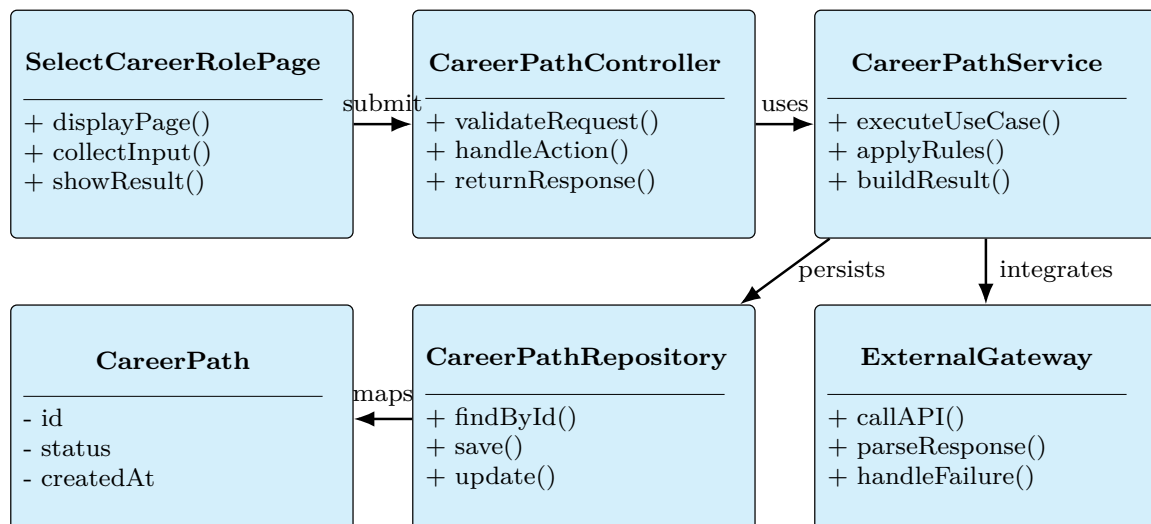


Figure IV.35: Class diagram for UC16 Select Career Role

3.4.1.8 Sequence Diagram

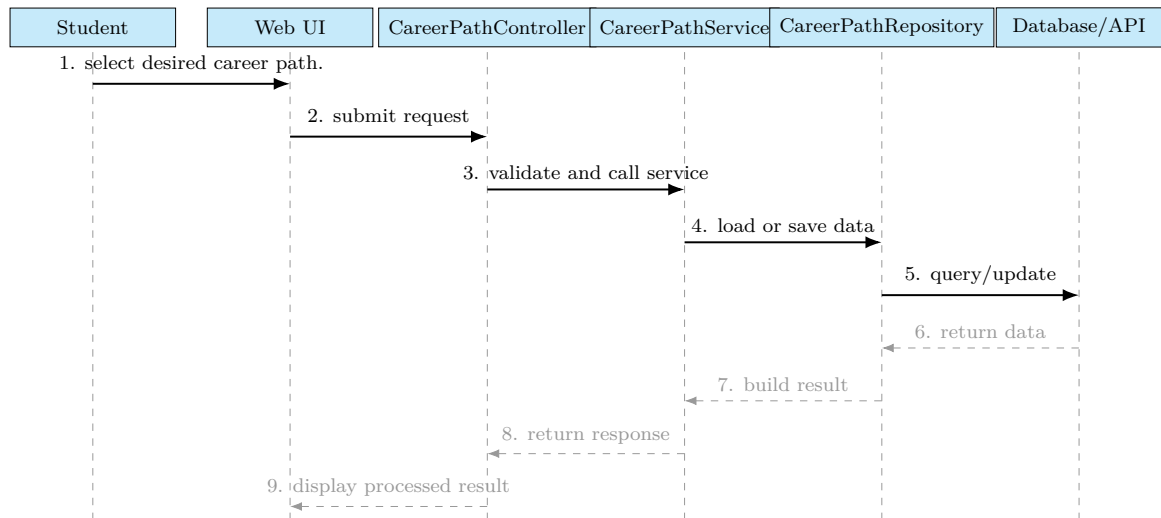


Figure IV.36: Sequence diagram for UC16 Select Career Role

3.4.2 UC17 Update Career Goal

3.4.2.1 Requirement Traceability

- **Use case ID:** UC17.
- **Use case name:** Update Career Goal.
- **Actor:** Student.
- **Requirement description:** Modify career objective.
- **Related table(s):** CareerPaths, SkillNodes, CareerPathSkills.

3.4.2.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.4.2.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.4.2.4 Main Processing Flow

1. The actor opens `UpdateCareerGoalPage` and performs the action: modify career objective..

2. The frontend validates required fields, prepares the request DTO, and sends it to **CareerPathController**.
3. **CareerPathController** checks authentication, authorization, request format, and path parameters.
4. **CareerPathService** loads related data through **CareerPathRepository** and applies business rules for Update Career Goal.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in **CareerPaths**, **SkillNodes**, **CareerPathSkills** and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.4.2.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.4.2.6 Participating Components

- **Boundary/UI – UpdateCareerGoalPage:** collects input and displays the final result for Update Career Goal.
- **Controller – CareerPathController:** receives REST requests and validates request DTOs.
- **Service – CareerPathService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – CareerPath:** represents the main persisted domain object.
- **Repository – CareerPathRepository:** provides data access operations for the related table(s).

3.4.2.7 Class Diagram

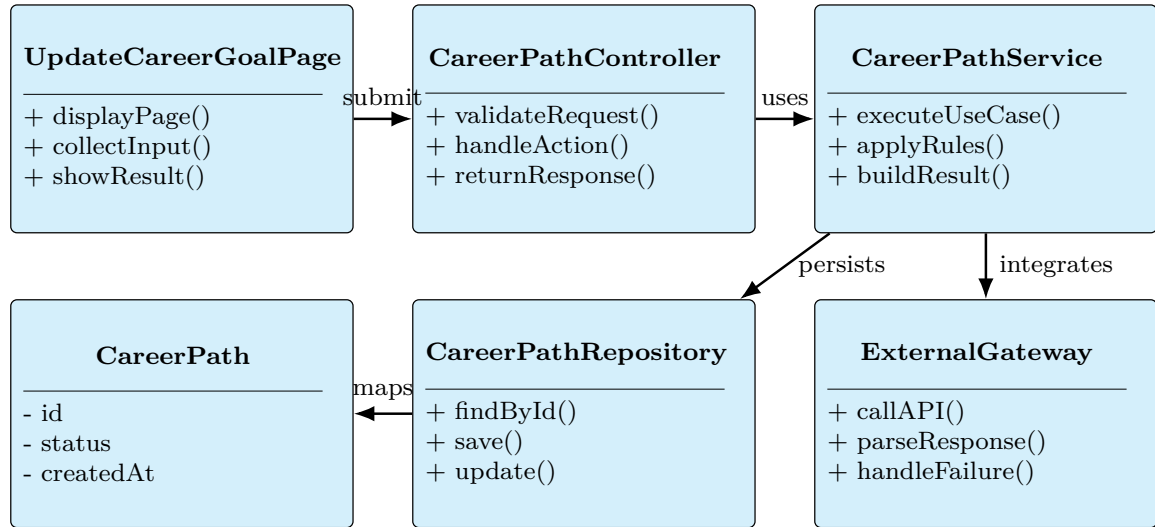


Figure IV.37: Class diagram for UC17 Update Career Goal

3.4.2.8 Sequence Diagram

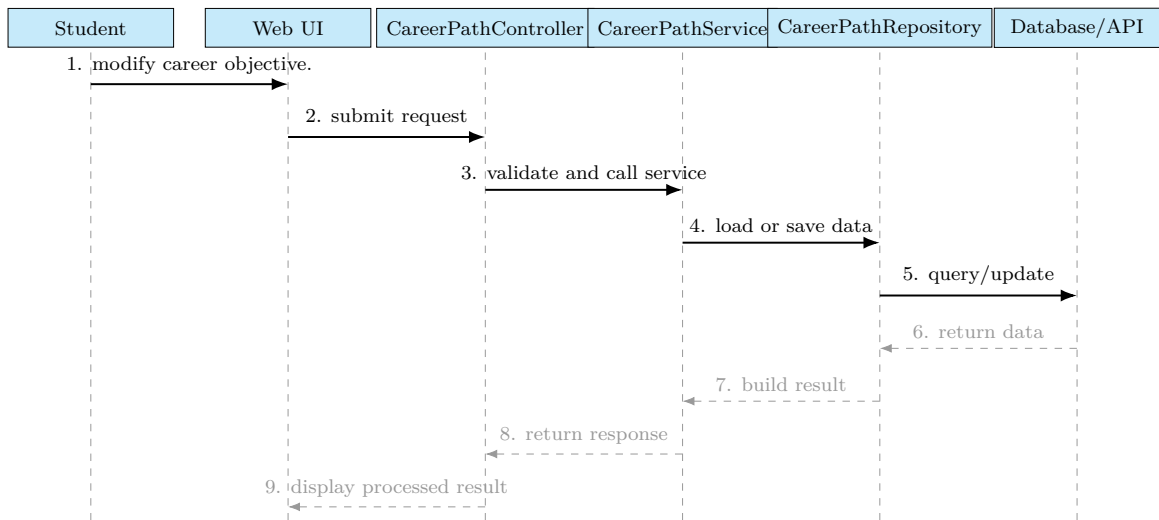


Figure IV.38: Sequence diagram for UC17 Update Career Goal

3.4.3 UC18 View Recommended Roles

3.4.3.1 Requirement Traceability

- **Use case ID:** UC18.
- **Use case name:** View Recommended Roles.
- **Actor:** Student.
- **Requirement description:** View suggested careers.
- **Related table(s):** CareerPaths, SkillNodes, CareerPathSkills.

3.4.3.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.4.3.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.4.3.4 Main Processing Flow

1. The actor opens `ViewRecommendedRolPage` and performs the action: view suggested careers..
2. The frontend validates required fields, prepares the request DTO, and sends it to `CareerPathController`.
3. `CareerPathController` checks authentication, authorization, request format, and path parameters.
4. `CareerPathService` loads related data through `CareerPathRepository` and applies business rules for View Recommended Roles.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in `CareerPaths`, `SkillNodes`, `CareerPathSkills` and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.4.3.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.4.3.6 Participating Components

- **Boundary/UI – ViewRecommendedRolPage:** collects input and displays the final result for View Recommended Roles.
- **Controller – CareerPathController:** receives REST requests and validates request DTOs.
- **Service – CareerPathService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – CareerPath:** represents the main persisted domain object.
- **Repository – CareerPathRepository:** provides data access operations for the related table(s).

3.4.3.7 Class Diagram

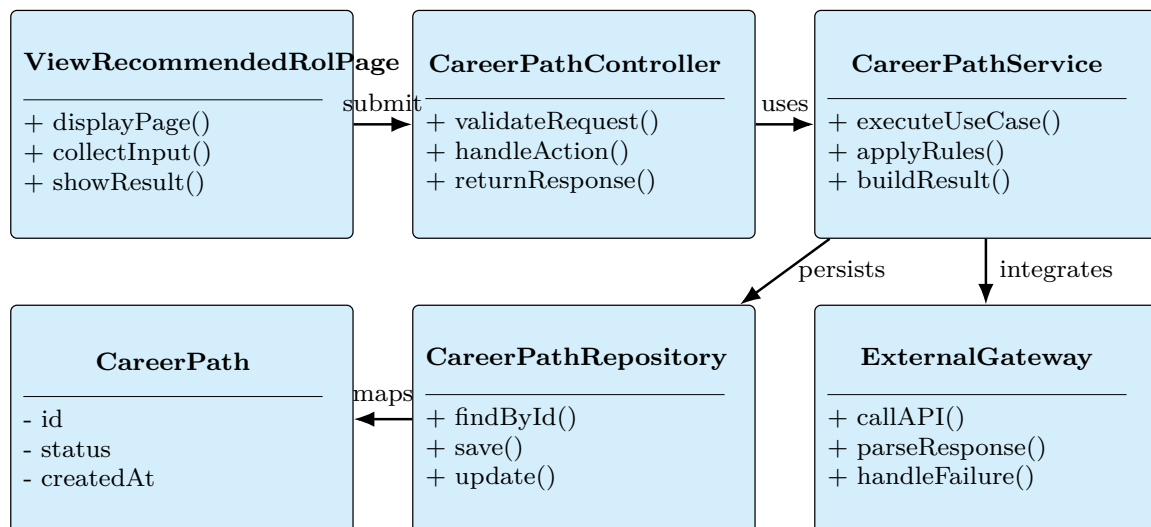


Figure IV.39: Class diagram for UC18 View Recommended Roles

3.4.3.8 Sequence Diagram

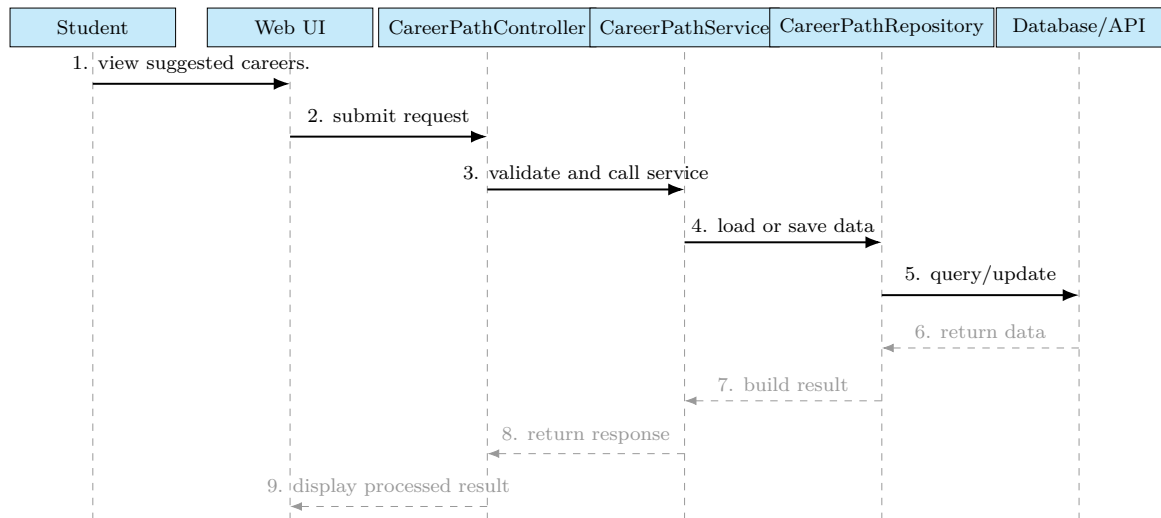


Figure IV.40: Sequence diagram for UC18 View Recommended Roles

3.4.4 UC19 Compare Career Paths

3.4.4.1 Requirement Traceability

- **Use case ID:** UC19.
- **Use case name:** Compare Career Paths.
- **Actor:** Student.
- **Requirement description:** Compare different career options.
- **Related table(s):** CareerPaths, SkillNodes, CareerPathSkills.

3.4.4.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.4.4.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.4.4.4 Main Processing Flow

1. The actor opens `CompareCareerPathsPage` and performs the action: compare different career options..

2. The frontend validates required fields, prepares the request DTO, and sends it to **CareerPathController**.
3. **CareerPathController** checks authentication, authorization, request format, and path parameters.
4. **CareerPathService** loads related data through **CareerPathRepository** and applies business rules for Compare Career Paths.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in **CareerPaths**, **SkillNodes**, **CareerPathSkills** and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.4.4.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.4.4.6 Participating Components

- **Boundary/UI – CompareCareerPathsPage:** collects input and displays the final result for Compare Career Paths.
- **Controller – CareerPathController:** receives REST requests and validates request DTOs.
- **Service – CareerPathService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – CareerPath:** represents the main persisted domain object.
- **Repository – CareerPathRepository:** provides data access operations for the related table(s).

3.4.4.7 Class Diagram

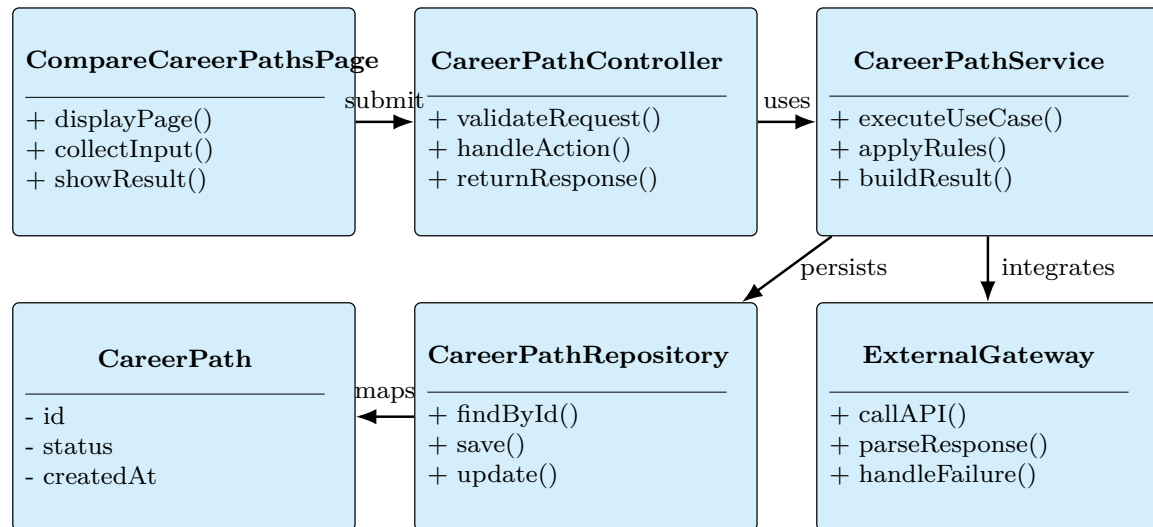


Figure IV.41: Class diagram for UC19 Compare Career Paths

3.4.4.8 Sequence Diagram

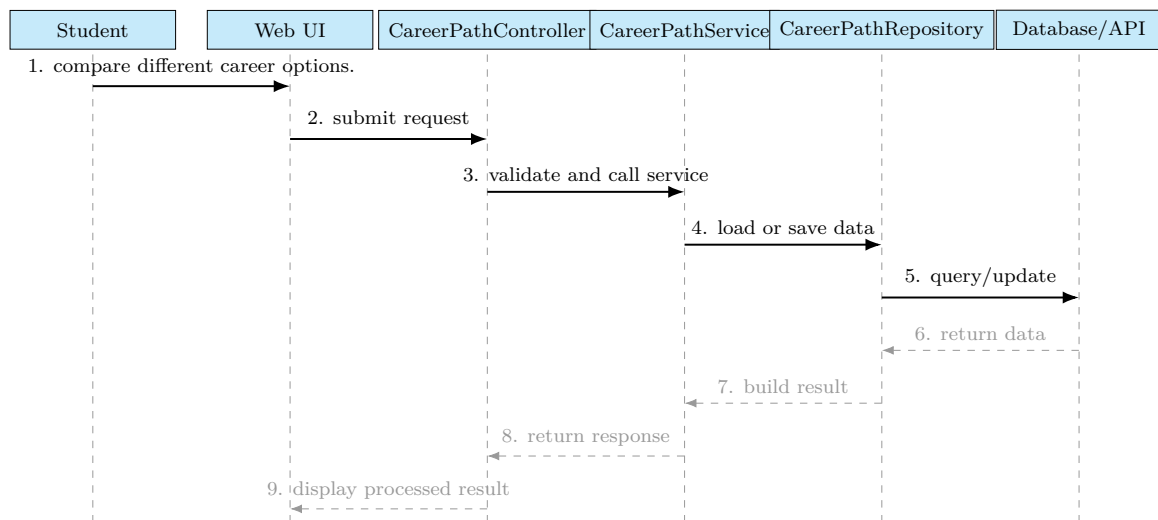


Figure IV.42: Sequence diagram for UC19 Compare Career Paths

3.4.5 UC20 View Required Skills

3.4.5.1 Requirement Traceability

- **Use case ID:** UC20.
- **Use case name:** View Required Skills.
- **Actor:** Student.
- **Requirement description:** View required competencies.
- **Related table(s):** CareerPaths, SkillNodes, CareerPathSkills.

3.4.5.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.4.5.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.4.5.4 Main Processing Flow

1. The actor opens `ViewRequiredSkillsPage` and performs the action: view required competencies..
2. The frontend validates required fields, prepares the request DTO, and sends it to `CareerPathController`.
3. `CareerPathController` checks authentication, authorization, request format, and path parameters.
4. `CareerPathService` loads related data through `CareerPathRepository` and applies business rules for View Required Skills.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in `CareerPaths`, `SkillNodes`, `CareerPathSkills` and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.4.5.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.4.5.6 Participating Components

- **Boundary/UI – ViewRequiredSkillsPage:** collects input and displays the final result for View Required Skills.
- **Controller – CareerPathController:** receives REST requests and validates request DTOs.
- **Service – CareerPathService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – CareerPath:** represents the main persisted domain object.
- **Repository – CareerPathRepository:** provides data access operations for the related table(s).

3.4.5.7 Class Diagram

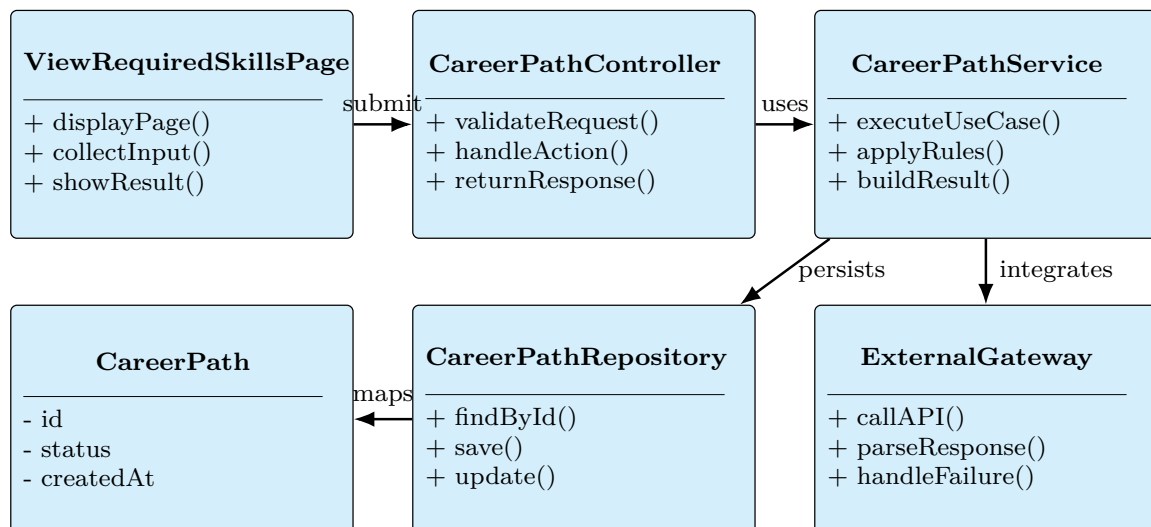


Figure IV.43: Class diagram for UC20 View Required Skills

3.4.5.8 Sequence Diagram

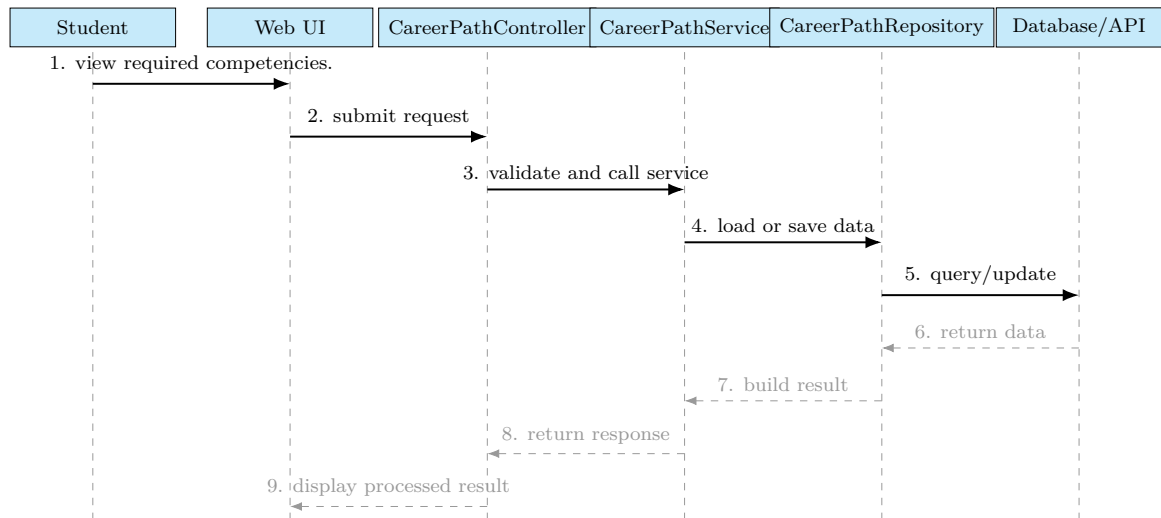


Figure IV.44: Sequence diagram for UC20 View Required Skills

3.5 Learning Roadmap and Course Management

This group contains use cases related to learning roadmap and course management. The design uses the layered architecture introduced in the system design: React boundary pages, Spring Boot controllers, services, repositories, database entities, and external gateways when required.

3.5.1 UC21 Generate Learning Roadmap

3.5.1.1 Requirement Traceability

- **Use case ID:** UC21.
- **Use case name:** Generate Learning Roadmap.
- **Actor:** Student.
- **Requirement description:** Generate personalized roadmap.
- **Related table(s):** Roadmaps, RoadmapProgress, CareerPathSkills.

3.5.1.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.5.1.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.5.1.4 Main Processing Flow

1. The actor opens **GenerateLearningRoPage** and performs the action: generate personalized roadmap..
2. The frontend validates required fields, prepares the request DTO, and sends it to **RoadmapController**.
3. **RoadmapController** checks authentication, authorization, request format, and path parameters.
4. **RoadmapService** loads related data through **RoadmapRepository** and applies business rules for Generate Learning Roadmap.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in **Roadmaps**, **RoadmapProgress**, **CareerPathSkills** and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.5.1.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.5.1.6 Participating Components

- **Boundary/UI – GenerateLearningRoPage:** collects input and displays the final result for Generate Learning Roadmap.
- **Controller – RoadmapController:** receives REST requests and validates request DTOs.
- **Service – RoadmapService:** coordinates business logic, integration calls, and transaction boundaries.

- **Entity – Roadmap:** represents the main persisted domain object.
- **Repository – RoadmapRepository:** provides data access operations for the related table(s).

3.5.1.7 Class Diagram

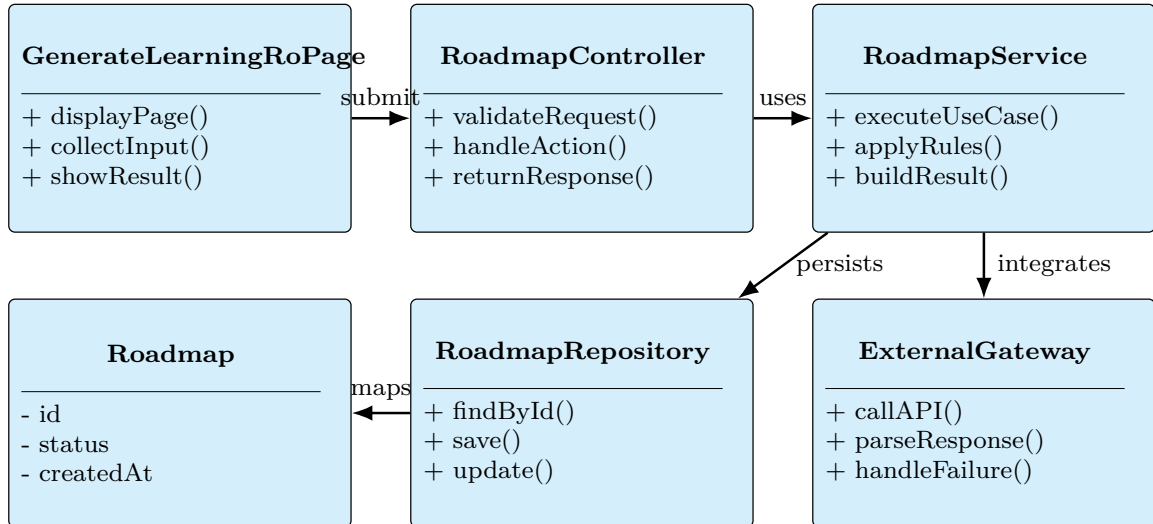


Figure IV.45: Class diagram for UC21 Generate Learning Roadmap

3.5.1.8 Sequence Diagram

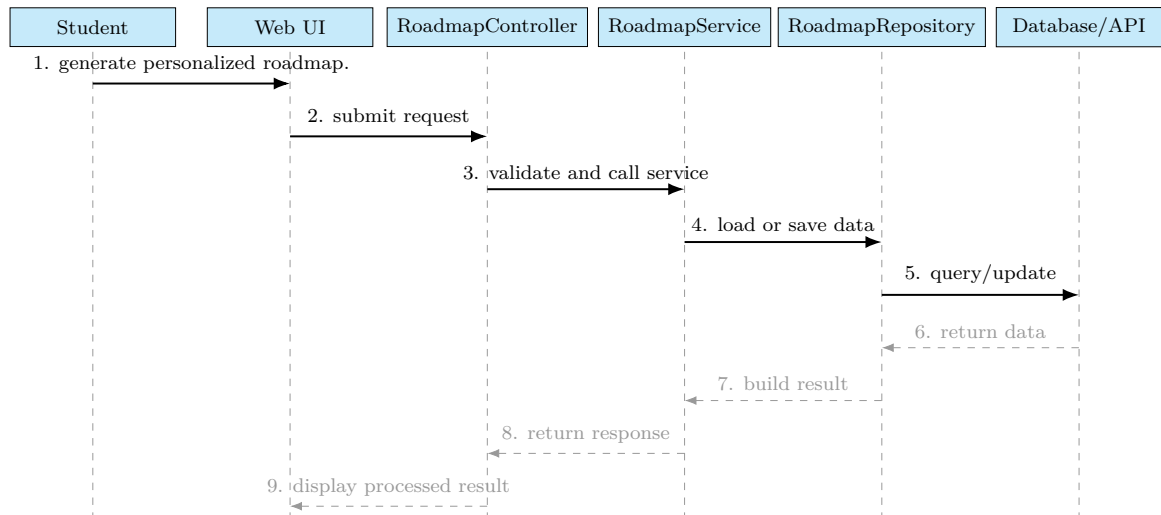


Figure IV.46: Sequence diagram for UC21 Generate Learning Roadmap

3.5.2 UC22 View Learning Roadmap

3.5.2.1 Requirement Traceability

- Use case ID: UC22.

- **Use case name:** View Learning Roadmap.
- **Actor:** Student.
- **Requirement description:** Display roadmap.
- **Related table(s):** Roadmaps, RoadmapProgress, CareerPathSkills.

3.5.2.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.5.2.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.5.2.4 Main Processing Flow

1. The actor opens `ViewLearningRoadmaPage` and performs the action: display roadmap..
2. The frontend validates required fields, prepares the request DTO, and sends it to `RoadmapController`.
3. `RoadmapController` checks authentication, authorization, request format, and path parameters.
4. `RoadmapService` loads related data through `RoadmapRepository` and applies business rules for View Learning Roadmap.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in `Roadmaps`, `RoadmapProgress`, `CareerPathSkills` and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.5.2.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.

- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.5.2.6 Participating Components

- **Boundary/UI – ViewLearningRoadmaPage:** collects input and displays the final result for View Learning Roadmap.
- **Controller – RoadmapController:** receives REST requests and validates request DTOs.
- **Service – RoadmapService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – Roadmap:** represents the main persisted domain object.
- **Repository – RoadmapRepository:** provides data access operations for the related table(s).

3.5.2.7 Class Diagram

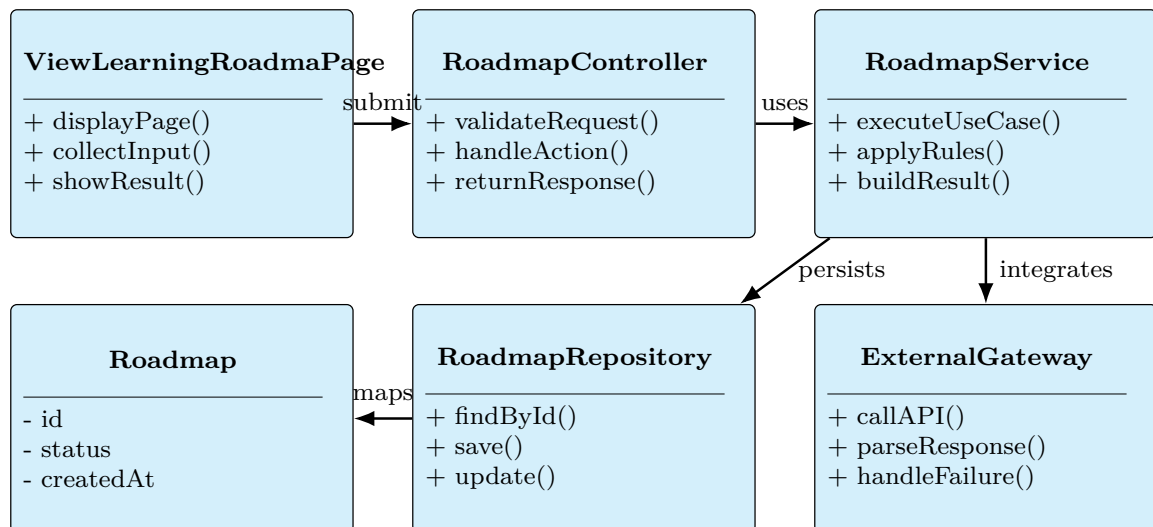


Figure IV.47: Class diagram for UC22 View Learning Roadmap

3.5.2.8 Sequence Diagram

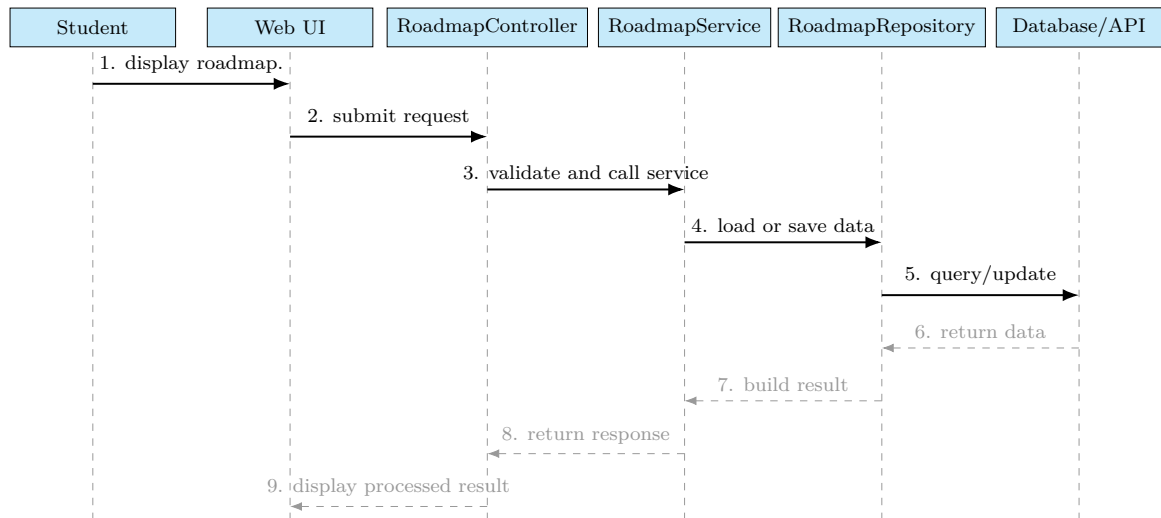


Figure IV.48: Sequence diagram for UC22 View Learning Roadmap

3.5.3 UC23 Update Roadmap Progress

3.5.3.1 Requirement Traceability

- **Use case ID:** UC23.
- **Use case name:** Update Roadmap Progress.
- **Actor:** Student.
- **Requirement description:** Update completed milestones.
- **Related table(s):** Roadmaps, RoadmapProgress, CareerPathSkills.

3.5.3.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.5.3.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.5.3.4 Main Processing Flow

1. The actor opens `UpdateRoadmapProgrPage` and performs the action: update completed milestones..

2. The frontend validates required fields, prepares the request DTO, and sends it to `RoadmapController`.
3. `RoadmapController` checks authentication, authorization, request format, and path parameters.
4. `RoadmapService` loads related data through `RoadmapRepository` and applies business rules for Update Roadmap Progress.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in `Roadmaps`, `RoadmapProgress`, `CareerPathSkills` and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.5.3.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.5.3.6 Participating Components

- **Boundary/UI – `UpdateRoadmapProgrPage`:** collects input and displays the final result for Update Roadmap Progress.
- **Controller – `RoadmapController`:** receives REST requests and validates request DTOs.
- **Service – `RoadmapService`:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – `Roadmap`:** represents the main persisted domain object.
- **Repository – `RoadmapRepository`:** provides data access operations for the related table(s).

3.5.3.7 Class Diagram

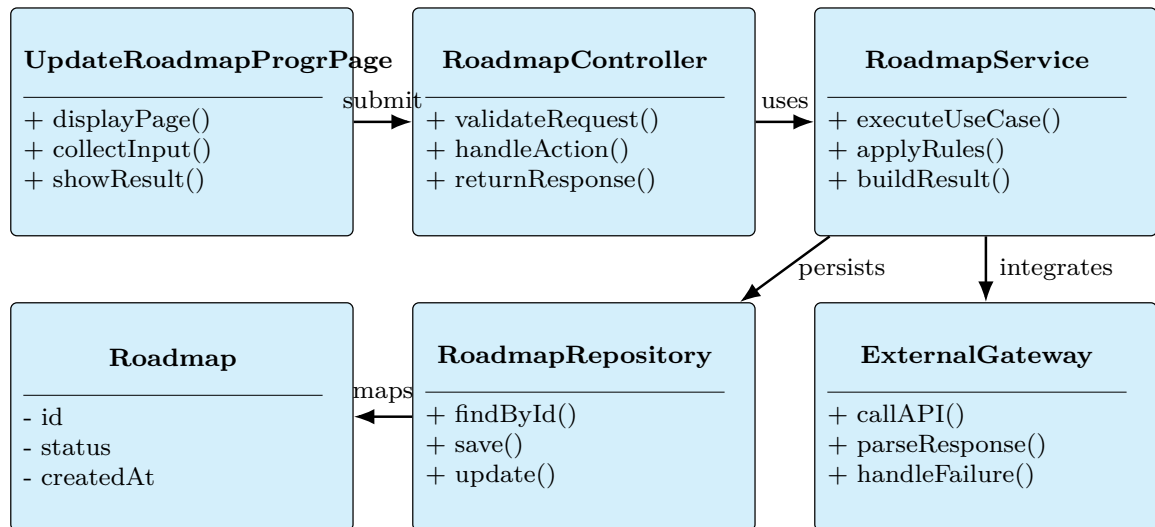


Figure IV.49: Class diagram for UC23 Update Roadmap Progress

3.5.3.8 Sequence Diagram

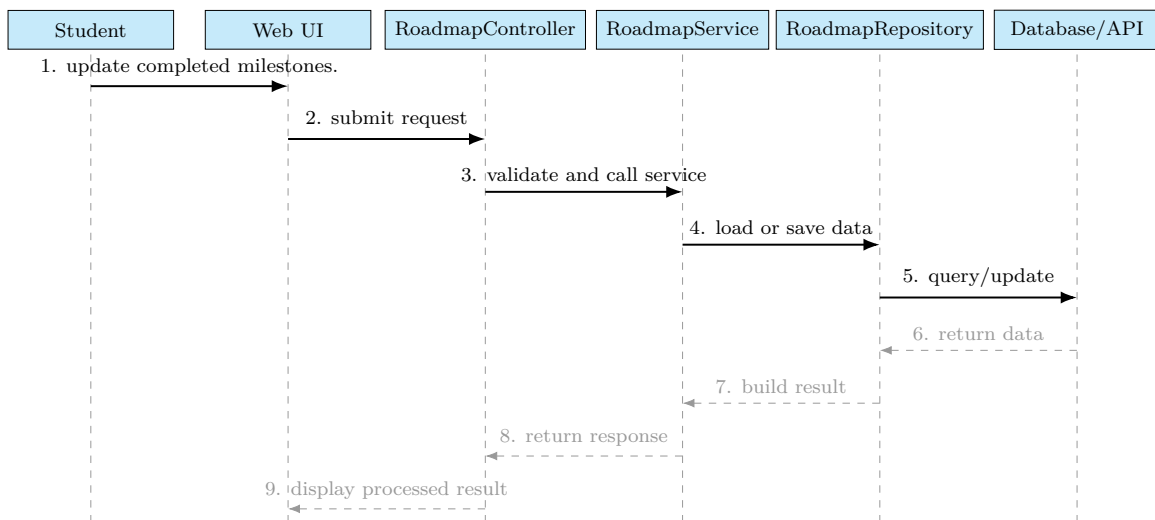


Figure IV.50: Sequence diagram for UC23 Update Roadmap Progress

3.5.4 UC24 View Learning Progress

3.5.4.1 Requirement Traceability

- **Use case ID:** UC24.
- **Use case name:** View Learning Progress.
- **Actor:** Student.
- **Requirement description:** Monitor learning status.
- **Related table(s):** Roadmaps, RoadmapProgress, CareerPathSkills.

3.5.4.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.5.4.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.5.4.4 Main Processing Flow

1. The actor opens `ViewLearningProgrePage` and performs the action: monitor learning status..
2. The frontend validates required fields, prepares the request DTO, and sends it to `RoadmapController`.
3. `RoadmapController` checks authentication, authorization, request format, and path parameters.
4. `RoadmapService` loads related data through `RoadmapRepository` and applies business rules for View Learning Progress.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in `Roadmaps`, `RoadmapProgress`, `CareerPathSkills` and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.5.4.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.5.4.6 Participating Components

- **Boundary/UI – ViewLearningProgrePage:** collects input and displays the final result for View Learning Progress.
- **Controller – RoadmapController:** receives REST requests and validates request DTOs.
- **Service – RoadmapService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – Roadmap:** represents the main persisted domain object.
- **Repository – RoadmapRepository:** provides data access operations for the related table(s).

3.5.4.7 Class Diagram

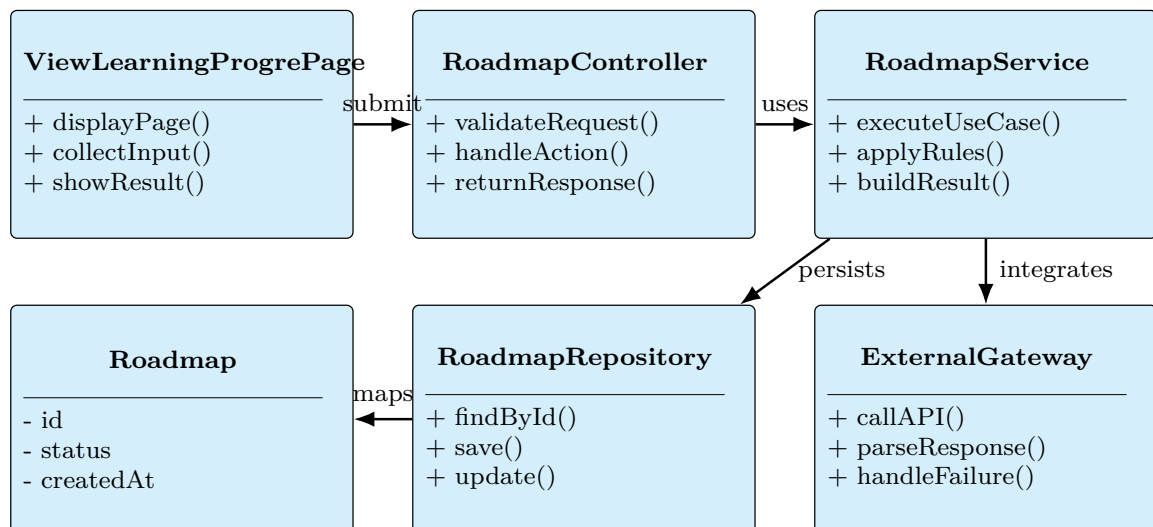


Figure IV.51: Class diagram for UC24 View Learning Progress

3.5.4.8 Sequence Diagram

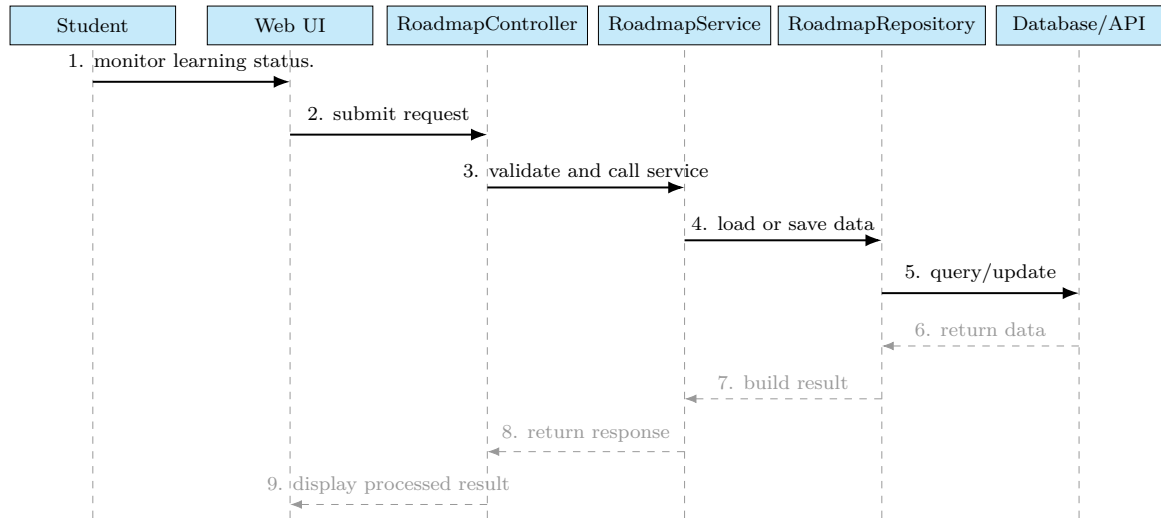


Figure IV.52: Sequence diagram for UC24 View Learning Progress

3.5.5 UC25 Receive Learning Reminder

3.5.5.1 Requirement Traceability

- **Use case ID:** UC25.
- **Use case name:** Receive Learning Reminder.
- **Actor:** Student.
- **Requirement description:** Receive notifications.
- **Related table(s):** Roadmaps, RoadmapProgress, CareerPathSkills.

3.5.5.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.5.5.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.5.5.4 Main Processing Flow

1. The actor opens `ReceiveLearningRemPage` and performs the action: receive notifications..

2. The frontend validates required fields, prepares the request DTO, and sends it to `RoadmapController`.
3. `RoadmapController` checks authentication, authorization, request format, and path parameters.
4. `RoadmapService` loads related data through `RoadmapRepository` and applies business rules for Receive Learning Reminder.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in `Roadmaps`, `RoadmapProgress`, `CareerPathSkills` and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.5.5.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.5.5.6 Participating Components

- **Boundary/UI – `ReceiveLearningRemPage`:** collects input and displays the final result for Receive Learning Reminder.
- **Controller – `RoadmapController`:** receives REST requests and validates request DTOs.
- **Service – `RoadmapService`:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – `Roadmap`:** represents the main persisted domain object.
- **Repository – `RoadmapRepository`:** provides data access operations for the related table(s).

3.5.5.7 Class Diagram

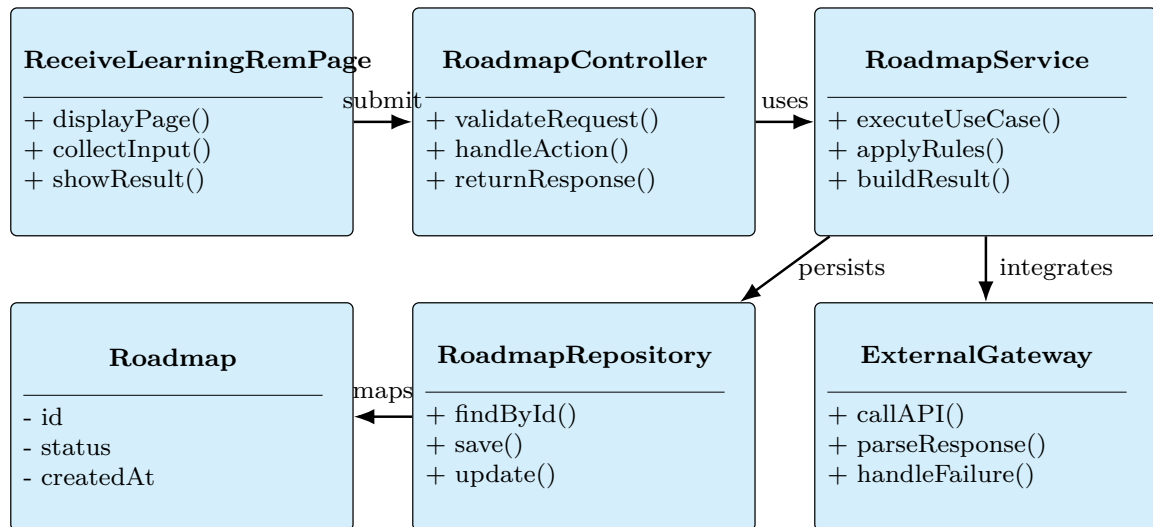


Figure IV.53: Class diagram for UC25 Receive Learning Reminder

3.5.5.8 Sequence Diagram

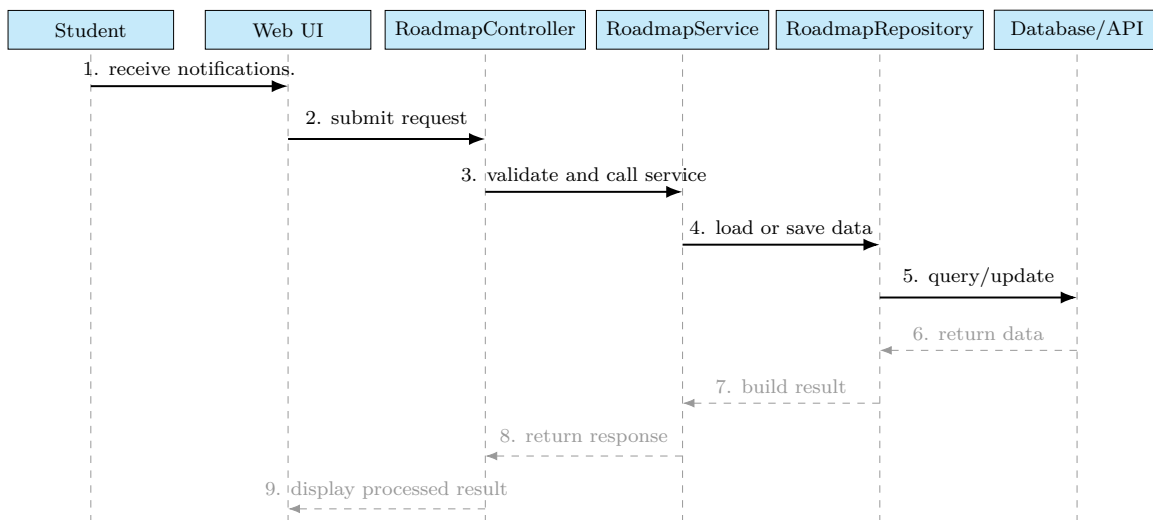


Figure IV.54: Sequence diagram for UC25 Receive Learning Reminder

3.5.6 UC26 Search Learning Resources

3.5.6.1 Requirement Traceability

- **Use case ID:** UC26.
- **Use case name:** Search Learning Resources.
- **Actor:** Student.
- **Requirement description:** Search learning materials.
- **Related table(s):** LearningResources, StudentCourses, SkillNodes.

3.5.6.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.5.6.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.5.6.4 Main Processing Flow

1. The actor opens `SearchLearningResoPage` and performs the action: search learning materials..
2. The frontend validates required fields, prepares the request DTO, and sends it to `ResourceController`.
3. `ResourceController` checks authentication, authorization, request format, and path parameters.
4. `ResourceService` loads related data through `LearningResourceRepo` and applies business rules for Search Learning Resources.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in `LearningResources`, `StudentCourses`, `SkillNodes` and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.5.6.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.5.6.6 Participating Components

- **Boundary/UI – SearchLearningResoPage:** collects input and displays the final result for Search Learning Resources.
- **Controller – ResourceController:** receives REST requests and validates request DTOs.
- **Service – ResourceService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – LearningResource:** represents the main persisted domain object.
- **Repository – LearningResourceRepo:** provides data access operations for the related table(s).

3.5.6.7 Class Diagram

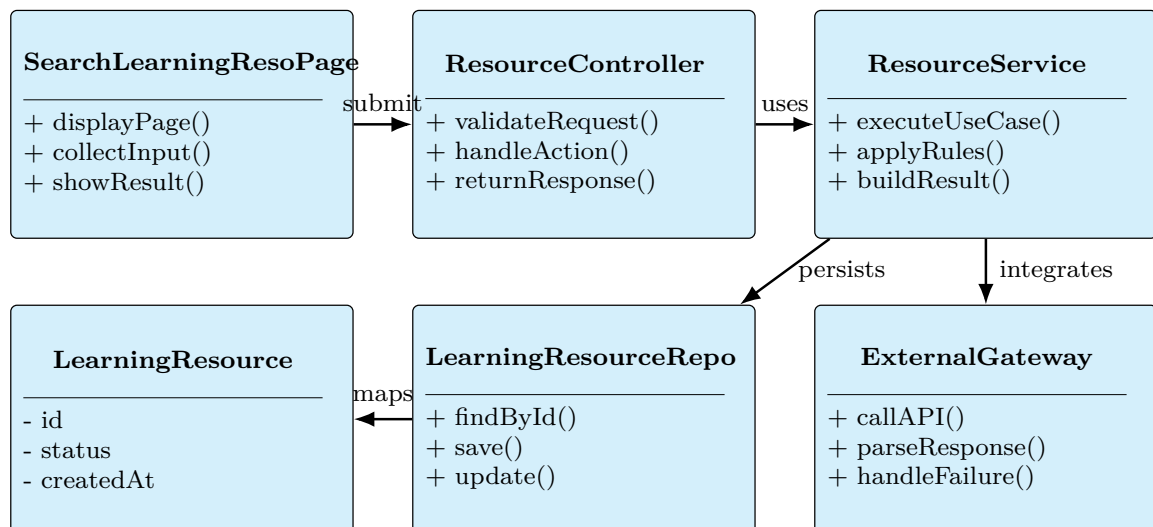


Figure IV.55: Class diagram for UC26 Search Learning Resources

3.5.6.8 Sequence Diagram

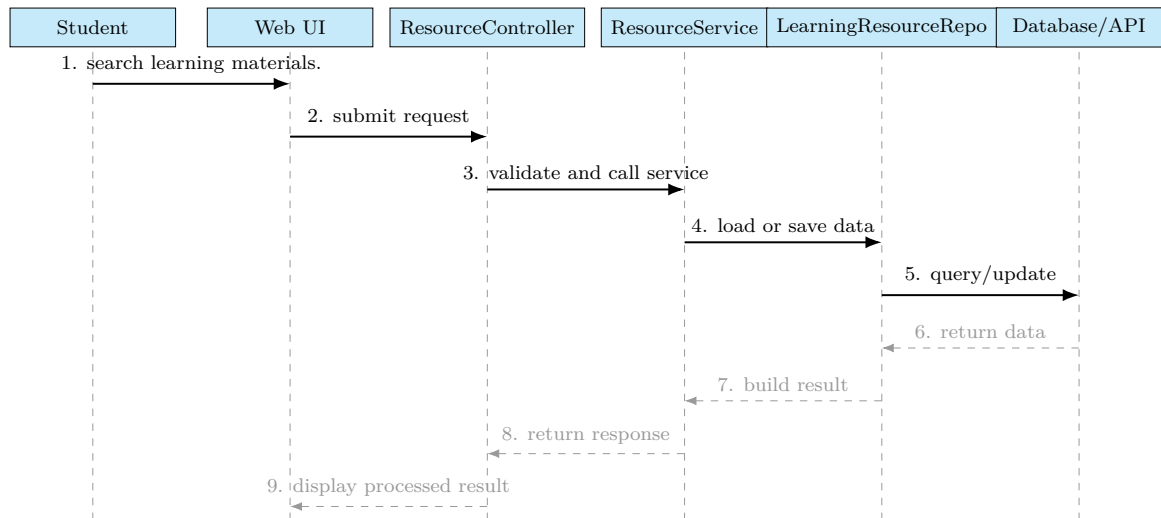


Figure IV.56: Sequence diagram for UC26 Search Learning Resources

3.5.7 UC27 View Recommended Courses

3.5.7.1 Requirement Traceability

- **Use case ID:** UC27.
- **Use case name:** View Recommended Courses.
- **Actor:** Student.
- **Requirement description:** View recommended courses.
- **Related table(s):** LearningResources, StudentCourses, SkillNodes.

3.5.7.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.5.7.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.5.7.4 Main Processing Flow

1. The actor opens `ViewRecommendedCouPage` and performs the action: view recommended courses..

2. The frontend validates required fields, prepares the request DTO, and sends it to **ResourceController**.
3. **ResourceController** checks authentication, authorization, request format, and path parameters.
4. **ResourceService** loads related data through **LearningResourceRepo** and applies business rules for View Recommended Courses.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in **LearningResources**, **StudentCourses**, **SkillNodes** and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.5.7.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.5.7.6 Participating Components

- **Boundary/UI – ViewRecommendedCouPage:** collects input and displays the final result for View Recommended Courses.
- **Controller – ResourceController:** receives REST requests and validates request DTOs.
- **Service – ResourceService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – LearningResource:** represents the main persisted domain object.
- **Repository – LearningResourceRepo:** provides data access operations for the related table(s).

3.5.7.7 Class Diagram

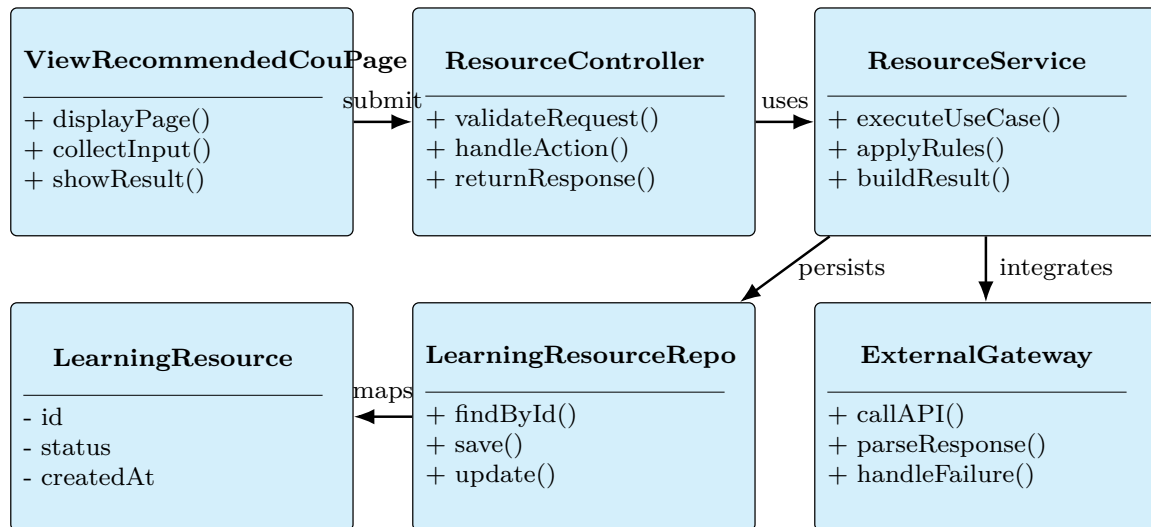


Figure IV.57: Class diagram for UC27 View Recommended Courses

3.5.7.8 Sequence Diagram

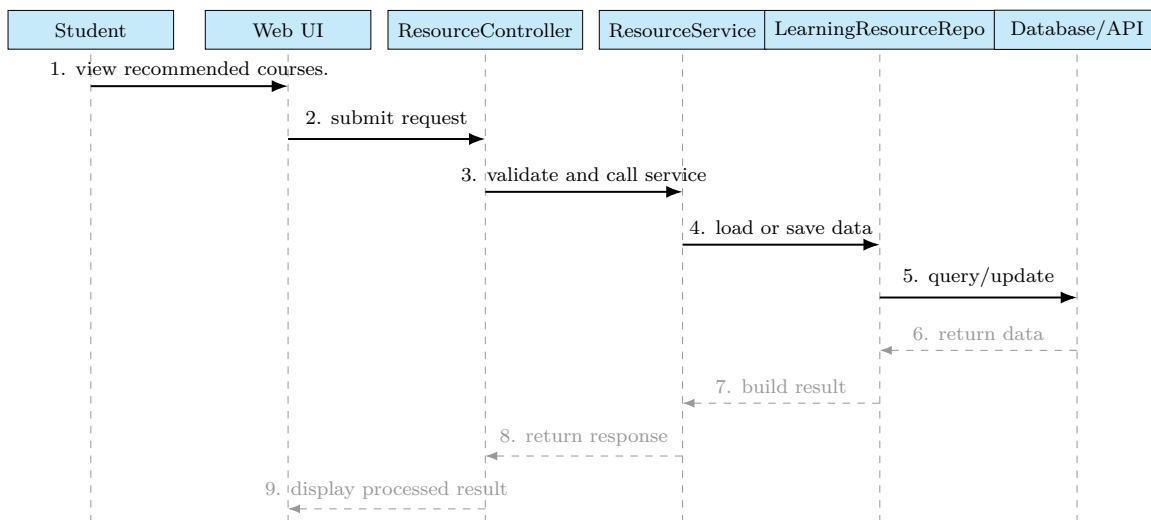


Figure IV.58: Sequence diagram for UC27 View Recommended Courses

3.5.8 UC28 Save Favorite Courses

3.5.8.1 Requirement Traceability

- **Use case ID:** UC28.
- **Use case name:** Save Favorite Courses.
- **Actor:** Student.
- **Requirement description:** Bookmark courses.
- **Related table(s):** LearningResources, StudentCourses, SkillNodes.

3.5.8.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.5.8.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.5.8.4 Main Processing Flow

1. The actor opens `SaveFavoriteCoursePage` and performs the action: bookmark courses..
2. The frontend validates required fields, prepares the request DTO, and sends it to `ResourceController`.
3. `ResourceController` checks authentication, authorization, request format, and path parameters.
4. `ResourceService` loads related data through `LearningResourceRepo` and applies business rules for Save Favorite Courses.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in `LearningResources`, `StudentCourses`, `SkillNodes` and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.5.8.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.5.8.6 Participating Components

- **Boundary/UI – SaveFavoriteCoursePage:** collects input and displays the final result for Save Favorite Courses.
- **Controller – ResourceController:** receives REST requests and validates request DTOs.
- **Service – ResourceService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – LearningResource:** represents the main persisted domain object.
- **Repository – LearningResourceRepo:** provides data access operations for the related table(s).

3.5.8.7 Class Diagram

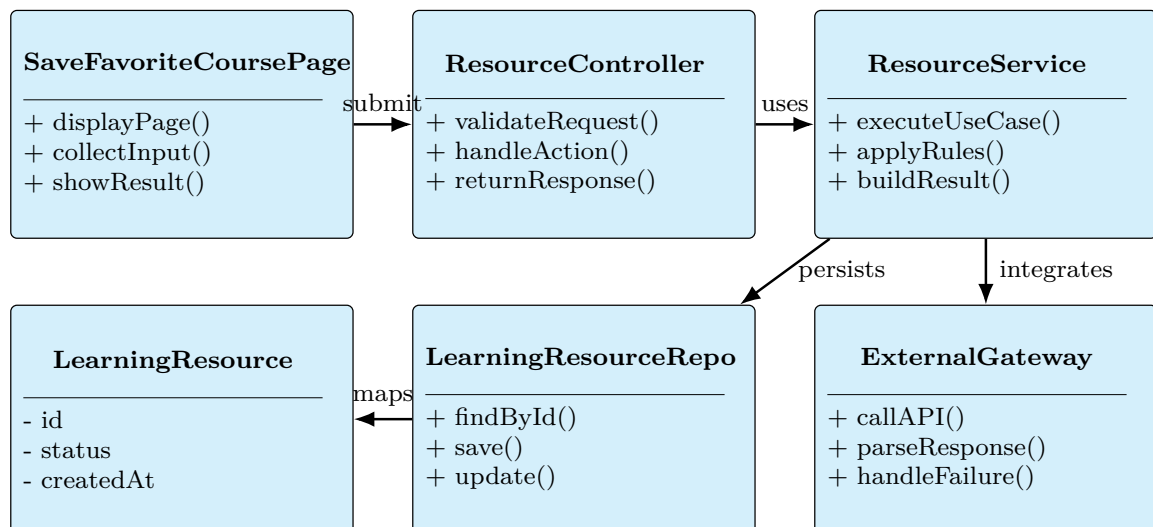


Figure IV.59: Class diagram for UC28 Save Favorite Courses

3.5.8.8 Sequence Diagram

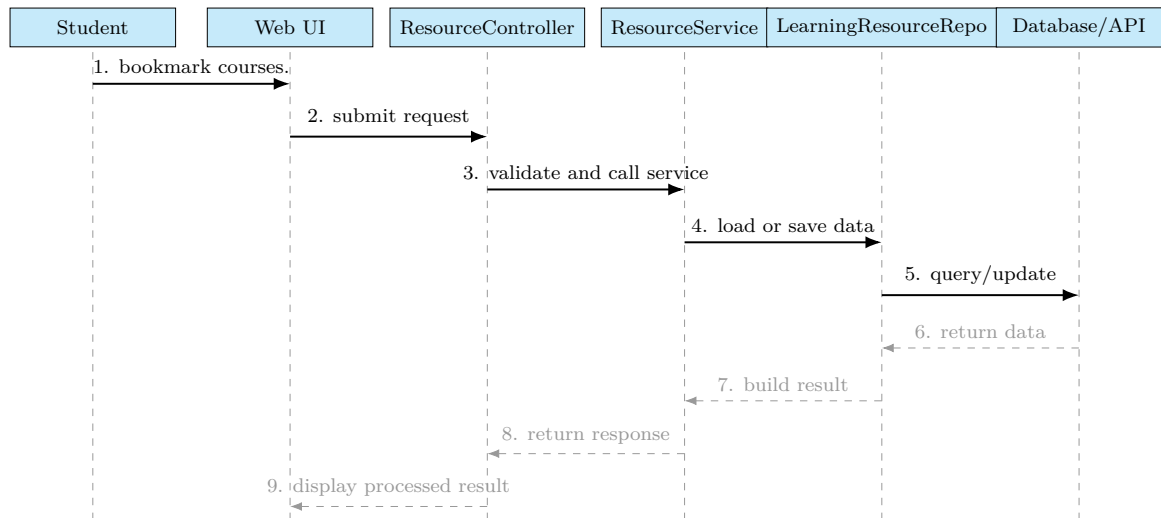


Figure IV.60: Sequence diagram for UC28 Save Favorite Courses

3.5.9 UC29 Register External Course

3.5.9.1 Requirement Traceability

- **Use case ID:** UC29.
- **Use case name:** Register External Course.
- **Actor:** Student.
- **Requirement description:** Enroll in external course.
- **Related table(s):** LearningResources, StudentCourses, SkillNodes.

3.5.9.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.5.9.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.5.9.4 Main Processing Flow

1. The actor opens `RegisterExternalCoPage` and performs the action: enroll in external course..

2. The frontend validates required fields, prepares the request DTO, and sends it to **ResourceController**.
3. **ResourceController** checks authentication, authorization, request format, and path parameters.
4. **ResourceService** loads related data through **LearningResourceRepo** and applies business rules for Register External Course.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in **LearningResources**, **StudentCourses**, **SkillNodes** and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.5.9.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.5.9.6 Participating Components

- **Boundary/UI – RegisterExternalCoPage:** collects input and displays the final result for Register External Course.
- **Controller – ResourceController:** receives REST requests and validates request DTOs.
- **Service – ResourceService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – LearningResource:** represents the main persisted domain object.
- **Repository – LearningResourceRepo:** provides data access operations for the related table(s).

3.5.9.7 Class Diagram

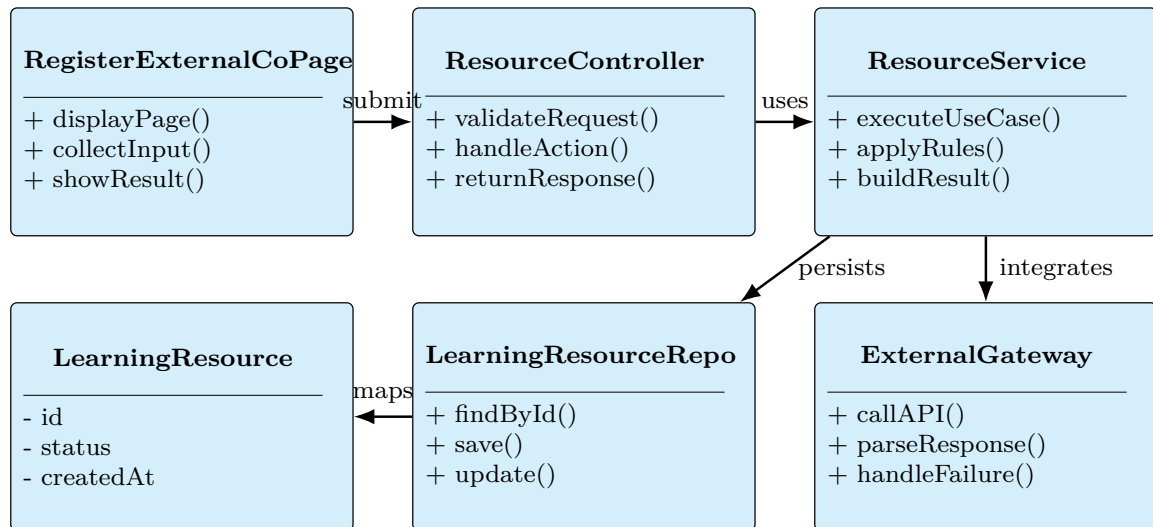


Figure IV.61: Class diagram for UC29 Register External Course

3.5.9.8 Sequence Diagram

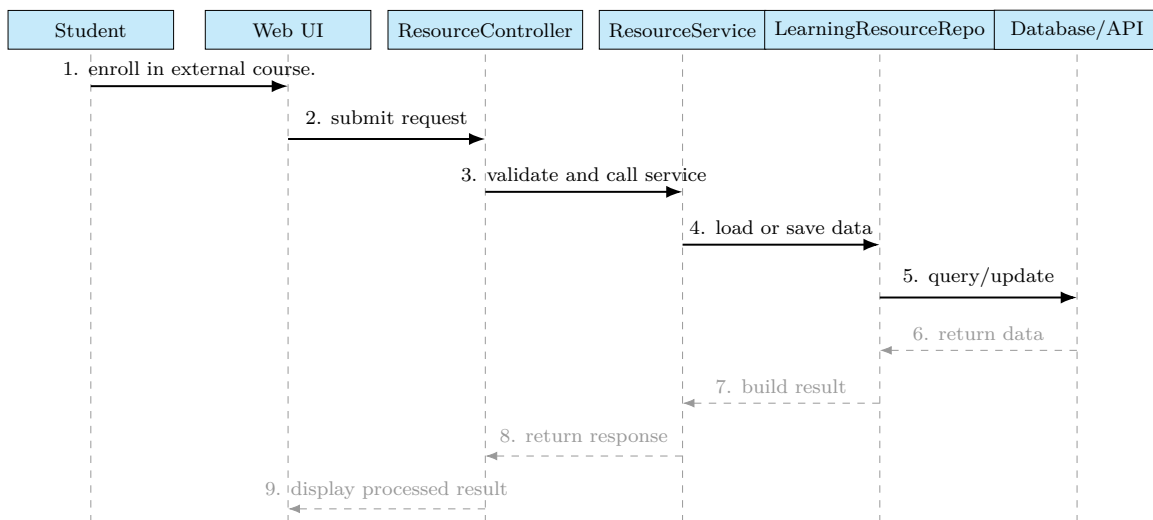


Figure IV.62: Sequence diagram for UC29 Register External Course

3.5.10 UC30 Track Course Completion

3.5.10.1 Requirement Traceability

- **Use case ID:** UC30.
- **Use case name:** Track Course Completion.
- **Actor:** Student.
- **Requirement description:** Track completed courses.
- **Related table(s):** LearningResources, StudentCourses, SkillNodes.

3.5.10.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.5.10.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.5.10.4 Main Processing Flow

1. The actor opens `TrackCourseComplePage` and performs the action: track completed courses..
2. The frontend validates required fields, prepares the request DTO, and sends it to `ResourceController`.
3. `ResourceController` checks authentication, authorization, request format, and path parameters.
4. `ResourceService` loads related data through `LearningResourceRepo` and applies business rules for Track Course Completion.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in `LearningResources`, `StudentCourses`, `SkillNodes` and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.5.10.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.5.10.6 Participating Components

- **Boundary/UI – TrackCourseComplePage:** collects input and displays the final result for Track Course Completion.
- **Controller – ResourceController:** receives REST requests and validates request DTOs.
- **Service – ResourceService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – LearningResource:** represents the main persisted domain object.
- **Repository – LearningResourceRepo:** provides data access operations for the related table(s).

3.5.10.7 Class Diagram

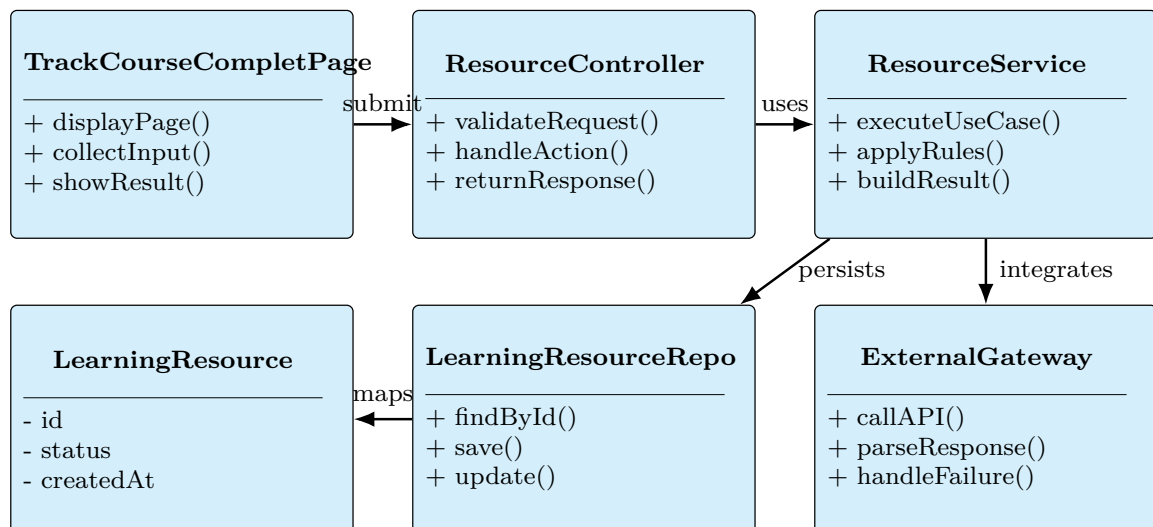


Figure IV.63: Class diagram for UC30 Track Course Completion

3.5.10.8 Sequence Diagram

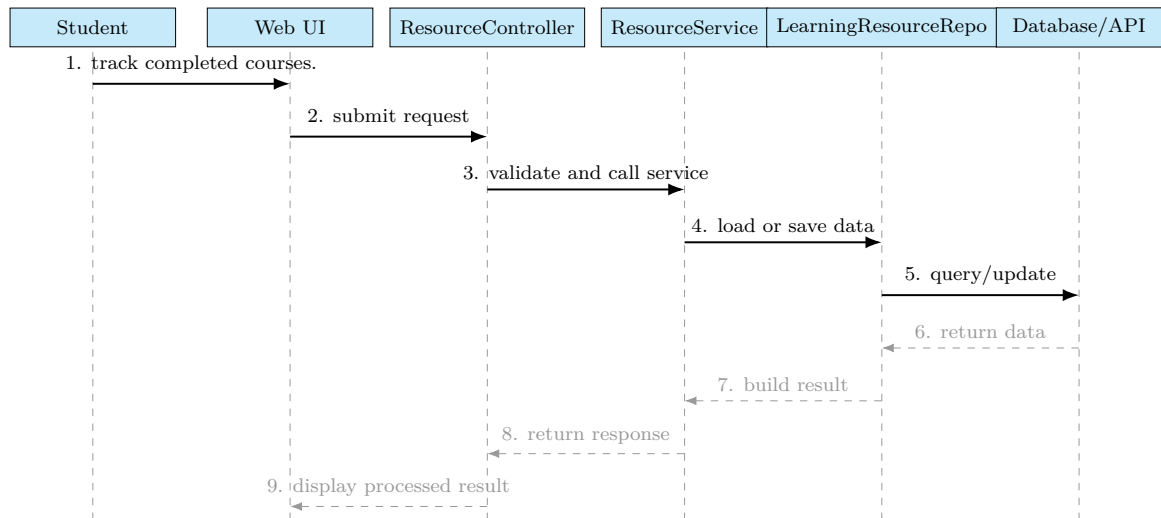


Figure IV.64: Sequence diagram for UC30 Track Course Completion

3.6 Market Trend Management

This group contains use cases related to market trend management. The design uses the layered architecture introduced in the system design: React boundary pages, Spring Boot controllers, services, repositories, database entities, and external gateways when required.

3.6.1 UC31 View Market Trends

3.6.1.1 Requirement Traceability

- **Use case ID:** UC31.
- **Use case name:** View Market Trends.
- **Actor:** Student.
- **Requirement description:** View labor market trends.
- **Related table(s):** JobTrends, CareerPaths.

3.6.1.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.6.1.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.6.1.4 Main Processing Flow

1. The actor opens **ViewMarketTrendsPage** and performs the action: view labor market trends..
2. The frontend validates required fields, prepares the request DTO, and sends it to **MarketTrendController**.
3. **MarketTrendController** checks authentication, authorization, request format, and path parameters.
4. **MarketTrendService** loads related data through **JobTrendRepository** and applies business rules for View Market Trends.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in **JobTrends**, **CareerPaths** and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.6.1.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.6.1.6 Participating Components

- **Boundary/UI – ViewMarketTrendsPage:** collects input and displays the final result for View Market Trends.
- **Controller – MarketTrendController:** receives REST requests and validates request DTOs.
- **Service – MarketTrendService:** coordinates business logic, integration calls, and transaction boundaries.

- **Entity – JobTrend:** represents the main persisted domain object.
- **Repository – JobTrendRepository:** provides data access operations for the related table(s).

3.6.1.7 Class Diagram

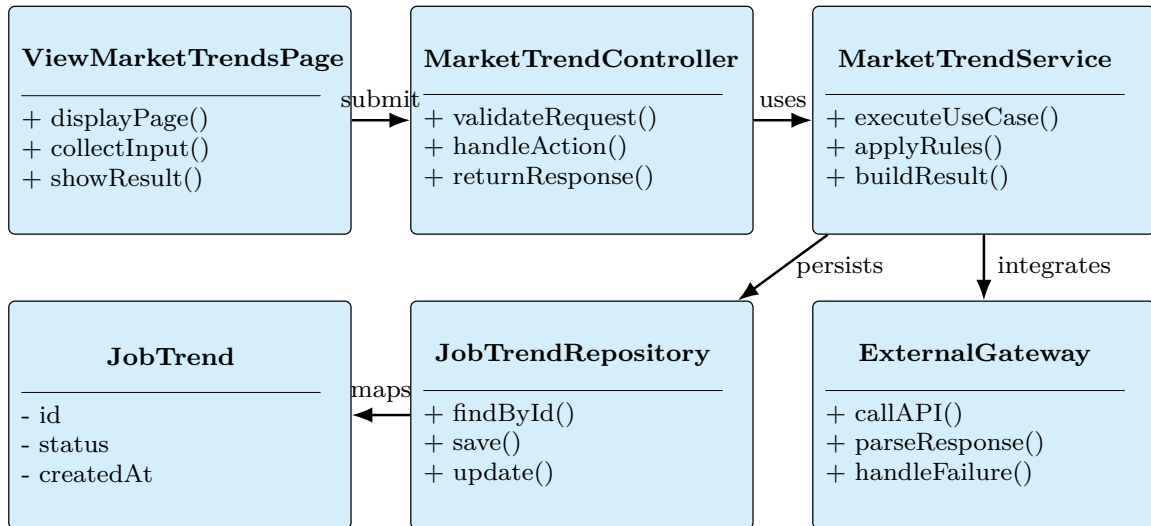


Figure IV.65: Class diagram for UC31 View Market Trends

3.6.1.8 Sequence Diagram

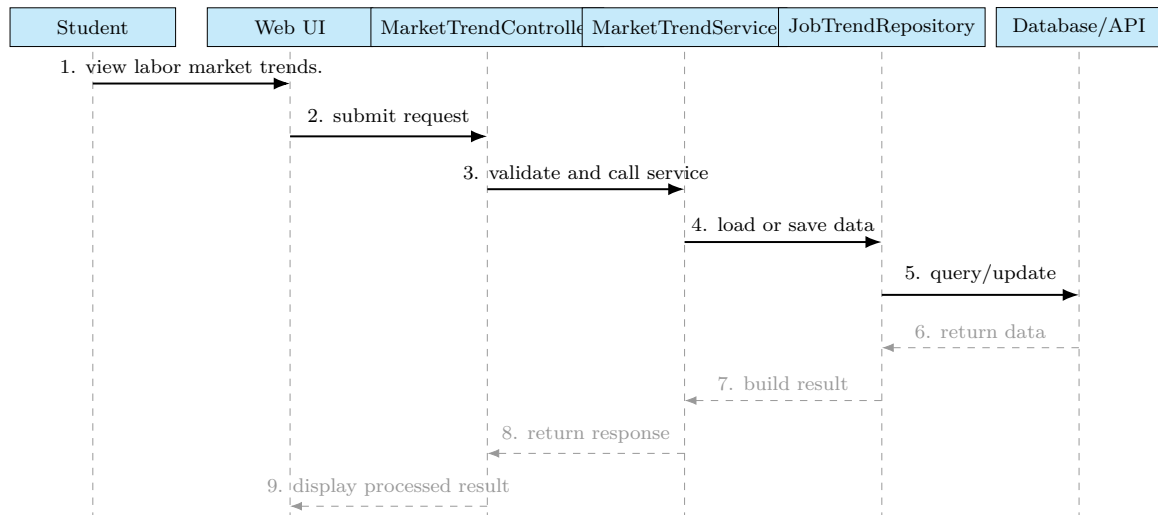


Figure IV.66: Sequence diagram for UC31 View Market Trends

3.6.2 UC32 Search Job Opportunities

3.6.2.1 Requirement Traceability

- **Use case ID:** UC32.

- **Use case name:** Search Job Opportunities.
- **Actor:** Student.
- **Requirement description:** Search job postings.
- **Related table(s):** JobTrends, CareerPaths.

3.6.2.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.6.2.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.6.2.4 Main Processing Flow

1. The actor opens `SearchJobOpportuniPage` and performs the action: search job postings..
2. The frontend validates required fields, prepares the request DTO, and sends it to `MarketTrendController`.
3. `MarketTrendController` checks authentication, authorization, request format, and path parameters.
4. `MarketTrendService` loads related data through `JobTrendRepository` and applies business rules for Search Job Opportunities.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in `JobTrends`, `CareerPaths` and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.6.2.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.

- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.6.2.6 Participating Components

- **Boundary/UI – SearchJobOpportuniPage:** collects input and displays the final result for Search Job Opportunities.
- **Controller – MarketTrendController:** receives REST requests and validates request DTOs.
- **Service – MarketTrendService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – JobTrend:** represents the main persisted domain object.
- **Repository – JobTrendRepository:** provides data access operations for the related table(s).

3.6.2.7 Class Diagram

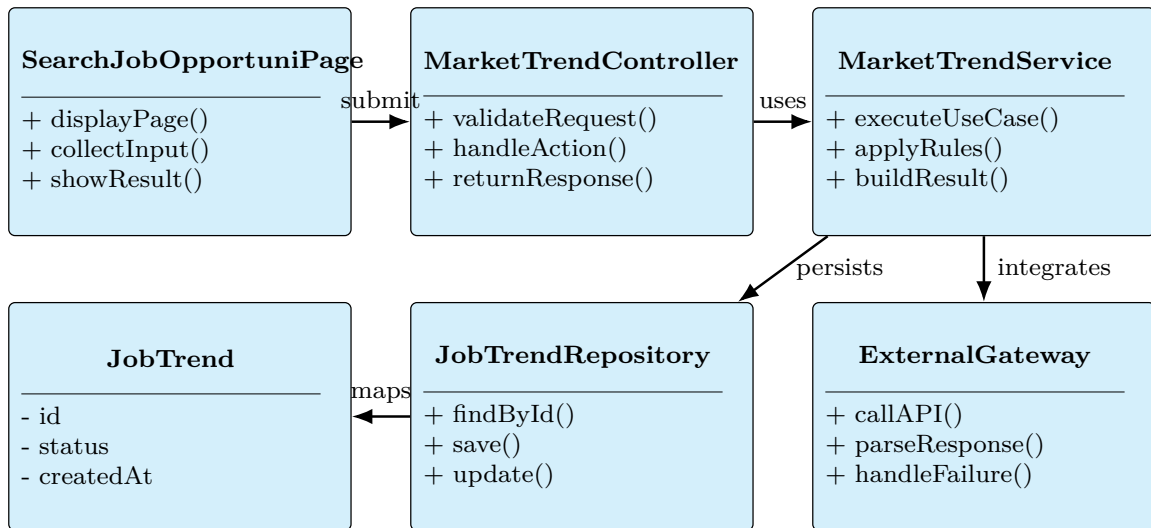


Figure IV.67: Class diagram for UC32 Search Job Opportunities

3.6.2.8 Sequence Diagram

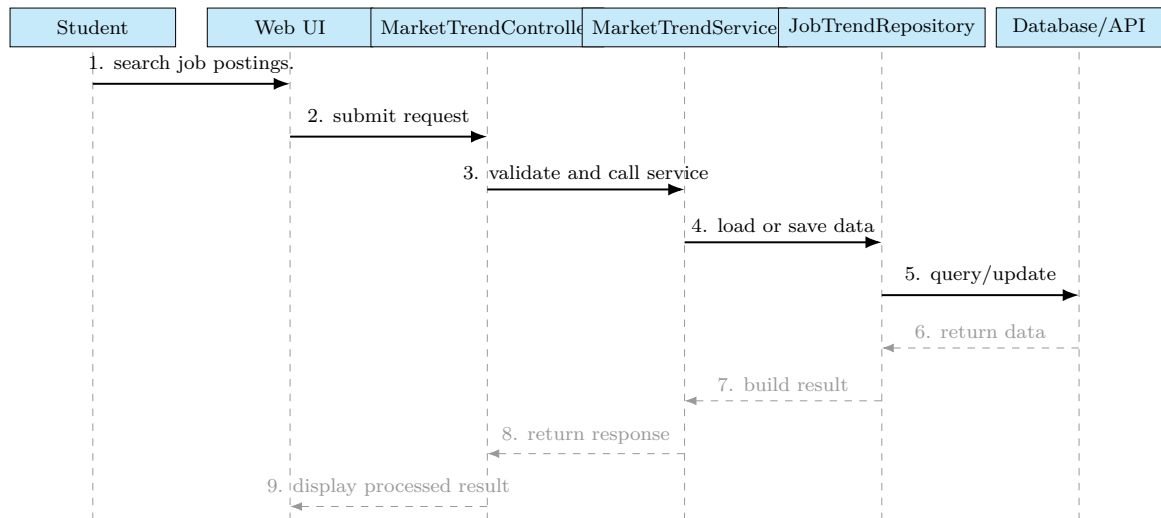


Figure IV.68: Sequence diagram for UC32 Search Job Opportunities

3.6.3 UC33 View Salary Information

3.6.3.1 Requirement Traceability

- **Use case ID:** UC33.
- **Use case name:** View Salary Information.
- **Actor:** Student.
- **Requirement description:** Display salary statistics.
- **Related table(s):** JobTrends, CareerPaths.

3.6.3.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.6.3.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.6.3.4 Main Processing Flow

1. The actor opens `ViewSalaryInformatPage` and performs the action: display salary statistics..

2. The frontend validates required fields, prepares the request DTO, and sends it to **MarketTrendController**.
3. **MarketTrendController** checks authentication, authorization, request format, and path parameters.
4. **MarketTrendService** loads related data through **JobTrendRepository** and applies business rules for View Salary Information.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in **JobTrends**, **CareerPaths** and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.6.3.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.6.3.6 Participating Components

- **Boundary/UI – ViewSalaryInformatPage:** collects input and displays the final result for View Salary Information.
- **Controller – MarketTrendController:** receives REST requests and validates request DTOs.
- **Service – MarketTrendService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – JobTrend:** represents the main persisted domain object.
- **Repository – JobTrendRepository:** provides data access operations for the related table(s).

3.6.3.7 Class Diagram

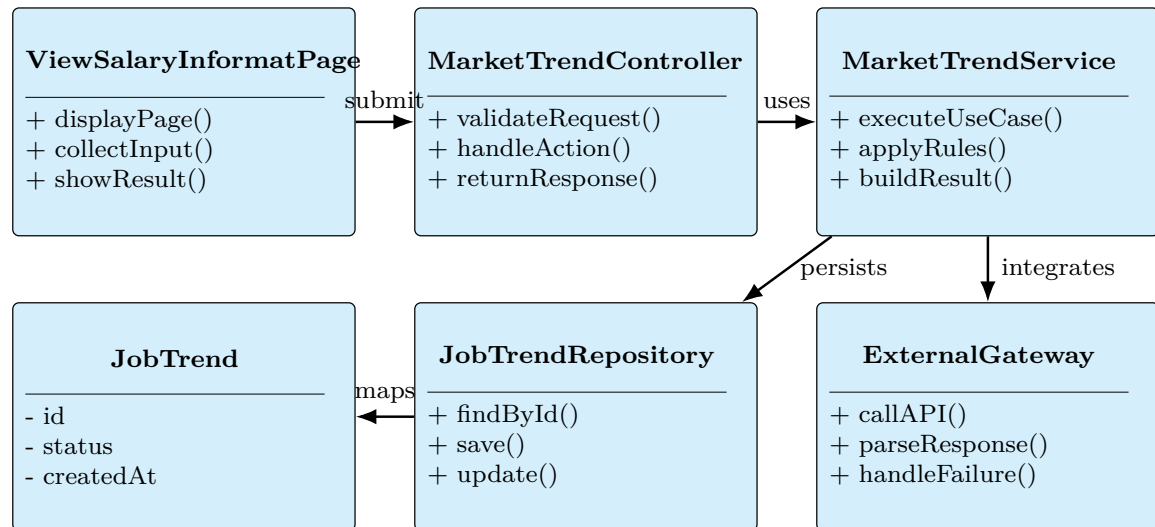


Figure IV.69: Class diagram for UC33 View Salary Information

3.6.3.8 Sequence Diagram

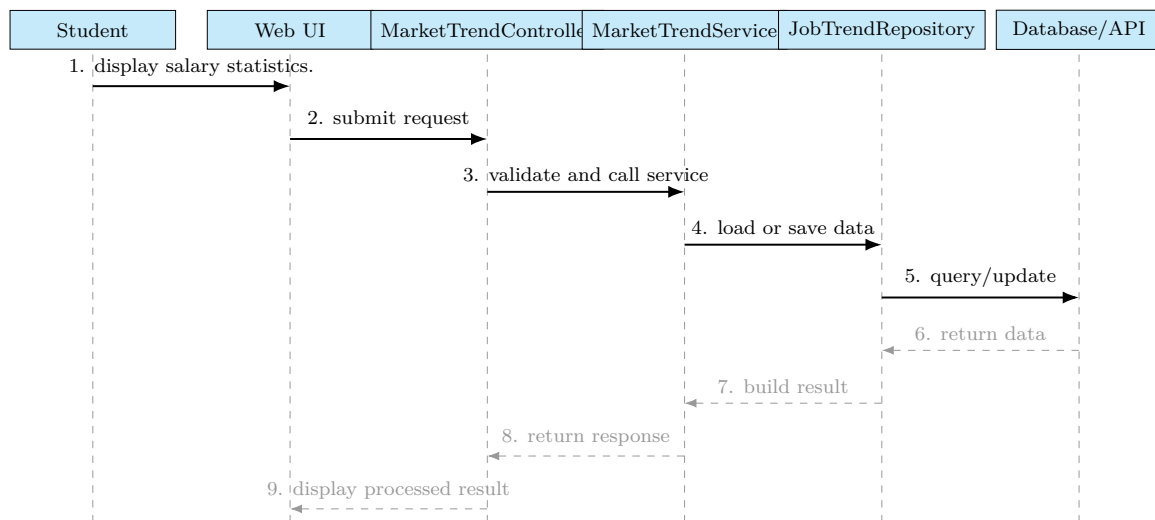


Figure IV.70: Sequence diagram for UC33 View Salary Information

3.6.4 UC34 View Technology Trends

3.6.4.1 Requirement Traceability

- **Use case ID:** UC34.
- **Use case name:** View Technology Trends.
- **Actor:** Student.
- **Requirement description:** Display technology trends.
- **Related table(s):** JobTrends, CareerPaths.

3.6.4.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.6.4.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.6.4.4 Main Processing Flow

1. The actor opens `ViewTechnologyTrenPage` and performs the action: display technology trends..
2. The frontend validates required fields, prepares the request DTO, and sends it to `MarketTrendController`.
3. `MarketTrendController` checks authentication, authorization, request format, and path parameters.
4. `MarketTrendService` loads related data through `JobTrendRepository` and applies business rules for View Technology Trends.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in `JobTrends`, `CareerPaths` and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.6.4.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.6.4.6 Participating Components

- **Boundary/UI – ViewTechnologyTrenPage:** collects input and displays the final result for View Technology Trends.
- **Controller – MarketTrendController:** receives REST requests and validates request DTOs.
- **Service – MarketTrendService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – JobTrend:** represents the main persisted domain object.
- **Repository – JobTrendRepository:** provides data access operations for the related table(s).

3.6.4.7 Class Diagram

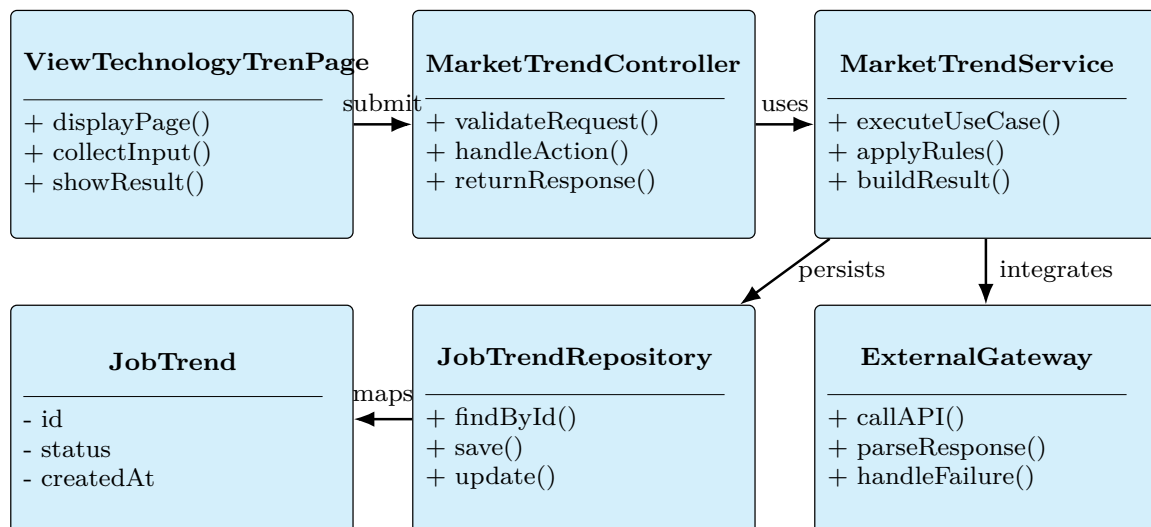


Figure IV.71: Class diagram for UC34 View Technology Trends

3.6.4.8 Sequence Diagram

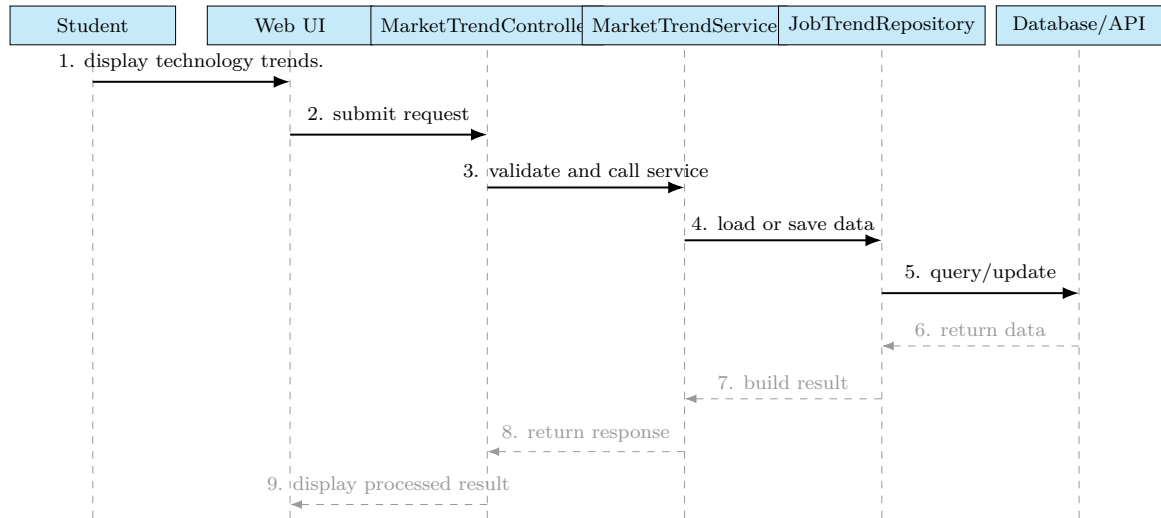


Figure IV.72: Sequence diagram for UC34 View Technology Trends

3.6.5 UC35 Receive Job Recommendation

3.6.5.1 Requirement Traceability

- **Use case ID:** UC35.
- **Use case name:** Receive Job Recommendation.
- **Actor:** Student.
- **Requirement description:** Receive recommended jobs.
- **Related table(s):** JobTrends, CareerPaths.

3.6.5.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.6.5.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.6.5.4 Main Processing Flow

1. The actor opens `ReceiveJobRecommenPage` and performs the action: receive recommended jobs..

2. The frontend validates required fields, prepares the request DTO, and sends it to **MarketTrendController**.
3. **MarketTrendController** checks authentication, authorization, request format, and path parameters.
4. **MarketTrendService** loads related data through **JobTrendRepository** and applies business rules for Receive Job Recommendation.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in **JobTrends**, **CareerPaths** and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.6.5.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.6.5.6 Participating Components

- **Boundary/UI – ReceiveJobRecommenPage:** collects input and displays the final result for Receive Job Recommendation.
- **Controller – MarketTrendController:** receives REST requests and validates request DTOs.
- **Service – MarketTrendService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – JobTrend:** represents the main persisted domain object.
- **Repository – JobTrendRepository:** provides data access operations for the related table(s).

3.6.5.7 Class Diagram

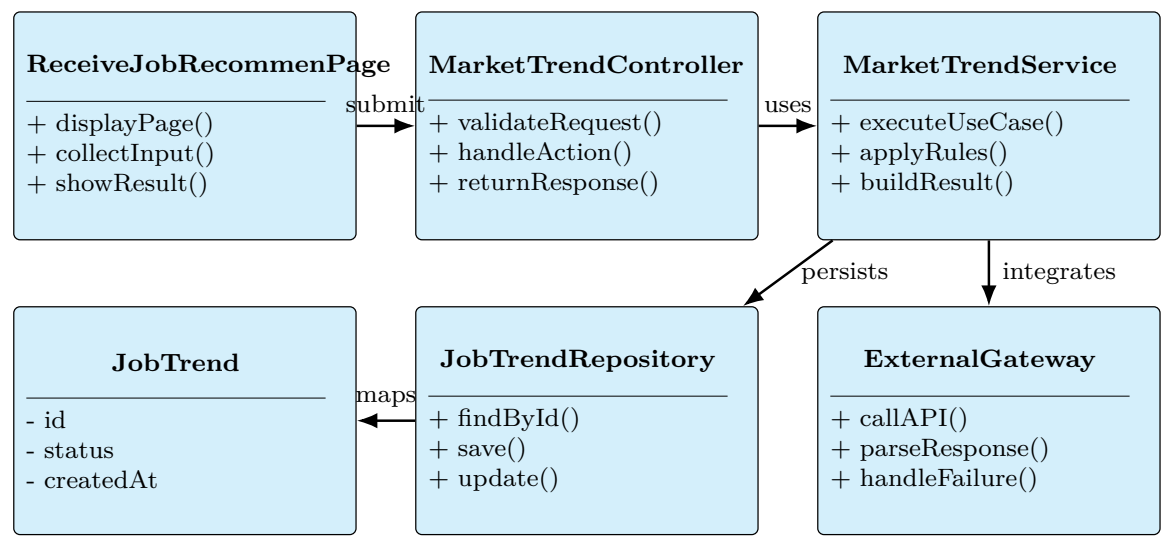


Figure IV.73: Class diagram for UC35 Receive Job Recommendation

3.6.5.8 Sequence Diagram

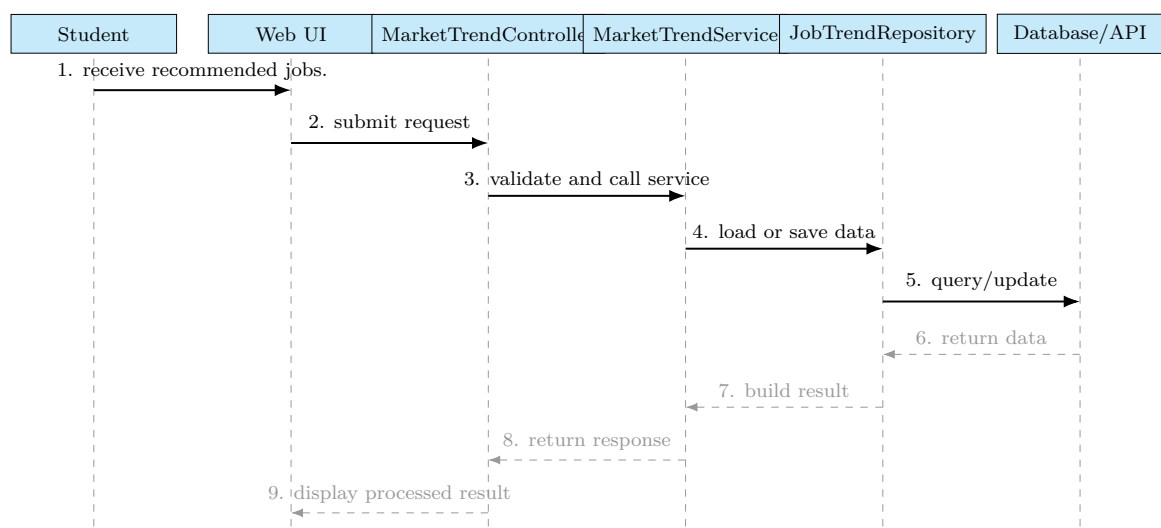


Figure IV.74: Sequence diagram for UC35 Receive Job Recommendation

3.7 AI Mentor and Analysis Logging

This group contains use cases related to ai mentor and analysis logging. The design uses the layered architecture introduced in the system design: React boundary pages, Spring Boot controllers, services, repositories, database entities, and external gateways when required.

3.7.1 UC36 Chat with AI Mentor

3.7.1.1 Requirement Traceability

- **Use case ID:** UC36.
- **Use case name:** Chat with AI Mentor.
- **Actor:** Student.
- **Requirement description:** Ask AI for learning advice.
- **Related table(s):** `AIConfigurations`, `AIAnalysisLogs`, `StudentProfiles`.

3.7.1.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.7.1.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.7.1.4 Main Processing Flow

1. The actor opens `ChatwithAIMentorPage` and performs the action: ask AI for learning advice..
2. The frontend validates required fields, prepares the request DTO, and sends it to `AIController`.
3. `AIController` checks authentication, authorization, request format, and path parameters.
4. `AIAnalysisService` loads related data through `AIAnalysisLogRepo` and applies business rules for Chat with AI Mentor.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in `AIConfigurations`, `AIAnalysisLogs`, `StudentProfiles` and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.7.1.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.7.1.6 Participating Components

- **Boundary/UI – ChatwithAIMentorPage:** collects input and displays the final result for Chat with AI Mentor.
- **Controller – AIController:** receives REST requests and validates request DTOs.
- **Service – AIAnalysisService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – AIAnalysisLog:** represents the main persisted domain object.
- **Repository – AIAnalysisLogRepo:** provides data access operations for the related table(s).

3.7.1.7 Class Diagram

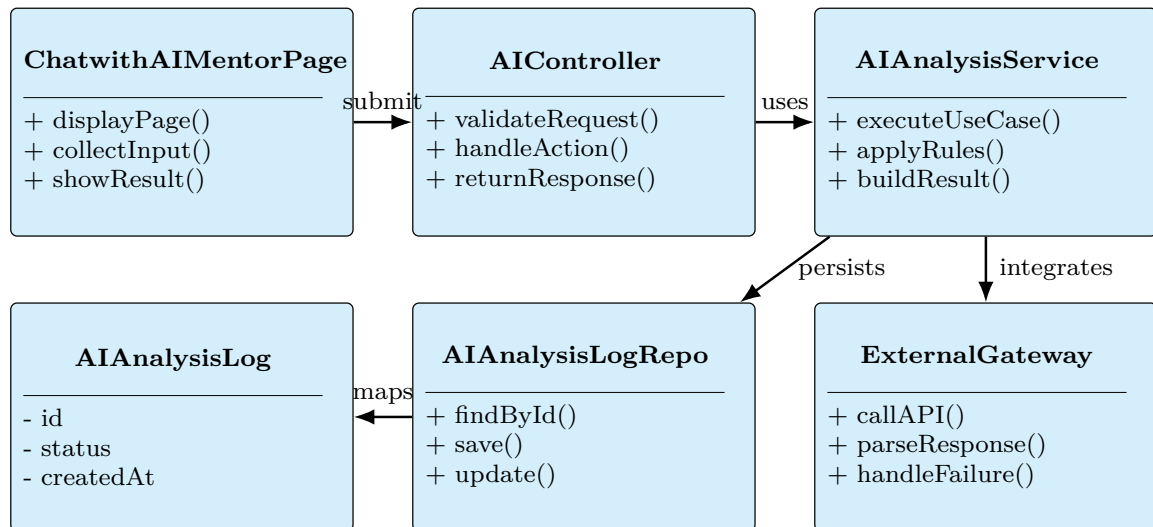


Figure IV.75: Class diagram for UC36 Chat with AI Mentor

3.7.1.8 Sequence Diagram

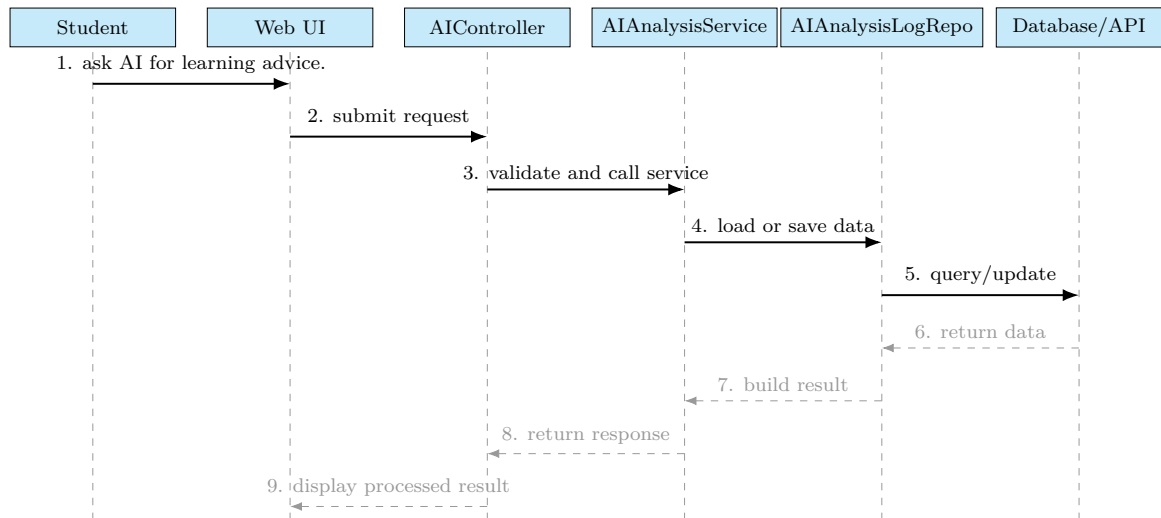


Figure IV.76: Sequence diagram for UC36 Chat with AI Mentor

3.7.2 UC37 Ask Career Questions

3.7.2.1 Requirement Traceability

- **Use case ID:** UC37.
- **Use case name:** Ask Career Questions.
- **Actor:** Student.
- **Requirement description:** Ask career-related questions.
- **Related table(s):** AIConfigurations, AIAnalysisLogs, StudentProfiles.

3.7.2.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.7.2.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.7.2.4 Main Processing Flow

1. The actor opens `AskCareerQuestionsPage` and performs the action: ask career-related questions..

2. The frontend validates required fields, prepares the request DTO, and sends it to **AIController**.
3. **AIController** checks authentication, authorization, request format, and path parameters.
4. **AIAnalysisService** loads related data through **AIAnalysisLogRepo** and applies business rules for Ask Career Questions.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in **AIConfigurations**, **AIAnalysisLogs**, **StudentProfiles** and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.7.2.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.7.2.6 Participating Components

- **Boundary/UI – AskCareerQuestionsPage:** collects input and displays the final result for Ask Career Questions.
- **Controller – AIController:** receives REST requests and validates request DTOs.
- **Service – AIAnalysisService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – AIAnalysisLog:** represents the main persisted domain object.
- **Repository – AIAnalysisLogRepo:** provides data access operations for the related table(s).

3.7.2.7 Class Diagram

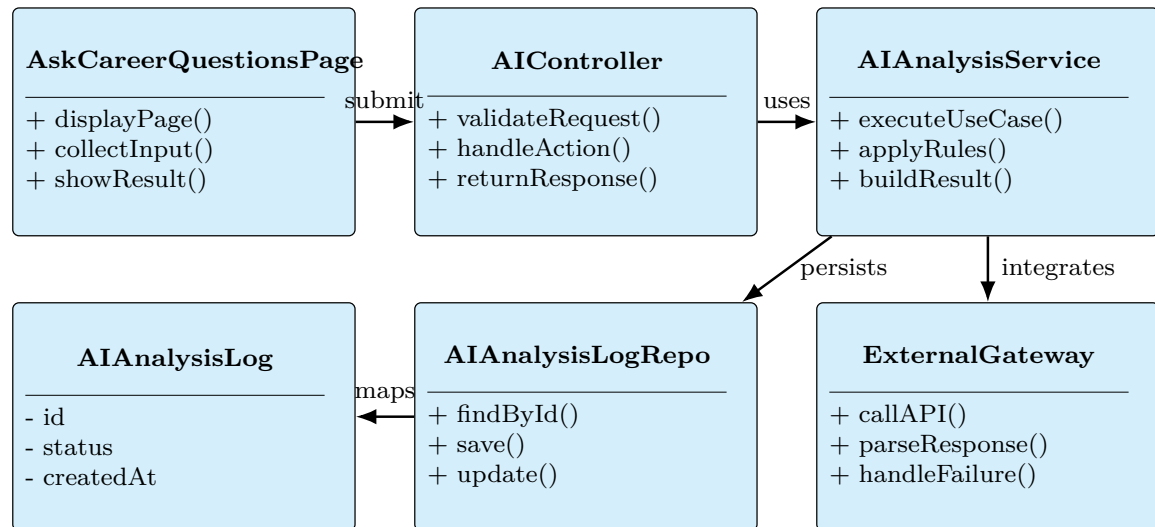


Figure IV.77: Class diagram for UC37 Ask Career Questions

3.7.2.8 Sequence Diagram

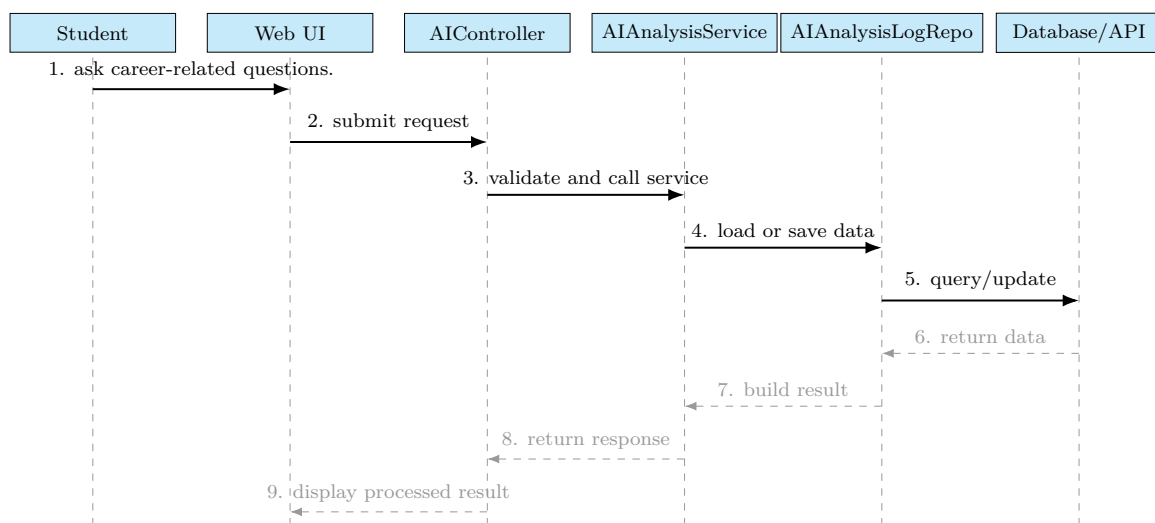


Figure IV.78: Sequence diagram for UC37 Ask Career Questions

3.7.3 UC38 Receive AI Feedback

3.7.3.1 Requirement Traceability

- **Use case ID:** UC38.
- **Use case name:** Receive AI Feedback.
- **Actor:** Student.
- **Requirement description:** Receive AI feedback.
- **Related table(s):** AIConfigurations, AIAnalysisLogs, StudentProfiles.

3.7.3.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.7.3.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.7.3.4 Main Processing Flow

1. The actor opens `ReceiveAIFeedbackPage` and performs the action: receive AI feedback..
2. The frontend validates required fields, prepares the request DTO, and sends it to `AIController`.
3. `AIController` checks authentication, authorization, request format, and path parameters.
4. `AIAnalysisService` loads related data through `AIAnalysisLogRepo` and applies business rules for Receive AI Feedback.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in `AIConfigurations`, `AIAnalysisLogs`, `StudentProfiles` and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.7.3.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.7.3.6 Participating Components

- **Boundary/UI – ReceiveAIFeedbackPage:** collects input and displays the final result for Receive AI Feedback.
- **Controller – AIController:** receives REST requests and validates request DTOs.
- **Service – AIAnalysisService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – AIAnalysisLog:** represents the main persisted domain object.
- **Repository – AIAnalysisLogRepo:** provides data access operations for the related table(s).

3.7.3.7 Class Diagram

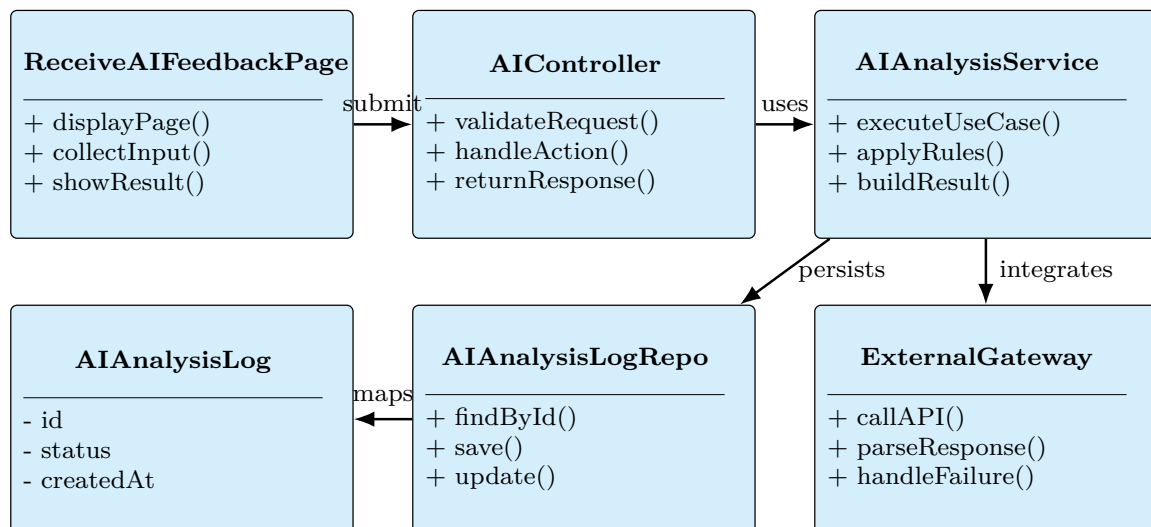


Figure IV.79: Class diagram for UC38 Receive AI Feedback

3.7.3.8 Sequence Diagram

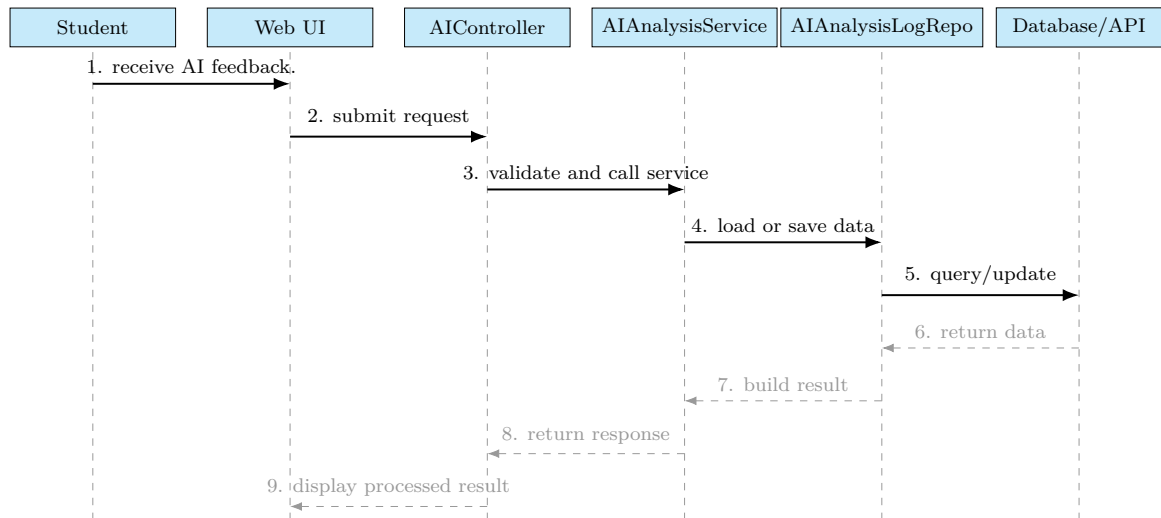


Figure IV.80: Sequence diagram for UC38 Receive AI Feedback

3.8 E-Portfolio and Mentor Review

This group contains use cases related to e-portfolio and mentor review. The design uses the layered architecture introduced in the system design: React boundary pages, Spring Boot controllers, services, repositories, database entities, and external gateways when required.

3.8.1 UC39 Generate E-Portfolio

3.8.1.1 Requirement Traceability

- **Use case ID:** UC39.
- **Use case name:** Generate E-Portfolio.
- **Actor:** Student.
- **Requirement description:** Generate portfolio automatically.
- **Related table(s):** Portfolios, GitHubRepositories, ConsultationFeedbacks.

3.8.1.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.8.1.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.8.1.4 Main Processing Flow

1. The actor opens **GenerateEPortfolioPage** and performs the action: generate portfolio automatically..
2. The frontend validates required fields, prepares the request DTO, and sends it to **PortfolioController**.
3. **PortfolioController** checks authentication, authorization, request format, and path parameters.
4. **PortfolioService** loads related data through **PortfolioRepository** and applies business rules for Generate E-Portfolio.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in **Portfolios**, **GitHubRepositories**, **ConsultationFeedbacks** and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.8.1.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.8.1.6 Participating Components

- **Boundary/UI – GenerateEPortfolioPage:** collects input and displays the final result for Generate E-Portfolio.
- **Controller – PortfolioController:** receives REST requests and validates request DTOs.
- **Service – PortfolioService:** coordinates business logic, integration calls, and transaction boundaries.

- **Entity – Portfolio:** represents the main persisted domain object.
- **Repository – PortfolioRepository:** provides data access operations for the related table(s).

3.8.1.7 Class Diagram

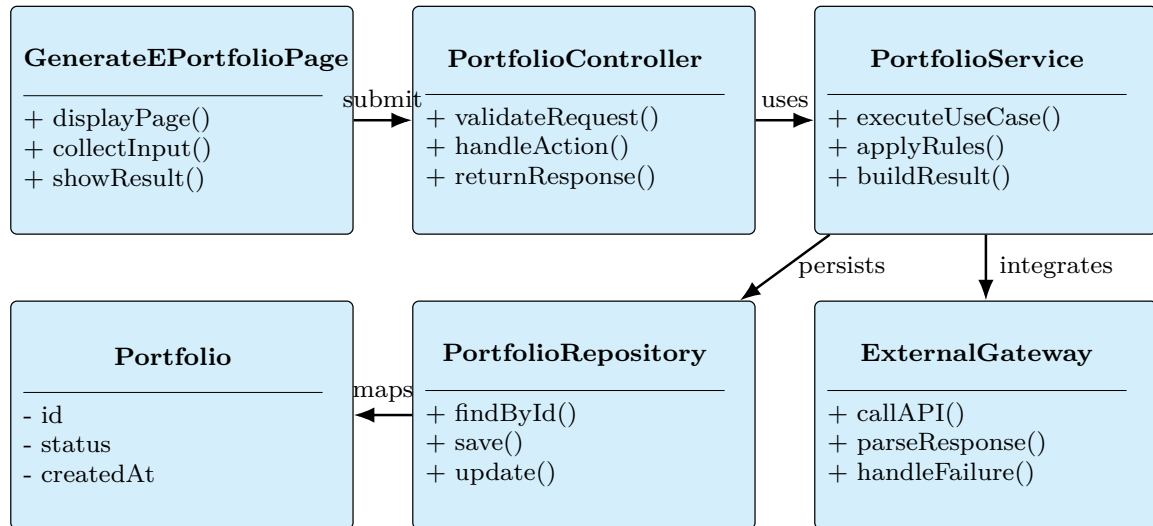


Figure IV.81: Class diagram for UC39 Generate E-Portfolio

3.8.1.8 Sequence Diagram

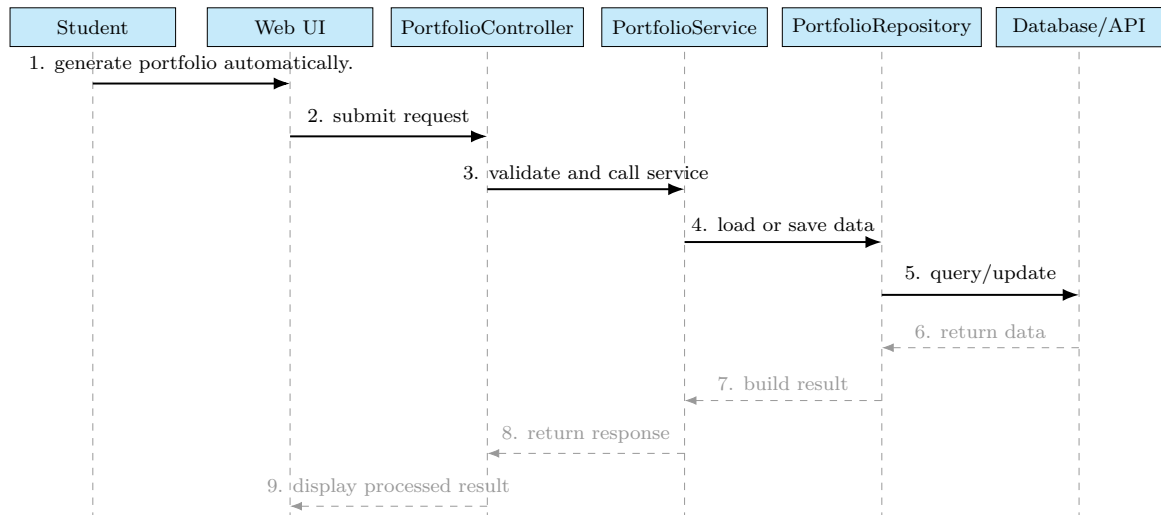


Figure IV.82: Sequence diagram for UC39 Generate E-Portfolio

3.8.2 UC40 Share Portfolio

3.8.2.1 Requirement Traceability

- **Use case ID:** UC40.

- **Use case name:** Share Portfolio.
- **Actor:** Student.
- **Requirement description:** Share portfolio link.
- **Related table(s):** Portfolios, GitHubRepositories, ConsultationFeedbacks.

3.8.2.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.8.2.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.8.2.4 Main Processing Flow

1. The actor opens `SharePortfolioPage` and performs the action: share portfolio link..
2. The frontend validates required fields, prepares the request DTO, and sends it to `PortfolioController`.
3. `PortfolioController` checks authentication, authorization, request format, and path parameters.
4. `PortfolioService` loads related data through `PortfolioRepository` and applies business rules for Share Portfolio.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in `Portfolios`, `GitHubRepositories`, `ConsultationFeedbacks` and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.8.2.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.

- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.8.2.6 Participating Components

- **Boundary/UI – SharePortfolioPage:** collects input and displays the final result for Share Portfolio.
- **Controller – PortfolioController:** receives REST requests and validates request DTOs.
- **Service – PortfolioService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – Portfolio:** represents the main persisted domain object.
- **Repository – PortfolioRepository:** provides data access operations for the related table(s).

3.8.2.7 Class Diagram

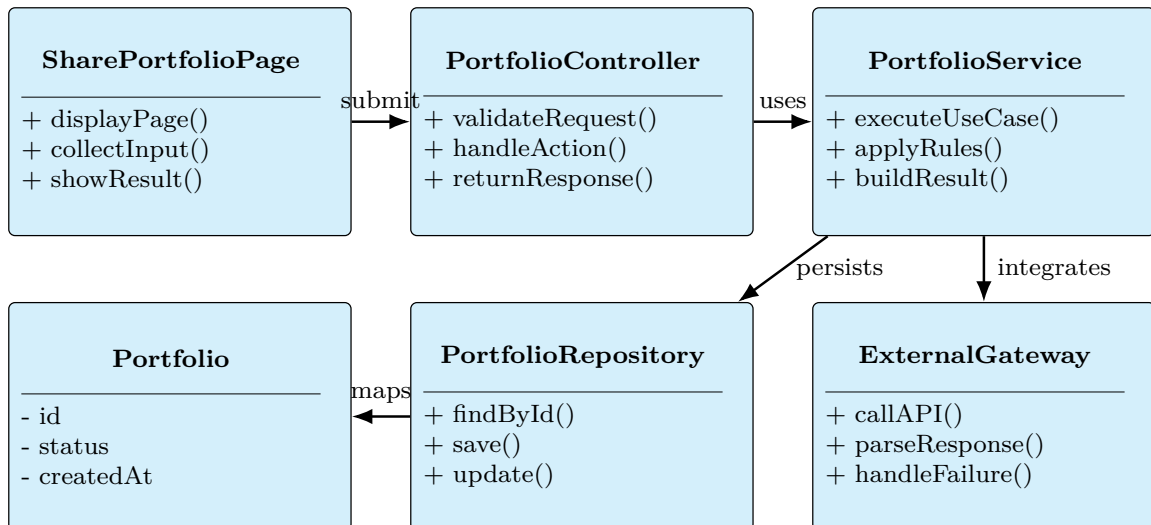


Figure IV.83: Class diagram for UC40 Share Portfolio

3.8.2.8 Sequence Diagram

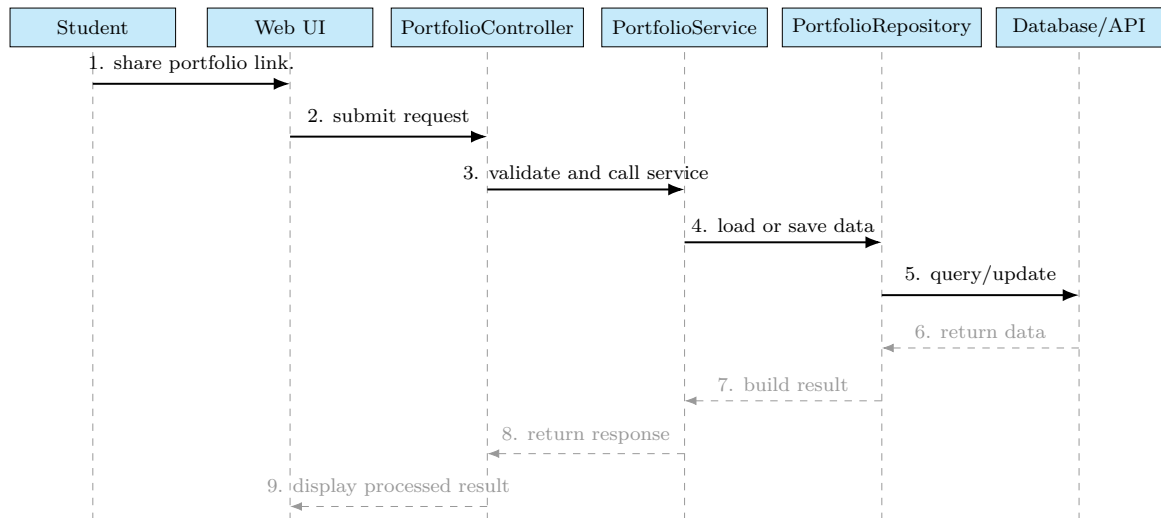


Figure IV.84: Sequence diagram for UC40 Share Portfolio

3.9 Authentication and Account Access

This group contains use cases related to authentication and account access. The design uses the layered architecture introduced in the system design: React boundary pages, Spring Boot controllers, services, repositories, database entities, and external gateways when required.

3.9.1 UC41 Login

3.9.1.1 Requirement Traceability

- **Use case ID:** UC41.
- **Use case name:** Login.
- **Actor:** Academic Counselor.
- **Requirement description:** Login to counselor dashboard.
- **Related table(s):** Users.

3.9.1.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.9.1.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.9.1.4 Main Processing Flow

1. The actor opens **LoginPage** and performs the action: login to counselor dashboard..
2. The frontend validates required fields, prepares the request DTO, and sends it to **AuthController**.
3. **AuthController** checks authentication, authorization, request format, and path parameters.
4. **AuthService** loads related data through **UserRepository** and applies business rules for Login.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in **Users** and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.9.1.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.9.1.6 Participating Components

- **Boundary/UI – LoginPage:** collects input and displays the final result for Login.
- **Controller – AuthController:** receives REST requests and validates request DTOs.
- **Service – AuthService:** coordinates business logic, integration calls, and transaction boundaries.

- **Entity – User:** represents the main persisted domain object.
- **Repository – UserRepository:** provides data access operations for the related table(s).

3.9.1.7 Class Diagram

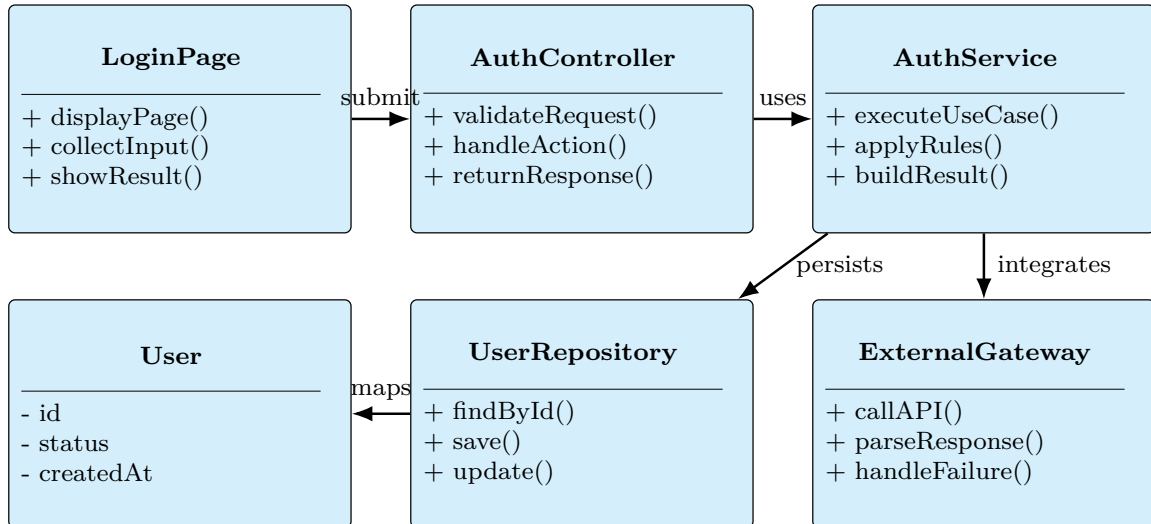


Figure IV.85: Class diagram for UC41 Login

3.9.1.8 Sequence Diagram

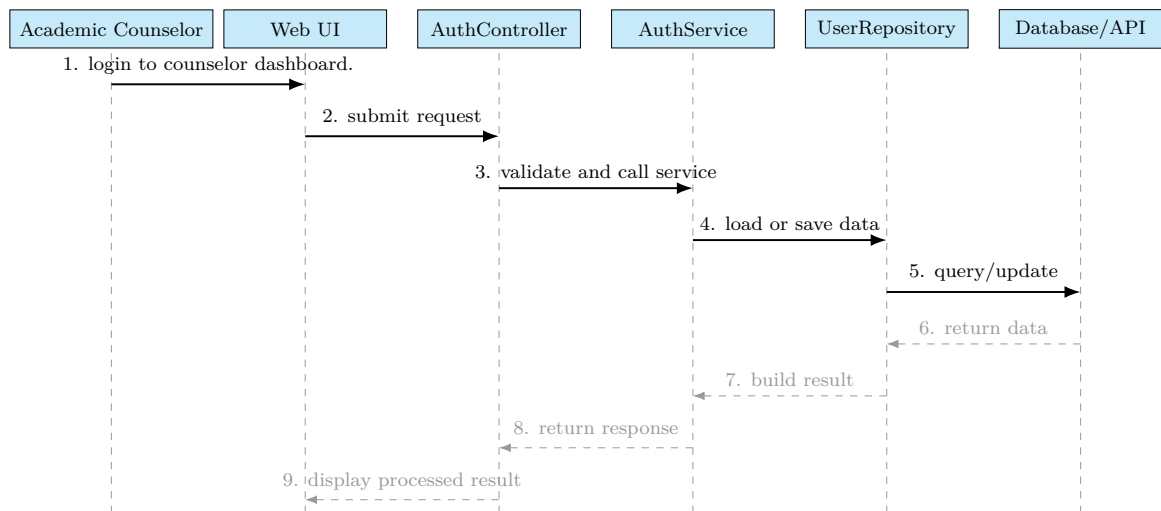


Figure IV.86: Sequence diagram for UC41 Login

3.10 Consultation and Feedback Management

This group contains use cases related to consultation and feedback management. The design uses the layered architecture introduced in the system design: React boundary pages, Spring Boot controllers, services, repositories, database entities, and external gateways when required.

3.10.1 UC42 View Student Profile

3.10.1.1 Requirement Traceability

- **Use case ID:** UC42.
- **Use case name:** View Student Profile.
- **Actor:** Academic Counselor.
- **Requirement description:** View student profile.
- **Related table(s):** ConsultationSessions, ConsultationFeedbacks, StaffProfiles.

3.10.1.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in **Users.Role** must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.10.1.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.10.1.4 Main Processing Flow

1. The actor opens **ViewStudentProfilePage** and performs the action: view student profile..
2. The frontend validates required fields, prepares the request DTO, and sends it to **ConsultationController**.
3. **ConsultationController** checks authentication, authorization, request format, and path parameters.
4. **ConsultationService** loads related data through **ConsultationSessionRepo** and applies business rules for View Student Profile.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in **ConsultationSessions**, **ConsultationFeedbacks**, **StaffProfiles** and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.10.1.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.10.1.6 Participating Components

- **Boundary/UI – ViewStudentProfilePage:** collects input and displays the final result for View Student Profile.
- **Controller – ConsultationController:** receives REST requests and validates request DTOs.
- **Service – ConsultationService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – ConsultationSession:** represents the main persisted domain object.
- **Repository – ConsultationSessionRepo:** provides data access operations for the related table(s).

3.10.1.7 Class Diagram

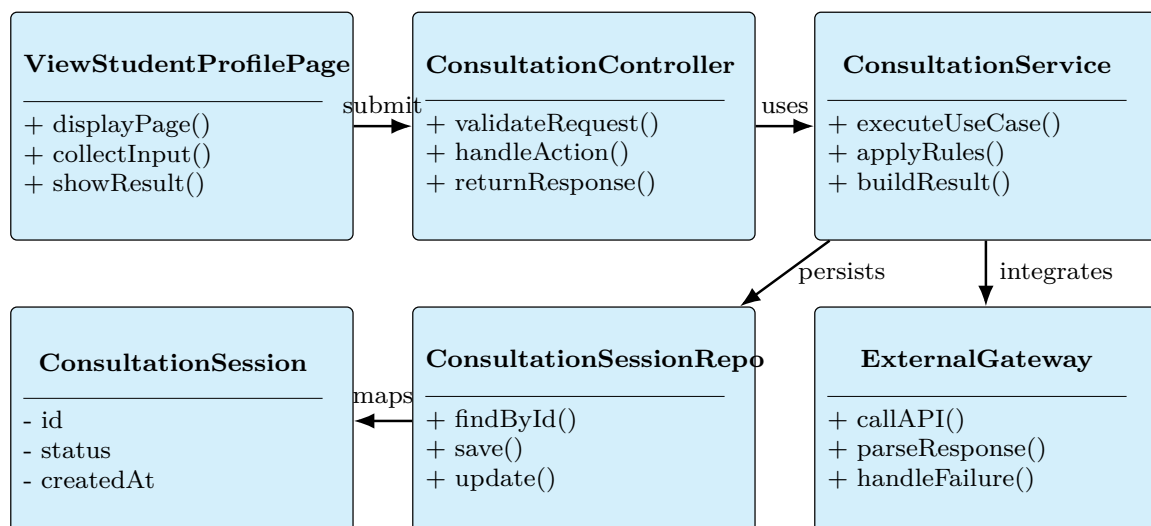


Figure IV.87: Class diagram for UC42 View Student Profile

3.10.1.8 Sequence Diagram

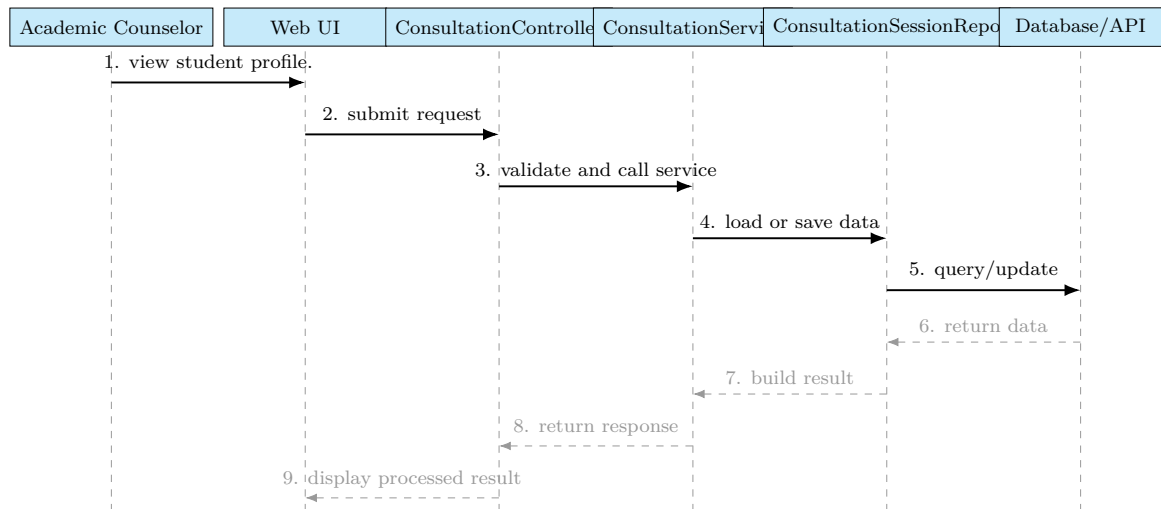


Figure IV.88: Sequence diagram for UC42 View Student Profile

3.10.2 UC43 Review Skill Gap

3.10.2.1 Requirement Traceability

- **Use case ID:** UC43.
- **Use case name:** Review Skill Gap.
- **Actor:** Academic Counselor.
- **Requirement description:** Review skill gap report.
- **Related table(s):** ConsultationSessions, ConsultationFeedbacks, StaffProfiles.

3.10.2.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.10.2.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.10.2.4 Main Processing Flow

1. The actor opens `ReviewSkillGapPage` and performs the action: review skill gap report..

2. The frontend validates required fields, prepares the request DTO, and sends it to **ConsultationController**.
3. **ConsultationController** checks authentication, authorization, request format, and path parameters.
4. **ConsultationService** loads related data through **ConsultationSessionRepo** and applies business rules for Review Skill Gap.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in **ConsultationSessions**, **ConsultationFeedbacks**, **StaffProfiles** and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.10.2.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.10.2.6 Participating Components

- **Boundary/UI – ReviewSkillGapPage:** collects input and displays the final result for Review Skill Gap.
- **Controller – ConsultationController:** receives REST requests and validates request DTOs.
- **Service – ConsultationService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – ConsultationSession:** represents the main persisted domain object.
- **Repository – ConsultationSessionRepo:** provides data access operations for the related table(s).

3.10.2.7 Class Diagram

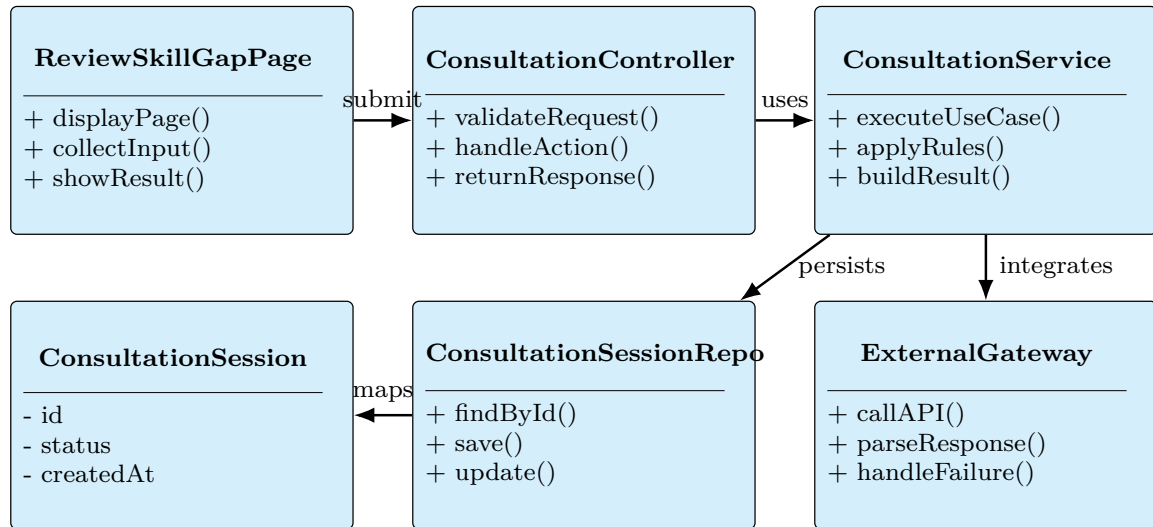


Figure IV.89: Class diagram for UC43 Review Skill Gap

3.10.2.8 Sequence Diagram

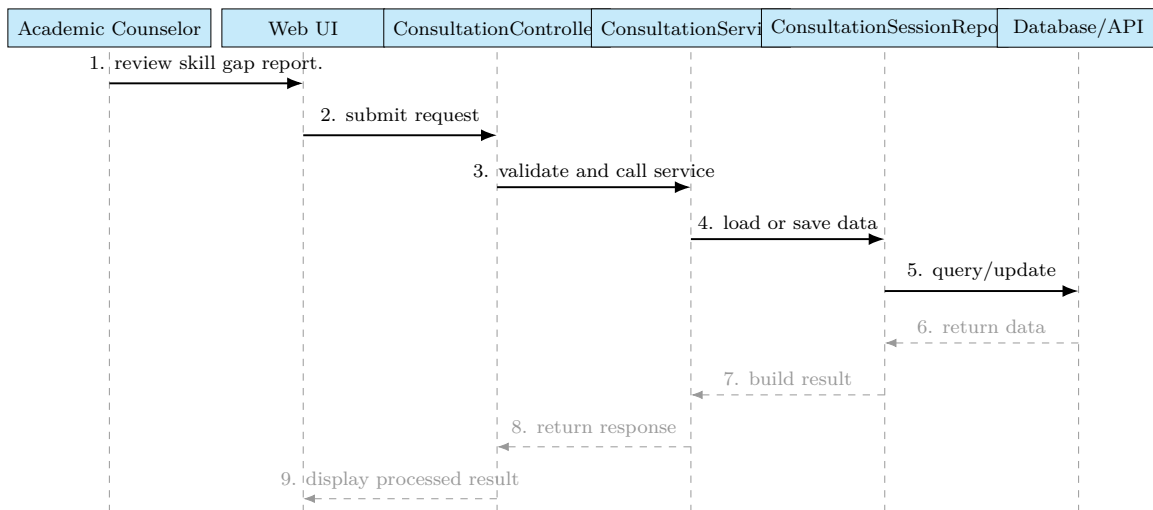


Figure IV.90: Sequence diagram for UC43 Review Skill Gap

3.10.3 UC44 Recommend Career Path

3.10.3.1 Requirement Traceability

- **Use case ID:** UC44.
- **Use case name:** Recommend Career Path.
- **Actor:** Academic Counselor.
- **Requirement description:** Recommend career orientation.
- **Related table(s):** ConsultationSessions, ConsultationFeedbacks, StaffProfiles.

3.10.3.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.10.3.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.10.3.4 Main Processing Flow

1. The actor opens `RecommendCareerPatPage` and performs the action: recommend career orientation..
2. The frontend validates required fields, prepares the request DTO, and sends it to `ConsultationController`.
3. `ConsultationController` checks authentication, authorization, request format, and path parameters.
4. `ConsultationService` loads related data through `ConsultationSessionRepo` and applies business rules for Recommend Career Path.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in `ConsultationSessions`, `ConsultationFeedbacks`, `StaffProfiles` and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.10.3.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.10.3.6 Participating Components

- **Boundary/UI – RecommendCareerPatPage:** collects input and displays the final result for Recommend Career Path.
- **Controller – ConsultationController:** receives REST requests and validates request DTOs.
- **Service – ConsultationService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – ConsultationSession:** represents the main persisted domain object.
- **Repository – ConsultationSessionRepo:** provides data access operations for the related table(s).

3.10.3.7 Class Diagram

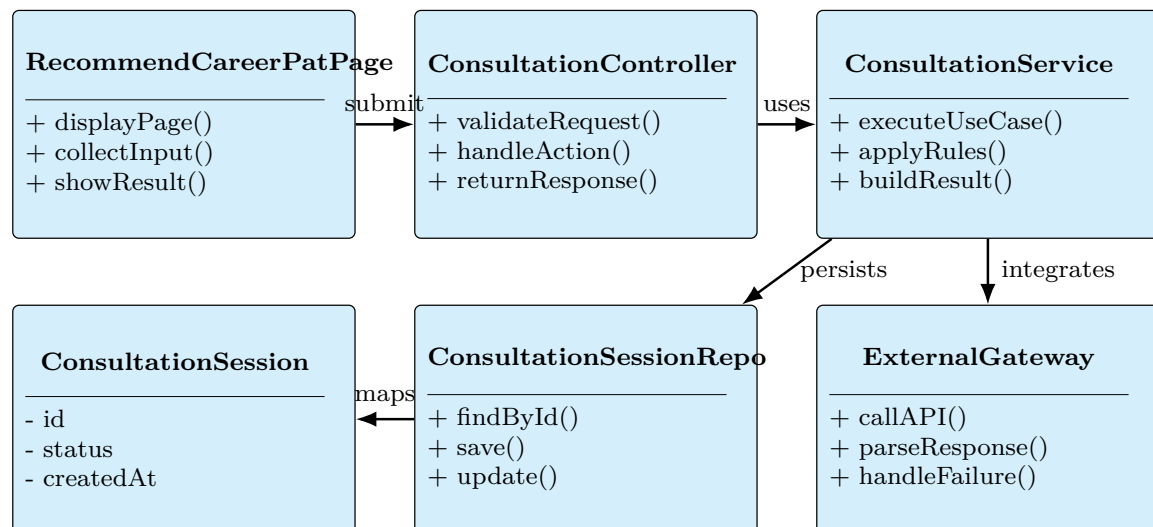


Figure IV.91: Class diagram for UC44 Recommend Career Path

3.10.3.8 Sequence Diagram

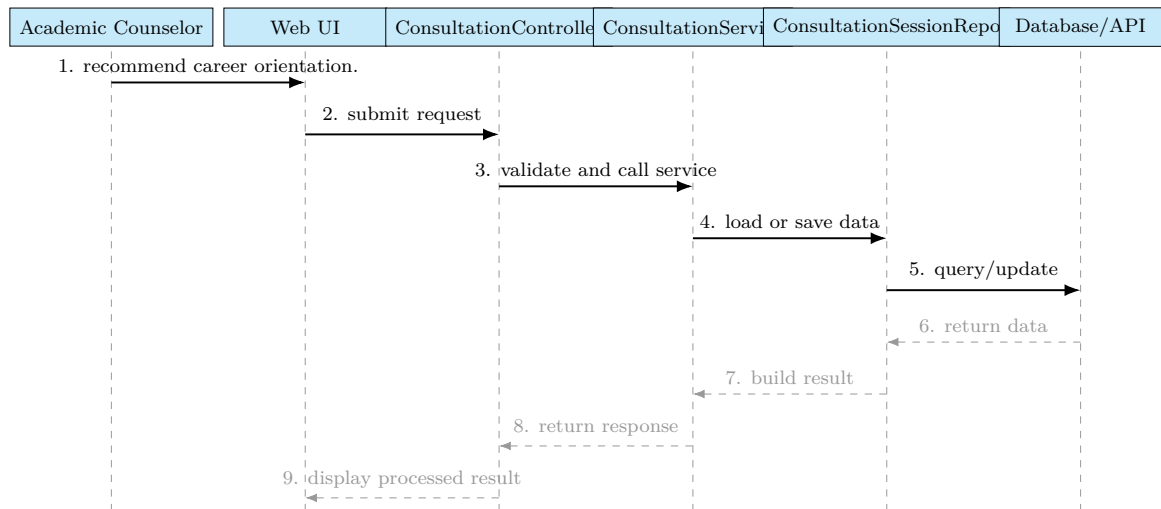


Figure IV.92: Sequence diagram for UC44 Recommend Career Path

3.10.4 UC45 Recommend Courses

3.10.4.1 Requirement Traceability

- **Use case ID:** UC45.
- **Use case name:** Recommend Courses.
- **Actor:** Academic Counselor.
- **Requirement description:** Recommend learning resources.
- **Related table(s):** ConsultationSessions, ConsultationFeedbacks, StaffProfiles.

3.10.4.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.10.4.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.10.4.4 Main Processing Flow

1. The actor opens `RecommendCoursesPage` and performs the action: recommend learning resources..

2. The frontend validates required fields, prepares the request DTO, and sends it to **ConsultationController**.
3. **ConsultationController** checks authentication, authorization, request format, and path parameters.
4. **ConsultationService** loads related data through **ConsultationSessionRepo** and applies business rules for Recommend Courses.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in **ConsultationSessions**, **ConsultationFeedbacks**, **StaffProfiles** and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.10.4.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.10.4.6 Participating Components

- **Boundary/UI – RecommendCoursesPage:** collects input and displays the final result for Recommend Courses.
- **Controller – ConsultationController:** receives REST requests and validates request DTOs.
- **Service – ConsultationService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – ConsultationSession:** represents the main persisted domain object.
- **Repository – ConsultationSessionRepo:** provides data access operations for the related table(s).

3.10.4.7 Class Diagram

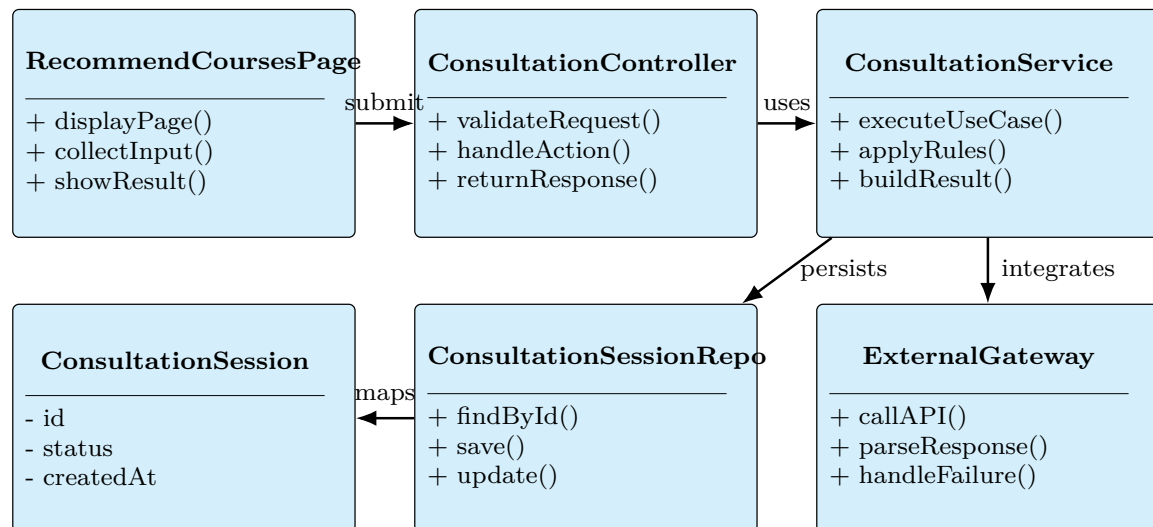


Figure IV.93: Class diagram for UC45 Recommend Courses

3.10.4.8 Sequence Diagram

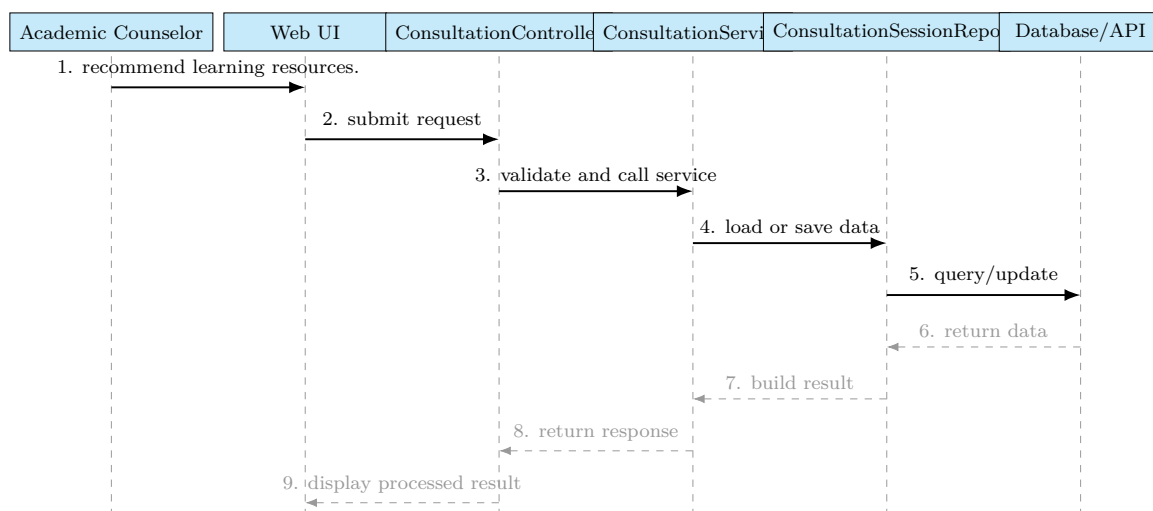


Figure IV.94: Sequence diagram for UC45 Recommend Courses

3.10.5 UC46 Track Student Progress

3.10.5.1 Requirement Traceability

- **Use case ID:** UC46.
- **Use case name:** Track Student Progress.
- **Actor:** Academic Counselor.
- **Requirement description:** Track learning progress.
- **Related table(s):** ConsultationSessions, ConsultationFeedbacks, StaffProfiles.

3.10.5.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.10.5.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.10.5.4 Main Processing Flow

1. The actor opens `TrackStudentProgrePage` and performs the action: track learning progress..
2. The frontend validates required fields, prepares the request DTO, and sends it to `ConsultationController`.
3. `ConsultationController` checks authentication, authorization, request format, and path parameters.
4. `ConsultationService` loads related data through `ConsultationSessionRepo` and applies business rules for Track Student Progress.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in `ConsultationSessions`, `ConsultationFeedbacks`, `StaffProfiles` and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.10.5.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.10.5.6 Participating Components

- **Boundary/UI – TrackStudentProgrePage:** collects input and displays the final result for Track Student Progress.
- **Controller – ConsultationController:** receives REST requests and validates request DTOs.
- **Service – ConsultationService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – ConsultationSession:** represents the main persisted domain object.
- **Repository – ConsultationSessionRepo:** provides data access operations for the related table(s).

3.10.5.7 Class Diagram

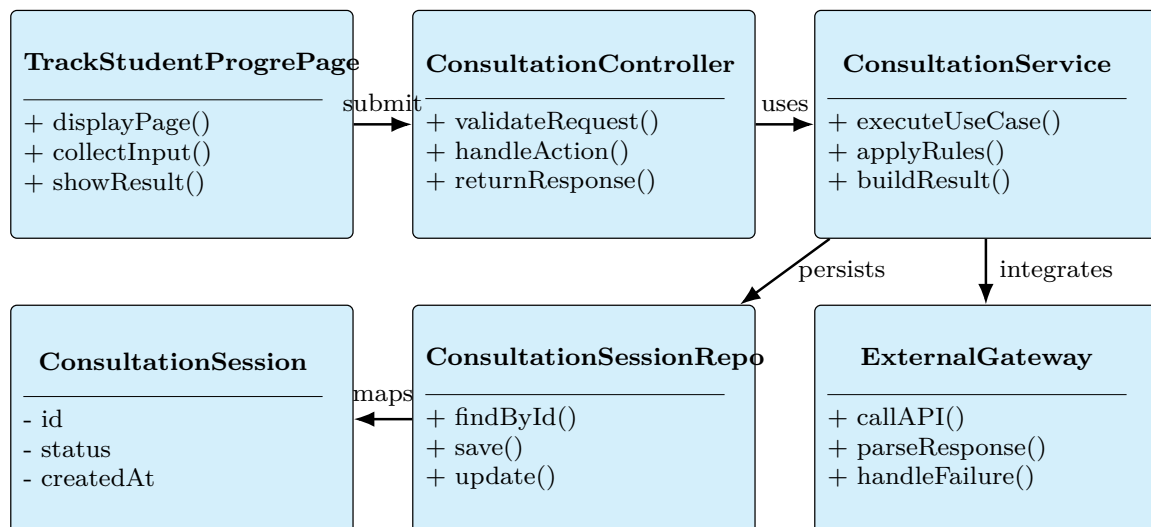


Figure IV.95: Class diagram for UC46 Track Student Progress

3.10.5.8 Sequence Diagram

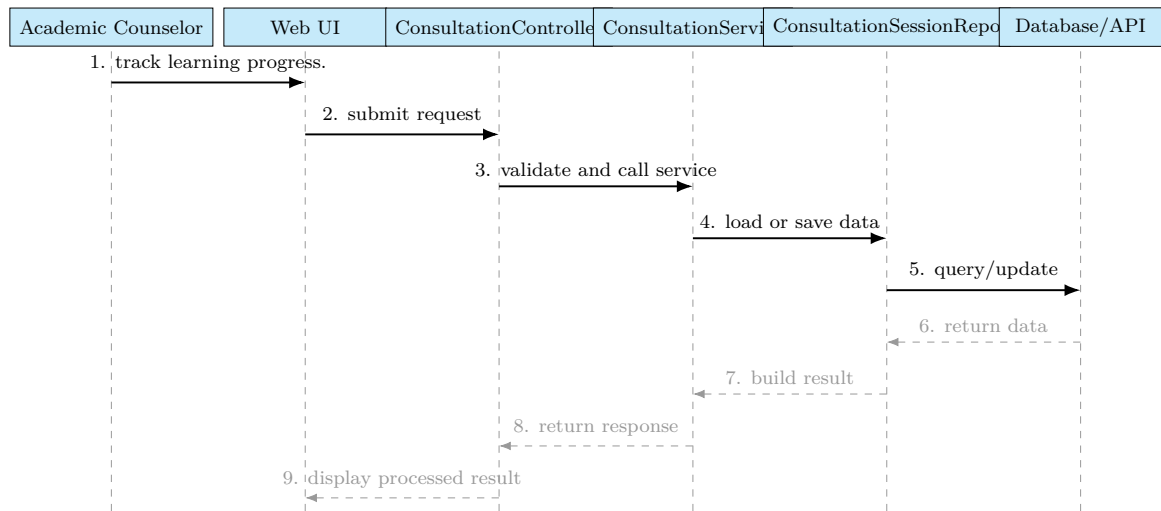


Figure IV.96: Sequence diagram for UC46 Track Student Progress

3.10.6 UC47 Comment Learning Plan

3.10.6.1 Requirement Traceability

- **Use case ID:** UC47.
- **Use case name:** Comment Learning Plan.
- **Actor:** Academic Counselor.
- **Requirement description:** Provide comments.
- **Related table(s):** ConsultationSessions, ConsultationFeedbacks, StaffProfiles.

3.10.6.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.10.6.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.10.6.4 Main Processing Flow

1. The actor opens `CommentLearningPlaPage` and performs the action: provide comments..

2. The frontend validates required fields, prepares the request DTO, and sends it to **ConsultationController**.
3. **ConsultationController** checks authentication, authorization, request format, and path parameters.
4. **ConsultationService** loads related data through **ConsultationSessionRepo** and applies business rules for Comment Learning Plan.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in **ConsultationSessions**, **ConsultationFeedbacks**, **StaffProfiles** and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.10.6.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.10.6.6 Participating Components

- **Boundary/UI – CommentLearningPlaPage:** collects input and displays the final result for Comment Learning Plan.
- **Controller – ConsultationController:** receives REST requests and validates request DTOs.
- **Service – ConsultationService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – ConsultationSession:** represents the main persisted domain object.
- **Repository – ConsultationSessionRepo:** provides data access operations for the related table(s).

3.10.6.7 Class Diagram

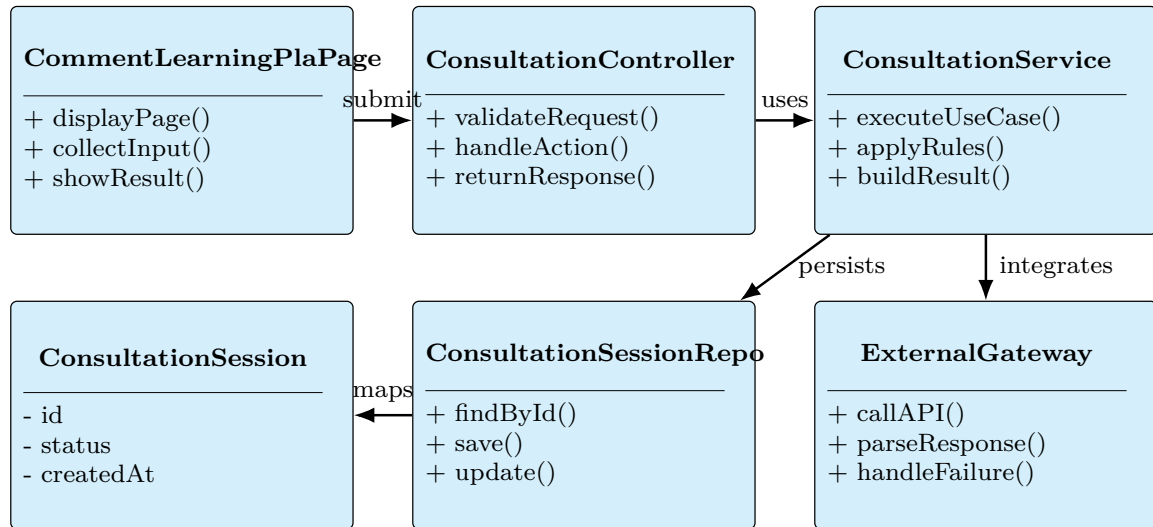


Figure IV.97: Class diagram for UC47 Comment Learning Plan

3.10.6.8 Sequence Diagram

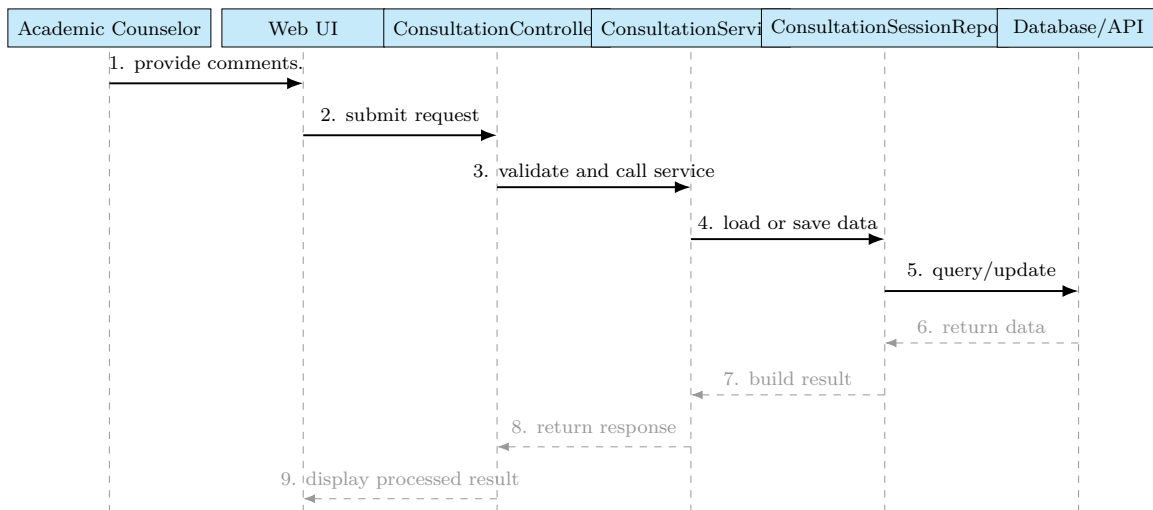


Figure IV.98: Sequence diagram for UC47 Comment Learning Plan

3.10.7 UC48 Schedule Consultation

3.10.7.1 Requirement Traceability

- **Use case ID:** UC48.
- **Use case name:** Schedule Consultation.
- **Actor:** Academic Counselor.
- **Requirement description:** Arrange consultation session.
- **Related table(s):** ConsultationSessions, ConsultationFeedbacks, StaffProfiles.

3.10.7.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.10.7.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.10.7.4 Main Processing Flow

1. The actor opens `ScheduleConsultatiPage` and performs the action: arrange consultation session..
2. The frontend validates required fields, prepares the request DTO, and sends it to `ConsultationController`.
3. `ConsultationController` checks authentication, authorization, request format, and path parameters.
4. `ConsultationService` loads related data through `ConsultationSessionRepo` and applies business rules for Schedule Consultation.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in `ConsultationSessions`, `ConsultationFeedbacks`, `StaffProfiles` and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.10.7.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.10.7.6 Participating Components

- **Boundary/UI – ScheduleConsultatiPage:** collects input and displays the final result for Schedule Consultation.
- **Controller – ConsultationController:** receives REST requests and validates request DTOs.
- **Service – ConsultationService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – ConsultationSession:** represents the main persisted domain object.
- **Repository – ConsultationSessionRepo:** provides data access operations for the related table(s).

3.10.7.7 Class Diagram

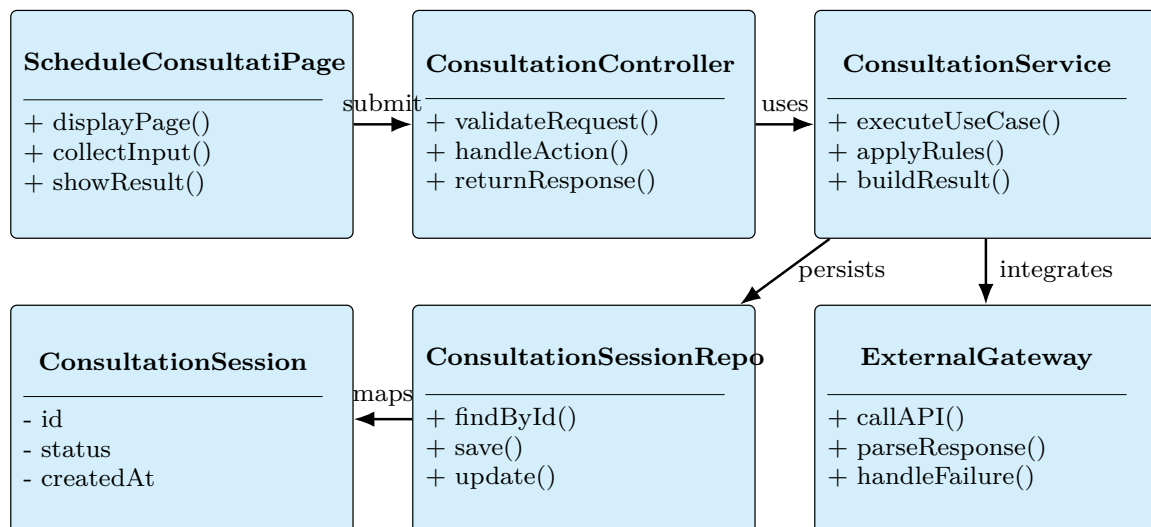


Figure IV.99: Class diagram for UC48 Schedule Consultation

3.10.7.8 Sequence Diagram

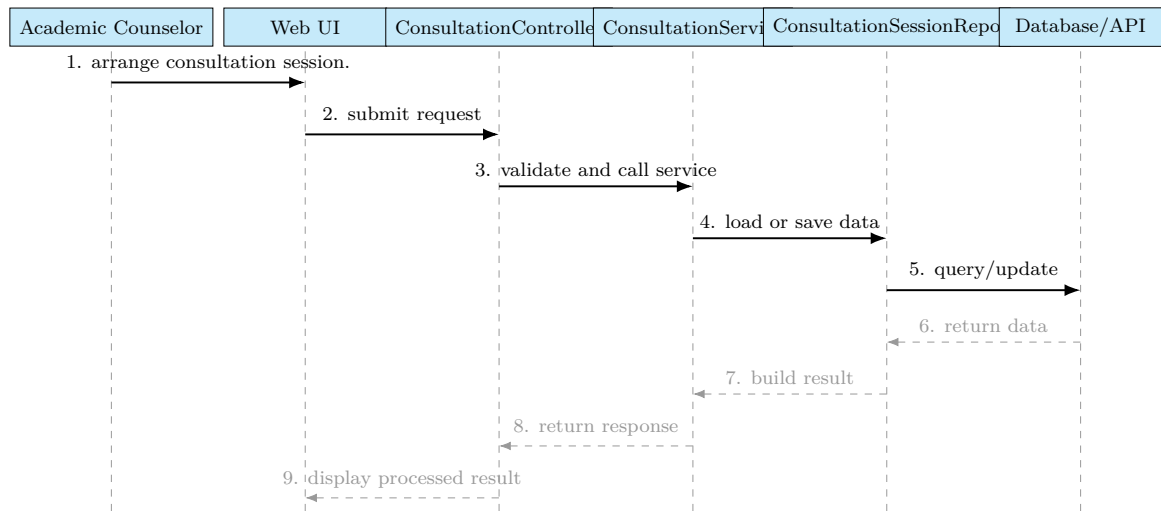


Figure IV.100: Sequence diagram for UC48 Schedule Consultation

3.10.8 UC49 View Consultation History

3.10.8.1 Requirement Traceability

- **Use case ID:** UC49.
- **Use case name:** View Consultation History.
- **Actor:** Academic Counselor.
- **Requirement description:** Review consultation history.
- **Related table(s):** ConsultationSessions, ConsultationFeedbacks, StaffProfiles.

3.10.8.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.10.8.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.10.8.4 Main Processing Flow

1. The actor opens `ViewConsultationHiPage` and performs the action: review consultation history..

2. The frontend validates required fields, prepares the request DTO, and sends it to `ConsultationController`.
3. `ConsultationController` checks authentication, authorization, request format, and path parameters.
4. `ConsultationService` loads related data through `ConsultationSessionRepo` and applies business rules for View Consultation History.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in `ConsultationSessions`, `ConsultationFeedbacks`, `StaffProfiles` and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.10.8.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.10.8.6 Participating Components

- **Boundary/UI – ViewConsultationHiPage:** collects input and displays the final result for View Consultation History.
- **Controller – ConsultationController:** receives REST requests and validates request DTOs.
- **Service – ConsultationService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – ConsultationSession:** represents the main persisted domain object.
- **Repository – ConsultationSessionRepo:** provides data access operations for the related table(s).

3.10.8.7 Class Diagram

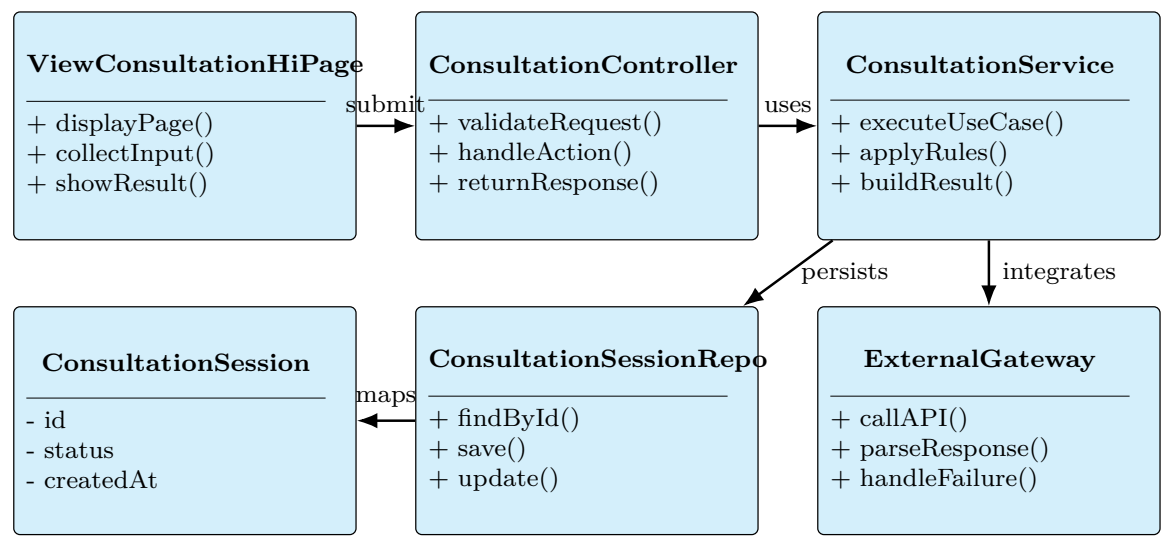


Figure IV.101: Class diagram for UC49 View Consultation History

3.10.8.8 Sequence Diagram

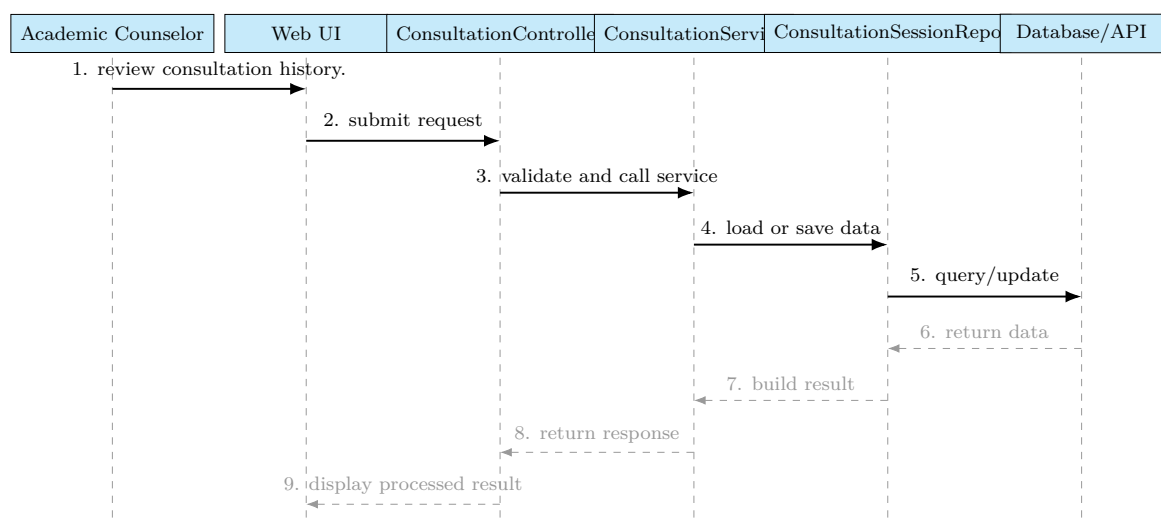


Figure IV.102: Sequence diagram for UC49 View Consultation History

3.11 Authentication and Account Access

This group contains use cases related to authentication and account access. The design uses the layered architecture introduced in the system design: React boundary pages, Spring Boot controllers, services, repositories, database entities, and external gateways when required.

3.11.1 UC50 Logout

3.11.1.1 Requirement Traceability

- **Use case ID:** UC50.
- **Use case name:** Logout.
- **Actor:** Academic Counselor.
- **Requirement description:** Logout.
- **Related table(s):** Users.

3.11.1.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in **Users.Role** must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.11.1.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.11.1.4 Main Processing Flow

1. The actor opens **LogoutPage** and performs the action: logout..
2. The frontend validates required fields, prepares the request DTO, and sends it to **AuthController**.
3. **AuthController** checks authentication, authorization, request format, and path parameters.
4. **AuthService** loads related data through **UserRepository** and applies business rules for Logout.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in **Users** and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.11.1.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.11.1.6 Participating Components

- **Boundary/UI – LogoutPage:** collects input and displays the final result for Logout.
- **Controller – AuthController:** receives REST requests and validates request DTOs.
- **Service – AuthService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – User:** represents the main persisted domain object.
- **Repository – UserRepository:** provides data access operations for the related table(s).

3.11.1.7 Class Diagram

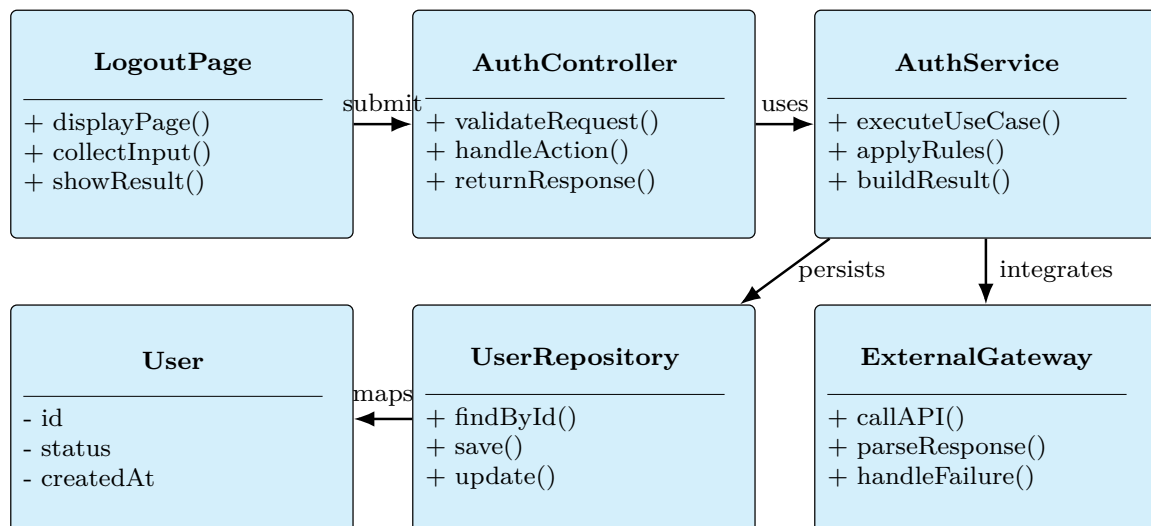


Figure IV.103: Class diagram for UC50 Logout

3.11.1.8 Sequence Diagram

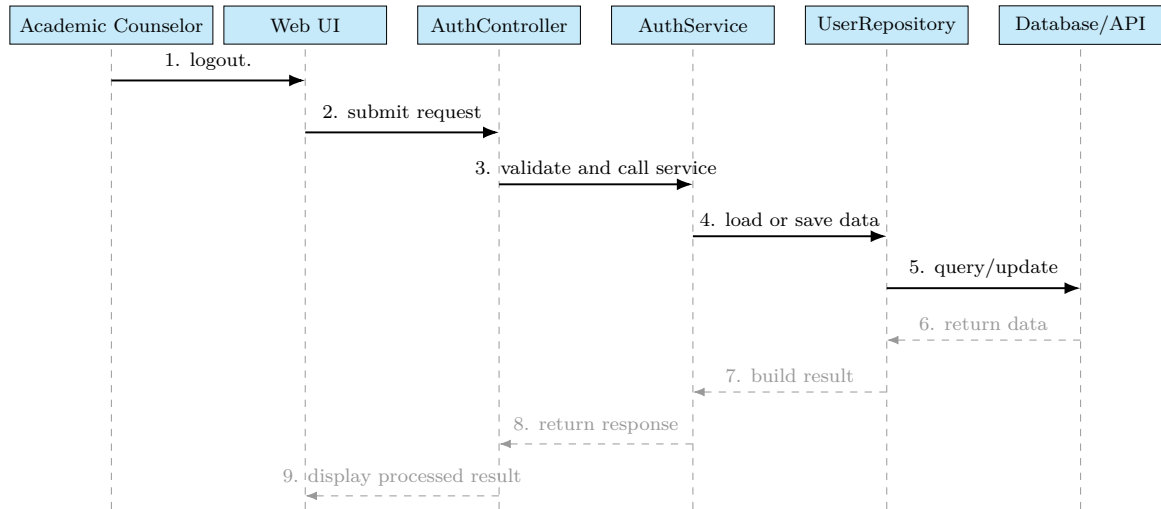


Figure IV.104: Sequence diagram for UC50 Logout

3.11.2 UC51 Login

3.11.2.1 Requirement Traceability

- **Use case ID:** UC51.
- **Use case name:** Login.
- **Actor:** Industry Mentor.
- **Requirement description:** Login to mentor dashboard.
- **Related table(s):** Users.

3.11.2.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in **Users.Role** must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.11.2.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.11.2.4 Main Processing Flow

1. The actor opens **LoginPage** and performs the action: login to mentor dashboard..

2. The frontend validates required fields, prepares the request DTO, and sends it to **AuthController**.
3. **AuthController** checks authentication, authorization, request format, and path parameters.
4. **AuthService** loads related data through **UserRepository** and applies business rules for Login.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in **Users** and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.11.2.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.11.2.6 Participating Components

- **Boundary/UI – LoginPage:** collects input and displays the final result for Login.
- **Controller – AuthController:** receives REST requests and validates request DTOs.
- **Service – AuthService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – User:** represents the main persisted domain object.
- **Repository – UserRepository:** provides data access operations for the related table(s).

3.11.2.7 Class Diagram

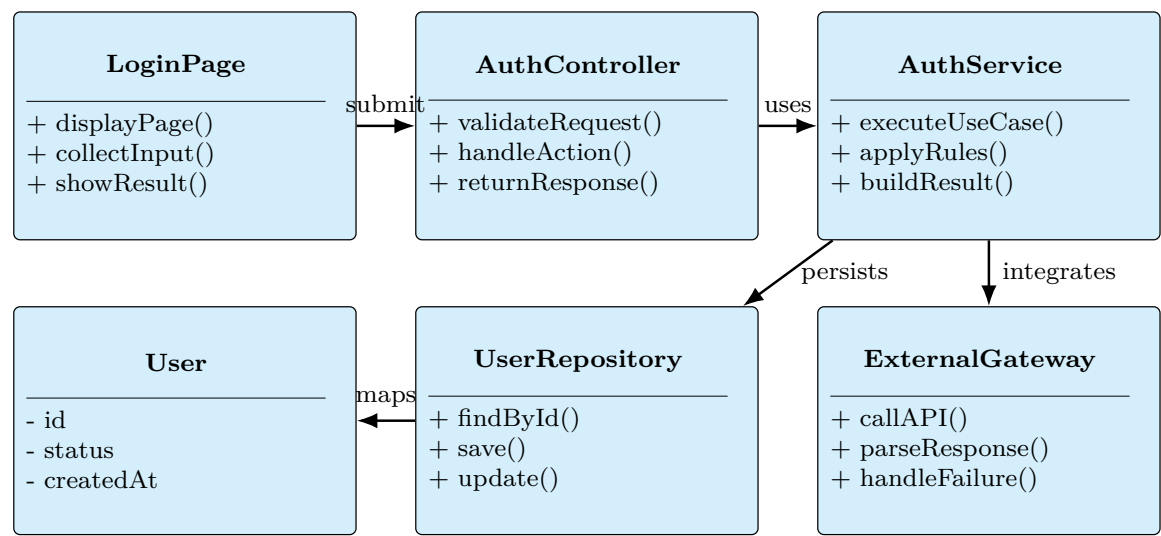


Figure IV.105: Class diagram for UC51 Login

3.11.2.8 Sequence Diagram

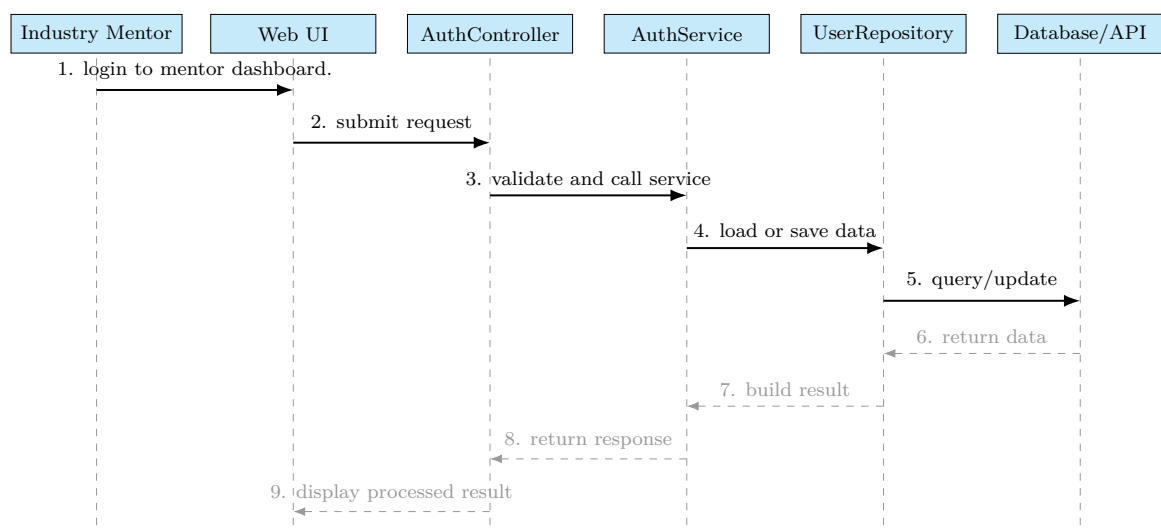


Figure IV.106: Sequence diagram for UC51 Login

3.12 E-Portfolio and Mentor Review

This group contains use cases related to e-portfolio and mentor review. The design uses the layered architecture introduced in the system design: React boundary pages, Spring Boot controllers, services, repositories, database entities, and external gateways when required.

3.12.1 UC52 View Student Portfolio

3.12.1.1 Requirement Traceability

- **Use case ID:** UC52.
- **Use case name:** View Student Portfolio.
- **Actor:** Industry Mentor.
- **Requirement description:** Review portfolio.
- **Related table(s):** Portfolios, GitHubRepositories, ConsultationFeedbacks.

3.12.1.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.12.1.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.12.1.4 Main Processing Flow

1. The actor opens `ViewStudentPortfolPage` and performs the action: review portfolio..
2. The frontend validates required fields, prepares the request DTO, and sends it to `PortfolioController`.
3. `PortfolioController` checks authentication, authorization, request format, and path parameters.
4. `PortfolioService` loads related data through `PortfolioRepository` and applies business rules for View Student Portfolio.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in `Portfolios`, `GitHubRepositories`, `ConsultationFeedbacks` and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.12.1.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.12.1.6 Participating Components

- **Boundary/UI – ViewStudentPortfolPage:** collects input and displays the final result for View Student Portfolio.
- **Controller – PortfolioController:** receives REST requests and validates request DTOs.
- **Service – PortfolioService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – Portfolio:** represents the main persisted domain object.
- **Repository – PortfolioRepository:** provides data access operations for the related table(s).

3.12.1.7 Class Diagram

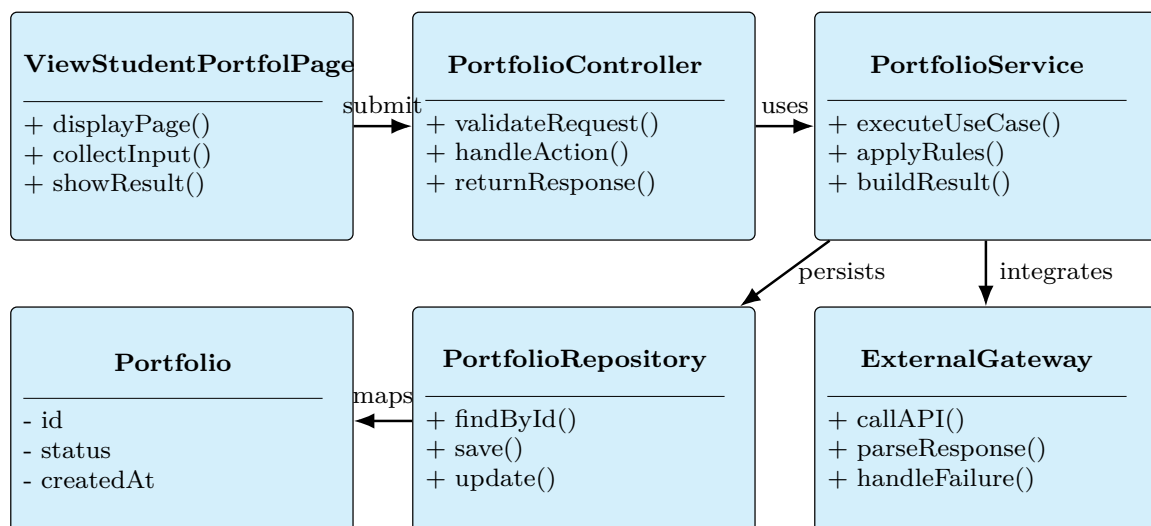


Figure IV.107: Class diagram for UC52 View Student Portfolio

3.12.1.8 Sequence Diagram

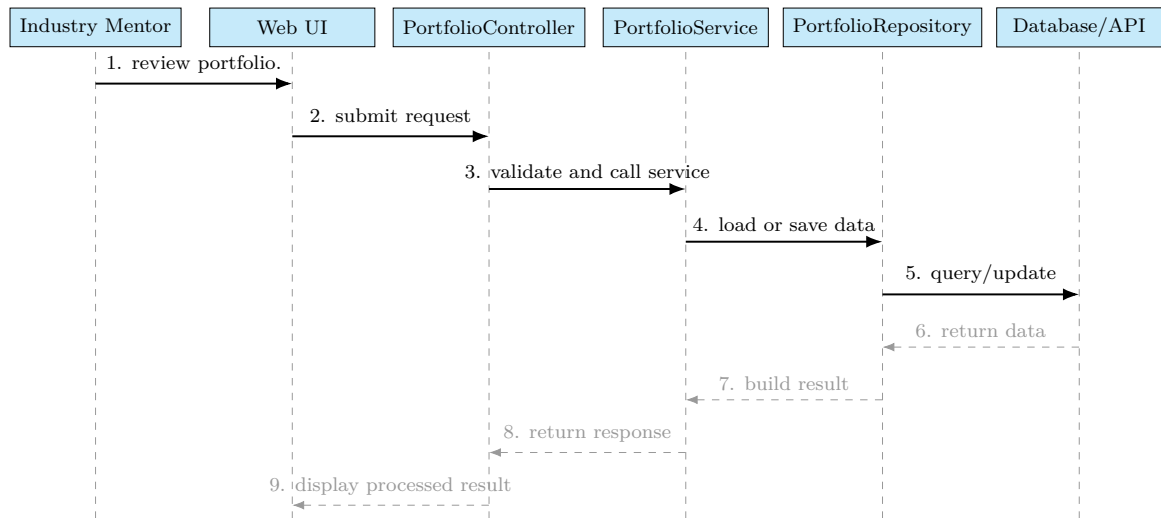


Figure IV.108: Sequence diagram for UC52 View Student Portfolio

3.12.2 UC53 Review Projects

3.12.2.1 Requirement Traceability

- **Use case ID:** UC53.
- **Use case name:** Review Projects.
- **Actor:** Industry Mentor.
- **Requirement description:** Review projects.
- **Related table(s):** Portfolios, GitHubRepositories, ConsultationFeedbacks.

3.12.2.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.12.2.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.12.2.4 Main Processing Flow

1. The actor opens `ReviewProjectsPage` and performs the action: review projects..

2. The frontend validates required fields, prepares the request DTO, and sends it to **PortfolioController**.
3. **PortfolioController** checks authentication, authorization, request format, and path parameters.
4. **PortfolioService** loads related data through **PortfolioRepository** and applies business rules for Review Projects.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in **Portfolios**, **GitHubRepositories**, **ConsultationFeedbacks** and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.12.2.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.12.2.6 Participating Components

- **Boundary/UI – ReviewProjectsPage:** collects input and displays the final result for Review Projects.
- **Controller – PortfolioController:** receives REST requests and validates request DTOs.
- **Service – PortfolioService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – Portfolio:** represents the main persisted domain object.
- **Repository – PortfolioRepository:** provides data access operations for the related table(s).

3.12.2.7 Class Diagram

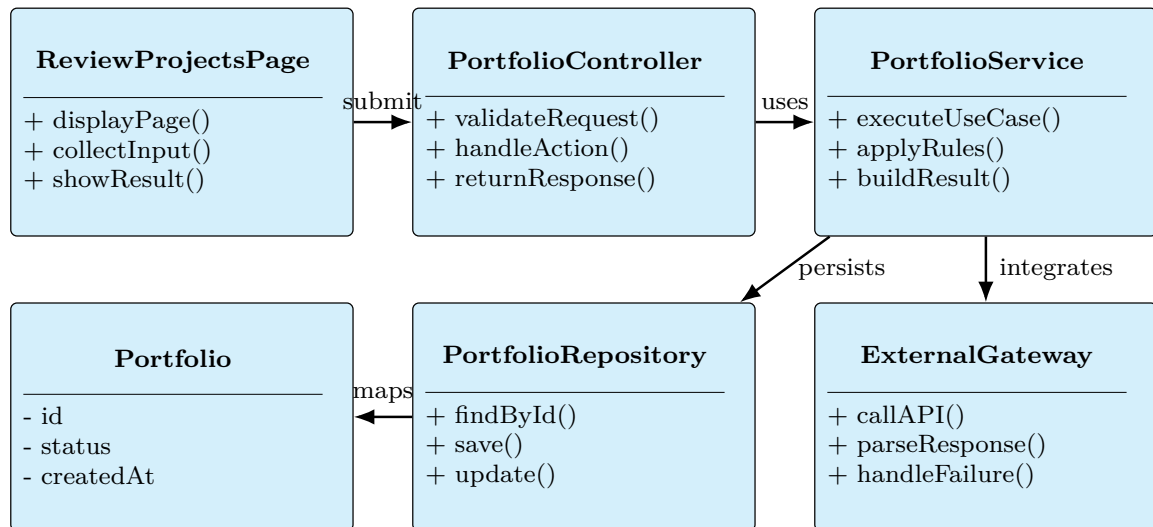


Figure IV.109: Class diagram for UC53 Review Projects

3.12.2.8 Sequence Diagram

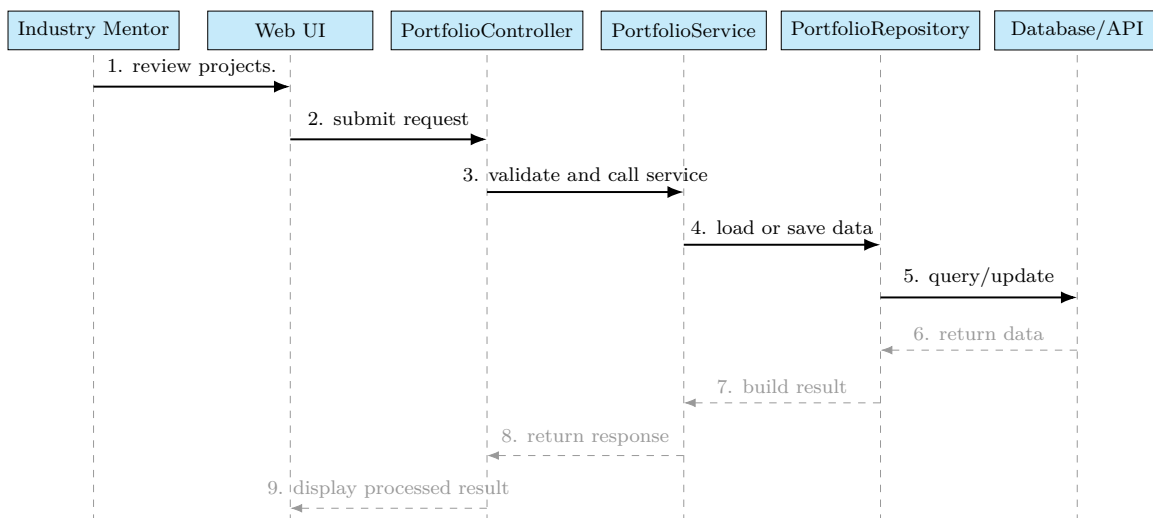


Figure IV.110: Sequence diagram for UC53 Review Projects

3.12.3 UC54 Recommend Technologies

3.12.3.1 Requirement Traceability

- **Use case ID:** UC54.
- **Use case name:** Recommend Technologies.
- **Actor:** Industry Mentor.
- **Requirement description:** Suggest technologies.
- **Related table(s):** Portfolios, GitHubRepositories, ConsultationFeedbacks.

3.12.3.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.12.3.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.12.3.4 Main Processing Flow

1. The actor opens `RecommendTechnologPage` and performs the action: suggest technologies..
2. The frontend validates required fields, prepares the request DTO, and sends it to `PortfolioController`.
3. `PortfolioController` checks authentication, authorization, request format, and path parameters.
4. `PortfolioService` loads related data through `PortfolioRepository` and applies business rules for Recommend Technologies.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in `Portfolios`, `GitHubRepositories`, `ConsultationFeedbacks` and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.12.3.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.12.3.6 Participating Components

- **Boundary/UI – RecommendTechnologyPage:** collects input and displays the final result for Recommend Technologies.
- **Controller – PortfolioController:** receives REST requests and validates request DTOs.
- **Service – PortfolioService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – Portfolio:** represents the main persisted domain object.
- **Repository – PortfolioRepository:** provides data access operations for the related table(s).

3.12.3.7 Class Diagram

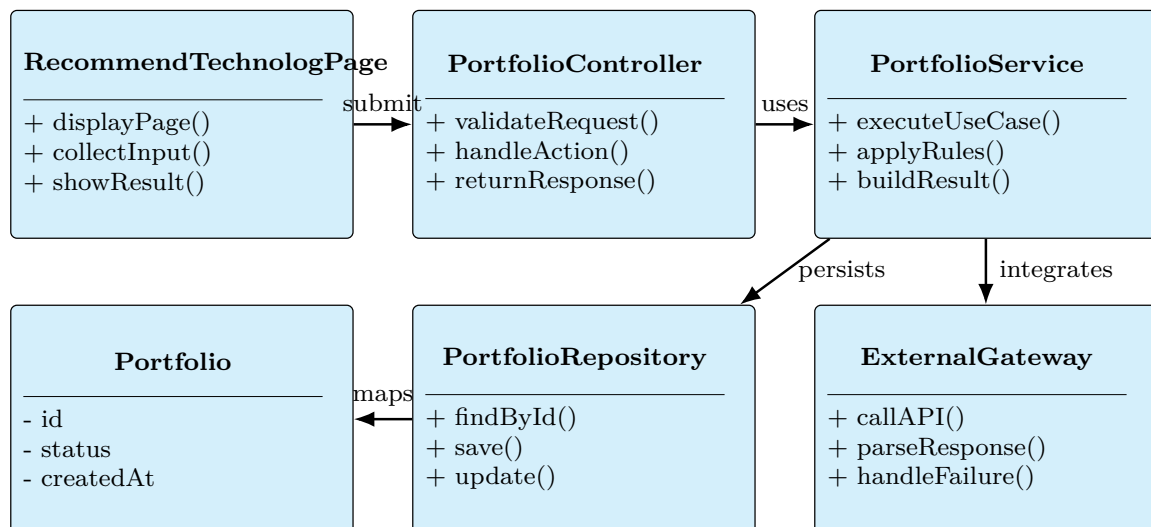


Figure IV.111: Class diagram for UC54 Recommend Technologies

3.12.3.8 Sequence Diagram

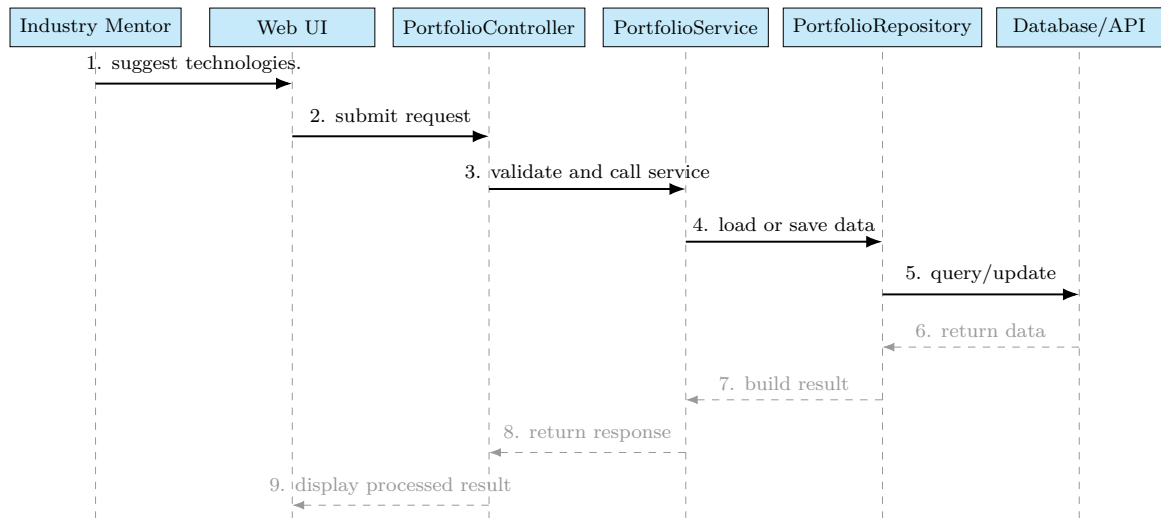


Figure IV.112: Sequence diagram for UC54 Recommend Technologies

3.12.4 UC55 Give Technical Advice

3.12.4.1 Requirement Traceability

- **Use case ID:** UC55.
- **Use case name:** Give Technical Advice.
- **Actor:** Industry Mentor.
- **Requirement description:** Provide technical guidance.
- **Related table(s):** Portfolios, GitHubRepositories, ConsultationFeedbacks.

3.12.4.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.12.4.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.12.4.4 Main Processing Flow

1. The actor opens `GiveTechnicalAdvicPage` and performs the action: provide technical guidance..

2. The frontend validates required fields, prepares the request DTO, and sends it to **PortfolioController**.
3. **PortfolioController** checks authentication, authorization, request format, and path parameters.
4. **PortfolioService** loads related data through **PortfolioRepository** and applies business rules for Give Technical Advice.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in **Portfolios**, **GitHubRepositories**, **ConsultationFeedbacks** and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.12.4.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.12.4.6 Participating Components

- **Boundary/UI – GiveTechnicalAdvicePage:** collects input and displays the final result for Give Technical Advice.
- **Controller – PortfolioController:** receives REST requests and validates request DTOs.
- **Service – PortfolioService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – Portfolio:** represents the main persisted domain object.
- **Repository – PortfolioRepository:** provides data access operations for the related table(s).

3.12.4.7 Class Diagram

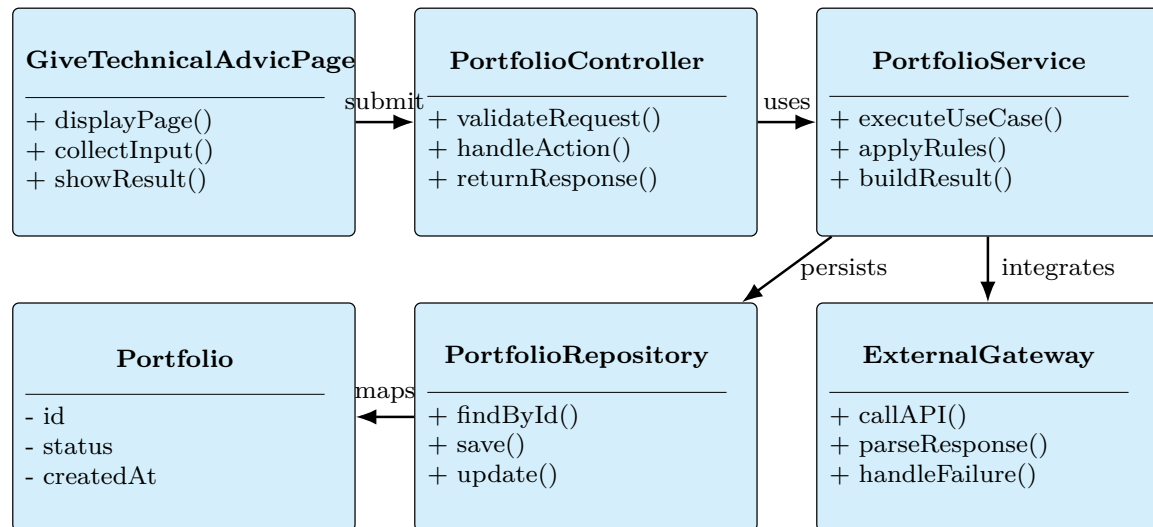


Figure IV.113: Class diagram for UC55 Give Technical Advice

3.12.4.8 Sequence Diagram

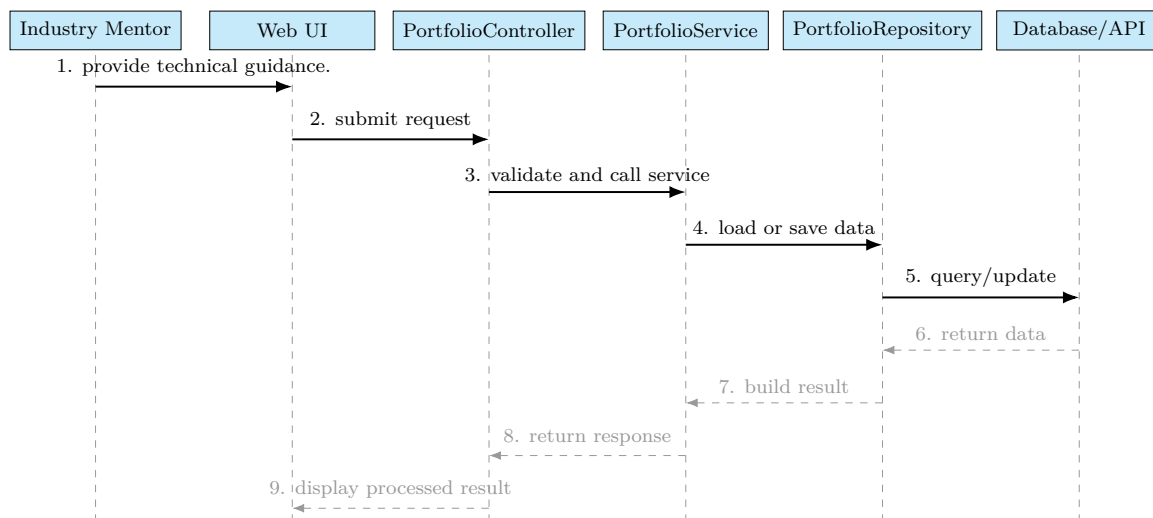


Figure IV.114: Sequence diagram for UC55 Give Technical Advice

3.12.5 UC56 Evaluate Progress

3.12.5.1 Requirement Traceability

- **Use case ID:** UC56.
- **Use case name:** Evaluate Progress.
- **Actor:** Industry Mentor.
- **Requirement description:** Evaluate learning outcome.
- **Related table(s):** Portfolios, GitHubRepositories, ConsultationFeedbacks.

3.12.5.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.12.5.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.12.5.4 Main Processing Flow

1. The actor opens `EvaluateProgressPage` and performs the action: evaluate learning outcome..
2. The frontend validates required fields, prepares the request DTO, and sends it to `PortfolioController`.
3. `PortfolioController` checks authentication, authorization, request format, and path parameters.
4. `PortfolioService` loads related data through `PortfolioRepository` and applies business rules for Evaluate Progress.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in `Portfolios`, `GitHubRepositories`, `ConsultationFeedbacks` and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.12.5.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.12.5.6 Participating Components

- **Boundary/UI – EvaluateProgressPage:** collects input and displays the final result for Evaluate Progress.
- **Controller – PortfolioController:** receives REST requests and validates request DTOs.
- **Service – PortfolioService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – Portfolio:** represents the main persisted domain object.
- **Repository – PortfolioRepository:** provides data access operations for the related table(s).

3.12.5.7 Class Diagram

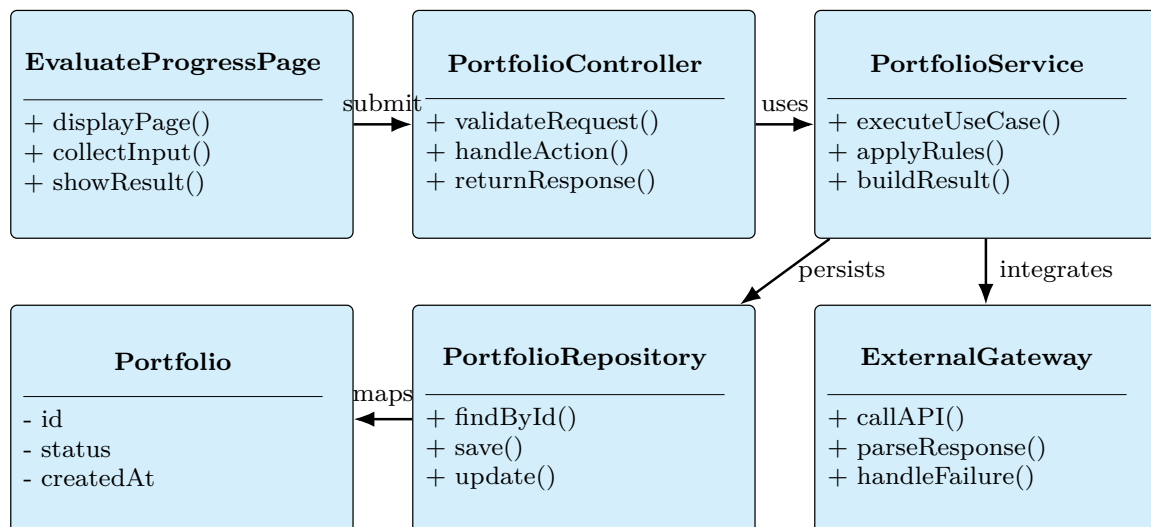


Figure IV.115: Class diagram for UC56 Evaluate Progress

3.12.5.8 Sequence Diagram

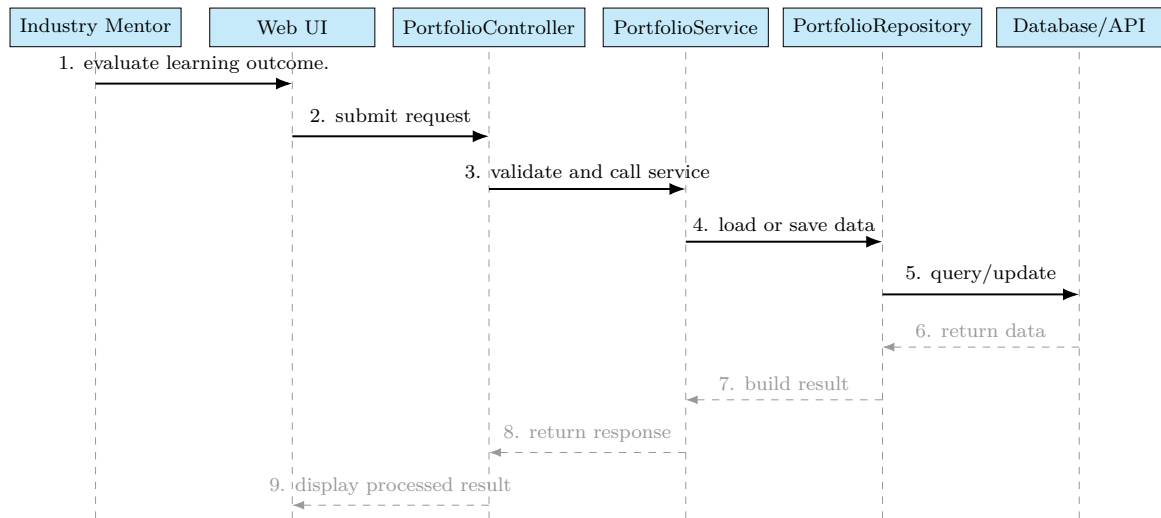


Figure IV.116: Sequence diagram for UC56 Evaluate Progress

3.12.6 UC57 Comment Projects

3.12.6.1 Requirement Traceability

- **Use case ID:** UC57.
- **Use case name:** Comment Projects.
- **Actor:** Industry Mentor.
- **Requirement description:** Comment projects.
- **Related table(s):** Portfolios, GitHubRepositories, ConsultationFeedbacks.

3.12.6.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.12.6.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.12.6.4 Main Processing Flow

1. The actor opens `CommentProjectsPage` and performs the action: comment projects..

2. The frontend validates required fields, prepares the request DTO, and sends it to **PortfolioController**.
3. **PortfolioController** checks authentication, authorization, request format, and path parameters.
4. **PortfolioService** loads related data through **PortfolioRepository** and applies business rules for Comment Projects.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in **Portfolios**, **GitHubRepositories**, **ConsultationFeedbacks** and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.12.6.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.12.6.6 Participating Components

- **Boundary/UI – CommentProjectsPage:** collects input and displays the final result for Comment Projects.
- **Controller – PortfolioController:** receives REST requests and validates request DTOs.
- **Service – PortfolioService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – Portfolio:** represents the main persisted domain object.
- **Repository – PortfolioRepository:** provides data access operations for the related table(s).

3.12.6.7 Class Diagram

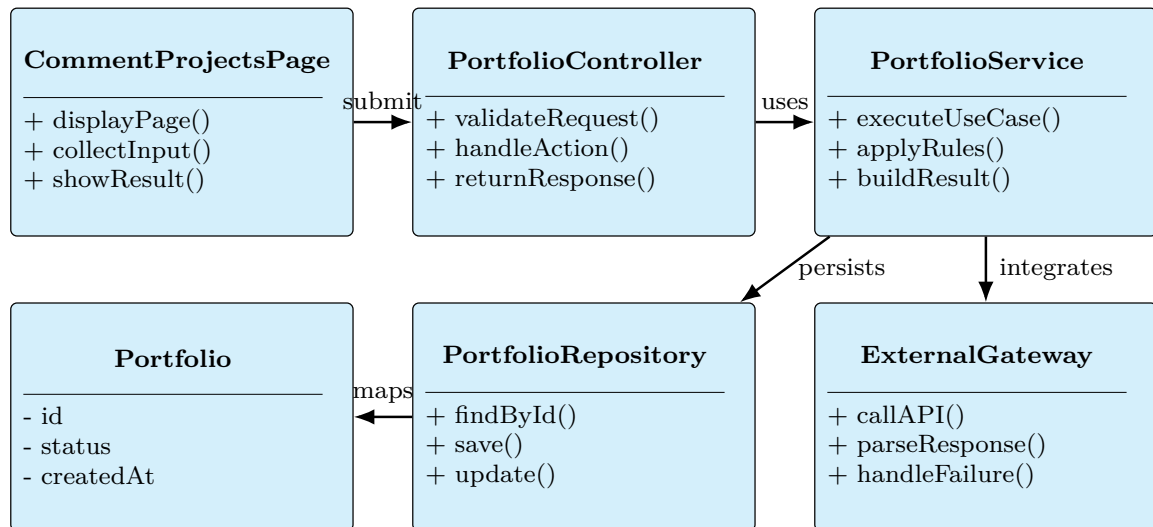


Figure IV.117: Class diagram for UC57 Comment Projects

3.12.6.8 Sequence Diagram

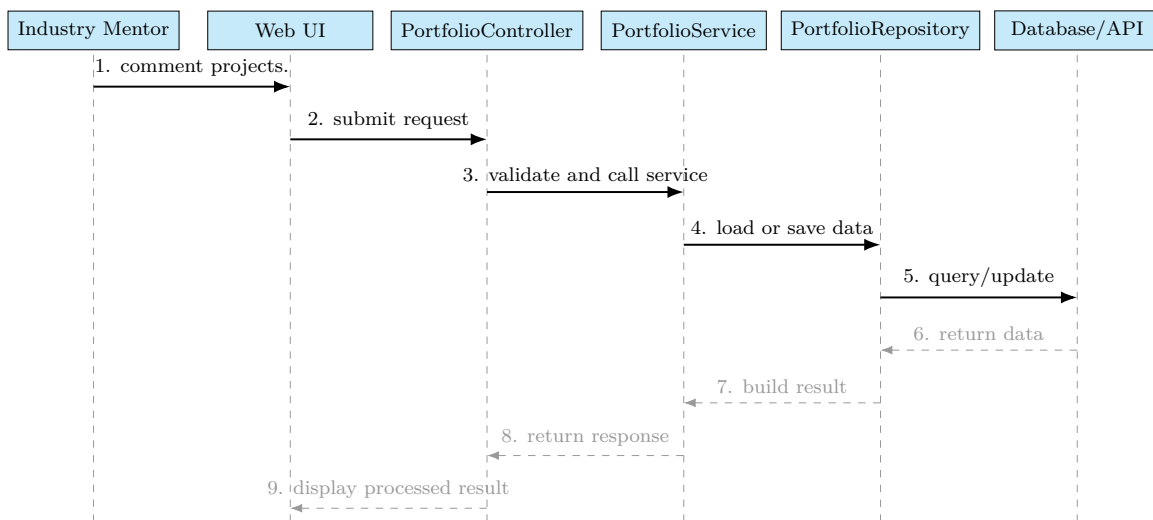


Figure IV.118: Sequence diagram for UC57 Comment Projects

3.12.7 UC58 Approve Portfolio

3.12.7.1 Requirement Traceability

- **Use case ID:** UC58.
- **Use case name:** Approve Portfolio.
- **Actor:** Industry Mentor.
- **Requirement description:** Approve portfolio quality.
- **Related table(s):** Portfolios, GitHubRepositories, ConsultationFeedbacks.

3.12.7.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.12.7.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.12.7.4 Main Processing Flow

1. The actor opens `ApprovePortfolioPage` and performs the action: approve portfolio quality..
2. The frontend validates required fields, prepares the request DTO, and sends it to `PortfolioController`.
3. `PortfolioController` checks authentication, authorization, request format, and path parameters.
4. `PortfolioService` loads related data through `PortfolioRepository` and applies business rules for Approve Portfolio.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in `Portfolios`, `GitHubRepositories`, `ConsultationFeedbacks` and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.12.7.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.12.7.6 Participating Components

- **Boundary/UI – ApprovePortfolioPage:** collects input and displays the final result for Approve Portfolio.
- **Controller – PortfolioController:** receives REST requests and validates request DTOs.
- **Service – PortfolioService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – Portfolio:** represents the main persisted domain object.
- **Repository – PortfolioRepository:** provides data access operations for the related table(s).

3.12.7.7 Class Diagram

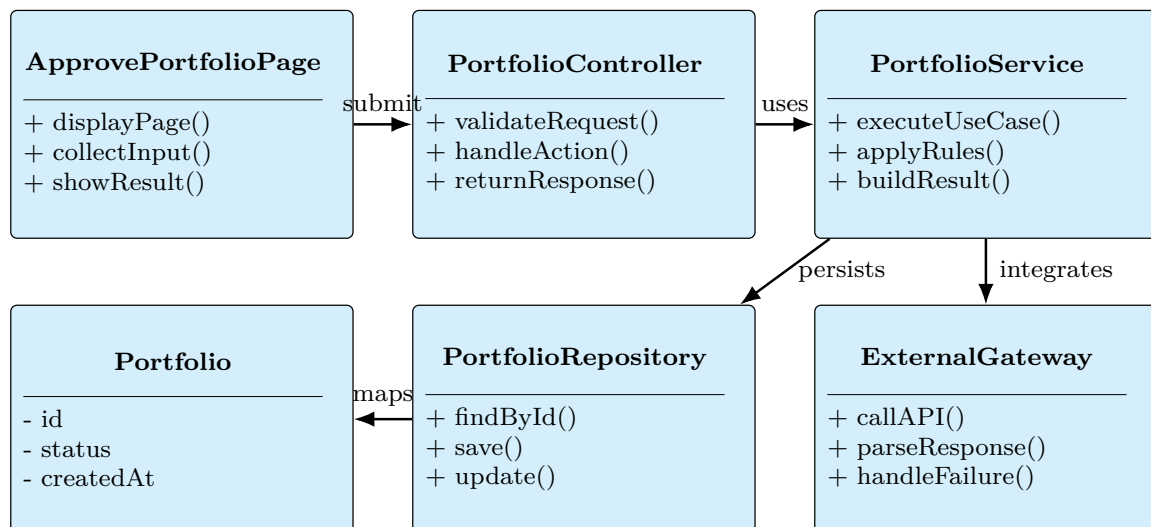


Figure IV.119: Class diagram for UC58 Approve Portfolio

3.12.7.8 Sequence Diagram

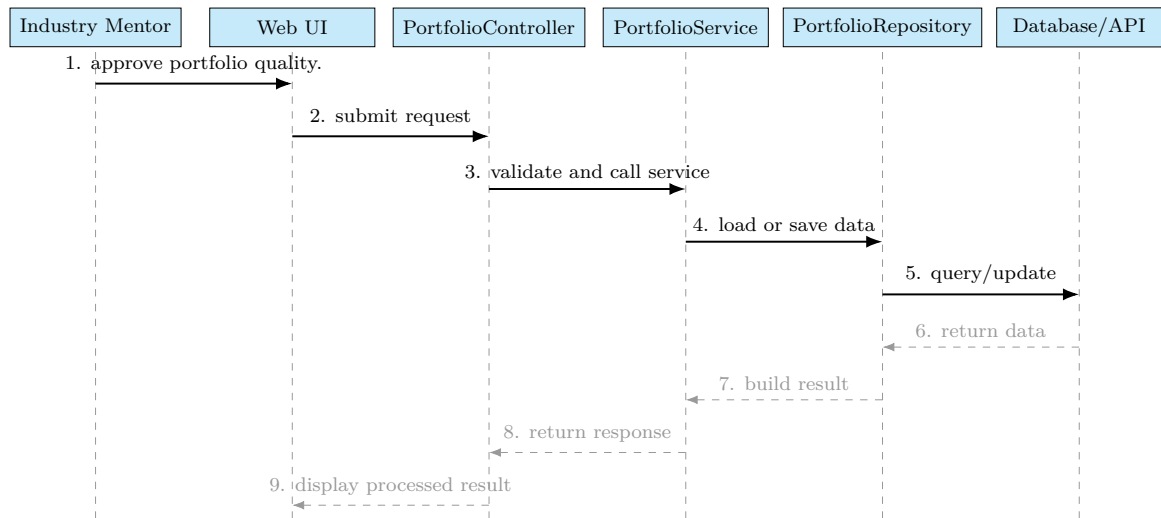


Figure IV.120: Sequence diagram for UC58 Approve Portfolio

3.12.8 UC59 Suggest Improvements

3.12.8.1 Requirement Traceability

- **Use case ID:** UC59.
- **Use case name:** Suggest Improvements.
- **Actor:** Industry Mentor.
- **Requirement description:** Suggest improvements.
- **Related table(s):** Portfolios, GitHubRepositories, ConsultationFeedbacks.

3.12.8.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.12.8.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.12.8.4 Main Processing Flow

1. The actor opens `SuggestImprovementPage` and performs the action: suggest improvements..

2. The frontend validates required fields, prepares the request DTO, and sends it to `PortfolioController`.
3. `PortfolioController` checks authentication, authorization, request format, and path parameters.
4. `PortfolioService` loads related data through `PortfolioRepository` and applies business rules for Suggest Improvements.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in `Portfolios`, `GitHubRepositories`, `ConsultationFeedbacks` and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.12.8.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.12.8.6 Participating Components

- **Boundary/UI – `SuggestImprovementPage`:** collects input and displays the final result for Suggest Improvements.
- **Controller – `PortfolioController`:** receives REST requests and validates request DTOs.
- **Service – `PortfolioService`:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – `Portfolio`:** represents the main persisted domain object.
- **Repository – `PortfolioRepository`:** provides data access operations for the related table(s).

3.12.8.7 Class Diagram

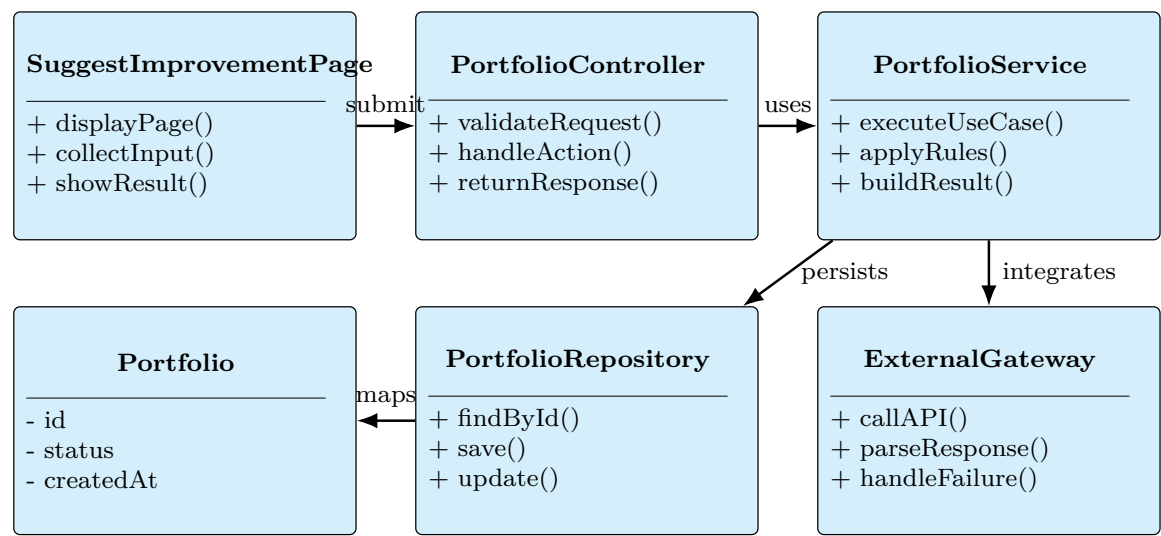


Figure IV.121: Class diagram for UC59 Suggest Improvements

3.12.8.8 Sequence Diagram

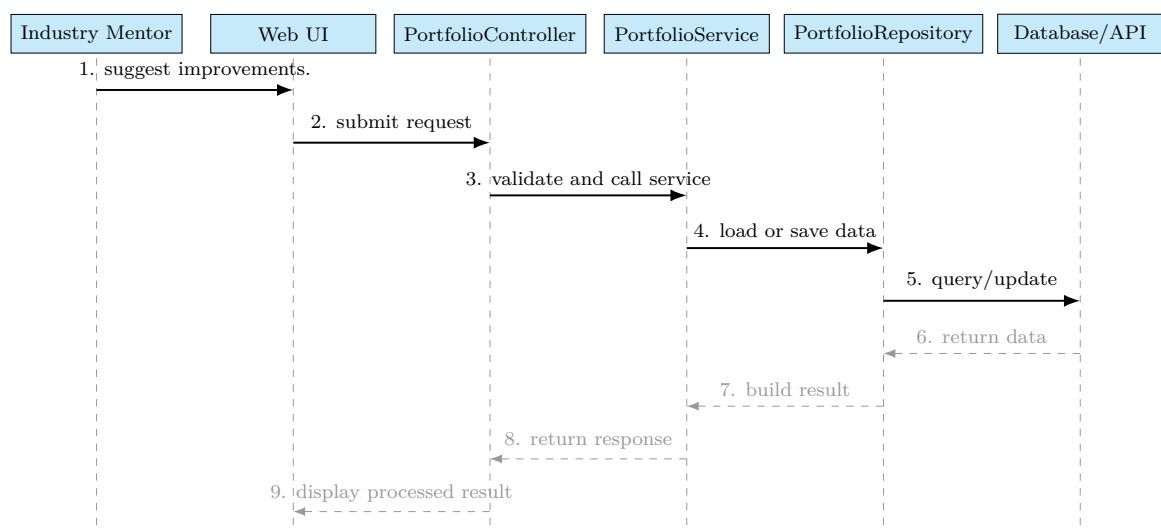


Figure IV.122: Sequence diagram for UC59 Suggest Improvements

3.13 Authentication and Account Access

This group contains use cases related to authentication and account access. The design uses the layered architecture introduced in the system design: React boundary pages, Spring Boot controllers, services, repositories, database entities, and external gateways when required.

3.13.1 UC60 Logout

3.13.1.1 Requirement Traceability

- **Use case ID:** UC60.
- **Use case name:** Logout.
- **Actor:** Industry Mentor.
- **Requirement description:** Logout.
- **Related table(s):** Users.

3.13.1.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in **Users.Role** must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.13.1.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.13.1.4 Main Processing Flow

1. The actor opens **LogoutPage** and performs the action: logout..
2. The frontend validates required fields, prepares the request DTO, and sends it to **AuthController**.
3. **AuthController** checks authentication, authorization, request format, and path parameters.
4. **AuthService** loads related data through **UserRepository** and applies business rules for Logout.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in **Users** and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.13.1.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.13.1.6 Participating Components

- **Boundary/UI – LogoutPage:** collects input and displays the final result for Logout.
- **Controller – AuthController:** receives REST requests and validates request DTOs.
- **Service – AuthService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – User:** represents the main persisted domain object.
- **Repository – UserRepository:** provides data access operations for the related table(s).

3.13.1.7 Class Diagram

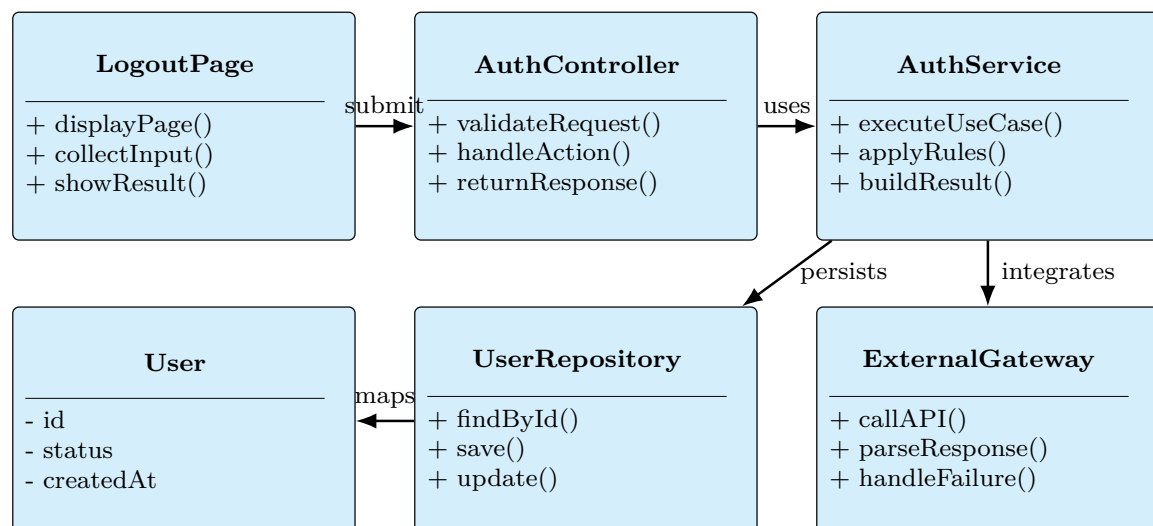


Figure IV.123: Class diagram for UC60 Logout

3.13.1.8 Sequence Diagram

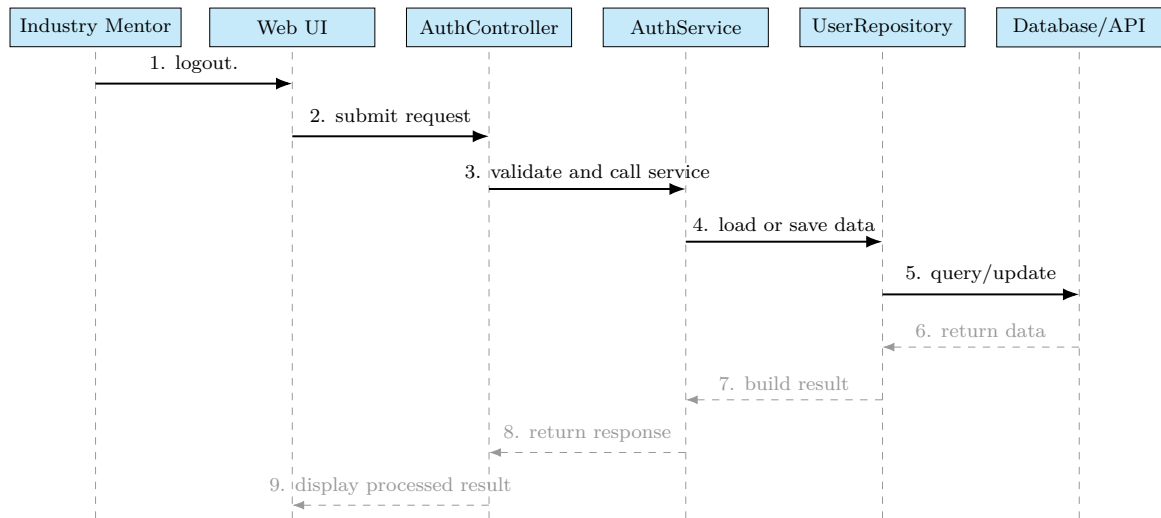


Figure IV.124: Sequence diagram for UC60 Logout

3.13.2 UC61 Login

3.13.2.1 Requirement Traceability

- **Use case ID:** UC61.
- **Use case name:** Login.
- **Actor:** Administrator.
- **Requirement description:** Login administrator.
- **Related table(s):** Users.

3.13.2.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.13.2.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.13.2.4 Main Processing Flow

1. The actor opens `LoginPage` and performs the action: login administrator..

2. The frontend validates required fields, prepares the request DTO, and sends it to **AuthController**.
3. **AuthController** checks authentication, authorization, request format, and path parameters.
4. **AuthService** loads related data through **UserRepository** and applies business rules for Login.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in **Users** and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.13.2.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.13.2.6 Participating Components

- **Boundary/UI – LoginPage:** collects input and displays the final result for Login.
- **Controller – AuthController:** receives REST requests and validates request DTOs.
- **Service – AuthService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – User:** represents the main persisted domain object.
- **Repository – UserRepository:** provides data access operations for the related table(s).

3.13.2.7 Class Diagram

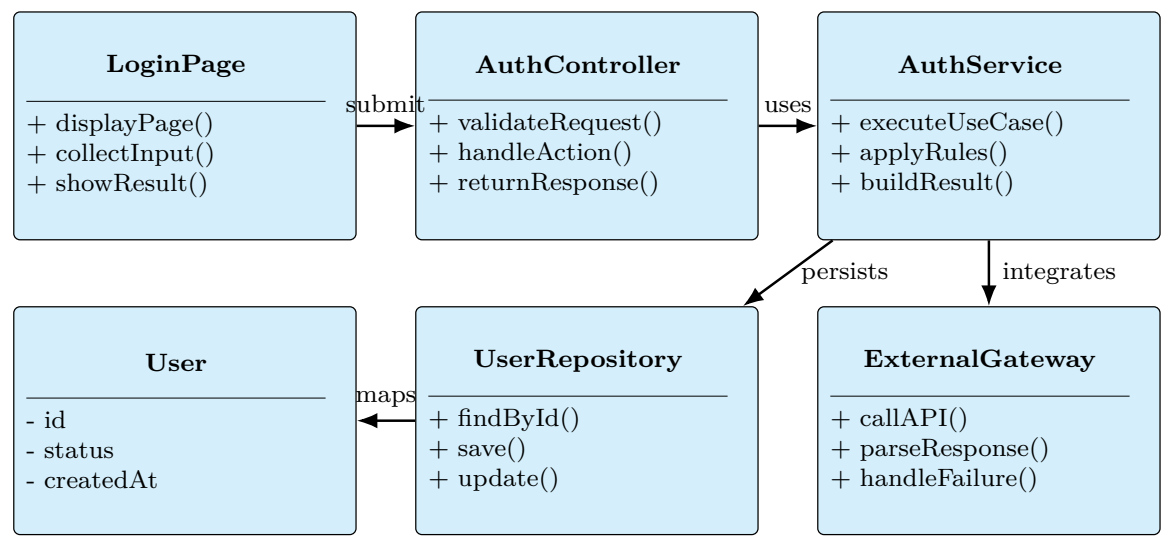


Figure IV.125: Class diagram for UC61 Login

3.13.2.8 Sequence Diagram

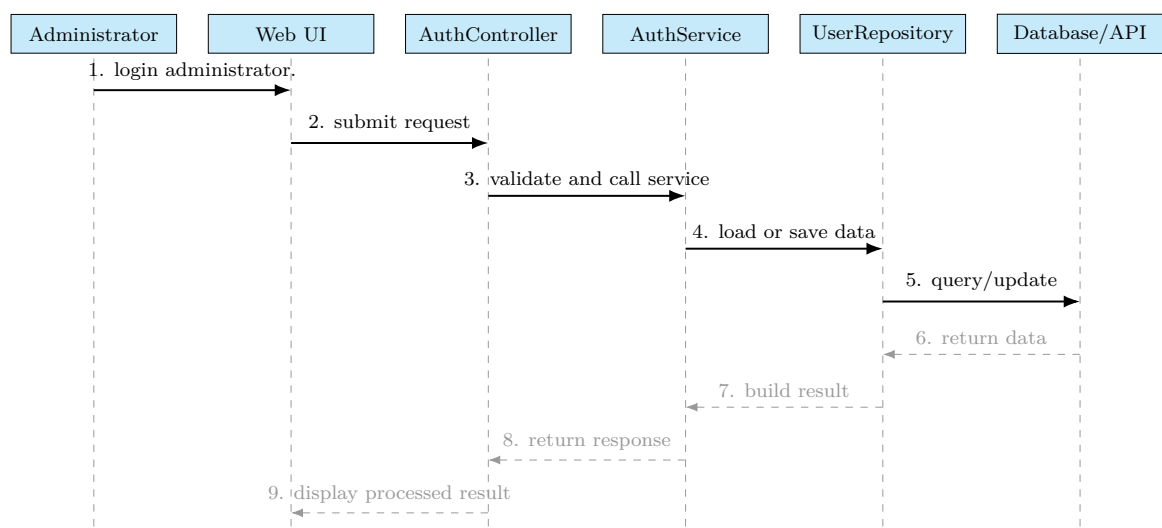


Figure IV.126: Sequence diagram for UC61 Login

3.14 Administration and Resource Management

This group contains use cases related to administration and resource management. The design uses the layered architecture introduced in the system design: React boundary pages, Spring Boot controllers, services, repositories, database entities, and external gateways when required.

3.14.1 UC62 Manage Users

3.14.1.1 Requirement Traceability

- **Use case ID:** UC62.
- **Use case name:** Manage Users.
- **Actor:** Administrator.
- **Requirement description:** Manage user accounts.
- **Related table(s):** Users, StaffProfiles, Notifications.

3.14.1.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in **Users.Role** must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.14.1.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.14.1.4 Main Processing Flow

1. The actor opens **ManageUsersPage** and performs the action: manage user accounts..
2. The frontend validates required fields, prepares the request DTO, and sends it to **AdminController**.
3. **AdminController** checks authentication, authorization, request format, and path parameters.
4. **AdminService** loads related data through **UserRepository** and applies business rules for Manage Users.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in **Users**, **StaffProfiles**, **Notifications** and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.14.1.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.14.1.6 Participating Components

- **Boundary/UI – ManageUsersPage:** collects input and displays the final result for Manage Users.
- **Controller – AdminController:** receives REST requests and validates request DTOs.
- **Service – AdminService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – User:** represents the main persisted domain object.
- **Repository – UserRepository:** provides data access operations for the related table(s).

3.14.1.7 Class Diagram

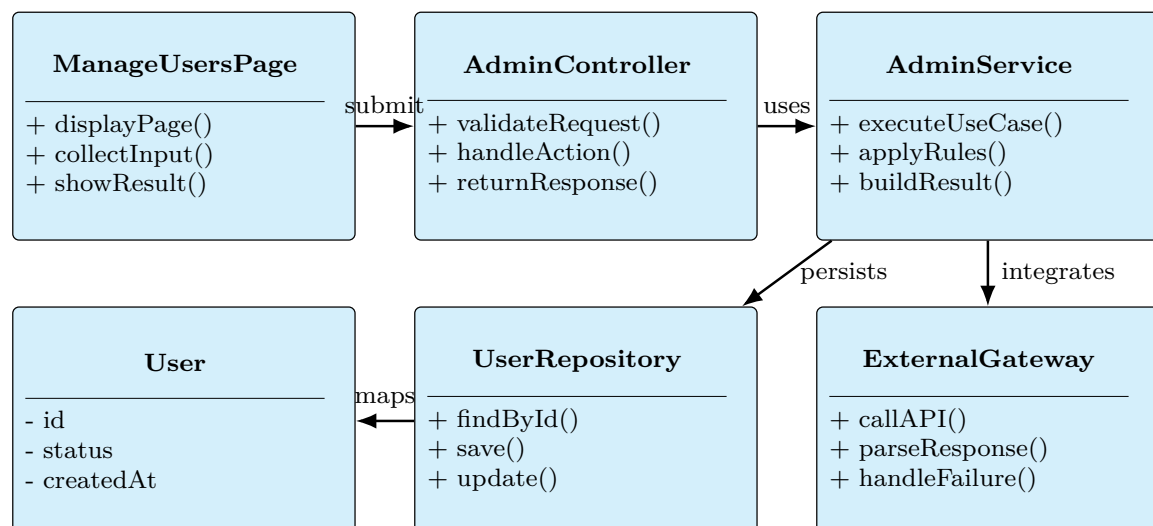


Figure IV.127: Class diagram for UC62 Manage Users

3.14.1.8 Sequence Diagram

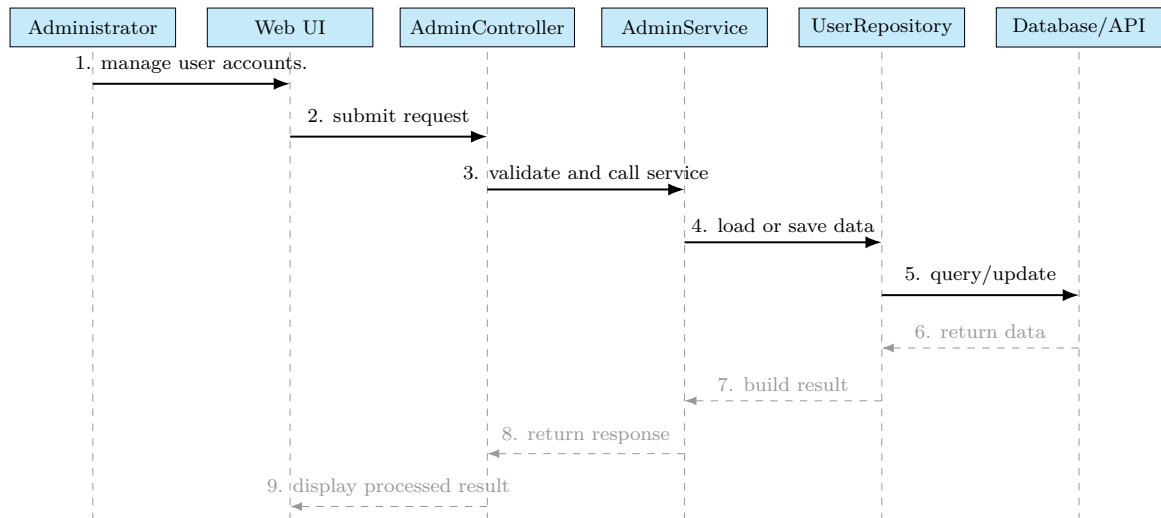


Figure IV.128: Sequence diagram for UC62 Manage Users

3.15 Career Path and Skill Management

This group contains use cases related to career path and skill management. The design uses the layered architecture introduced in the system design: React boundary pages, Spring Boot controllers, services, repositories, database entities, and external gateways when required.

3.15.1 UC63 Manage Career Roles

3.15.1.1 Requirement Traceability

- **Use case ID:** UC63.
- **Use case name:** Manage Career Roles.
- **Actor:** Administrator.
- **Requirement description:** Manage career roles.
- **Related table(s):** CareerPaths, SkillNodes, CareerPathSkills.

3.15.1.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.15.1.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.15.1.4 Main Processing Flow

1. The actor opens **ManageCareerRolesPage** and performs the action: manage career roles..
2. The frontend validates required fields, prepares the request DTO, and sends it to **CareerPathController**.
3. **CareerPathController** checks authentication, authorization, request format, and path parameters.
4. **CareerPathService** loads related data through **CareerPathRepository** and applies business rules for Manage Career Roles.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in **CareerPaths**, **SkillNodes**, **CareerPathSkills** and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.15.1.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.15.1.6 Participating Components

- **Boundary/UI – ManageCareerRolesPage:** collects input and displays the final result for Manage Career Roles.
- **Controller – CareerPathController:** receives REST requests and validates request DTOs.
- **Service – CareerPathService:** coordinates business logic, integration calls, and transaction boundaries.

- **Entity – CareerPath:** represents the main persisted domain object.
- **Repository – CareerPathRepository:** provides data access operations for the related table(s).

3.15.1.7 Class Diagram

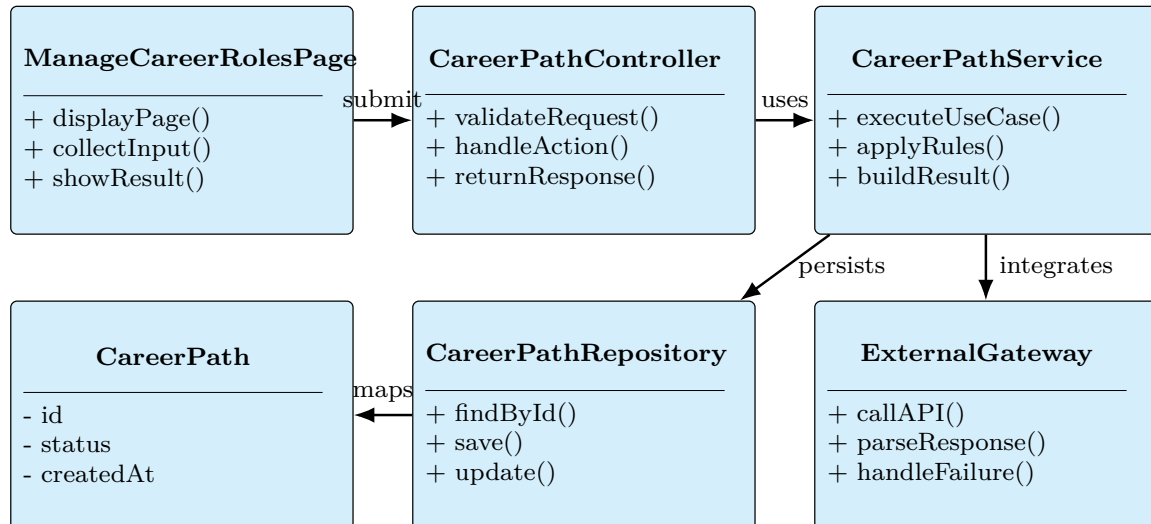


Figure IV.129: Class diagram for UC63 Manage Career Roles

3.15.1.8 Sequence Diagram

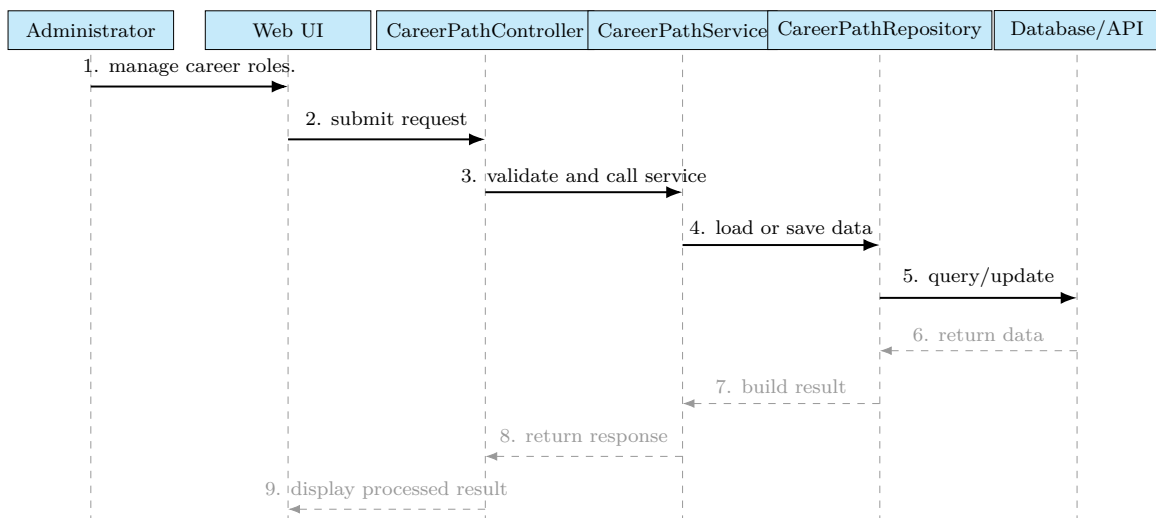


Figure IV.130: Sequence diagram for UC63 Manage Career Roles

3.15.2 UC64 Manage Skill Nodes

3.15.2.1 Requirement Traceability

- **Use case ID:** UC64.

- **Use case name:** Manage Skill Nodes.
- **Actor:** Administrator.
- **Requirement description:** Manage skill graph.
- **Related table(s):** CareerPaths, SkillNodes, CareerPathSkills.

3.15.2.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.15.2.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.15.2.4 Main Processing Flow

1. The actor opens `ManageSkillNodesPage` and performs the action: manage skill graph..
2. The frontend validates required fields, prepares the request DTO, and sends it to `CareerPathController`.
3. `CareerPathController` checks authentication, authorization, request format, and path parameters.
4. `CareerPathService` loads related data through `CareerPathRepository` and applies business rules for Manage Skill Nodes.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in `CareerPaths`, `SkillNodes`, `CareerPathSkills` and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.15.2.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.

- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.15.2.6 Participating Components

- **Boundary/UI – ManageSkillNodesPage:** collects input and displays the final result for Manage Skill Nodes.
- **Controller – CareerPathController:** receives REST requests and validates request DTOs.
- **Service – CareerPathService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – CareerPath:** represents the main persisted domain object.
- **Repository – CareerPathRepository:** provides data access operations for the related table(s).

3.15.2.7 Class Diagram

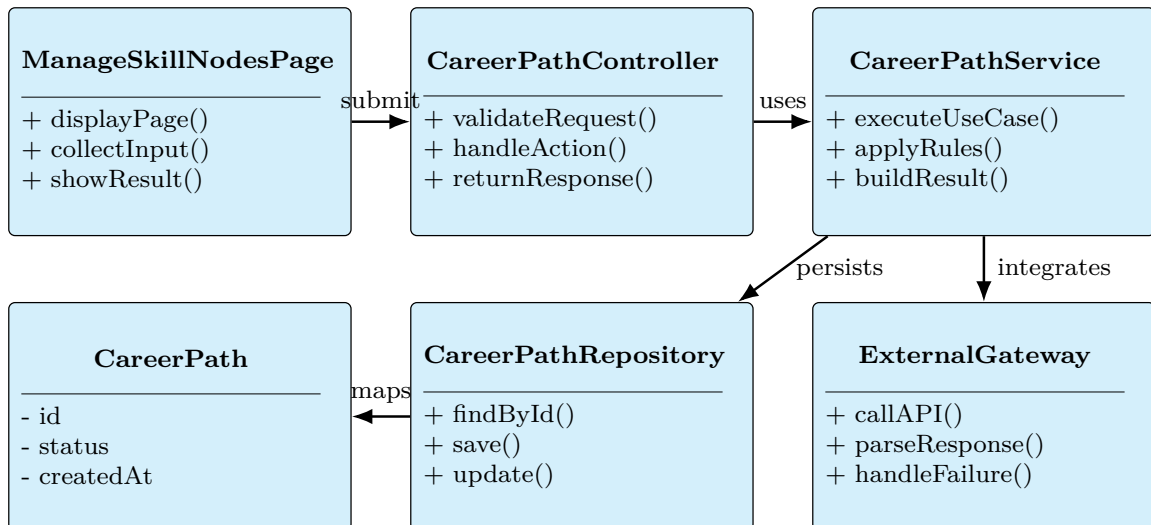


Figure IV.131: Class diagram for UC64 Manage Skill Nodes

3.15.2.8 Sequence Diagram

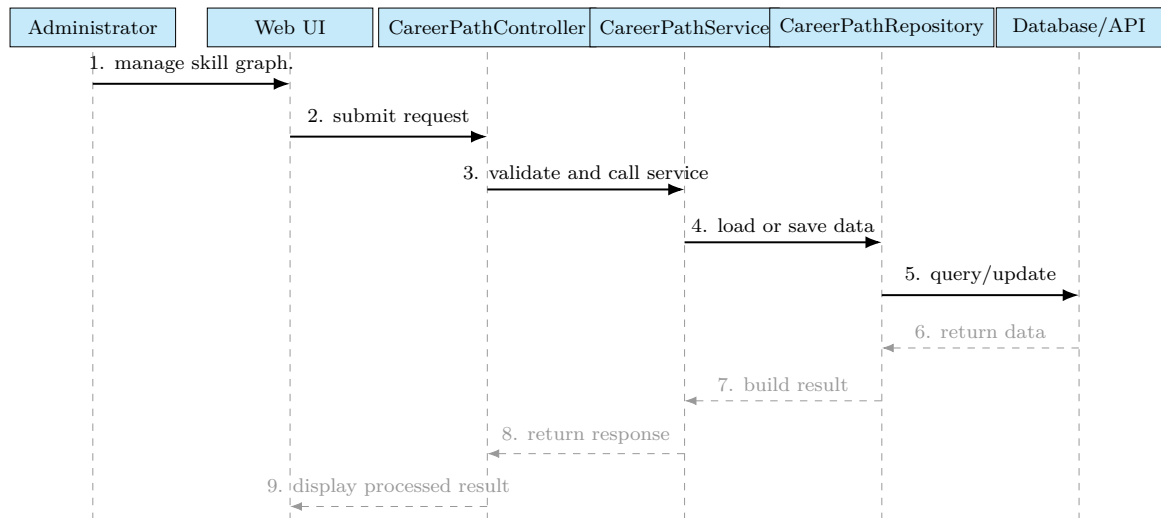


Figure IV.132: Sequence diagram for UC64 Manage Skill Nodes

3.16 Administration and Resource Management

This group contains use cases related to administration and resource management. The design uses the layered architecture introduced in the system design: React boundary pages, Spring Boot controllers, services, repositories, database entities, and external gateways when required.

3.16.1 UC65 Manage Courses

3.16.1.1 Requirement Traceability

- **Use case ID:** UC65.
- **Use case name:** Manage Courses.
- **Actor:** Administrator.
- **Requirement description:** Manage courses.
- **Related table(s):** LearningResources, SkillNodes.

3.16.1.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.16.1.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.16.1.4 Main Processing Flow

1. The actor opens **ManageCoursesPage** and performs the action: manage courses..
2. The frontend validates required fields, prepares the request DTO, and sends it to **ResourceAdminController**.
3. **ResourceAdminController** checks authentication, authorization, request format, and path parameters.
4. **ResourceAdminService** loads related data through **LearningResourceRepo** and applies business rules for Manage Courses.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in **LearningResources**, **SkillNodes** and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.16.1.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.16.1.6 Participating Components

- **Boundary/UI – ManageCoursesPage:** collects input and displays the final result for Manage Courses.
- **Controller – ResourceAdminController:** receives REST requests and validates request DTOs.
- **Service – ResourceAdminService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – LearningResource:** represents the main persisted domain object.
- **Repository – LearningResourceRepo:** provides data access operations for the related table(s).

3.16.1.7 Class Diagram

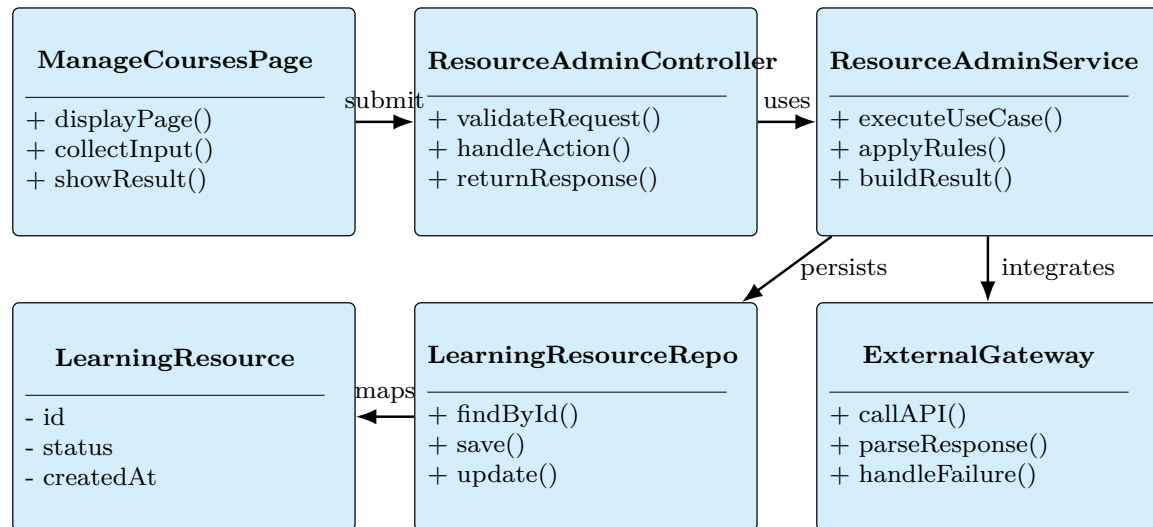


Figure IV.133: Class diagram for UC65 Manage Courses

3.16.1.8 Sequence Diagram

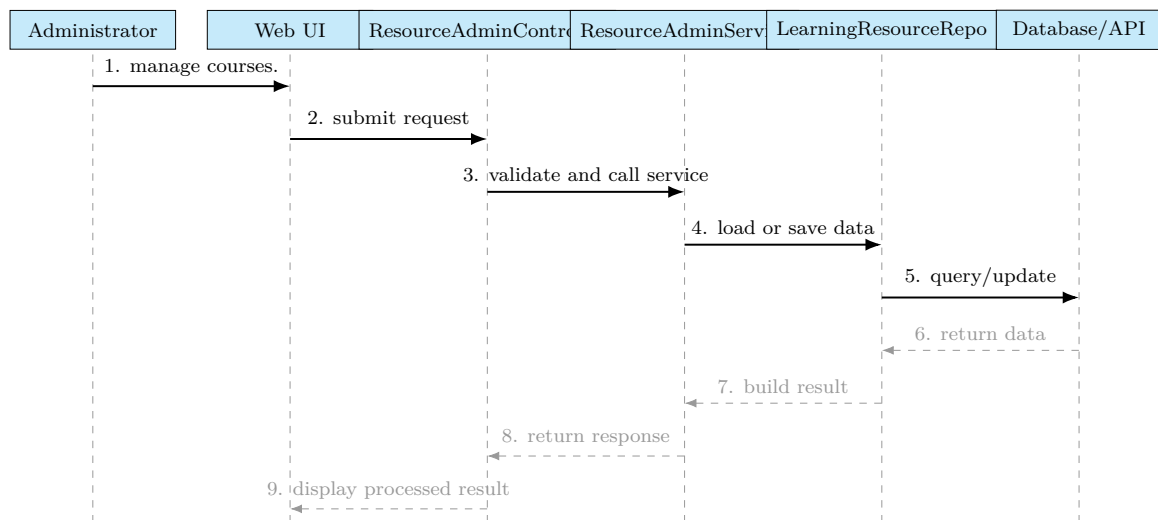


Figure IV.134: Sequence diagram for UC65 Manage Courses

3.16.2 UC66 Manage Learning Resources

3.16.2.1 Requirement Traceability

- **Use case ID:** UC66.
- **Use case name:** Manage Learning Resources.
- **Actor:** Administrator.
- **Requirement description:** Manage learning resources.
- **Related table(s):** LearningResources, SkillNodes.

3.16.2.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.16.2.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.16.2.4 Main Processing Flow

1. The actor opens `ManageLearningResoPage` and performs the action: manage learning resources..
2. The frontend validates required fields, prepares the request DTO, and sends it to `ResourceAdminController`.
3. `ResourceAdminController` checks authentication, authorization, request format, and path parameters.
4. `ResourceAdminService` loads related data through `LearningResourceRepo` and applies business rules for Manage Learning Resources.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in `LearningResources`, `SkillNodes` and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.16.2.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.16.2.6 Participating Components

- **Boundary/UI – ManageLearningResoPage:** collects input and displays the final result for Manage Learning Resources.
- **Controller – ResourceAdminController:** receives REST requests and validates request DTOs.
- **Service – ResourceAdminService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – LearningResource:** represents the main persisted domain object.
- **Repository – LearningResourceRepo:** provides data access operations for the related table(s).

3.16.2.7 Class Diagram

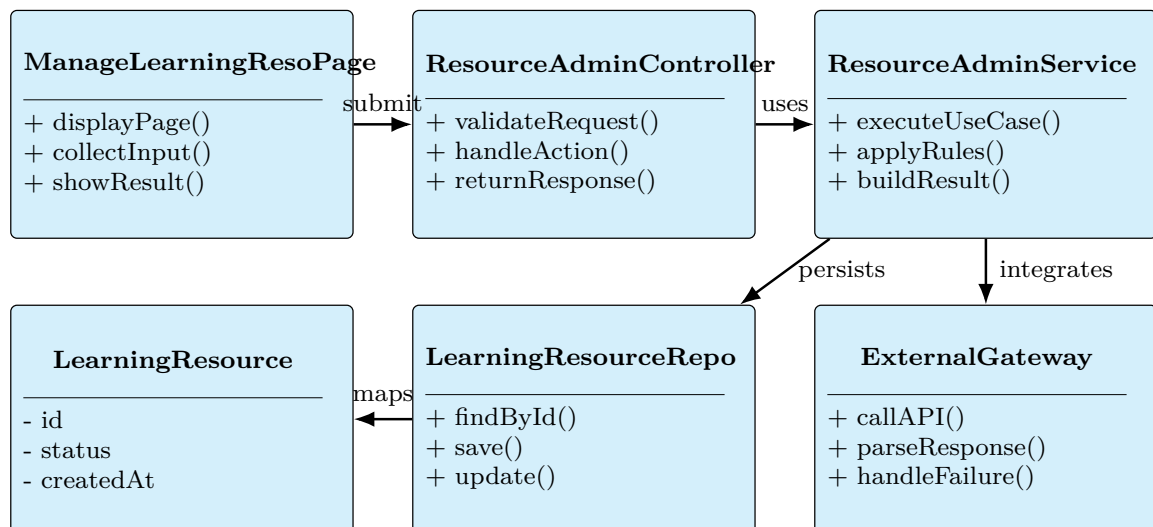


Figure IV.135: Class diagram for UC66 Manage Learning Resources

3.16.2.8 Sequence Diagram

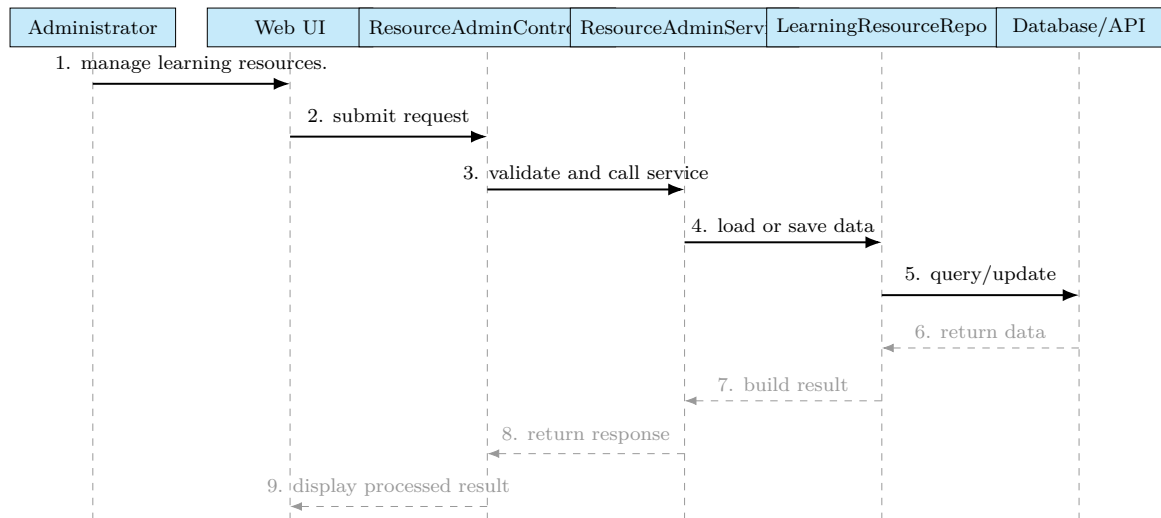


Figure IV.136: Sequence diagram for UC66 Manage Learning Resources

3.17 Market Trend Management

This group contains use cases related to market trend management. The design uses the layered architecture introduced in the system design: React boundary pages, Spring Boot controllers, services, repositories, database entities, and external gateways when required.

3.17.1 UC67 Manage Job Trend Data

3.17.1.1 Requirement Traceability

- **Use case ID:** UC67.
- **Use case name:** Manage Job Trend Data.
- **Actor:** Administrator.
- **Requirement description:** Manage job market data.
- **Related table(s):** JobTrends, CareerPaths.

3.17.1.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.17.1.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.17.1.4 Main Processing Flow

1. The actor opens **ManageJobTrendDataPage** and performs the action: manage job market data..
2. The frontend validates required fields, prepares the request DTO, and sends it to **MarketTrendController**.
3. **MarketTrendController** checks authentication, authorization, request format, and path parameters.
4. **MarketTrendService** loads related data through **JobTrendRepository** and applies business rules for Manage Job Trend Data.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in **JobTrends**, **CareerPaths** and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.17.1.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.17.1.6 Participating Components

- **Boundary/UI – ManageJobTrendDataPage:** collects input and displays the final result for Manage Job Trend Data.
- **Controller – MarketTrendController:** receives REST requests and validates request DTOs.
- **Service – MarketTrendService:** coordinates business logic, integration calls, and transaction boundaries.

- **Entity – JobTrend:** represents the main persisted domain object.
- **Repository – JobTrendRepository:** provides data access operations for the related table(s).

3.17.1.7 Class Diagram

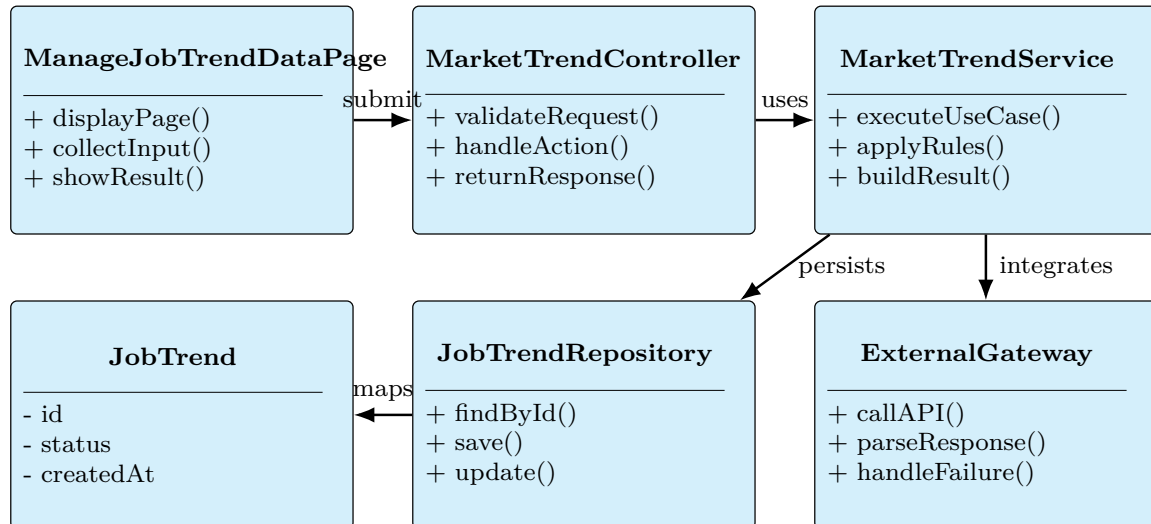


Figure IV.137: Class diagram for UC67 Manage Job Trend Data

3.17.1.8 Sequence Diagram

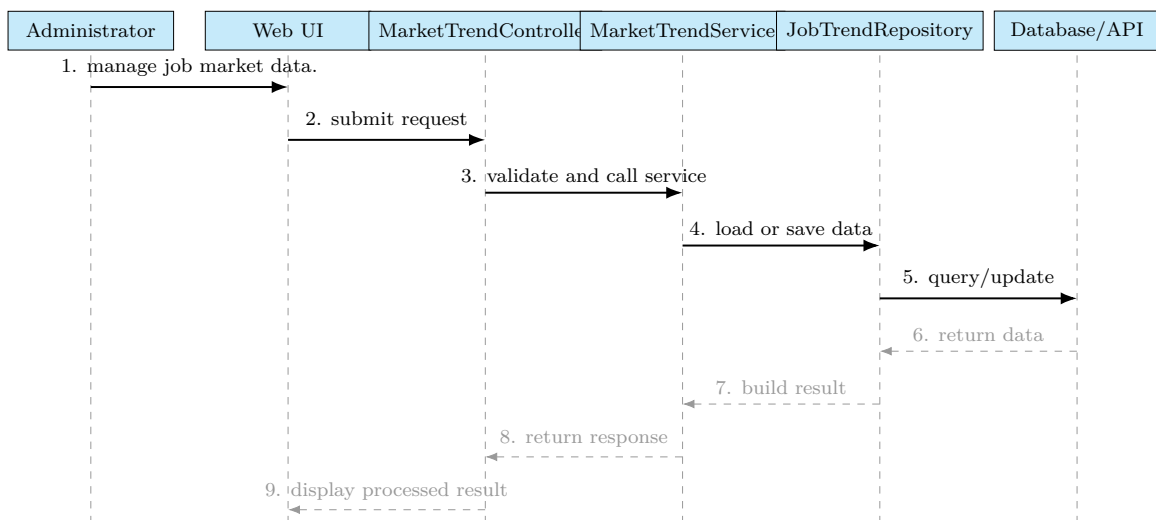


Figure IV.138: Sequence diagram for UC67 Manage Job Trend Data

3.18 AI and Market Configuration

This group contains use cases related to ai and market configuration. The design uses the layered architecture introduced in the system design: React boundary pages, Spring Boot controllers, services, repositories, database entities, and external gateways when required.

3.18.1 UC68 Manage AI Configuration

3.18.1.1 Requirement Traceability

- **Use case ID:** UC68.
- **Use case name:** Manage AI Configuration.
- **Actor:** Administrator.
- **Requirement description:** Configure AI engine.
- **Related table(s):** AIConfigurations.

3.18.1.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in **Users.Role** must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.18.1.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.18.1.4 Main Processing Flow

1. The actor opens **ManageAIConfiguratPage** and performs the action: configure AI engine..
2. The frontend validates required fields, prepares the request DTO, and sends it to **AIConfigController**.
3. **AIConfigController** checks authentication, authorization, request format, and path parameters.
4. **AIConfigService** loads related data through **AIConfigRepository** and applies business rules for Manage AI Configuration.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in **AIConfigurations** and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.18.1.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.18.1.6 Participating Components

- **Boundary/UI – ManageAIConfiguratPage:** collects input and displays the final result for Manage AI Configuration.
- **Controller – AIConfigController:** receives REST requests and validates request DTOs.
- **Service – AIConfigService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – AIConfiguration:** represents the main persisted domain object.
- **Repository – AIConfigRepository:** provides data access operations for the related table(s).

3.18.1.7 Class Diagram

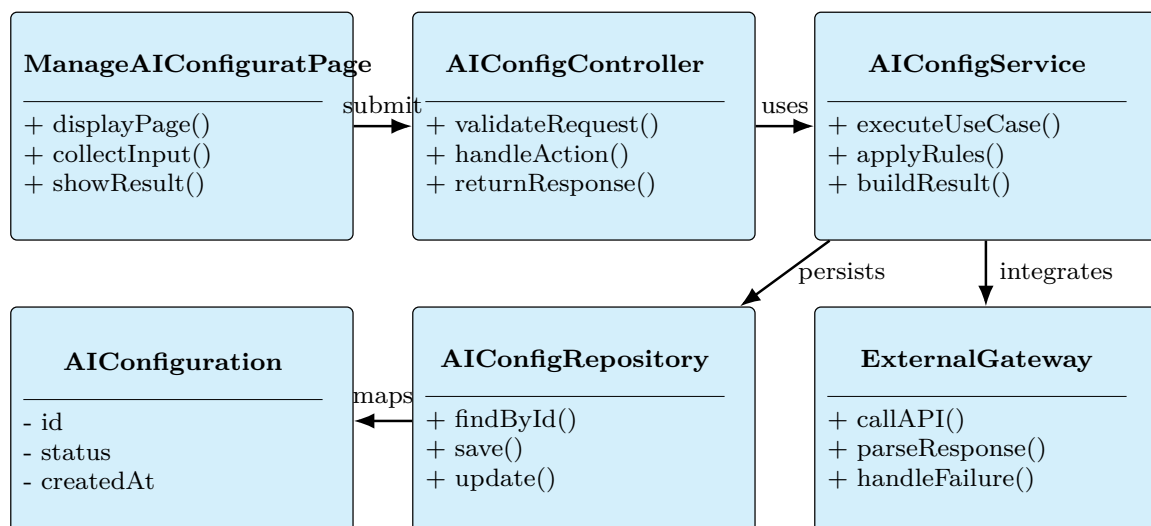


Figure IV.139: Class diagram for UC68 Manage AI Configuration

3.18.1.8 Sequence Diagram

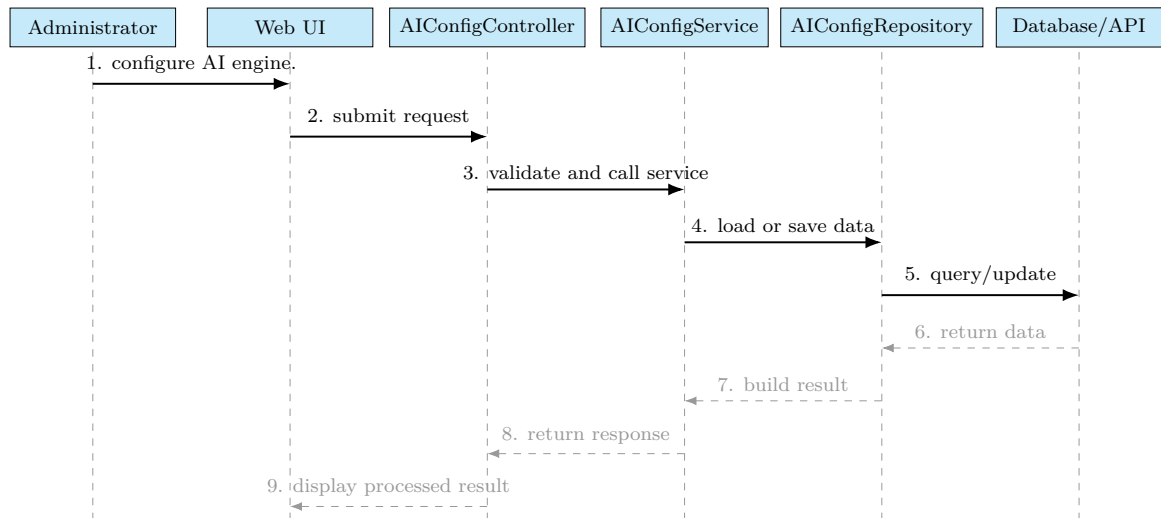


Figure IV.140: Sequence diagram for UC68 Manage AI Configuration

3.19 Administration and Resource Management

This group contains use cases related to administration and resource management. The design uses the layered architecture introduced in the system design: React boundary pages, Spring Boot controllers, services, repositories, database entities, and external gateways when required.

3.19.1 UC69 Manage Reports

3.19.1.1 Requirement Traceability

- **Use case ID:** UC69.
- **Use case name:** Manage Reports.
- **Actor:** Administrator.
- **Requirement description:** Generate reports.
- **Related table(s):** Users, StaffProfiles, Notifications.

3.19.1.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in **Users.Role** must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.19.1.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.19.1.4 Main Processing Flow

1. The actor opens **ManageReportsPage** and performs the action: generate reports..
2. The frontend validates required fields, prepares the request DTO, and sends it to **AdminController**.
3. **AdminController** checks authentication, authorization, request format, and path parameters.
4. **AdminService** loads related data through **UserRepository** and applies business rules for Manage Reports.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in **Users**, **StaffProfiles**, **Notifications** and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.19.1.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.19.1.6 Participating Components

- **Boundary/UI – ManageReportsPage:** collects input and displays the final result for Manage Reports.
- **Controller – AdminController:** receives REST requests and validates request DTOs.
- **Service – AdminService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – User:** represents the main persisted domain object.
- **Repository – UserRepository:** provides data access operations for the related table(s).

3.19.1.7 Class Diagram

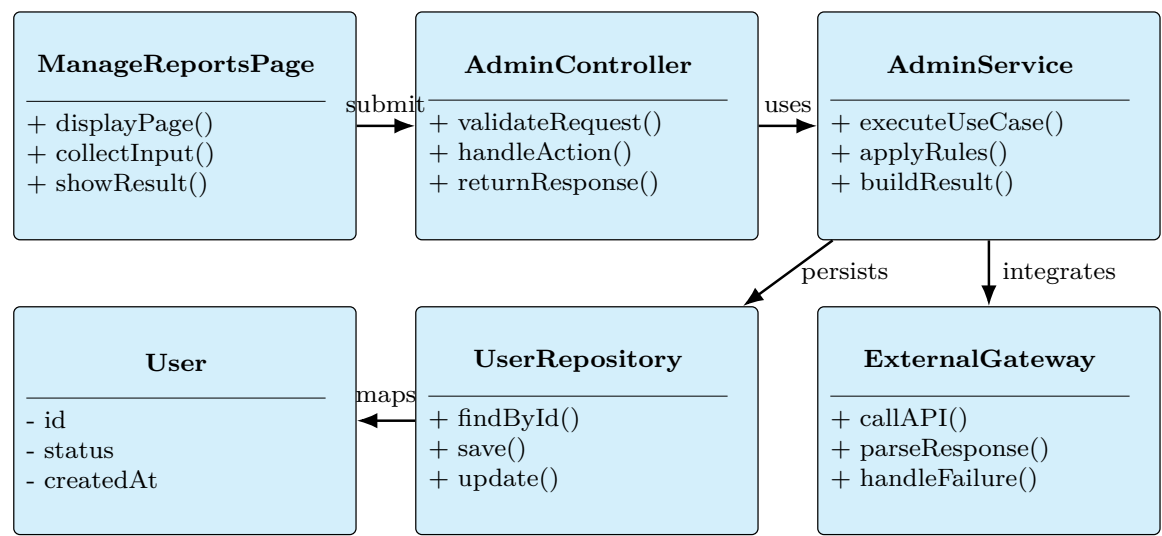


Figure IV.141: Class diagram for UC69 Manage Reports

3.19.1.8 Sequence Diagram

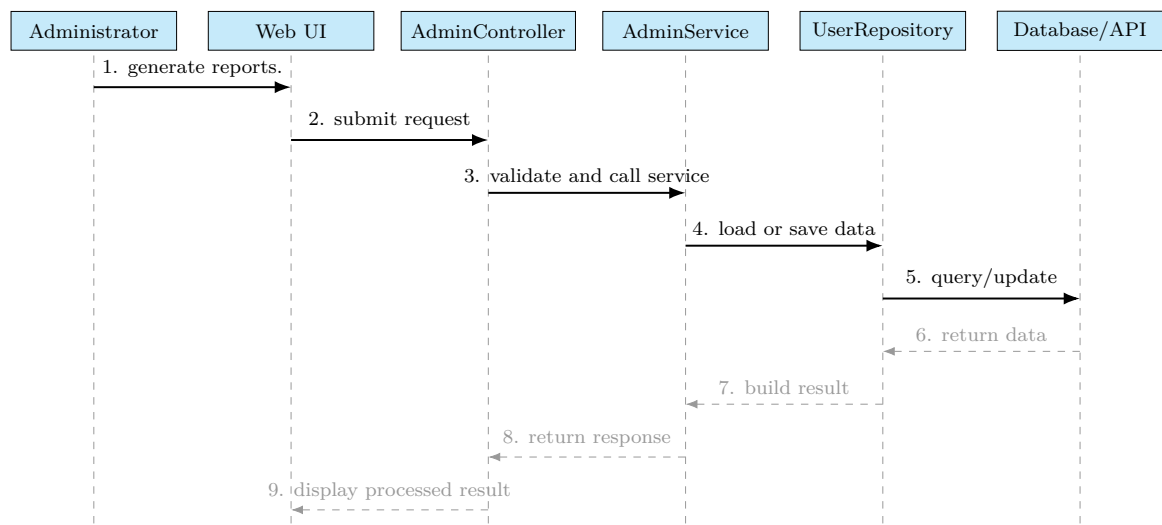


Figure IV.142: Sequence diagram for UC69 Manage Reports

3.20 Authentication and Account Access

This group contains use cases related to authentication and account access. The design uses the layered architecture introduced in the system design: React boundary pages, Spring Boot controllers, services, repositories, database entities, and external gateways when required.

3.20.1 UC70 Logout

3.20.1.1 Requirement Traceability

- **Use case ID:** UC70.
- **Use case name:** Logout.
- **Actor:** Administrator.
- **Requirement description:** Logout.
- **Related table(s):** Users.

3.20.1.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in **Users.Role** must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.20.1.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.20.1.4 Main Processing Flow

1. The actor opens **LogoutPage** and performs the action: logout..
2. The frontend validates required fields, prepares the request DTO, and sends it to **AuthController**.
3. **AuthController** checks authentication, authorization, request format, and path parameters.
4. **AuthService** loads related data through **UserRepository** and applies business rules for Logout.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in **Users** and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.20.1.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.20.1.6 Participating Components

- **Boundary/UI – LogoutPage:** collects input and displays the final result for Logout.
- **Controller – AuthController:** receives REST requests and validates request DTOs.
- **Service – AuthService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – User:** represents the main persisted domain object.
- **Repository – UserRepository:** provides data access operations for the related table(s).

3.20.1.7 Class Diagram

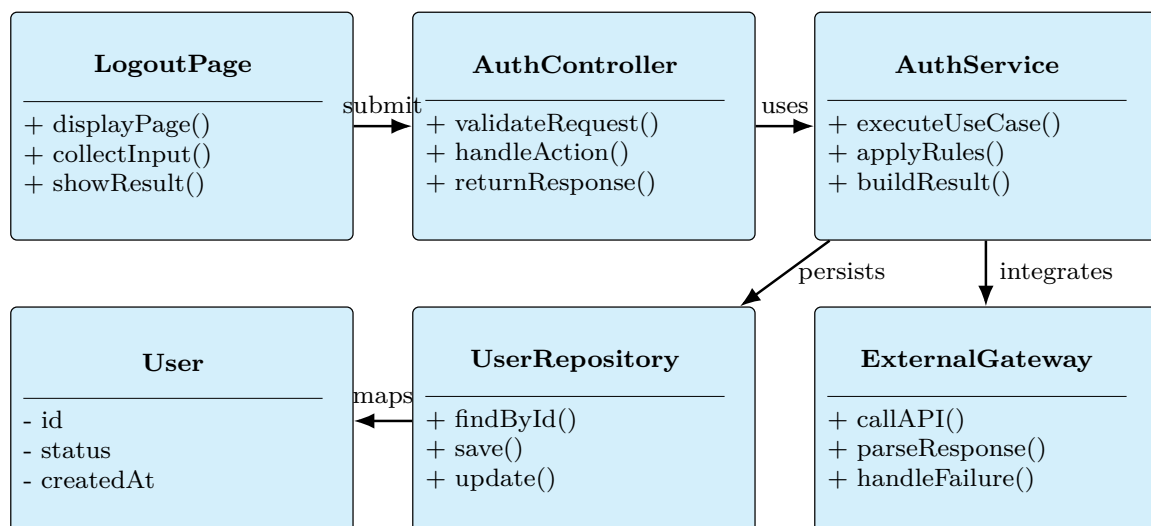


Figure IV.143: Class diagram for UC70 Logout

3.20.1.8 Sequence Diagram

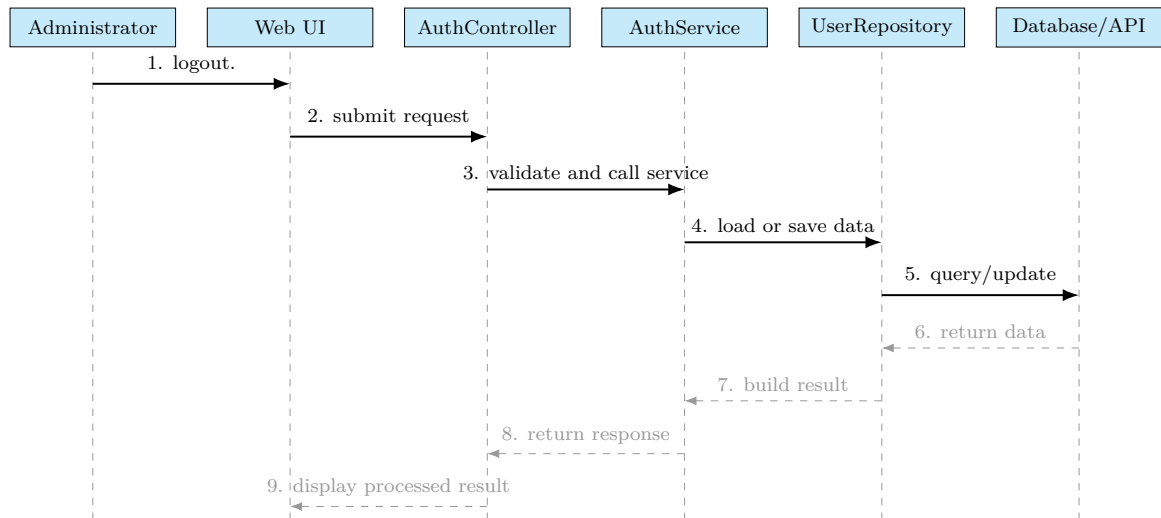


Figure IV.144: Sequence diagram for UC70 Logout

3.21 AI Mentor and Analysis Logging

This group contains use cases related to ai mentor and analysis logging. The design uses the layered architecture introduced in the system design: React boundary pages, Spring Boot controllers, services, repositories, database entities, and external gateways when required.

3.21.1 UC71 Analyze Transcript

3.21.1.1 Requirement Traceability

- **Use case ID:** UC71.
- **Use case name:** Analyze Transcript.
- **Actor:** AI Recommendation Engine.
- **Requirement description:** Analyze transcript.
- **Related table(s):** AIConfigurations, AIAnalysisLogs, StudentProfiles.

3.21.1.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.21.1.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.21.1.4 Main Processing Flow

1. The actor opens **AnalyzeTranscriptPage** and performs the action: analyze transcript..
2. The frontend validates required fields, prepares the request DTO, and sends it to **AIController**.
3. **AIController** checks authentication, authorization, request format, and path parameters.
4. **AIAnalysisService** loads related data through **AIAnalysisLogRepo** and applies business rules for Analyze Transcript.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in **AIConfigurations**, **AIAnalysisLogs**, **StudentProfiles** and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.21.1.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.21.1.6 Participating Components

- **Boundary/UI – AnalyzeTranscriptPage:** collects input and displays the final result for Analyze Transcript.
- **Controller – AIController:** receives REST requests and validates request DTOs.
- **Service – AIAnalysisService:** coordinates business logic, integration calls, and transaction boundaries.

- **Entity – AIAnalysisLog:** represents the main persisted domain object.
- **Repository – AIAnalysisLogRepo:** provides data access operations for the related table(s).

3.21.1.7 Class Diagram

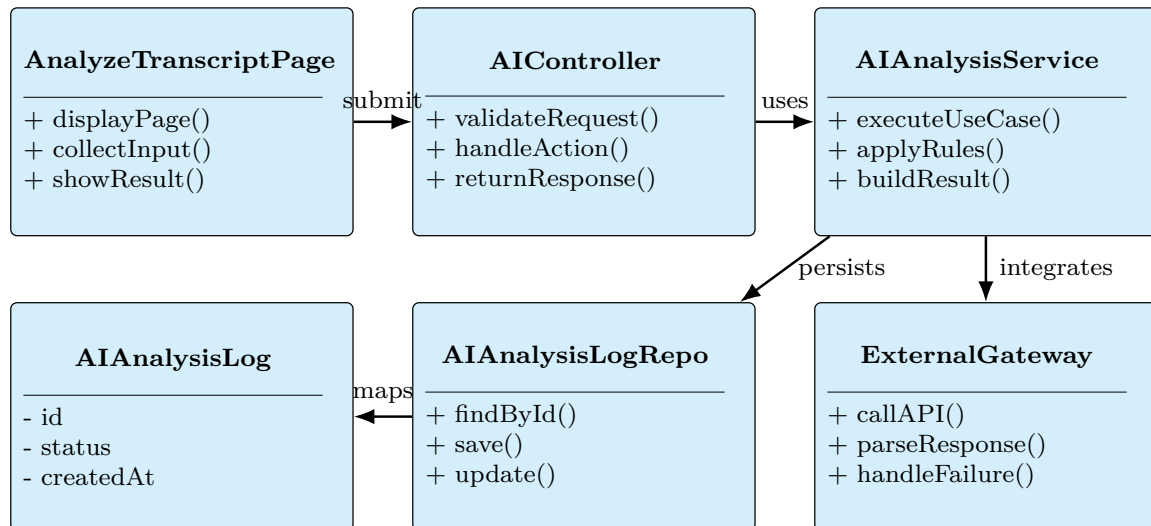


Figure IV.145: Class diagram for UC71 Analyze Transcript

3.21.1.8 Sequence Diagram

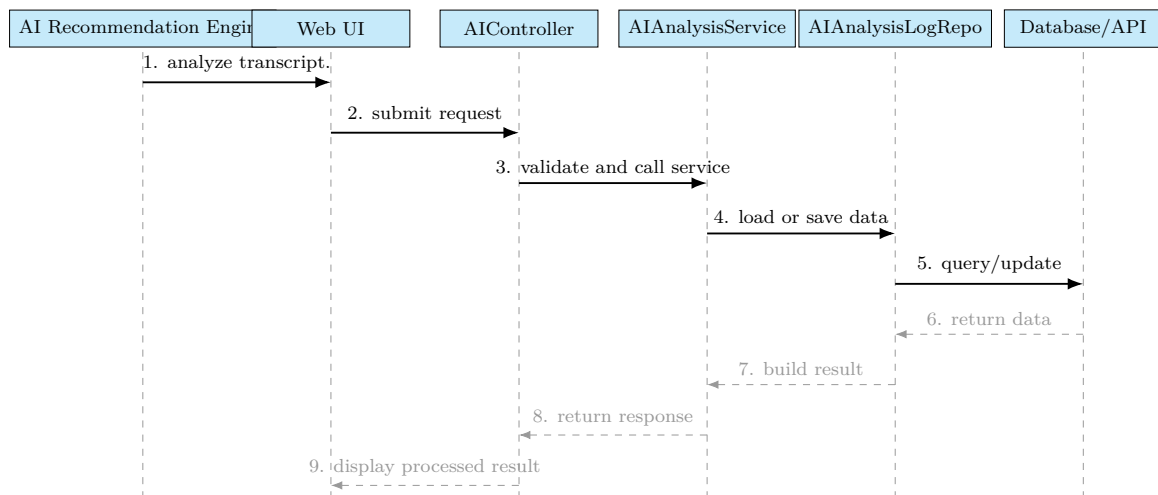


Figure IV.146: Sequence diagram for UC71 Analyze Transcript

3.21.2 UC72 Analyze GitHub Repository

3.21.2.1 Requirement Traceability

- **Use case ID:** UC72.
- **Use case name:** Analyze GitHub Repository.

- **Actor:** AI Recommendation Engine.
- **Requirement description:** Analyze GitHub repository.
- **Related table(s):** `AIConfigurations`, `AIAnalysisLogs`, `StudentProfiles`.

3.21.2.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.21.2.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.21.2.4 Main Processing Flow

1. The actor opens `AnalyzeGitHubReposPage` and performs the action: analyze GitHub repository..
2. The frontend validates required fields, prepares the request DTO, and sends it to `AIController`.
3. `AIController` checks authentication, authorization, request format, and path parameters.
4. `AIAnalysisService` loads related data through `AIAnalysisLogRepo` and applies business rules for Analyze GitHub Repository.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in `AIConfigurations`, `AIAnalysisLogs`, `StudentProfiles` and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.21.2.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.21.2.6 Participating Components

- **Boundary/UI – AnalyzeGitHubReposPage:** collects input and displays the final result for Analyze GitHub Repository.
- **Controller – AIController:** receives REST requests and validates request DTOs.
- **Service – AIAnalysisService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – AIAnalysisLog:** represents the main persisted domain object.
- **Repository – AIAnalysisLogRepo:** provides data access operations for the related table(s).

3.21.2.7 Class Diagram

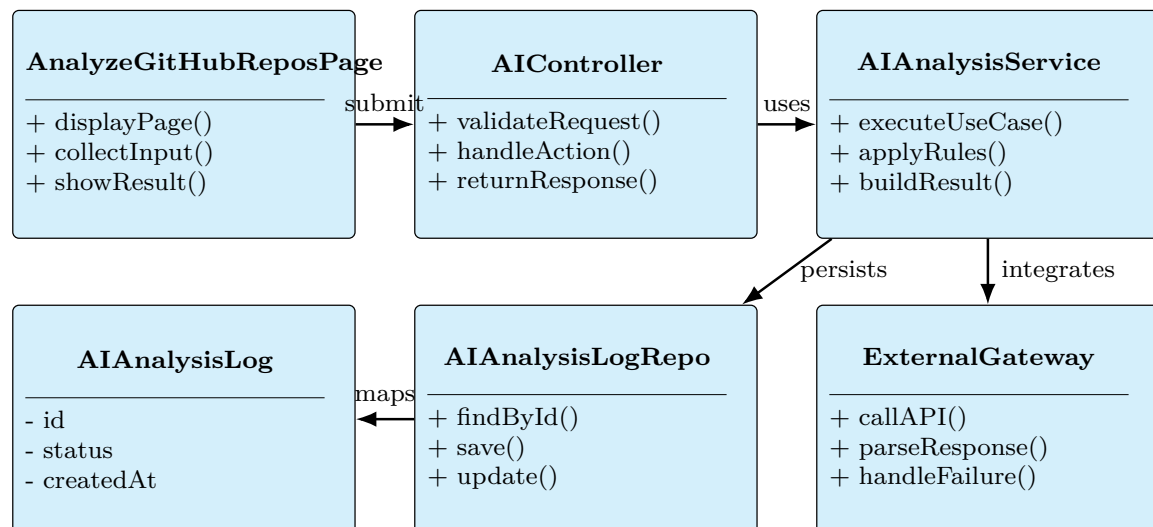


Figure IV.147: Class diagram for UC72 Analyze GitHub Repository

3.21.2.8 Sequence Diagram

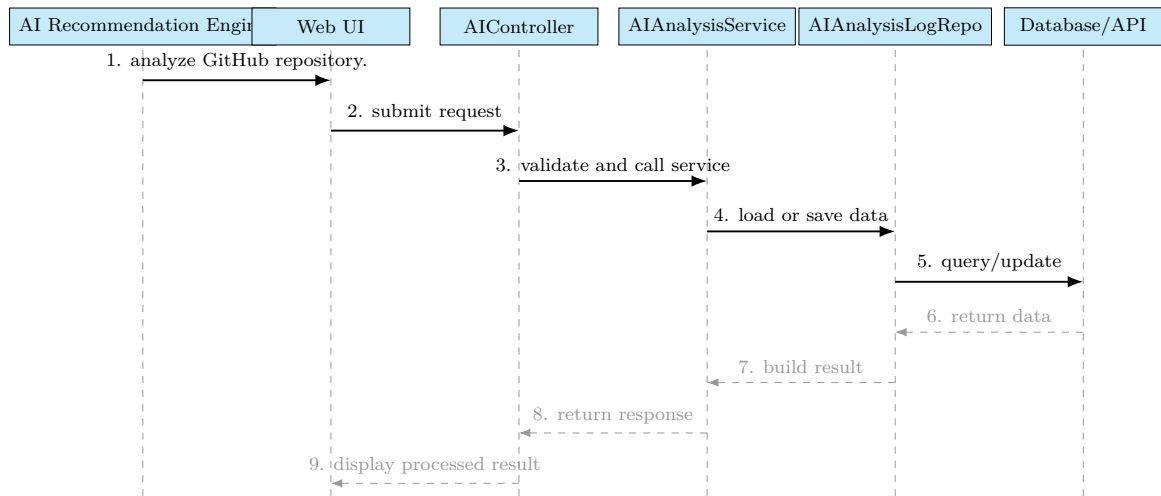


Figure IV.148: Sequence diagram for UC72 Analyze GitHub Repository

3.22 Learning Roadmap and Course Management

This group contains use cases related to learning roadmap and course management. The design uses the layered architecture introduced in the system design: React boundary pages, Spring Boot controllers, services, repositories, database entities, and external gateways when required.

3.22.1 UC73 Generate Personalized Roadmap

3.22.1.1 Requirement Traceability

- **Use case ID:** UC73.
- **Use case name:** Generate Personalized Roadmap.
- **Actor:** AI Recommendation Engine.
- **Requirement description:** Generate roadmap.
- **Related table(s):** Roadmaps, RoadmapProgress, CareerPathSkills.

3.22.1.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.22.1.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.22.1.4 Main Processing Flow

1. The actor opens **GeneratePersonalizPage** and performs the action: generate roadmap..
2. The frontend validates required fields, prepares the request DTO, and sends it to **RoadmapController**.
3. **RoadmapController** checks authentication, authorization, request format, and path parameters.
4. **RoadmapService** loads related data through **RoadmapRepository** and applies business rules for Generate Personalized Roadmap.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in **Roadmaps**, **RoadmapProgress**, **CareerPathSkills** and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.22.1.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.22.1.6 Participating Components

- **Boundary/UI – GeneratePersonalizPage:** collects input and displays the final result for Generate Personalized Roadmap.
- **Controller – RoadmapController:** receives REST requests and validates request DTOs.
- **Service – RoadmapService:** coordinates business logic, integration calls, and transaction boundaries.

- **Entity – Roadmap:** represents the main persisted domain object.
- **Repository – RoadmapRepository:** provides data access operations for the related table(s).

3.22.1.7 Class Diagram

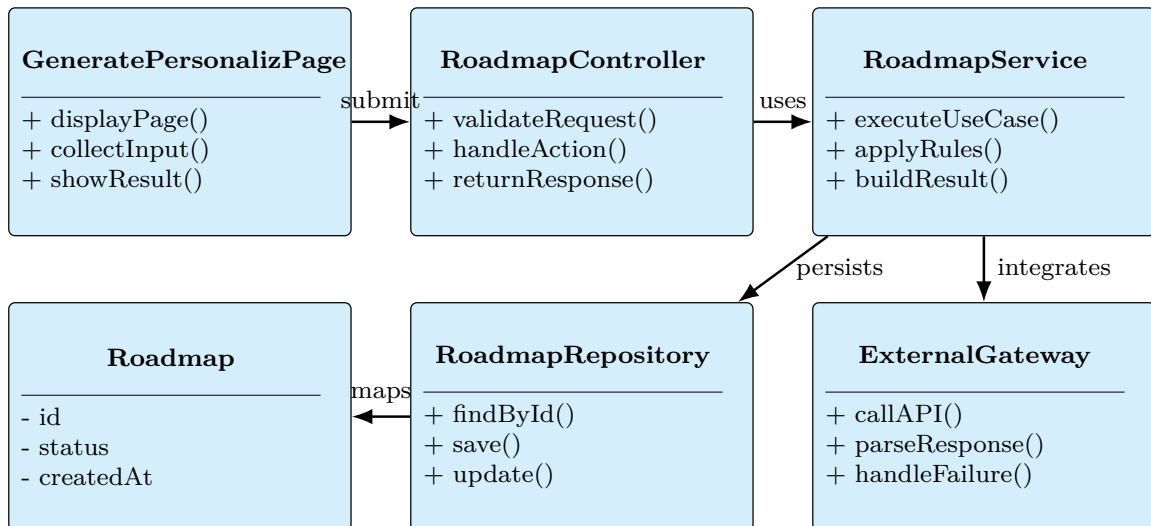


Figure IV.149: Class diagram for UC73 Generate Personalized Roadmap

3.22.1.8 Sequence Diagram

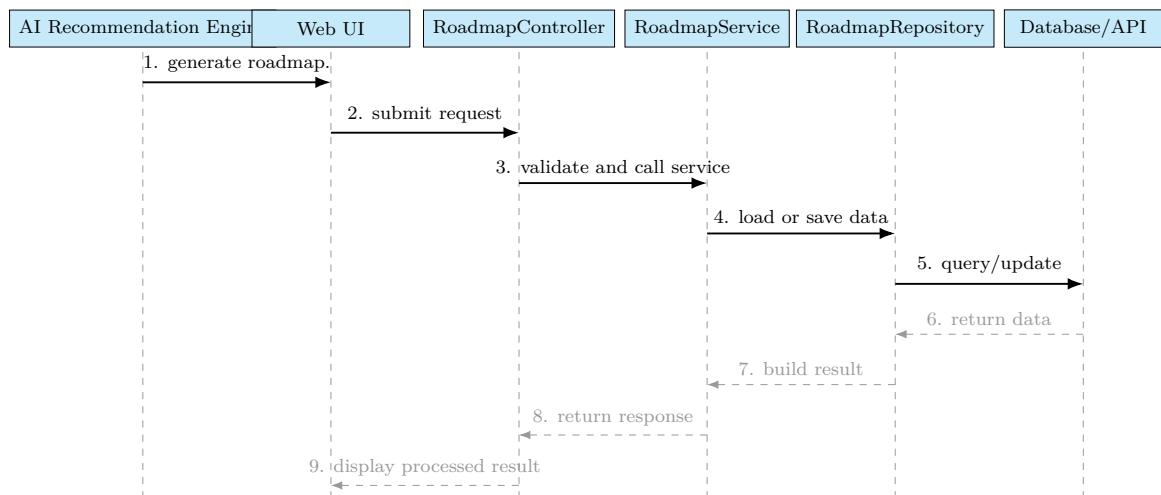


Figure IV.150: Sequence diagram for UC73 Generate Personalized Roadmap

3.22.2 UC74 Analyze Skill Gap

3.22.2.1 Requirement Traceability

- **Use case ID:** UC74.
- **Use case name:** Analyze Skill Gap.

- **Actor:** AI Recommendation Engine.
- **Requirement description:** Detect missing skills.
- **Related table(s):** Roadmaps, RoadmapProgress, CareerPathSkills.

3.22.2.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.22.2.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.22.2.4 Main Processing Flow

1. The actor opens `AnalyzeSkillGapPage` and performs the action: detect missing skills..
2. The frontend validates required fields, prepares the request DTO, and sends it to `RoadmapController`.
3. `RoadmapController` checks authentication, authorization, request format, and path parameters.
4. `RoadmapService` loads related data through `RoadmapRepository` and applies business rules for Analyze Skill Gap.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in `Roadmaps`, `RoadmapProgress`, `CareerPathSkills` and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.22.2.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.22.2.6 Participating Components

- **Boundary/UI – AnalyzeSkillGapPage:** collects input and displays the final result for Analyze Skill Gap.
- **Controller – RoadmapController:** receives REST requests and validates request DTOs.
- **Service – RoadmapService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – Roadmap:** represents the main persisted domain object.
- **Repository – RoadmapRepository:** provides data access operations for the related table(s).

3.22.2.7 Class Diagram

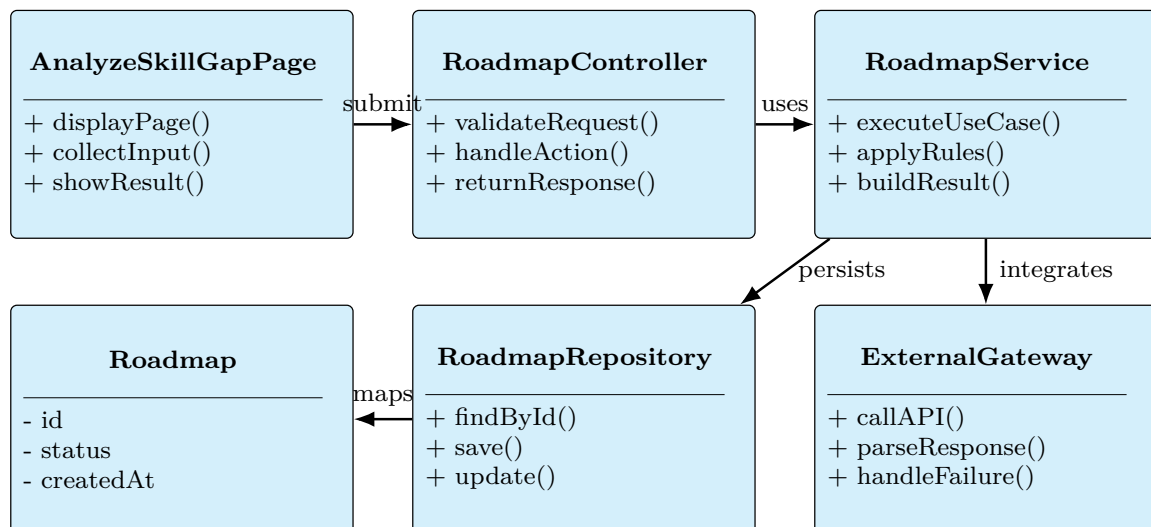


Figure IV.151: Class diagram for UC74 Analyze Skill Gap

3.22.2.8 Sequence Diagram

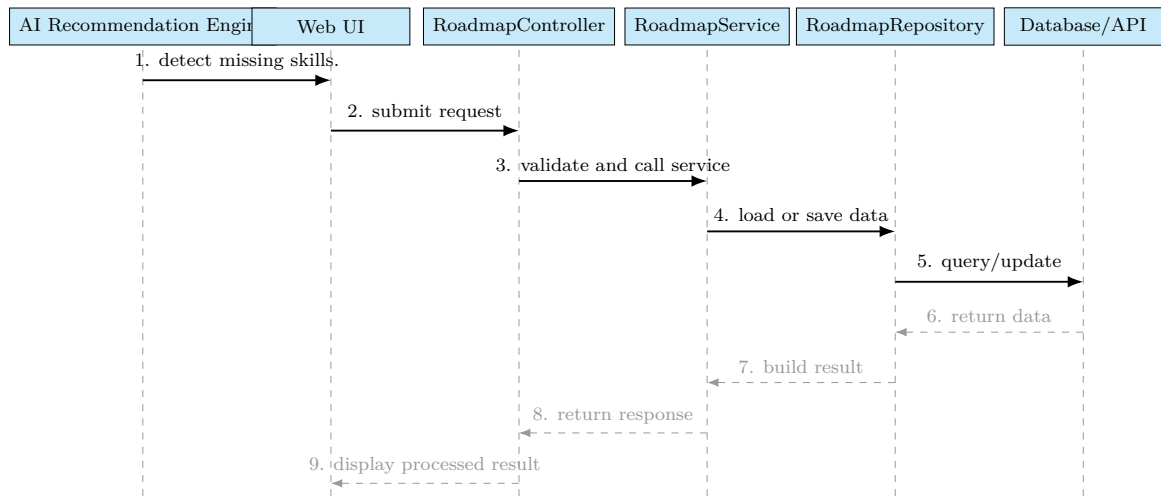


Figure IV.152: Sequence diagram for UC74 Analyze Skill Gap

3.22.3 UC75 Recommend Courses

3.22.3.1 Requirement Traceability

- **Use case ID:** UC75.
- **Use case name:** Recommend Courses.
- **Actor:** AI Recommendation Engine.
- **Requirement description:** Recommend learning resources.
- **Related table(s):** LearningResources, StudentCourses, SkillNodes.

3.22.3.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in **Users.Role** must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.22.3.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.22.3.4 Main Processing Flow

1. The actor opens **RecommendCoursesPage** and performs the action: recommend learning resources..

2. The frontend validates required fields, prepares the request DTO, and sends it to **ResourceController**.
3. **ResourceController** checks authentication, authorization, request format, and path parameters.
4. **ResourceService** loads related data through **LearningResourceRepo** and applies business rules for Recommend Courses.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in **LearningResources**, **StudentCourses**, **SkillNodes** and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.22.3.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.22.3.6 Participating Components

- **Boundary/UI – RecommendCoursesPage:** collects input and displays the final result for Recommend Courses.
- **Controller – ResourceController:** receives REST requests and validates request DTOs.
- **Service – ResourceService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – LearningResource:** represents the main persisted domain object.
- **Repository – LearningResourceRepo:** provides data access operations for the related table(s).

3.22.3.7 Class Diagram

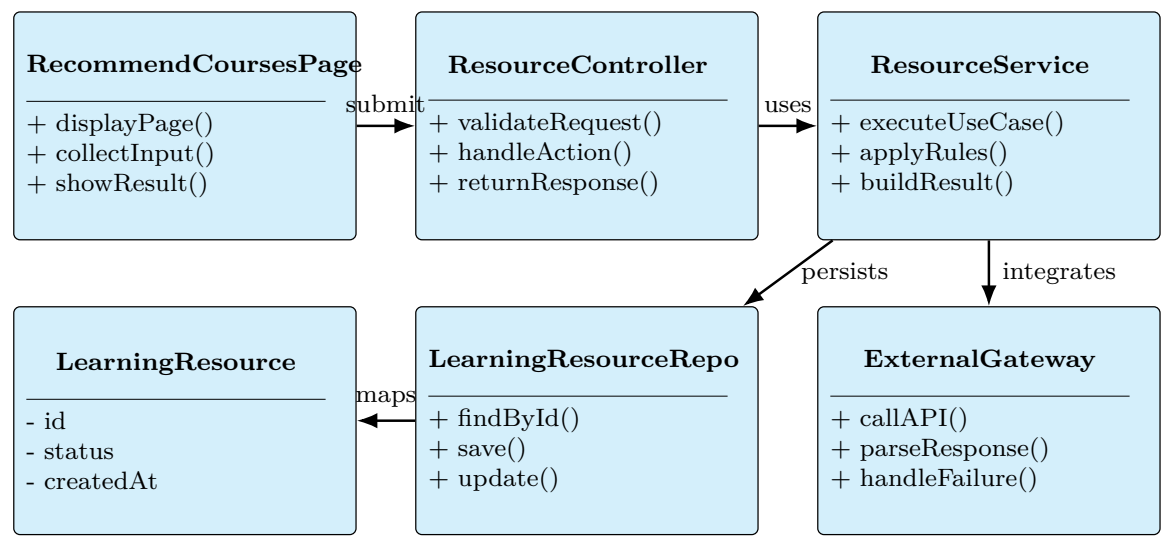


Figure IV.153: Class diagram for UC75 Recommend Courses

3.22.3.8 Sequence Diagram

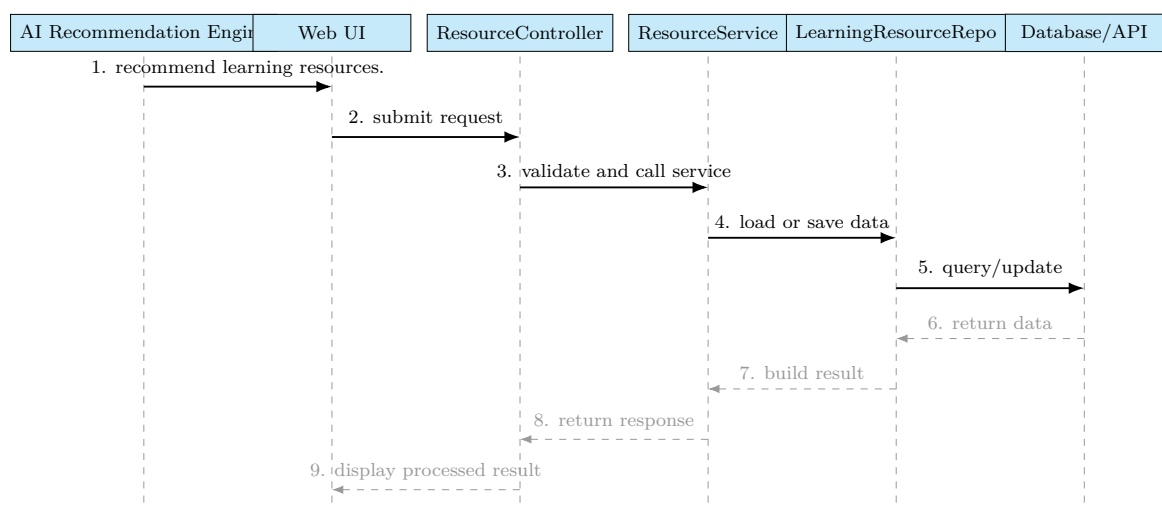


Figure IV.154: Sequence diagram for UC75 Recommend Courses

3.23 AI Mentor and Analysis Logging

This group contains use cases related to ai mentor and analysis logging. The design uses the layered architecture introduced in the system design: React boundary pages, Spring Boot controllers, services, repositories, database entities, and external gateways when required.

3.23.1 UC76 Generate Portfolio Summary

3.23.1.1 Requirement Traceability

- **Use case ID:** UC76.
- **Use case name:** Generate Portfolio Summary.
- **Actor:** AI Recommendation Engine.
- **Requirement description:** Summarize portfolio.
- **Related table(s):** AIConfigurations, AIAnalysisLogs, StudentProfiles.

3.23.1.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in **Users.Role** must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.23.1.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.23.1.4 Main Processing Flow

1. The actor opens **GeneratePortfolioSPage** and performs the action: summarize portfolio..
2. The frontend validates required fields, prepares the request DTO, and sends it to **AIController**.
3. **AIController** checks authentication, authorization, request format, and path parameters.
4. **AIAnalysisService** loads related data through **AIAnalysisLogRepo** and applies business rules for Generate Portfolio Summary.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in **AIConfigurations**, **AIAnalysisLogs**, **StudentProfiles** and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.23.1.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.23.1.6 Participating Components

- **Boundary/UI – GeneratePortfolioSPage:** collects input and displays the final result for Generate Portfolio Summary.
- **Controller – AIController:** receives REST requests and validates request DTOs.
- **Service – AIAnalysisService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – AIAnalysisLog:** represents the main persisted domain object.
- **Repository – AIAnalysisLogRepo:** provides data access operations for the related table(s).

3.23.1.7 Class Diagram

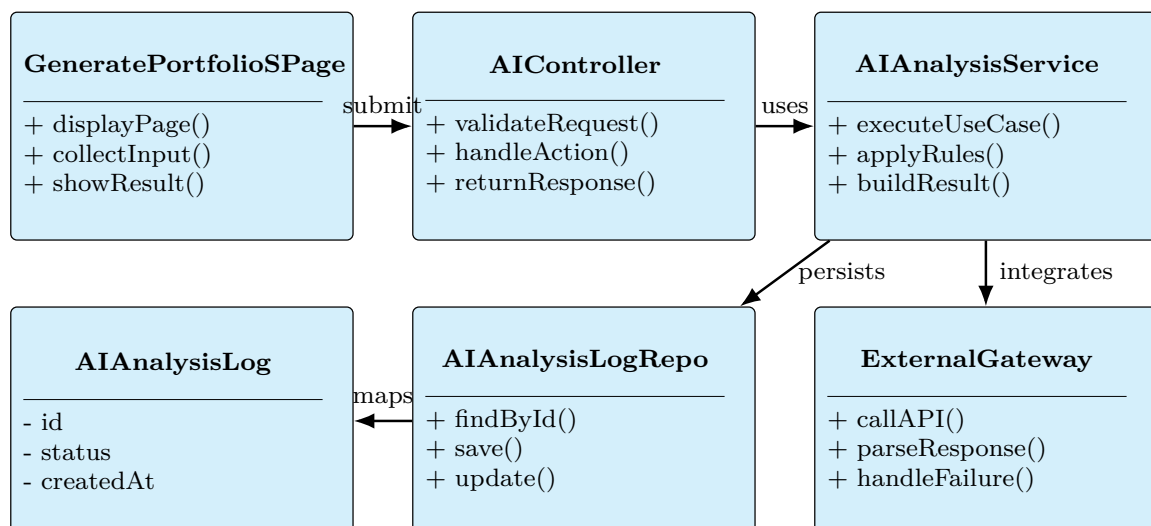


Figure IV.155: Class diagram for UC76 Generate Portfolio Summary

3.23.1.8 Sequence Diagram

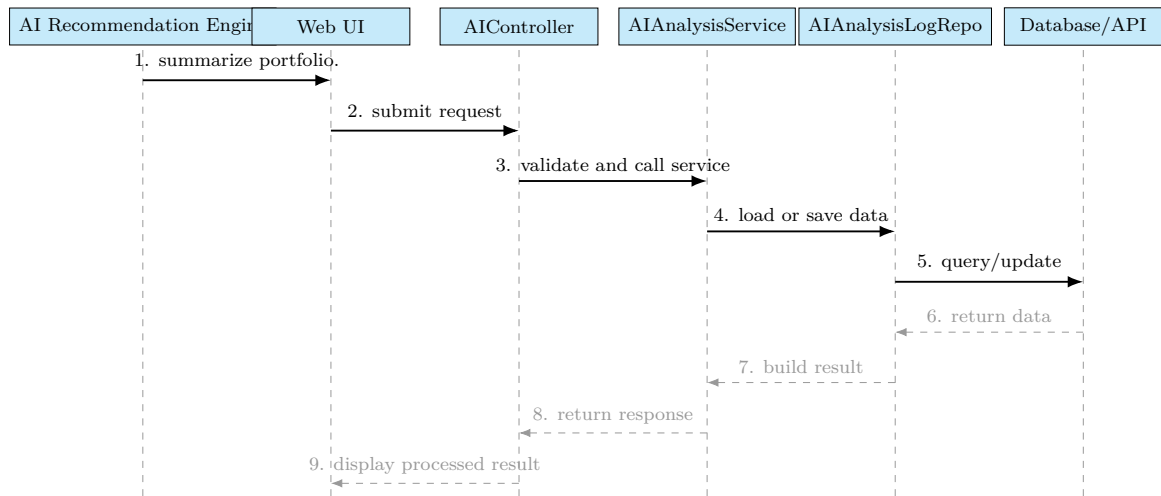


Figure IV.156: Sequence diagram for UC76 Generate Portfolio Summary

3.24 Authentication and Account Access

This group contains use cases related to authentication and account access. The design uses the layered architecture introduced in the system design: React boundary pages, Spring Boot controllers, services, repositories, database entities, and external gateways when required.

3.24.1 UC77 Authenticate User

3.24.1.1 Requirement Traceability

- **Use case ID:** UC77.
- **Use case name:** Authenticate User.
- **Actor:** Google OAuth.
- **Requirement description:** Authenticate user account.
- **Related table(s):** Users.

3.24.1.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.24.1.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.24.1.4 Main Processing Flow

1. The actor opens **AuthenticateUserPage** and performs the action: authenticate user account..
2. The frontend validates required fields, prepares the request DTO, and sends it to **AuthController**.
3. **AuthController** checks authentication, authorization, request format, and path parameters.
4. **AuthService** loads related data through **UserRepository** and applies business rules for Authenticate User.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in **Users** and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.24.1.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.24.1.6 Participating Components

- **Boundary/UI – AuthenticateUserPage:** collects input and displays the final result for Authenticate User.
- **Controller – AuthController:** receives REST requests and validates request DTOs.
- **Service – AuthService:** coordinates business logic, integration calls, and transaction boundaries.

- **Entity – User:** represents the main persisted domain object.
- **Repository – UserRepository:** provides data access operations for the related table(s).

3.24.1.7 Class Diagram

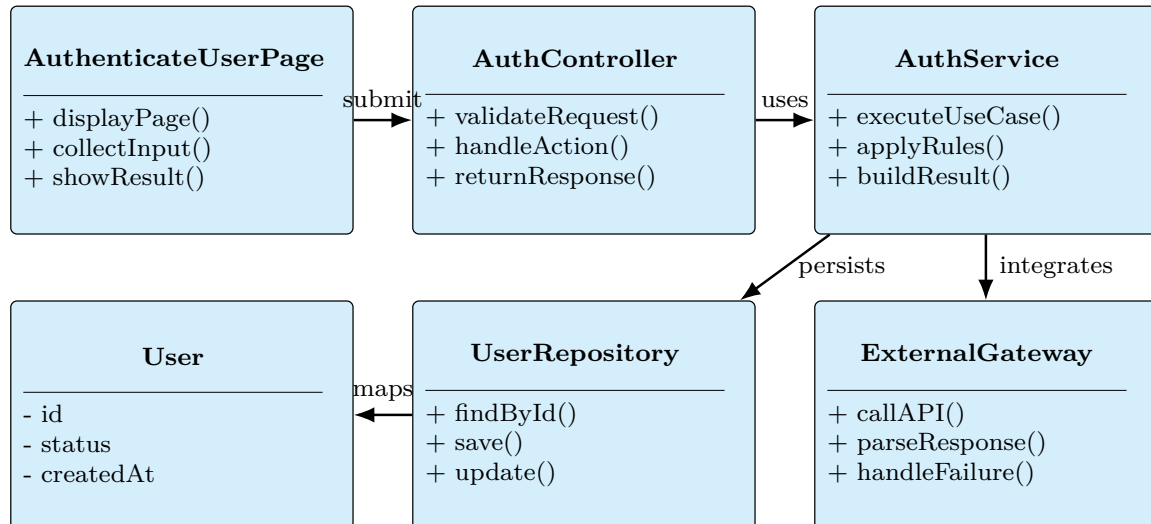


Figure IV.157: Class diagram for UC77 Authenticate User

3.24.1.8 Sequence Diagram

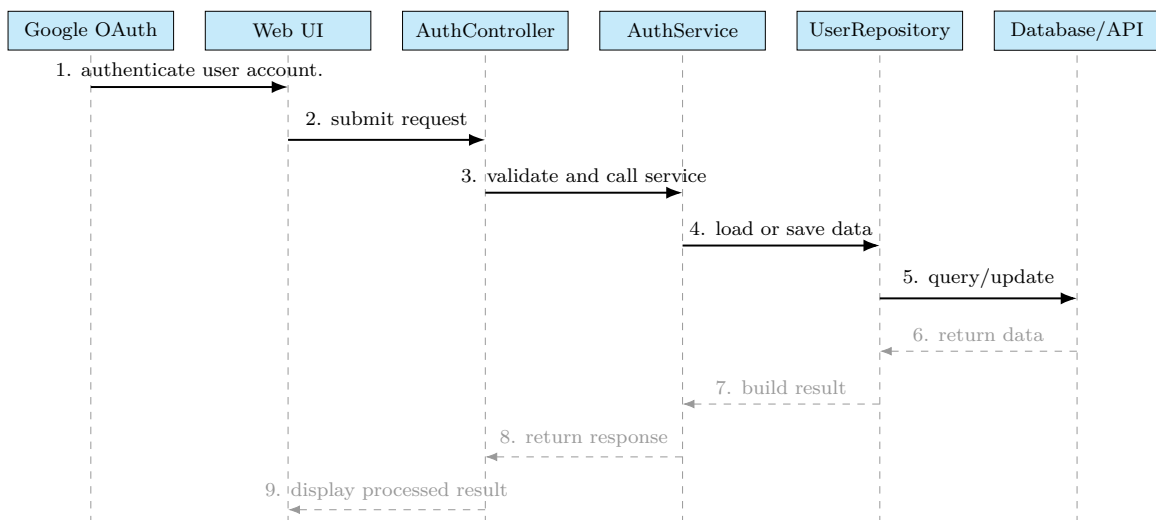


Figure IV.158: Sequence diagram for UC77 Authenticate User

3.25 Student Profile and GitHub Integration

This group contains use cases related to student profile and github integration. The design uses the layered architecture introduced in the system design: React boundary pages, Spring Boot controllers, services, repositories, database entities, and external gateways when required.

3.25.1 UC78 Retrieve Repository

3.25.1.1 Requirement Traceability

- **Use case ID:** UC78.
- **Use case name:** Retrieve Repository.
- **Actor:** GitHub API.
- **Requirement description:** Retrieve repository information.
- **Related table(s):** `StudentProfiles`, `GitHubRepositories`, `GitHubRepoLanguages`.

3.25.1.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.25.1.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.25.1.4 Main Processing Flow

1. The actor opens `RetrieveRepositoryPage` and performs the action: retrieve repository information..
2. The frontend validates required fields, prepares the request DTO, and sends it to `GitHubController`.
3. `GitHubController` checks authentication, authorization, request format, and path parameters.
4. `GitHubSyncService` loads related data through `GitHubRepositoryRepo` and applies business rules for Retrieve Repository.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in `StudentProfiles`, `GitHubRepositories`, `GitHubRepoLanguages` and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.25.1.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.25.1.6 Participating Components

- **Boundary/UI – RetrieveRepositoryPage:** collects input and displays the final result for Retrieve Repository.
- **Controller – GitHubController:** receives REST requests and validates request DTOs.
- **Service – GitHubSyncService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – GitHubRepository:** represents the main persisted domain object.
- **Repository – GitHubRepositoryRepo:** provides data access operations for the related table(s).

3.25.1.7 Class Diagram

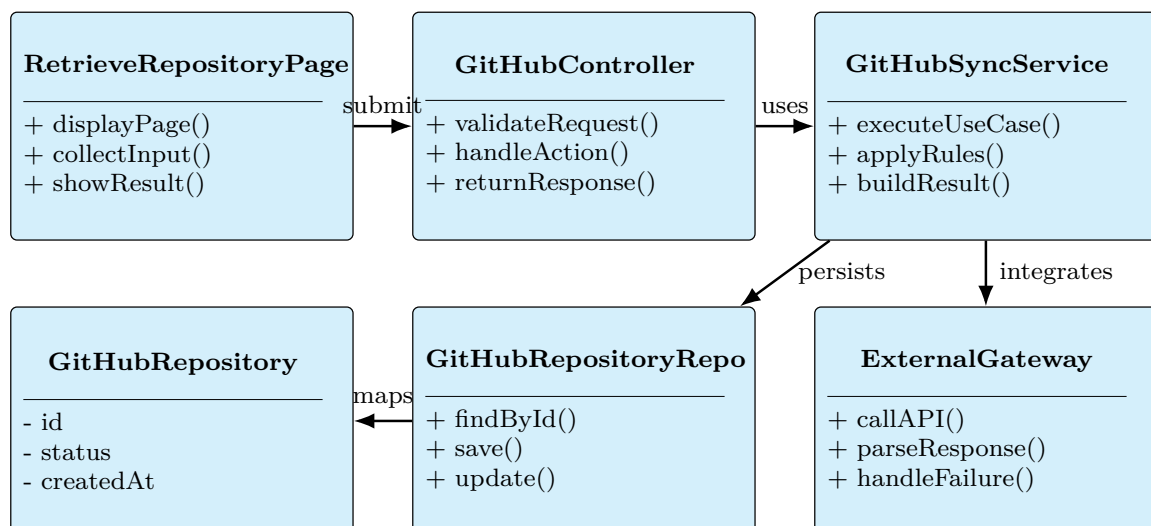


Figure IV.159: Class diagram for UC78 Retrieve Repository

3.25.1.8 Sequence Diagram

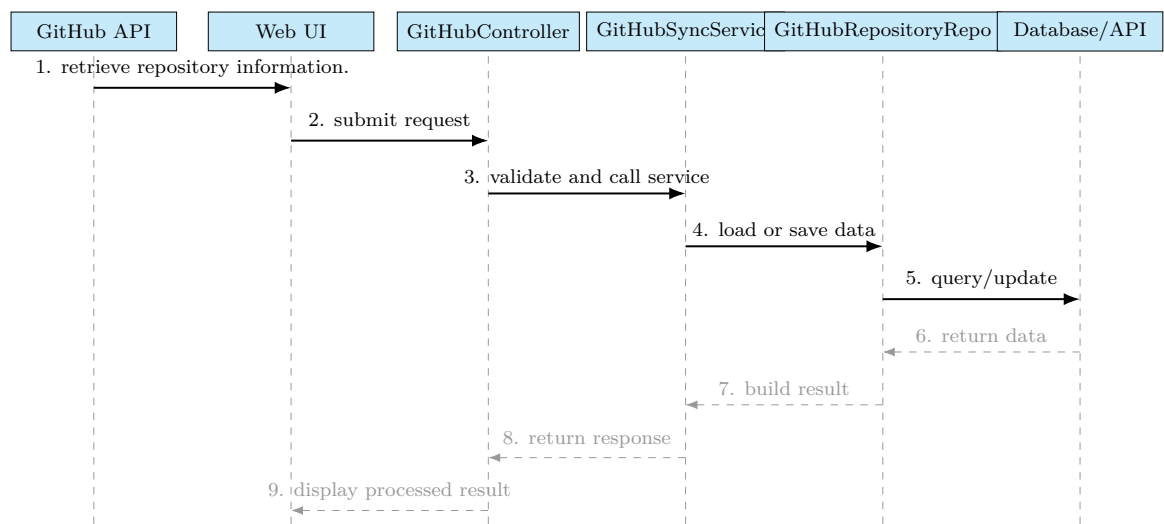


Figure IV.160: Sequence diagram for UC78 Retrieve Repository

3.25.2 UC79 Retrieve README

3.25.2.1 Requirement Traceability

- **Use case ID:** UC79.
- **Use case name:** Retrieve README.
- **Actor:** GitHub API.
- **Requirement description:** Retrieve README file.
- **Related table(s):** StudentProfiles, GitHubRepositories, GitHubRepoLanguages.

3.25.2.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.25.2.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.25.2.4 Main Processing Flow

1. The actor opens `RetrieveREADMEPage` and performs the action: retrieve README file..

2. The frontend validates required fields, prepares the request DTO, and sends it to `GitHubController`.
3. `GitHubController` checks authentication, authorization, request format, and path parameters.
4. `GitHubSyncService` loads related data through `GitHubRepositoryRepo` and applies business rules for Retrieve README.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in `StudentProfiles`, `GitHubRepositories`, `GitHubRepoLanguages` and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.25.2.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.25.2.6 Participating Components

- **Boundary/UI – RetrieveREADMEPage:** collects input and displays the final result for Retrieve README.
- **Controller – GitHubController:** receives REST requests and validates request DTOs.
- **Service – GitHubSyncService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – GitHubRepository:** represents the main persisted domain object.
- **Repository – GitHubRepositoryRepo:** provides data access operations for the related table(s).

3.25.2.7 Class Diagram

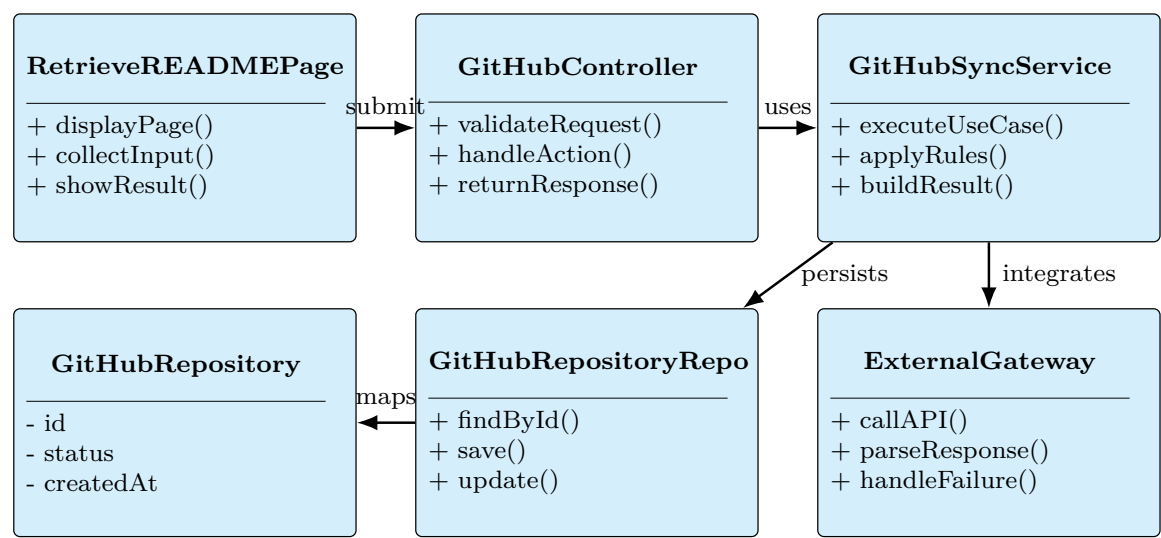


Figure IV.161: Class diagram for UC79 Retrieve README

3.25.2.8 Sequence Diagram

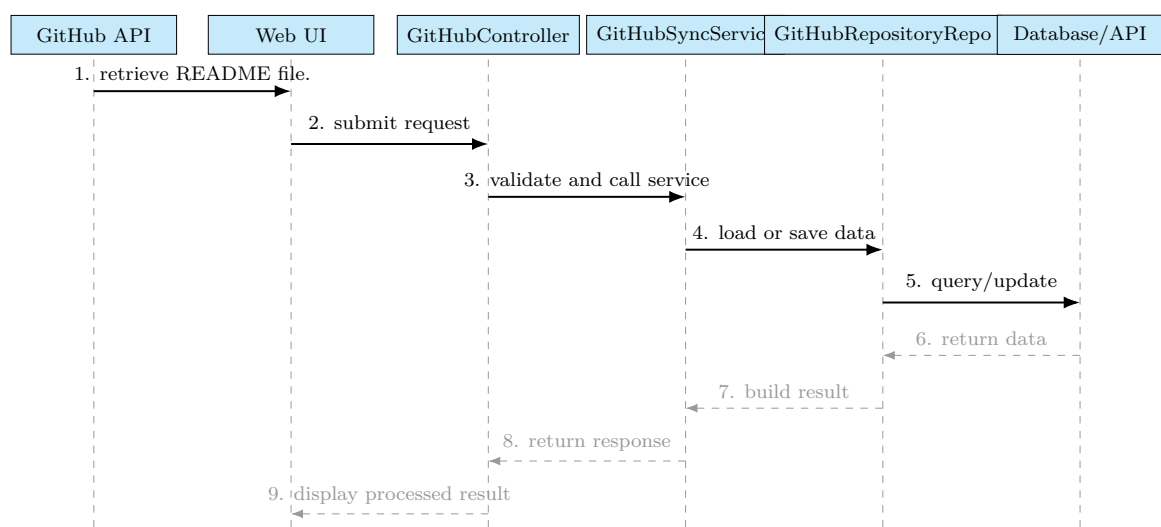


Figure IV.162: Sequence diagram for UC79 Retrieve README

3.25.3 UC80 Retrieve Tech Stack

3.25.3.1 Requirement Traceability

- Use case ID: UC80.
- Use case name: Retrieve Tech Stack.
- Actor: GitHub API.
- Requirement description: Retrieve technology stack.
- Related table(s): StudentProfiles, GitHubRepositories, GitHubRepoLanguages.

3.25.3.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.25.3.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.25.3.4 Main Processing Flow

1. The actor opens `RetrieveTechStackPage` and performs the action: retrieve technology stack..
2. The frontend validates required fields, prepares the request DTO, and sends it to `GitHubController`.
3. `GitHubController` checks authentication, authorization, request format, and path parameters.
4. `GitHubSyncService` loads related data through `GitHubRepositoryRepo` and applies business rules for Retrieve Tech Stack.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in `StudentProfiles`, `GitHubRepositories`, `GitHubRepoLanguages` and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.25.3.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.25.3.6 Participating Components

- **Boundary/UI – RetrieveTechStackPage:** collects input and displays the final result for Retrieve Tech Stack.
- **Controller – GitHubController:** receives REST requests and validates request DTOs.
- **Service – GitHubSyncService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – GitHubRepository:** represents the main persisted domain object.
- **Repository – GitHubRepositoryRepo:** provides data access operations for the related table(s).

3.25.3.7 Class Diagram

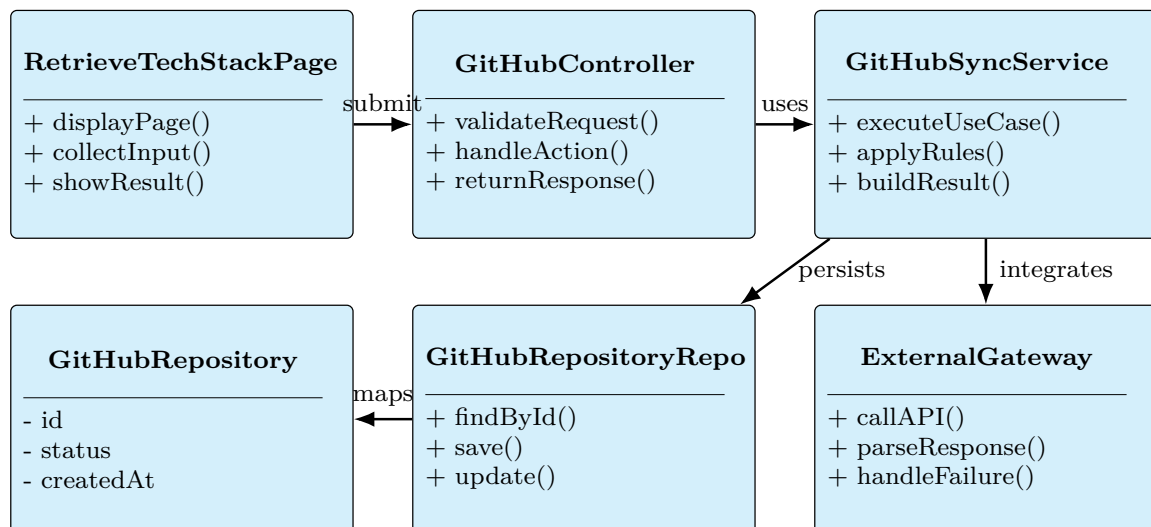


Figure IV.163: Class diagram for UC80 Retrieve Tech Stack

3.25.3.8 Sequence Diagram

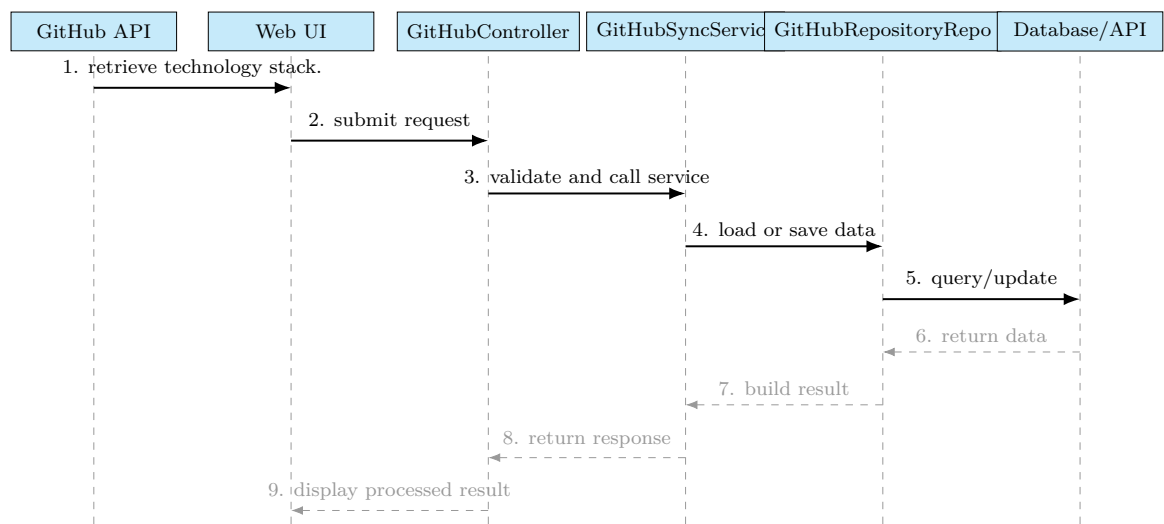


Figure IV.164: Sequence diagram for UC80 Retrieve Tech Stack

3.26 Market Trend Management

This group contains use cases related to market trend management. The design uses the layered architecture introduced in the system design: React boundary pages, Spring Boot controllers, services, repositories, database entities, and external gateways when required.

3.26.1 UC81 Retrieve Job Data

3.26.1.1 Requirement Traceability

- **Use case ID:** UC81.
- **Use case name:** Retrieve Job Data.
- **Actor:** Job Portal API.
- **Requirement description:** Retrieve job information.
- **Related table(s):** JobTrends, CareerPaths.

3.26.1.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.26.1.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.26.1.4 Main Processing Flow

1. The actor opens **RetrieveJobDataPage** and performs the action: retrieve job information..
2. The frontend validates required fields, prepares the request DTO, and sends it to **MarketTrendController**.
3. **MarketTrendController** checks authentication, authorization, request format, and path parameters.
4. **MarketTrendService** loads related data through **JobTrendRepository** and applies business rules for Retrieve Job Data.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in **JobTrends**, **CareerPaths** and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.26.1.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.
- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.26.1.6 Participating Components

- **Boundary/UI – RetrieveJobDataPage:** collects input and displays the final result for Retrieve Job Data.
- **Controller – MarketTrendController:** receives REST requests and validates request DTOs.
- **Service – MarketTrendService:** coordinates business logic, integration calls, and transaction boundaries.

- **Entity – JobTrend:** represents the main persisted domain object.
- **Repository – JobTrendRepository:** provides data access operations for the related table(s).

3.26.1.7 Class Diagram

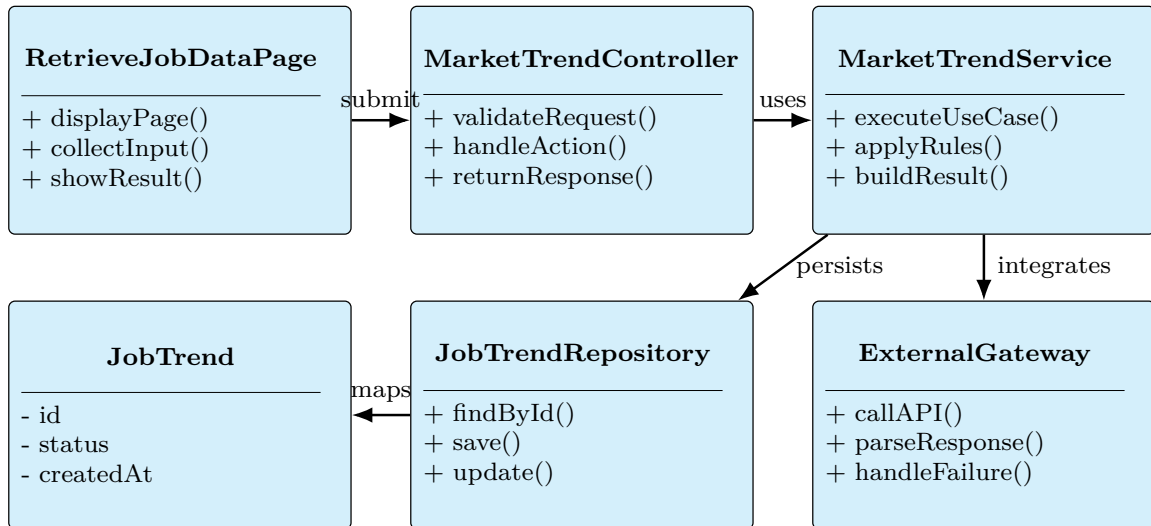


Figure IV.165: Class diagram for UC81 Retrieve Job Data

3.26.1.8 Sequence Diagram

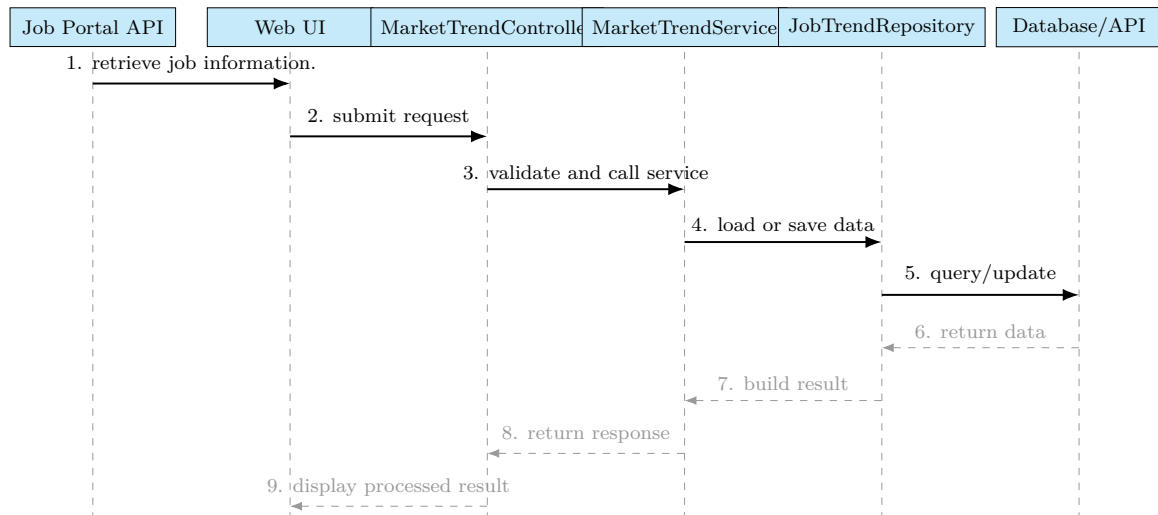


Figure IV.166: Sequence diagram for UC81 Retrieve Job Data

3.26.2 UC82 Analyze Market Trends

3.26.2.1 Requirement Traceability

- **Use case ID:** UC82.

- **Use case name:** Analyze Market Trends.
- **Actor:** Job Portal API.
- **Requirement description:** Analyze labor market trends.
- **Related table(s):** JobTrends, CareerPaths.

3.26.2.2 Preconditions

The actor has access to the corresponding interface. For protected functions, the actor must be authenticated, the account must be active, and the role stored in `Users.Role` must be allowed to perform this use case. Required master data such as career paths, skill nodes, learning resources, or staff profiles must already exist when the use case depends on them.

3.26.2.3 Postconditions

The system stores the created or updated data in the related database table(s), returns a response DTO to the frontend, and records logs or notifications when the use case requires auditing, AI analysis, reminder, or integration feedback.

3.26.2.4 Main Processing Flow

1. The actor opens `AnalyzeMarketTrendPage` and performs the action: analyze labor market trends..
2. The frontend validates required fields, prepares the request DTO, and sends it to `MarketTrendController`.
3. `MarketTrendController` checks authentication, authorization, request format, and path parameters.
4. `MarketTrendService` loads related data through `JobTrendRepository` and applies business rules for Analyze Market Trends.
5. If the use case needs external data, the service calls the proper gateway such as Google OAuth, GitHub API, AI service, or job portal API.
6. The service creates, updates, reads, or deletes records in `JobTrends`, `CareerPaths` and builds the response DTO.
7. The frontend displays the processed result and updates the visible state for the actor.

3.26.2.5 Alternative and Exception Flow

- If required input is missing or invalid, the controller returns validation errors and no database change is committed.
- If the actor does not have permission, the security layer returns an unauthorized or forbidden response.

- If a referenced record does not exist, the service returns a not-found response.
- If an external API fails, the service handles the failure, records the issue when applicable, and returns a retryable error message.

3.26.2.6 Participating Components

- **Boundary/UI – AnalyzeMarketTrendPage:** collects input and displays the final result for Analyze Market Trends.
- **Controller – MarketTrendController:** receives REST requests and validates request DTOs.
- **Service – MarketTrendService:** coordinates business logic, integration calls, and transaction boundaries.
- **Entity – JobTrend:** represents the main persisted domain object.
- **Repository – JobTrendRepository:** provides data access operations for the related table(s).

3.26.2.7 Class Diagram

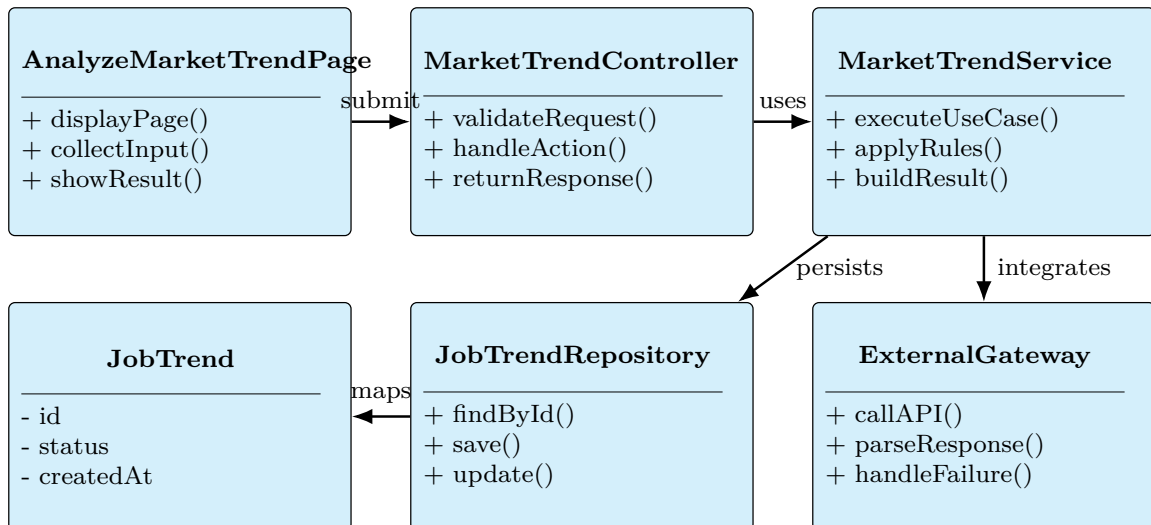


Figure IV.167: Class diagram for UC82 Analyze Market Trends

3.26.2.8 Sequence Diagram

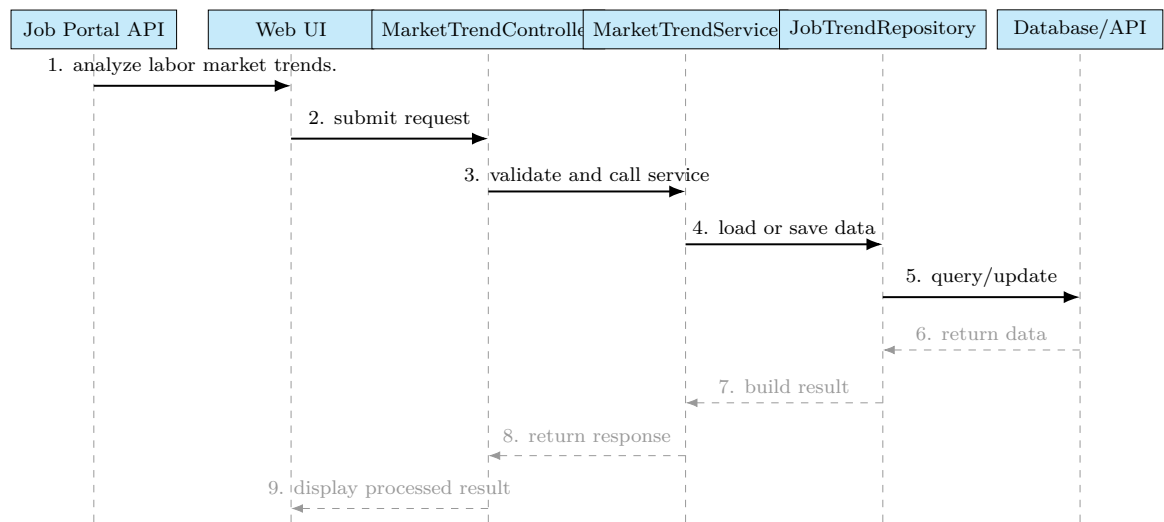


Figure IV.168: Sequence diagram for UC82 Analyze Market Trends

V. Software Testing Documentation

This chapter will be updated later.

VI. Requirement Appendix

1. Business Rules

The following business rules define the constraints and policies applied throughout the Personalized Career Orientation & Learning Roadmap Platform.

Table VI.1: Business Rules

ID	Rule Definition
BR-01	A student can own only one profile in the system.
BR-02	A student can select only one target career role at a time.
BR-03	A personalized roadmap can only be generated after a target career role has been selected.
BR-04	Each Skill Node belongs to exactly one Tech Path.
BR-05	A Skill Node cannot be marked as completed unless it exists in the student's roadmap.
BR-06	Roadmap progress is automatically recalculated whenever the completion status changes.
BR-07	AI recommendations are generated using the student's transcript, GitHub repositories, and selected career role.
BR-08	A student may upload multiple transcripts, but only the latest version is used for analysis.
BR-09	One GitHub account can only be linked to one system account.
BR-10	Only public GitHub repositories are synchronized.
BR-11	Repositories without a README.md file are not analyzed by the AI.
BR-12	Each repository is synchronized only once during a synchronization cycle.
BR-13	One AI-generated summary is created for every synchronized repository.
BR-14	Skill Gap Analysis compares only the skills required for the selected career role.
BR-15	Missing skills are prioritized according to industry demand and prerequisite relationships.
BR-16	Generated Skill Gap reports are stored for future reference.
BR-17	Market Pulse data is automatically updated every 24 hours.
BR-18	Only verified job postings are included in technology trend analysis.
BR-19	Technology popularity is calculated using keyword frequency extracted from job descriptions.
BR-20	All AI mentor conversations are stored in the user's chat history.
BR-21	Users may delete their chat history without affecting roadmap data.
BR-22	Each student owns exactly one shareable E-Portfolio URL.
BR-23	Generating a new E-Portfolio URL invalidates the previous one.
BR-24	Only authenticated users can access personalized recommendations.
BR-25	Google OAuth users must authenticate with a verified Google

2. Assumptions and Dependencies

This section describes the assumptions made during system development and the external dependencies required for system operation.

2.1 Assumptions

- Users have a stable Internet connection.
- GitHub repositories are publicly accessible.
- Uploaded academic transcripts are valid and readable.
- AI services (OpenAI or Gemini) are available during recommendation requests.
- External job portals provide accessible data for technology trend analysis.

2.2 Dependencies

Table VI.2: External Dependencies

Service	Purpose
OpenAI API / Gemini API	Generate AI career recommendations and chatbot responses.
GitHub REST API	Synchronize repositories and retrieve README files.
Google OAuth 2.0	User authentication.
Job Portal APIs	Collect job market information and technology trends.
PDF Generator	Export Skill Gap Analysis reports.

3. Data Dictionary

3.1 Student Profile

Table VI.3: Student Profile Entity

Attribute	Type	Description
StudentId	UUID	Unique identifier
FullName	String	Student full name
Email	String	Student email
GPA	Float	Grade Point Average
CareerRole	String	Selected career role
GitHubURL	String	GitHub profile link
Transcript	File	Uploaded transcript

3.2 Tech Path

Table VI.4: Tech Path Entity

Attribute	Type	Description
PathId	UUID	Unique identifier
PathName	String	Career path name
Description	Text	Career path description

3.3 Skill Node

Table VI.5: Skill Node Entity

Attribute	Type	Description
SkillId	UUID	Skill identifier
SkillName	String	Skill name
ParentSkill	UUID	Parent node
Status	Boolean	Completion status
Priority	Integer	Learning priority

3.4 Course Repository

Table VI.6: Course Repository Entity

Attribute	Type	Description
RepoId	UUID	Repository identifier
RepositoryName	String	Repository name
TechStack	Text	Technologies used
Summary	Text	AI-generated summary
RepositoryURL	String	GitHub URL

3.5 Job Trend

Table VI.7: Job Trend Entity

Attribute	Type	Description
TrendId	UUID	Trend identifier
Skill	String	Technology name
Frequency	Integer	Number of occurrences
GrowthRate	Float	Growth percentage
UpdatedAt	DateTime	Last updated time

4. Glossary

Table VI.8: Glossary

Term	Definition
AI Virtual Mentor	Artificial Intelligence chatbot providing personalized career advice.
Tech Path	Personalized career learning roadmap.
Skill Node	An individual technical skill in the roadmap.
Skill Gap	Missing skills compared to the target career.
Market Pulse	Technology trend analysis generated from job postings.
GitHub Sync	Synchronization of public GitHub repositories.
README Analysis	AI analysis of README.md files.
E-Portfolio	Automatically generated online portfolio.
Repository Summary	AI-generated project summary.
Career Role	Target software engineering position.

5. Requirement Traceability Matrix

Table VI.9: Requirement Traceability Matrix

Functional Requirement	Main Entity
FR1.1 – FR1.3	Student Profile, Mentor Session
FR2.1 – FR2.4	Tech Path, Skill Node
FR3.1 – FR3.3	Student Profile, Skill Node
FR4.1 – FR4.3	Job Trend
FR5.1 – FR5.3	Course Repository, Student Profile
FR6.1	Student Profile
FR6.2	Student Profile, Mentor Session

VII. Release Package & User Guides

This chapter presents the deliverable package of Release 1.0, the installation guide for setting up the **Personalized Career Orientation & Learning Roadmap Platform for Software Engineering Students** (also referred to as *SE Career Compass*), and the user manual describing how each actor operates the system.

1. Deliverable Package

Table [VII.1](#) lists the artifacts delivered together with Release 1.0 of the platform, along with the tools used to manage and store them throughout the project.

Table VII.1: Deliverable Package

tableheader No.	Deliverable Item	Description
1	Schedule / Task Tracking	Sprint schedule, backlog items, and daily tasks are tracked using Trello boards; meeting notes, requirement documents, and reports are stored on Google Drive .
2	Project Backlog	Trello boards (To Do / In Progress / Review / Done) are used to organize the product backlog, sprint backlog, and track the progress of user stories and tasks.
3	Source Code	The Release 1.0 demo (front-end prototype) is hosted on GitHub: - Repository: <code>phucvobao/Software-Engineering-Students</code> - Live demo: https://phucvobao.github.io/Software-Engineering-Students/
4	Database Script(s)	N/A for Release 1.0 — the demo build uses local (browser-side) persistence to simulate user data; the relational schema designed for the full system is provided in the Software Design chapter.
5	Final Report Document	This report, including the Project Charter, SRS, Software Design, Testing Documentation, and Release Package & User Guides, delivered as a PDF document.
6	Diagrams & Supporting Assets	Context Diagram, Use Case Diagram, ERD, and Class Diagram used throughout the report (folder <code>images/</code> and <code>diagrams/</code>).

2. Installation Guides

2.1 System Requirements

Because Release 1.0 is delivered as a lightweight, static web demo (HTML/CSS/JavaScript) hosted on GitHub Pages, the platform only requires a modern web browser and an internet connection to run. Table VII.2 summarizes the recommended environment for development and for end users.

Table VII.2: System Requirements

Component	Minimum	Recommended
CPU	Dual-core 1.6 GHz	Quad-core 2.0 GHz or higher
Memory	4 GB RAM	8 GB RAM or higher
Storage	200 MB free space	1 GB free space (for local clone and cache)
Network	5 Mbps	20 Mbps or higher
Web Browser	Chrome / Edge / Firefox (latest 2 major versions)	Google Chrome (latest stable version)
Development Tools	Node.js 18+, Git	Node.js 20 LTS, Git, Visual Studio Code

2.2 Installation Instructions

2.2.1 Option 1 — Access the Online Demo

The fastest way to explore Release 1.0 is through the hosted GitHub Pages demo, which requires no installation:

- Step 1: Open a web browser.
- Step 2: Navigate to <https://phucvobao.github.io/Software-Engineering-Students/>.
- Step 3: The landing page of *SE Career Compass* will load directly; no login credentials are required to explore the demo flows.

2.2.2 Option 2 — Run the Project Locally

Developers or evaluators who want to run and modify the source code can set up the project locally by following these steps:

- Step 1: Download & install **Git**: <https://git-scm.com/>.
- Step 2: Download & install **Node.js** (v18 or higher): <https://nodejs.org/>.
- Step 3: Download & install **Visual Studio Code**: <https://code.visualstudio.com/>.
- Step 4: Open a terminal and clone the repository:

```
git clone https://github.com/phucvobao/Software-Engineering-Students.git
```

- Step 5: Navigate into the project folder:

```
cd Software-Engineering-Students
```

- Step 6: If the project uses npm dependencies, install them:

```
npm install
```

- Step 7: Serve the site locally (for a static site, either open `index.html` directly in a browser or run a local server, e.g.):

```
npx serve .
```

- Step 8: Open the address shown in the terminal (e.g. `http://localhost:3000`) in a web browser to view the application.

2.2.3 Deployment

The demo is deployed automatically to **GitHub Pages** whenever changes are pushed to the repository's publishing branch, so the public URL always reflects the latest committed version of Release 1.0.

3. User Manual

3.1 Overview

The platform serves four internal actors, each with a dedicated working area, as summarized below.

Student

- Authentication: Register, Login (email/password or Google OAuth), Forgot Password, Logout.
- Profile: Manage profile, upload avatar, upload transcript, upload CV, link GitHub account.
- Career Orientation: Chat with the AI Virtual Mentor, view recommended career roles, compare career paths, view required skills.
- Learning Roadmap: Generate and view the personalized learning roadmap (skill tree), update progress, receive learning reminders.
- Skill Gap Analysis: View skill gap report and export it as a PDF.
- Market Pulse: View labor-market trends, technology trends, salary information, and job recommendations.
- E-Portfolio: Sync GitHub repositories, generate an e-portfolio, and share it via a public link.

Academic Counselor

- Login to the counselor workspace; view student profiles and skill gap reports; recommend career paths and courses; track student progress; comment on learning plans; schedule and review consultations.

Industry Mentor

- Login to the mentor workspace; view student e-portfolios and projects; recommend technologies and provide technical advice; evaluate progress; approve portfolios and suggest improvements.

Administrator

- Manage user accounts, career roles, the skill graph, courses, learning resources, job-trend data, AI configuration, and system reports.

3.2 Guideline

3.2.1 Login

- Step 1: Open the platform's landing page and click **Login**.

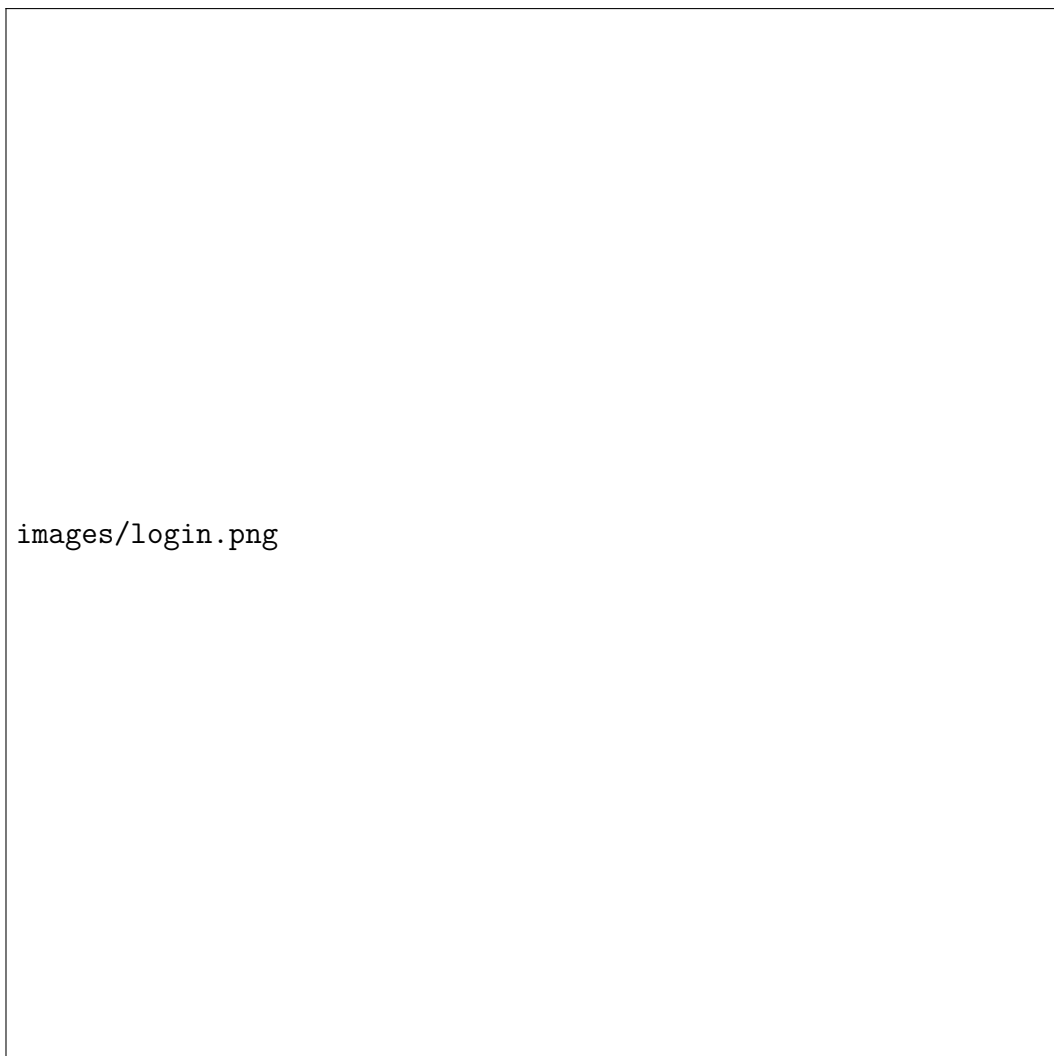


Figure VII.1: Login Screen

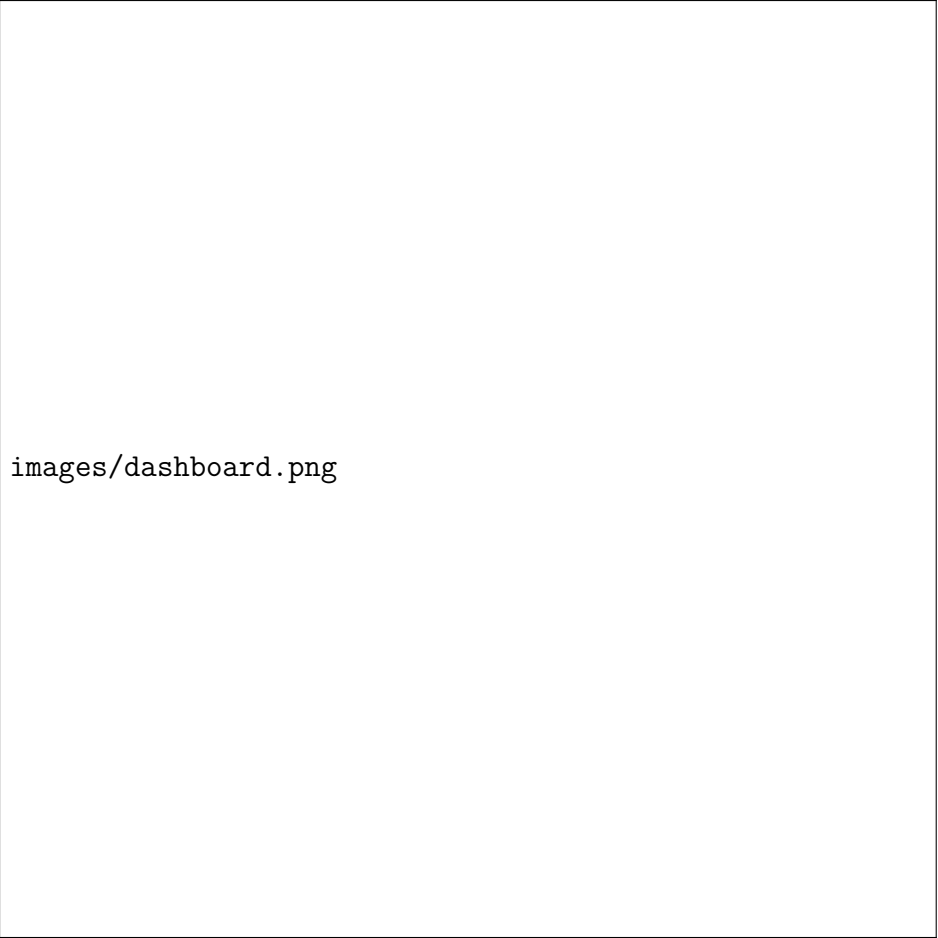
- Step 2: Sign in with a registered email/password, or click **Continue with Google** to authenticate via Google OAuth.



images/google-login.png

Figure VII.2: Continue with Google Authentication

- Step 3: Upon successful authentication, the user is redirected to the workspace corresponding to their role (Student, Academic Counselor, or Industry Mentor).



images/dashboard.png

Figure VII.3: Student Dashboard

3.2.2 Chat with the AI Virtual Mentor (FR1)

- Step 1: From the Student dashboard, select **AI Mentor**.

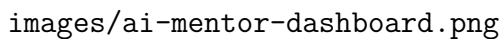
The image placeholder shows the text 'images/ai-mentor-dashboard.png' in a monospaced font, indicating the location of the AI Mentor Interface dashboard image.

Figure VII.4: AI Mentor Interface

- Step 2: Optionally attach context such as transcript data, GPA, or a linked GitHub profile so the mentor can personalize its guidance.
- Step 3: Type a career or learning question (e.g., "What should I learn to become a Cloud Architect?") and send it.
- Step 4: Review the AI Mentor's suggested career direction and recommended next steps directly in the chat window.

3.2.3 View and Update the Dynamic Learning Roadmap (FR2)

- Step 1: Select a target role (e.g., *Cloud Architect*) from the career selection screen.
- Step 2: Open **Roadmap** to view the hierarchical skill tree generated for that role.
- Step 3: Mark a skill node as completed once the corresponding learning milestone is finished; the completion percentage updates automatically.

3.2.4 Run a Skill Gap Analysis (FR3)

- Step 1: Open **Skill Gap Analysis** from the Student dashboard.
- Step 2: The system compares the student's current skills against the requirements of the selected target role.

- Step 3: Review the list of missing competencies and click **Export Report** to download the analysis as a PDF.

3.2.5 Check Market Pulse (FR4)

- Step 1: Open **Market Pulse** from the navigation menu.
- Step 2: Browse trending job keywords, in-demand technologies, and salary statistics aggregated from job-market sources such as LinkedIn and TopCV.

3.2.6 Build and Share an E-Portfolio (FR5)

- Step 1: Open **E-Portfolio** and click **Sync GitHub** to connect a GitHub account.
- Step 2: Select the repositories to include; the platform assembles a project story from each repository's activity.
- Step 3: Click **Generate Portfolio** to build the portfolio page, then click **Share** to obtain a public link that can be sent to recruiters or mentors.

3.2.7 Academic Counselor — Review a Student

- Step 1: Login to the counselor workspace and open **Student List**.
- Step 2: Select a student to view their profile and skill gap report.
- Step 3: Add a recommended career path or course, or leave a comment on the student's learning plan.

3.2.8 Industry Mentor — Evaluate a Portfolio

- Step 1: Login to the mentor workspace and open **Portfolio Reviews**.
- Step 2: Open a student's e-portfolio and review the listed projects.
- Step 3: Leave technical feedback, suggest improvements, or approve the portfolio.